

ADVANCED PYTHON PROGRAMMING

AUTHOR PROFILE



Dr.S.Moses Praveen Mukharjee is a distinguished Academician with over 24 years of Experience in the field of Computer Science Education. He is currently serving as a Professor in the Department of Computer Science and Applications at East Point College of Higher Education, Bangalore. With a strong background in both Teaching and Research. He has contributed significantly to the Academic Community through Numerous National and International Publications. He has published Papers in National, International, Scopus and Web of Science Journals. His expertise spans various domains of Computer Science and he is committed to Advancing Knowledge and Fostering Innovation in the Field. A dedicated Educator and Researcher. He has continued to inspire and Mentor Students, shaping the next generation of Technology Professionals.



Dr.K.Adishesha currently working as Professor and Coordinator in Department of Computer Science and Application at SEA College of Science Commerce and Arts (Autonomous). is a distinguished academician and researcher in Computer Science and Application with over 25 years of Teaching experience and Industry exposure. He holds a Ph.D. in Computer Science with a specialization in Data Analytics and Machine Learning. He has contributed extensively to higher education through teaching, research, curriculum development, and academic leadership. His areas of expertise include Artificial Intelligence, Machine Learning, Data Science, Data Mining, Business Analytics, Internet of Things, and Cloud Computing. He has authored multiple textbooks and published numerous research papers in reputed national and international journals. He is also an active contributor to Conferences, Faculty development programs, and Institutional initiatives. Recognised with several awards for academic excellence, he is committed to fostering innovation, critical thinking, and practical knowledge among students, making significant contributions to the advancement of computer science education and research.

 **SURYA**
PUBLICATION
Coimbatore, Tamilnadu, India.
www.suryapublication.com



ISBN 978-81-69406-97-0

MRP: ₹400

ADVANCED PYTHON PROGRAMMING

ADVANCED PYTHON PROGRAMMING



Dr.S.Moses Praveen Mukharjee
Dr.K.Adishesha

Advanced Python Programming

Dr. Moses Praveen Mukherjee

MCA, B.Ed., Ph.D.

*Professor, Department of Computer Applications,
East Point College of Higher Education
Avalahalli, Bangalore, India.*

Dr. K. Adishesha

BE(CES), MCA, M.Sc. (IT), MBA, UGC-NET, Ph.D.

*Professor and Coordinator in Computer Science and Application,
SEA college of Science Commerce and Arts (Autonomous),
K R Puram, Bangalore, India.*

PUBLISHED BY



SURYA PUBLICATION

**No.44, 7th Street, Arul Nagar,
Rathinapuri, Coimbatore – 641027.
Tamilnadu, India.**

Copyright © 2026 Surya Publication Inc. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or any information storage and retrieval system, without permission in writing from the publisher. Details on how to seek permission, further information about the Publisher's permissions policies and our arrangements with organizations such as the Copyright Clearance Center and the Copyright Licensing Agency, can be found at our website: suryapublication.com/permissions.

This book and the individual contributions contained in it are protected under copyright by the Publisher (other than as may be noted herein).

Publisher ©

The publisher recognizes and respects all marks used by companies, manufacturers, and developers as a means to distinguish their products. All brand names and product names mentioned in this book are trademarks or service marks of their respective companies. Any omission or misuse (of any kind) of service marks or trademarks, etc. is not an attempt to infringe on the property of others.

Edition: First

Year: 2026

ISBN: 978-81-69406-97-0



For information visit our website at
www.suryapublication.com

Published by:



Phone: +91 93631 38729

Mail : suryabookpublicitions@gmail.com

No.44, 7th Street, Arul Nagar,

Rathinapuri, Coimbatore – 641027.



AUTHOR PROFILE



Dr.S.Moses Praveen Mukharjee is a distinguished Academician with over 24 years of Experience in the field of Computer Science Education. He is currently serving as a Professor in the Department of Computer Science and Applications at East Point College of Higher Education, Bangalore. With a strong background in both Teaching and Research. He has contributed significantly to the Academic Community through Numerous National and International Publications. He has published Papers in National, International, Scopus and Web of Science Journals. His expertise spans various domains of Computer Science and he is committed to Advancing Knowledge and Fostering Innovation in the Field. A dedicated Educator and Researcher. He has continued to inspire and Mentor Students, shaping the next generation of Technology Professionals.



Dr.K.Adishesha currently working as Professor and Coordinator in Department of Computer Science and Application at SEA College of Science Commerce and Arts (Autonomous). is a distinguished academician and researcher in Computer Science and Application with over 25 years of Teaching experience and Industry exposure. He holds a Ph.D. in Computer Science with a specialization in Data Analytics and Machine Learning. He has contributed extensively to higher education through teaching, research, curriculum development, and academic leadership. His areas of expertise include Artificial Intelligence, Machine Learning, Data Science, Data Mining, Business Analytics, Internet of Things, and Cloud Computing. He has authored multiple textbooks and published numerous research papers in reputed national and international journals. He is also an active contributor to Conferences, Faculty development programs, and Institutional initiatives. Recognised with several awards for academic excellence, he is committed to fostering innovation, critical thinking, and practical knowledge among students, making significant contributions to the advancement of computer science education and research.

PREFACE

Python has become one of the most powerful and widely used programming languages in the world. Its simplicity makes it easy to learn, but its capabilities make it strong enough to build complex and advanced applications. From web development to data science, automation, and artificial intelligence, Python continues to play a key role in modern technology.

This book, *Advanced Python Programming*, is designed to take your knowledge beyond the basics. It focuses on helping readers understand how to write efficient, clean, and scalable Python code. The aim is to make advanced concepts easier to understand by using simple language, clear explanations, and practical examples. In this book, you will explore important topics such as object-oriented programming, decorators, generators, file handling, exception management, and working with libraries and frameworks. It also introduces concepts like multithreading, performance optimization, and best coding practices that are essential for real-world applications.

This book is suitable for students, developers, and anyone who already has a basic understanding of Python and wants to improve their skills. It helps you think like a programmer and solve problems in a more structured and effective way.

The goal of this book is not just to teach advanced features, but to build confidence in using Python for real-world projects. I hope this book supports your learning journey and inspires you to create powerful and innovative solutions using Python.

Chapter 1: Python Basics

Introduction to Python Programming	1
Installing Python	2
Features of Python	4
Python Programming	6
• Identifiers	7
• Keywords	11
• Statements and Expressions	12
• Variables	16
• Operators	20
• Precedence and Association	26
• Data Types	30
• Indentation	36
• Comments	37
• Built-in Functions	38
Console Input and Console Output	42
Type Conversions	43
Python Libraries	45
Importing Libraries with Examples	48
Control Flow Statements-	49
• if, else, elif, while loop, break,	56
• continue statements,	60
• for loop Statement;	61
• range () and exit () functions	65

Chapter 2: Functions in Python Programming

Python functions:	67
Function return Types	72
Function with parameters	74
Function scope	75
Local and Global functions	80
Function composition	84
User Defined Functions	89
Function Defining	93
Function Calling	95
Types of Functions,	98
Function Arguments	101

Lambda Function	103
Importing Modules.....	106
Recursion	

Chapter 3: Python Data Structures

String, Lists, Tuples, Dictionaries, Set, Arrays	113
String Functions:	120
• String slices,	125
• Immutability,	128
• String functions and methods.....	130
• String module	132
Lists:	135
list operations,	139
• List slices	141
• List methods	144
• List loop	147
• Mutability	149
• Aliasing	150
• cloning lists	152
• list parameters	153
Tuples:	154
• Operations and methods	158
• Tuple assignment	163
• Tuple as return value.....	170
Dictionaries:	172
• Dictionaries assignment	175
• Dictionaries as return value	179
Advanced list processing	180
• list comprehension.	185
• Lists as arrays	187

Chapter 4: Regular Expression

Regular Expression	189
Python RegEx	190
Concept of regular expression,	208
Various types of regular expressions.....	210
Metacharacters	216
Using match function.....	217
Special Sequences	222
Sets.....	223

Chapter 5: Files, Modules and Packages

Files and Exception	227
File Types.....	223
Text files	235
Reading and Writing files	238
Format operator.....	242
Command line arguments	247
Errors and exceptions	250
Handling exceptions	252
Modules	256
Python Packages	258
• Creating and Accessing Packages	260
• Python Packages for Web Frameworks.....	262

Chapter 6: Database connectivity

Python Database	273
Relational Databases (RDBMS)	274
Non-relational Databases (NoSQL)	278
Database Programming:	283
• Connecting to database	286
• Creating Tables	288
• Insert	291
• Update	293
• Delete and Read operations	294
• Transaction Control,	296
Disconnecting from a database, Exception Handling in Databases.....	299

Chapter 7: Data Analysis using Python

Introduction to Data Analysis	303
Introduction to NumPy,	321
Array Creation using NumPy	326
Operations on Arrays	330
Introduction to Pandas	334
Series and Data Frames.....	337
Creating Data Frames from Excel Sheet and .csv file	343
Dictionary and Tuples	346
Operations on Data Frames	351

Chapter 8: Data Visualisation using Python

Data Visualisation	355
Introduction to Data Visualisation	356
Matplotlib Library	365
Different Types of Charts using Pyplot-	373
• Line chart	389
• Bar chart	399
• Histogram	405
• Pie chart.....	410
• Scatter Plot.....	417
• Box Plot.....	421
• Area Plot.....	425
• Heatmap	431
Python Programming Lab Manual	436

CHAPTER 1

PYTHON BASICS

Introduction to Python Programming

Python is a high-level programming language known for its simple and readable syntax. It has the following features:

- Allows writing programs with fewer lines of code, improving readability.
- Automatically detects variable types at runtime, eliminating the need for explicit declarations.
- Used in web development, data analysis, automation and many other fields.
- Supports object-oriented, functional and procedural programming styles.
- Dynamically typed and has automatic garbage collection.

Understanding Hello World Program

This is a comment. It will not be executed.

```
print("Hello, World!")
```

Output

Hello, World!

How does this work:



- `print()` is a built-in Python function that instructs the computer to display text on the screen.
- `"Hello, World!"` is a string, which is a sequence of text. Strings are enclosed in quotes (either single `'` or double `"`).
- The scope of code blocks is defined using Indentation (spaces or tabs). This is different from C, C++ and Java which use braces `{}` to create a block of code.

Installing Python

The first step towards learning Python is to install it on your system. Whether you're using Windows, macOS, or Linux, this guide walks you through the process of downloading and installing the latest Python 3 version.

- Python is released under the Python Software Foundation License (PSF License), an open-source permissive license, and can be downloaded from python.org.
- Most Operating Systems, notably Linux, provide a package manager through which you can directly install Python.

Install on Windows

Note: The following steps apply to both Windows 10 and Windows 11.

Step 1: Download Python for Windows

- Open your browser and visit the Python download page: python.org/downloads.
- Select the latest Python 3 version, such as Python 3.13.1 (or whichever is the latest stable version available).
- Click on the version to download the installer (a .exe file for Windows).



Step 2: Run the Python Installer

Once the download is complete, run the installer program. On Windows, it will typically be a .exe file.

Install Python

- Make sure to mark Add Python to PATH otherwise you will have to do it explicitly. It will start installing Python on Windows.
- After installation is complete click on Close.

Step 3: Verify Installation

Once the installation is complete, we can check the version of Python installed by using the following command in command prompt:

`python --version`

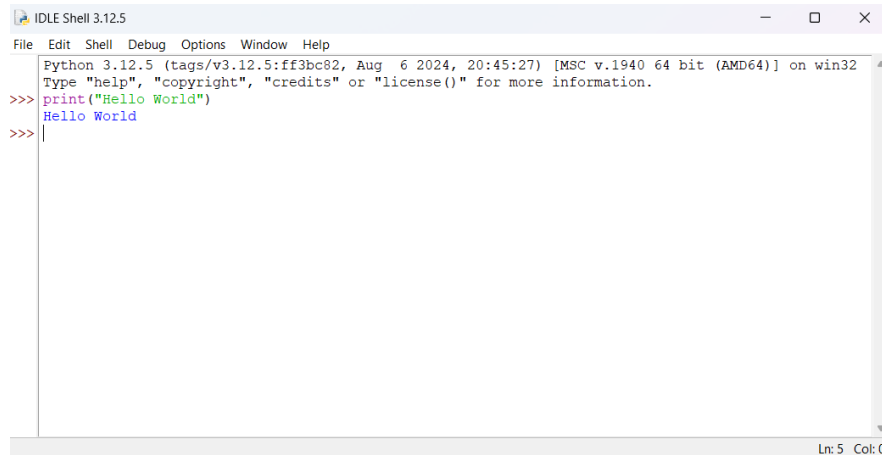
```
C:\Users\gfg0753>python --version
Python 3.13.5
```

Verifying Installation

If a Python version is displayed, Python is installed successfully. If not, repeat the above installation steps.

Running Python Shell on Windows

After installing Python, we can launch the Python Shell by searching for "IDLE" in the "Start" menu and clicking on it.



Python Shell

Install on MacOS

Installing Python on macOS can be done via Homebrew, a popular package manager for macOS.

Step 1: Install Python Using Homebrew

- Open the Terminal application (located in Applications > Utilities).
- Run the following command to install Python via Homebrew:

```
brew install python3
```

Step 2: Verify the Installation

Once the installation is complete, we can check the version of Python installed by using the following command:

```
python3 --version
```

The terminal will display the installed version, confirming that Python is ready to use.

Running Python Shell on MacOS

To run Python Shell on MacOS type "python3" in terminal (you can search it using Spotlight) and press enter.

For a more detailed guide on how to install Python on MacOS, visit: [Stepwise guide to Install Python on Mac](#)

Install on Linux

Most Linux distributions come with Python pre-installed. If you're running Linux and want to check if Python is installed:

Step 1: Check for Python Installation

Open a terminal using Ctrl+Alt+T and type:

```
$ python --version
```

If Python 3 is installed, you'll see something like:

```
Python 3.x.x
```

If by any chance, Python is not pre-installed, we can install it by using the following steps:

Step 2: Install or Upgrade Python on Linux

Open the terminal and run the following commands:

```
sudo apt install software-properties-common -y
```

```
sudo add-apt-repository ppa:deadsnakes/ppa
```

```
sudo apt update
```

```
sudo apt install python3.13
```

This will install Python 3.13 on your Linux machine.

Step 3: Verify Python Installation

To verify the installation, run the following command:

```
python3 --version
```

Running Python Shell on Linux

To launch Python Shell on linux, open the terminal using "Ctrl + Alt + T" and then type "python3" and press enter.

Features of Python

Python is a dynamic, high-level, free open source, and interpreted programming language. It supports object-oriented programming as well as procedural-oriented programming. In Python, we don't need to declare the type of variable because it is a dynamically typed language. For example, `x = 10` Here, `x` can be anything such as String, int, etc. In this article we will see what characteristics describe the python programming language

Features in Python

In this section we will see what are the features of Python programming language:

1. Free and Open Source

Python language is freely available at the official website and you can download it from the given download link below click on the **Download Python** keyword. Download Python Since it is open-source, this means that source code is also available to the public. So you can download it, use it as well as share it.

2. Easy to code

Python is a high-level programming language. Python is very easy to learn the language as compared to other languages like C, C#, Javascript, Java, etc. It is very easy to code in the Python language and anybody can learn Python basics in a few hours or days. It is also a developer-friendly language.

3. Easy to Read

As you will see, learning Python is quite simple. As was already established, Python's syntax is really straightforward. The code block is defined by the indentations rather than by semicolons or brackets.

4. Object-Oriented Language

One of the key features of Python is Object-Oriented programming. Python supports object-oriented language and concepts of classes, object encapsulation, etc.

5. GUI Programming Support

Graphical User interfaces can be made using a module such as PyQt5, PyQt4, wxPython, or Tk in Python. PyQt5 is the most popular option for creating graphical apps with Python.

6. High-Level Language

Python is a high-level language. When we write programs in Python, we do not need to remember the system architecture, nor do we need to manage the memory.

7. Large Community Support

Python has gained popularity over the years. Our questions are constantly answered by the enormous StackOverflow community. These websites have already provided answers to many questions about Python, so Python users can consult them as needed.

8. Easy to Debug

Excellent information for mistake tracing. You will be able to quickly identify and correct the majority of your program's issues once you understand how to interpret Python's error traces. Simply by glancing at the code, you can determine what it is designed to perform.

9. Python is a Portable language

Python language is also a portable language. For example, if we have Python code for Windows and if we want to run this code on other platforms such as Linux, Unix, and Mac then we do not need to change it, we can run this code on any platform.

10. Python is an Integrated language

Python is also an Integrated language because we can easily integrate Python with other languages like C, C++, etc.

11. Interpreted Language:

Python is an Interpreted Language because Python code is executed line by line at a time. like other languages C, C++, Java, etc. there is no need to compile Python code this makes it easier to debug our code. The source code of Python is converted into an immediate form called **bytecode**.

12. Large Standard Library

Python has a large standard library that provides a rich set of modules and functions so you do not have to write your own code for every single thing. There are many libraries present in Python such as regular expressions, unit-testing, web browsers, etc.

13. Dynamically Typed Language

Python is a dynamically-typed language. That means the type (for example- int, double, long, etc.) for a variable is decided at run time not in advance because of this feature we don't need to specify the type of variable.

14. Frontend and backend development

With a new project py script, you can run and write Python codes in HTML with the help of some simple tags <py-script>, <py-env>, etc. This will help you do frontend development work

in Python like javascript. Backend is the strong forte of Python it's extensively used for this work cause of its frameworks like Django and Flask.

15. Allocating Memory Dynamically

In Python, the variable data type does not need to be specified. The memory is automatically allocated to a variable at runtime when it is given a value. Developers do not need to write `int y = 18` if the integer value 15 is set to `y`. You may just type `y=18`.

Python Latest Update

The latest Python version is 3.12.6 and in this Python versions improve an efficiency, developer experience, and performance.

- **Improved Performance:** Python 3.12 introduces several optimizations, such as support for the BOLT binary optimizer, which provides an overall 5% performance improvement.
- **Enhanced Error Messages:** This version includes better error messages, with more detailed suggestions for fixing common typos and mistakes, making it more user-friendly, especially for beginners.
- **New Debugging Features:** A new debugging and profiling API was introduced, which will be especially useful for developers when profiling their applications.
- **F-String Flexibility:** Python's f-string syntax has become more flexible, allowing for more powerful and readable string formatting capabilities.
- **Type Annotation Enhancements:** Python 3.12 also introduces more advanced type annotations, especially for generic classes, making code more readable and easier to maintain.
- **Easy to learn and use:** Python's simple syntax makes it beginner-friendly
- **Extensive libraries and frameworks:** Offers rich libraries for web development, data science, AI, etc.
- **Versatile:** Supports multiple programming paradigms and applications
- **Cross-platform:** Works seamlessly across Windows, macOS, and Linux
- **Great for rapid prototyping:** Quick syntax allows for fast idea implementation

Python Applications in Real World

- Web Development
- Data Science and Analytics
- Artificial Intelligence and Machine Learning
- Automation and Scripting

Python Programming

Python is one of the most popular programming languages. It's simple to use, packed with features and supported by a wide range of libraries and frameworks. Its clean syntax makes it beginner-friendly.

- A high-level language, used in data science, automation, AI, web development and more.
- Known for its readability, which means code is easier to write, understand and maintain.

- Backed by strong library support, we don't have to build everything from scratch.

Basic Code Example

The following is a simple program that displays the message "Hello, World!" on the screen.
`Print("Hello World!")`

Output

Hello World!

Why Learn Python?

- Requires fewer lines of code compared to other programming languages like Java.
- Provides libraries and frameworks such as Django and Flask for web development and tools like Pandas, TensorFlow and Scikit-learn for artificial intelligence, machine learning and data analysis.
- Cross-platform, works on Windows, Mac and Linux without major changes.
- Used by top tech companies like Google, Netflix and NASA.
- Many Python coding job opportunities in Software Development, Data Science and AI/ML.

Identifiers

Python Keywords are reserved words with fixed meanings that define Python's syntax. Cannot be used as names. Python Identifiers are user-defined names for variables, functions, or classes. Must follow naming rules (no digits at start, only _ allowed).

Keywords in Python

- Predefined and reserved words with special meanings.
- Used to define the syntax and structure of Python code.
- Cannot be used as identifiers, variables, or function names.
- Written in lowercase, except True and False.
- Python 3.11 has 35 keywords.
- The keyword module provides:
 - **iskeyword()** → checks if a string is a keyword.
 - **kwlist** → returns the list of all keywords.

Rules for Keywords in Python

- Python keywords cannot be used as identifiers.
- All the keywords in Python should be in lowercase except True and False.

List of Python Keywords

Category	Keywords
Value Keywords	True, False, None
Operator Keywords	and, or, not, is, in

Control Flow Keywords	if, else, elif, for, while, break, continue, pass, try, except, finally, raise, assert
Function and Class	def, return, lambda, yield, class
Context Management	with, as
Import and Module	import, from
Scope and Namespace	global, nonlocal
Async Programming	async, await

Getting List of all Python keywords

We can also get all the keyword names using the below code.

```
import keyword
```

```
print(keyword.kwlist)
```

Output

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', ...]
```

Identifiers in Python

- User-defined names for variables, functions, classes, modules, etc.
- Can include letters, digits, and underscores (_).
- Case-sensitive → num, Num, and NUM are different identifiers.
- Python provides str.isidentifier() to check if a string is a valid identifier.

Rules for Naming Python Identifiers

- It cannot be a reserved python keyword.
- It should not contain white space.
- It can be a combination of A-Z, a-z, 0-9, or underscore.
- It should start with an alphabet character or an underscore (_).
- It should not contain any special character other than an underscore (_).

Examples of Python Identifiers

Valid identifiers:

- var1
- _var1
- _1_var
- var_1

Invalid Identifiers

- !var1
- 1var
- 1_var
- var#1
- var 1

Python Keywords and Identifiers Examples**Example 1: Example of and, or, not, True, False keywords.**

```
print("example of True, False, and, or, not keywords")
```

```
# compare two operands using and operator
print(True and True)
```

```
# compare two operands using or operator
print(True or False)
```

```
# use of not operator
print(not False)
```

Output

```
example of True, False, and, or, not keywords
True
True
True
```

Example 2: Example of a break, continue keywords and identifier.

```
# execute for loop
for i in range(1, 11):
```

```
    # print the value of i
    print(i)
```

```
    # check the value of i is less than 5
    # if i less than 5 then continue loop
    if i < 5:
        continue
```

```
    # if i greater than 5 then break loop
    else:
        break
```

Output

```
1
```

2
3
4
5

Example 3: example of for, in, if, elif, and else keywords.

```
# run for loop
for t in range(1, 5):
    # print one of t ==1
    if t == 1:
        print('One')
    # print two if t ==2
    elif t == 2:
        print('Two')
    else:
        print('else block execute')
```

Output

One
Two
else block execute
else block execute

Example 4: Example of def, if, and else keywords.

```
# define GFG() function using def keyword
def GFG():
    i=20
    # check I is odd or not
    # using if and else keyword
    if(I % 2 == 0):
        print("given number is even")
    else:
        print("given number is odd")

# call GFG() function
GFG()
```

Output

given number is even

Example 5: Example of try, except, raise.

```
Def fun(num):
    try:
        r = 1.0/num
    except:
```

```

    print('Exception raises')
    return
return r

print(fun(10))
print(fun(0))

```

Output

```

0.1
Exception raises
None

```

Keywords

dict.keys() method in Python returns a view object that contains all the keys of the dictionary. The returned object is dynamic, meaning any changes made to the dictionary are automatically reflected in the view. It is used when only keys are required from a dictionary.

Example: In this example, all keys of a dictionary are retrieved using keys().

```

d = {'A': 'Geeks', 'B': 'For', 'C': 'Geeks'}
print(d.keys())

```

Output

```
dict_keys(['A', 'B', 'C'])
```

Explanation: d.keys() returns a view object. It contains all keys present in dictionary d.

Syntax

```
dict_name.keys()
```

- **Parameters:** No parameter required.
- **Return Type:** Returns a dynamic view object containing dictionary keys.

Examples

Example 1: In this example, the keys() method is used inside a loop to iterate over dictionary keys.

```

d = {'A': 1, 'B': 2, 'C': 3}
for k in d.keys():
    print(k)

```

Output

```

A
B
C

```

Explanation: d.keys() returns all keys and the loop accesses each key one by one.

Example 2: In this example, the dynamic nature of the keys() view object is demonstrated.

```

d = {'A': 10, 'B': 20}
k = d.keys()
d['C'] = 30

```

```
print(k)
```

Output

```
dict_keys(['A', 'B', 'C'])
```

Explanation:

- `k = d.keys()` stores the view object.
- After adding `'C' = 30` in the dictionary, the change appears in `k` also.

Example 3: In this example, the `keys()` view object is converted into a list.

```
d = {'X': 100, 'Y': 200, 'Z': 300}
```

```
k = list(d.keys())
```

```
print(k)
```

Output

```
['X', 'Y', 'Z']
```

Explanation: `d.keys()` returns a view object, `list(d.keys())` converts it into a list. This allows indexing or list operations.

Statements and Expressions

An expression is a combination of operators and operands that is interpreted to produce some other value. In any programming language, an expression is evaluated as per the precedence of its operators. So that if there is more than one operator in an expression, their precedence decides which operation will be performed first. We have many different types of expressions in Python. Let's discuss all types along with some exemplar codes :

1. Constant Expressions: These are the expressions that have constant values only.

Example:

```
# Constant Expressions
```

```
x = 15 + 1.3
```

```
print(x)
```

Output

```
16.3
```

2. Arithmetic Expressions: An arithmetic expression is a combination of numeric values, operators, and sometimes parenthesis. The result of this type of expression is also a numeric value. The operators used in these expressions are arithmetic operators like addition, subtraction, etc. Here are some arithmetic operators in Python:

Operators	Syntax	Functioning
+	<code>x + y</code>	Addition
-	<code>x - y</code>	Subtraction
*	<code>x * y</code>	Multiplication
/	<code>x / y</code>	Division

//	x // y	Quotient
%	x % y	Remainder
**	x ** y	Exponentiation

Example:

Let's see an exemplar code of arithmetic expressions in Python :

```
# Arithmetic Expressions
```

```
x = 40
```

```
y = 12
```

```
add = x + y
```

```
sub = x - y
```

```
pro = x * y
```

```
div = x / y
```

```
print(add)
```

```
print(sub)
```

```
print(pro)
```

```
print(div)
```

Output

```
52
```

```
28
```

```
480
```

```
3.3333333333333335
```

3. Integral Expressions: These are the kind of expressions that produce only integer results after all computations and type conversions.

Example:

```
# Integral Expressions
```

```
a = 13
```

```
b = 12.0
```

```
c = a + int(b)
```

```
print(c)
```

Output

```
25
```

4. Floating Expressions: These are the kind of expressions which produce floating point numbers as result after all computations and type conversions.

Example:

```
# Floating Expressions
```

```
a = 13
```

```
b = 5
```

```
c = a / b
print(c)
```

Output

2.6

5. Relational Expressions: In these types of expressions, arithmetic expressions are written on both sides of relational operator ($>$, $<$, $>=$, $<=$). Those arithmetic expressions are evaluated first, and then compared as per relational operator and produce a boolean output in the end. These expressions are also called Boolean expressions.

Example:

```
# Relational Expressions
```

```
a = 21
b = 13
c = 40
d = 37
```

```
p = (a + b) >= (c - d)
print(p)
```

Output

True

6. Logical Expressions: These are kinds of expressions that result in either True or False. It basically specifies one or more conditions. For example, $(10 == 9)$ is a condition if 10 is equal to 9. As we know it is not correct, so it will return False. Studying logical expressions, we also come across some logical operators which can be seen in logical expressions most often. Here are some logical operators in Python:

Operator	Syntax	Functioning
and	P and Q	It returns true if both P and Q are true otherwise returns false
or	P or Q	It returns true if at least one of P and Q is true
not	not P	It returns true if condition P is false

Example:

Let's have a look at an exemplar code :

```
P = (10 == 9)
Q = (7 > 5)
```

```
# Logical Expressions
```

```
R = P and Q
S = P or Q
T = not P
```



```
print(R)
print(S)
print(T)
```

Output

```
False
True
True
```

7. Bitwise Expressions: These are the kind of expressions in which computations are performed at bit level.

Example:

```
# Bitwise Expressions
a = 12
```

```
x = a >> 2
y = a << 1
```

```
print(x, y)
```

Output

```
3 24
```

8. Combinational Expressions: We can also use different types of expressions in a single expression, and that will be termed as combinational expressions.

Example:

```
# Combinational Expressions
a = 16
b = 12
```

```
c = a + (b >> 1)
print(c)
```

Output

```
22
```

But when we combine different types of expressions or use multiple operators in a single expression, operator precedence comes into play.

Multiple operators in expression (Operator Precedence)

It's a quite simple process to get the result of an expression if there is only one operator in an expression. But if there is more than one operator in an expression, it may give different results on basis of the order of operators executed. To sort out these confusions, the operator precedence is defined. Operator Precedence simply defines the priority of operators that which

operator is to be executed first. Here we see the operator precedence in Python, where the operator higher in the list has more precedence or priority:

Precedence	Name	Operator
1	Parenthesis	() [] {}
2	Exponentiation	**
3	Unary plus or minus, complement	-a , +a , ~a
4	Multiply, Divide, Modulo	/ * // %
5	Addition & Subtraction	+ -
6	Shift Operators	>> <<
7	Bitwise AND	&
8	Bitwise XOR	^
9	Bitwise OR	
10	Comparison Operators	>= <= > <
11	Equality Operators	== !=
12	Assignment Operators	= += -= /= *=
13	Identity and membership operators	is, is not, in, not in
14	Logical Operators	and, or, not

So, if we have more than one operator in an expression, it is evaluated as per operator precedence. For example, if we have the expression “10 + 3 * 4”. Going without precedence it could have given two different outputs 22 or 52. But now looking at operator precedence, it must yield 22. Let’s discuss this with the help of a Python program:

Multi-operator expression

```
a = 10 + 3 * 4
print(a)
b = (10 + 3) * 4
print(b)
c = 10 + (3 * 4)
print(c)
```

Output

```
22
52
22
```

Hence, operator precedence plays an important role in the evaluation of a Python expression.

Variables

Variables are used to store data that can be referenced and manipulated during program execution. A variable is essentially a name that is assigned to a value.

- Unlike Java and many other languages, Python variables do not require explicit declaration of type.
- Type of the variable is inferred based on the value assigned.

```
x = 5
name = "Samantha"
print(x)
print(name)
```

Output

```
5
Samantha
```

Rules for Naming Variables

To use variables effectively, we must follow Python's naming rules:

1. Variable names can only contain letters, digits and underscores (_).
2. A variable name cannot start with a digit.
3. Variable names are case-sensitive like myVar and myvar are different.
4. Avoid using Keywords like if, else, for as variable names.

Below listed variable names are valid:

```
age = 21
_colour = "lilac"
total_score = 90
```

Below listed variables names are invalid:

```
1name = "Error" # Starts with a digit
class = 10      # 'class' is a reserved keyword
user-name = "Doe" # Contains a hyphen
```

Assigning Values to Variables

Basic Assignment: Variables are assigned values using the = operator.

```
x = 5
y = 3.14
z = "Hi"
```

Dynamic Typing: variables are dynamically typed, meaning the same variable can hold different types of values during execution.

```
x = 10
x = "Now a string"
```

Multiple Assignments

Assigning Same Value: allows assigning the same value to multiple variables in a single line, which can be useful for initializing variables with the same value.

```
a = b = c = 100
print(a, b, c)
```

Output

```
100 100 100
```

Assigning Different Values: we can assign different values to multiple variables simultaneously, making the code concise and easier to read.

```
x, y, z = 1, 2.5, "Python"
```

```
print(x, y, z)
```

Output

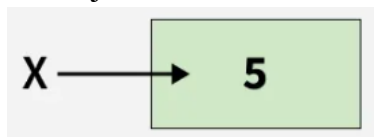
```
1 2.5 Python
```

Concept of Object Reference

Let us assign a variable x to value 5.

```
x = 5
```

When `x = 5` is executed, Python creates an object to represent the value 5 and makes x reference this object.

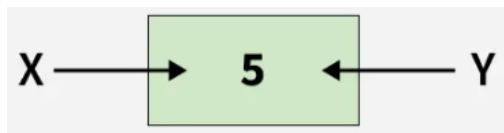


object reference

Now, let's assign another variable y to the variable x.

```
y = x
```

This statement creates y and references the same object as x, not x itself. This is called a Shared Reference, where multiple variables reference the same object.

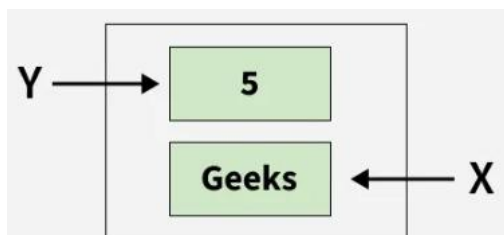


shared reference

Now, if we write

```
x = 'Geeks'
```

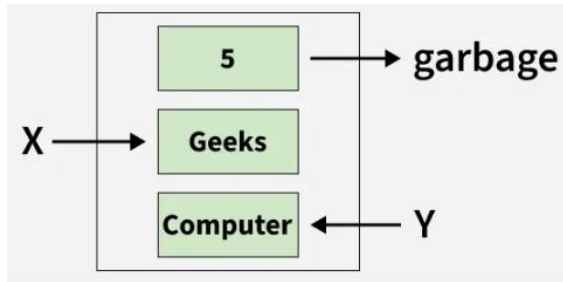
Python creates a new object for the value "Geeks" and makes x reference this new object.



shared reference example

The variable y remains unchanged, still referencing the original object 5. Now, If we assign a new value to y:

```
y = "Computer"
```



shared reference example

- Python creates yet another object for "Computer" and updates y to reference it.
- The original object 5 no longer has any references and becomes eligible for garbage collection.
- Python variables hold references to objects, not the actual objects themselves.
- Reassigning a variable does not affect other variables referencing the same object unless explicitly updated.

Understanding Variable Reassignment

In this example, we check whether modifying one variable affects another when both initially reference the same object.

```
x = 1
y = x
y = y + 1
```

```
print(x)
print(y)
```

Output

```
1
2
```

Explanation:

- Initially, both x and y reference the same object 1
- When `y = y + 1` is executed, python creates a new object 2
- y now references this new object and x still references the original object 1
- So, changing y does NOT affect x

Type and Casting a Variable

- Type of a variable can be determine using `type()` function which returns the type of the object passed to it.
- Type casting means converting the value of one data type into another. Python provides several built-in functions to facilitate casting, including `int()`, `float()` and `str()` among others.
- For example, `int()` converts compatible values to an integer, `float()` to a floating point number and `str()` to a string.

Deleting a Variable

We can remove a variable from the namespace using the `del` keyword. This deletes the variable and frees up the memory it was using.

```
x = 10
```

```
del x
```

```
print(x)
```

Output

```
ERROR!
```

```
Traceback (most recent call last):
```

```
File "<main.py>", line 3, in <module>
```

```
NameError: name 'x' is not defined
```

Explanation:

- `del x` removes the variable `x` from memory.
- After deletion, trying to access the variable `x` results in a `NameError` indicating that the variable no longer exists.

Practical Examples

1. **Swapping Two Variables:** Using multiple assignments, we can swap the values of two variables without needing a temporary variable.

```
a, b = 5, 10
```

```
a, b = b, a
```

```
print(a, b)
```

Output

```
10 5
```

2. **Counting Characters in a String:** Assign the results of multiple operations on a string to variables in one line.

```
Word = "Python"
```

```
length = len(word)
```

```
print("Length of the word:", length)
```

Output

```
Length of the word: 6
```

Operators

In Python programming, Operators in general are used to perform operations on values and variables.

- **Operators:** Special symbols like `-`, `+`, `*`, `/`, etc.
- **Operands:** Value on which the operator is applied.

Types of Operators in Python

Type	Operators
Arithmetic Operators	+, -, *, /, %
Relational Operators	<, <=, >, >=, ==, !=
Logical Operators	AND, OR, NOT
Bitwise operator	&, , ^, ~, <<, >>
Assignment Operator	=, +=, -=, *=, %=
Ternary Operator	x if condition else y
Identity Operators	is, is not

Python operators

Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication and division.

In Python, the division operator (/) returns a floating-point result, while floor division (//) returns an integer result.

```
a = 15
```

```
b = 4
```

```
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulus:", a % b)
print("Exponentiation:", a ** b)
```

Output

Addition: 19

Subtraction: 11

Multiplication: 60

Division: 3.75

Floor Division: 3

Modulus: 3

Exponentiation: 50625

Note: Refer to Differences between / and //.

Comparison Operators

Comparison(or Relational) operators compares values. It either returns True or False according to the condition.

```
a = 13
```

```
b = 33
```

```
print(a > b)
```

```
print(a < b)
```

```
print(a == b)
```

```
print(a != b)
```

```
print(a >= b)
```

```
print(a <= b)
```

Output

```
False
```

```
True
```

```
False
```

```
True
```

```
False
```

```
True
```

Logical Operators

Logical operators perform **Logical AND**, **Logical OR** and **Logical NOT** operations. It is used to combine conditional statements.

The precedence of Logical Operators in Python is as follows:

1. Logical not
2. logical and
3. logical or

```
a = True
```

```
b = False
```

```
print(a and b)
```

```
print(a or b)
```

```
print(not a)
```

Output

```
False
```

```
True
```

```
False
```

Bitwise Operators

Bitwise operators act on bits and perform bit-by-bit operations. These are used to operate on binary numbers.

Bitwise Operators in Python are as follows:

1. Bitwise NOT

2. Bitwise Shift
3. Bitwise AND
4. Bitwise XOR
5. Bitwise OR

```
a = 10
```

```
b = 4
```

```
print(a & b)
```

```
print(a | b)
```

```
print(~a)
```

```
print(a ^ b)
```

```
print(a >> 2)
```

```
print(a << 2)
```

Output

```
0
```

```
14
```

```
-11
```

```
14
```

```
2
```

```
40
```

Assignment Operators

Assignment operators are used to assign values to the variables. This operator is used to assign the value of the right side of the expression to the left side operand. **Example:**

```
a = 10
```

```
b = a
```

```
print(b)
```

```
b += a
```

```
print(b)
```

```
b -= a
```

```
print(b)
```

```
b *= a
```

```
print(b)
```

```
b <<= a
```

```
print(b)
```

Output

```
10
```

```
20
```

```
10
```

```
100
```

```
102400
```

Identity Operators

"**is**" and "**is not**" are the identity operators and both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

is True if the operands are identical

is not True if the operands are not identical

```
a = 10
```

```
b = 20
```

```
c = a
```

```
print(a is not b)
```

```
print(a is c)
```

Output

```
True
```

```
True
```

Membership Operators

"**in**" and "**not in**" are the membership operators that are used to test whether a value or variable is in a sequence.

in True if value is found in the sequence

not in True if value is not found in the sequence

```
x = 24
```

```
y = 20
```

```
my_list = [10, 20, 30, 40, 50]
```

```
if (x not in my_list):
```

```
    print("x is NOT present in given list")
```

```
else:
```

```
    print("x is present in given list")
```

```
if (y in my_list):
```

```
    print("y is present in given list")
```

```
else:
```

```
    print("y is NOT present in given list")
```

Output

```
x is NOT present in given list
```

```
y is present in given list
```

Ternary Operator

Ternary operators also known as conditional expressions are operators that evaluate something based on a condition being true or false. It was added to Python in version 2.5.

It simply allows testing a condition in a single line replacing the multiline if-else, making the code compact.

Syntax : [on_true] if [expression] else [on_false]

```
a, b = 10, 20
```

```
min = a if a < b else b
```

```
print(min)
```

Output

```
10
```

Precedence and Associativity of Operators

Operator precedence and associativity determine the priorities of the operator.

Operator Precedence

This is used in an expression with more than one operator with different precedence to determine which operation to perform first.

```
expr = 10 + 20 * 30
```

```
print(expr)
```

```
name = "Alex"
```

```
age = 0
```

```
if name == "Alex" or name == "John" and age >= 2:
```

```
    print("Hello! Welcome.")
```

```
else:
```

```
    print("Good Bye!!")
```

Output

```
610
```

```
Hello! Welcome.
```

Operator Associativity

If an expression contains two or more operators with the same precedence then Operator Associativity is used to determine. It can either be Left to Right or from Right to Left.

```
print(100 / 10 * 10)
```

```
print(5 - 2 + 3)
```

```
print(5 - (2 + 3))
```

```
print(2 ** 3 ** 2)
```

Output

```
100.0
```

```
6
```

```
0
```

```
512
```

Precedence and Association

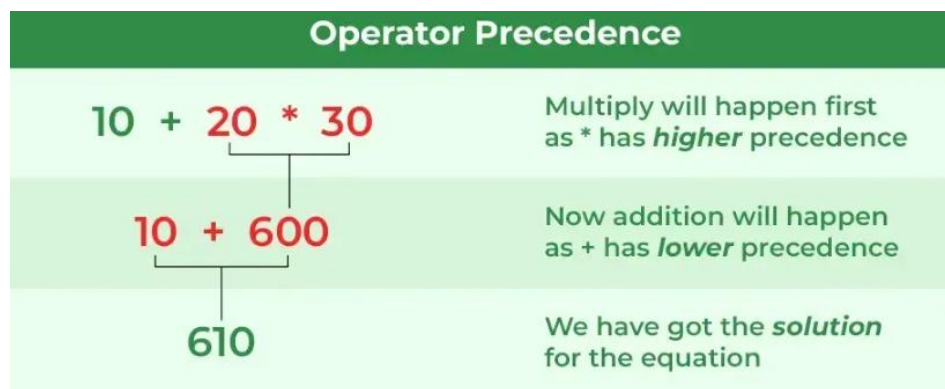
In Python, operators have different precedence levels, which determine order in which expressions are evaluated. If operators have same precedence, associativity decides whether they are evaluated left-to-right or right-to-left.

Operators Precedence

Operator precedence defines order in which Python evaluates different operators in an expression. When an expression has multiple operators, Python follows precedence rules to decide order of evaluation.

Expression:

$10 + 20 * 30$



Example: This code shows how Python evaluates + and * operators according to precedence rules.

```
# Precedence of '+' & '*'
expr = 10 + 20 * 30
print(expr)
```

Output

610

Precedence of Logical Operators

Logical operators (and, or, not) also follow precedence.

Example: This code demonstrates how and has higher precedence than or affecting conditional evaluation. Using parentheses can override default order.

```
# Precedence of 'or' & 'and'
name = "Alex"
age = 0
if name == "Alex" or name == "John" and age >= 2:
    print("Hello! Welcome.")
else:
    print("Good Bye!!")
```

Output

Hello! Welcome.

Explanation:

- **and** has higher precedence than **or**.
- So `name == "John"` and `age >= 2` is checked first.
- Since `name == "Alex"` is already true, whole condition passes.

To fix this, use parentheses:

`if (name == "Alex" or name == "John") and age >= 2:`

Here, parentheses ensure the **or** part is checked first.

Operator Precedence and Associativity List in Python

Consider following list of operator precedence and associativity in Python. It shows all operators from highest precedence to lowest precedence.

1. **()** : Parentheses (highest precedence) -> **Associativity**: Left to right
2. **x[index], x[index:index]** : Subscription, slicing -> **Associativity**: Left to right
3. **await x** : Await expression
4. ****** : Exponentiation -> **Associativity**: Right to left
5. **+x, -x, ~x** : Unary plus, unary minus, bitwise NOT -> **Associativity**: Right to left
6. ***, @, /, //, %** : Multiplication, matrix multiplication, division, floor division, remainder -> **Associativity**: Left to right
7. **+, -** : Addition and subtraction -> **Associativity**: Left to right
8. **<<, >>** : Bitwise shifts -> **Associativity**: Left to right
9. **&** : Bitwise AND -> **Associativity**: Left to right
10. **^** : Bitwise XOR -> **Associativity**: Left to right
11. **|** : Bitwise OR -> **Associativity**: Left to right
12. **in, not in, is, is not, <, <=, >, >=, !=, ==** : Comparisons, membership, identity tests -> **Associativity**: Left to right
13. **not x** : Boolean NOT -> **Associativity**: Right to left
14. **and** : Boolean AND -> **Associativity**: Left to right
15. **or** : Boolean OR -> **Associativity**: Left to right
16. **if-else** : Conditional expression -> **Associativity**: Right to left
17. **lambda** : Lambda expression
18. **:=** : Assignment expression (Walrus operator) -> **Associativity**: Right to left

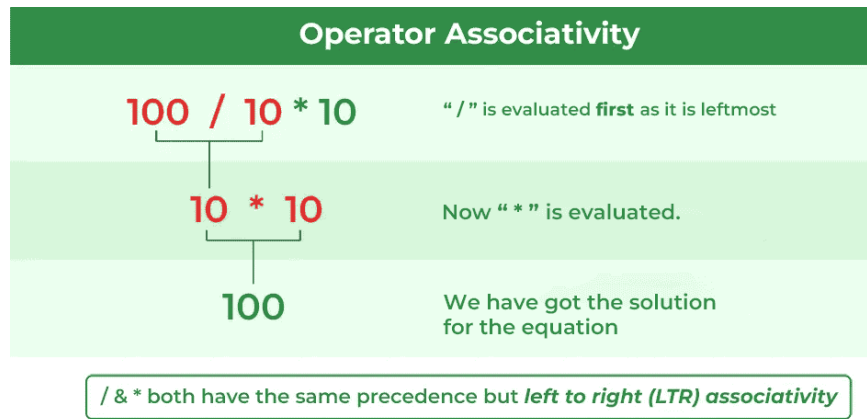
Note: Parentheses **()** have highest precedence and can override default order. Some operators, like `await` and `lambda`, do not have associativity.

Operators Associativity

If an expression contains two or more operators with same precedence then Operator Associativity is used. It can either be Left to Right or from Right to Left.

Expression:

`100 / 10 * 10`



Example 1: Demonstrates left-to-right associativity for division, multiplication, addition and subtraction.

Division and multiplication

```
print(100 / 10 * 10)
```

Addition and subtraction

```
print(5 - 2 + 3)
```

```
print(5 - (2 + 3))
```

Output

100.0

6

0

Explanation:

- **100 / 10 * 10:** division and multiplication have same precedence, so they are evaluated left to right: $(100 / 10) * 10 = 100.0$.
- **5 - 2 + 3:** subtraction and addition also have same precedence, so it goes left to right: $(5 - 2) + 3 = 6$.
- **5 - (2 + 3):** parentheses change the order, so $5 - 5 = 0$.

Example 2: Demonstrates right-to-left associativity for exponentiation (**).

Exponentiation is right-to-left

```
print(2 ** 3 ** 2)
```

Output

512

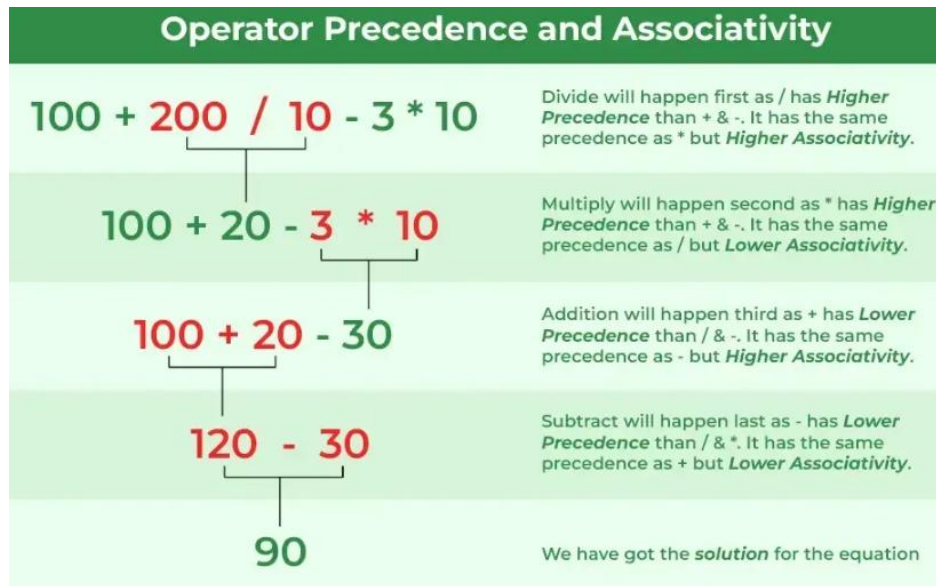
Explanation: Exponentiation (**) is evaluated right to left, so $2 ** (3 ** 2) = 2 ** 9 = 512$.

Operators Precedence and Associativity

Operators Precedence and Associativity are two main characteristics of operators that determine evaluation order of sub-expressions in the absence of brackets.

Expression:

$100 + 200 / 10 - 3 * 10$



Example: In this example, we calculate an expression containing addition, subtraction, multiplication and division to demonstrate how Python evaluates operators based on precedence and associativity.

```
Expression = 100 + 200 / 10 - 3 * 10
print(expression)
```

Output

90.0

Non-associative Operators

In Python, most operators have associativity, which means they are evaluated from left to right or right to left when they have same precedence. However, there are a few operators that are non-associative, meaning they cannot be chained together.

Example: This code demonstrates that = and += cannot be chained because they are non-associative operators.

```
A = 5
b = 10
c = 15
a = b = (a < b) += (b < c)
```

Output

```
a = b = (a < b) += (b < c)
^^
```

SyntaxError: invalid syntax

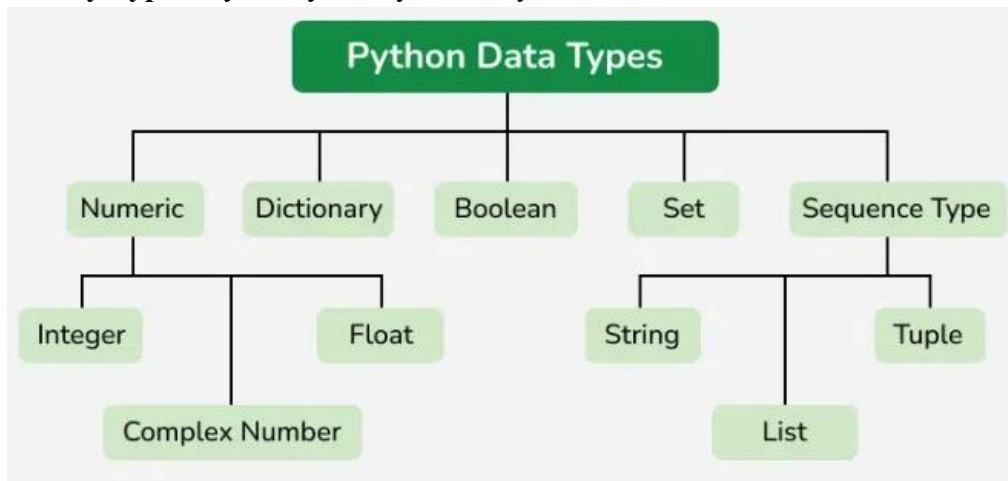
Explanation: = and += are non-associative; combining them in one expression causes a syntax error.

Data Types

Data types in Python are a way to classify data items. They represent the kind of value which determines what operations can be performed on that data. Since everything is an object in Python programming, Python data types are classes and variables are instances (objects) of these classes.

The following are standard or built-in data types in Python:

- **Numeric:** int, float, complex
- **Sequence Type:** string, list, tuple
- **Mapping Type:** dict
- **Boolean:** bool
- **Set Type:** set, frozenset
- **Binary Types:** bytes, bytearray, memoryview



Below code assigns variable 'x' different values of few Python data types - int, float, list, tuple and string. Each assignment replaces previous value, making 'x' take on data type and value of most recent assignment.

```

x = 50 # int
x = 60.5 # float
x = "Hello World" # string
x = ["geeks", "for", "geeks"] # list
x = ("geeks", "for", "geeks") # tuple
  
```

1. Numeric Data Types

Python numbers represent data that has a numeric value. A numeric value can be an integer, a floating number or even a complex number. These values are defined as int, float and complex classes.

- **Integers:** value is represented by int class. It contains positive or negative whole numbers (without fractions or decimals). There is no limit to how long an integer value can be.

- **Float:** value is represented by float class. It is a real number with a floating-point representation. It is specified by a decimal point. Optionally, character e or E followed by a positive or negative integer may be appended to specify scientific notation.
- **Complex Numbers:** It is represented by a complex class. It is specified as (real part) + (imaginary part)j. For example - 2+3j

```
a = 5
print(type(a))
```

```
b = 5.0
print(type(b))
```

```
c = 2 + 4j
print(type(c))
```

Output

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

2. Sequence Data Types

A sequence is an ordered collection of items, which can be of similar or different data types. Sequences allow storing of multiple values in an organized and efficient fashion. There are several sequence data types of Python:

String Data Type

Python Strings are arrays of bytes representing Unicode characters. In Python, there is no character data type, a character is a string of length one. It is represented by str class.

Strings in Python can be created using single quotes, double quotes or even triple quotes. We can access individual characters of a String using index.

```
s = 'Welcome to the Geeks World'
print(s)
```

```
# check data type
print(type(s))
```

```
# access string with index
print(s[1])
print(s[2])
print(s[-1]) # -1 refers to the last character, -2 is second last, and so on
```

Output

```
Welcome to the Geeks World
<class 'str'>
e
```

```
l
d
```

List Data Type

Lists are similar to arrays found in other languages. They are an ordered and mutable collection of items. It is very flexible as items in a list do not need to be of the same type.

Creating a List in Python: Lists can be created by just placing sequence inside the square brackets[].

```
# Empty list
```

```
a = []
```

```
# list with int values
```

```
a = [1, 2, 3]
```

```
print(a)
```

```
# list with mixed values int and String
```

```
b = ["Geeks", "For", "Geeks", 4, 5]
```

```
print(b)
```

Output

```
[1, 2, 3]
```

```
['Geeks', 'For', 'Geeks', 4, 5]
```

Access List Items: In order to access the list items refer to index number. In Python, negative sequence indexes represent positions from end of the array. Instead of having to compute offset as in List[len(List)-3], it is enough to just write List[-3]. Negative indexing means beginning from end, -1 refers to last item, -2 refers to second-last item, etc.

```
a = ["Geeks", "For", "Geeks"]
```

```
print("Accessing element from the list")
```

```
print(a[0])
```

```
print(a[2])
```

```
print("Accessing element using negative indexing")
```

```
print(a[-1])
```

```
print(a[-3])
```

Output

```
Accessing element from the list
```

```
Geeks
```

```
Geeks
```

```
Accessing element using negative indexing
```

```
Geeks
```

```
Geeks
```

Tuple Data Type

Tuple is an ordered collection of Python objects. The only difference between a tuple and a list is that tuples are immutable. Tuples cannot be modified after it is created.

Creating a Tuple in Python: tuples are created by placing a sequence of values separated by a 'comma' with or without the use of parentheses for grouping data sequence. Tuples can contain any number of elements and of any datatype (like strings, integers, lists, etc.).

Note: Tuples can also be created with a single element, but it is a bit tricky. Having one element in the parentheses is not sufficient, there must be a trailing '**comma**' to make it a tuple.

```
# initiate empty tuple
```

```
tup1 = ()
```

```
tup2 = ('Geeks', 'For')
```

```
print("\nTuple with the use of String: ", tup2)
```

Output

Tuple with the use of String: ('Geeks', 'For')

Note: The creation of a Python tuple without the use of parentheses is known as Tuple Packing.

Access Tuple Items: In order to access tuple items refer to the index number. Use the index operator [] to access an item in a tuple.

```
tup1 = (1, 2, 3, 4, 5)
```

```
# access tuple items
```

```
print(tup1[0])
```

```
print(tup1[-1])
```

```
print(tup1[-3])
```

Output

```
1
```

```
5
```

```
3
```

3. Boolean Data Type

Python Boolean Data type is one of the two built-in values, True or False. Boolean objects that are equal to True are truthy (true) and those equal to False are falsy (false). However non-Boolean objects can be evaluated in a Boolean context as well and determined to be true or false. It is denoted by class bool.

```
print(type(True))
```

```
print(type(False))
```

```
print(type(true))
```

Output:

```
<class 'bool'>
```

```
<class 'bool'>
```

```
Hangup (SIGHUP)
```

```
Traceback (most recent call last):
```

```
File "/home/guest/sandbox/Solution.py", line 3, in <module>
print(type(true))
^^^^
```

NameError: name 'true' is not defined. Did you mean: 'True'?

Truthy and Falsy Values

truthy and falsy values are values that evaluate to True or False in a Boolean context. Truthy values behave like True, while falsy values behave like False when used in conditions.

```
if 1:
```

```
    print("1 is truthy")
```

```
if not 0:
```

```
    print("0 is falsy")
```

Output

```
1 is truthy
```

```
0 is falsy
```

4. Set Data Type

Set is an unordered collection of data types that is iterable, mutable, and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

Create a Set in Python

Sets can be created by using the built-in set() function with an iterable object or a sequence by placing the sequence inside curly braces, separated by a 'comma'. The type of elements in a set need not be the same, various mixed-up data type values can also be passed to the set.

```
# initializing empty set
```

```
s1 = set()
```

```
s1 = set("GeeksForGeeks")
```

```
print("Set with the use of String: ", s1)
```

```
s2 = set(["Geeks", "For", "Geeks"])
```

```
print("Set with the use of List: ", s2)
```

Note: Sets in Python are unordered, so the order of elements when printed may vary each time the code is run.

Access Set Items

Set items cannot be accessed by referring to an index, since sets are unordered the items have no index. But we can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the keyword in.

```
set1 = set(["Geeks", "For", "Geeks"]) #Duplicates are removed automatically
```

```
print(set1)
```

```
# loop through set
```

```
for i in set1:
    print(i, end=" ") #prints elements one by one
```

```
# check if item exist in set
print("Geeks" in set1)
```

5. Dictionary Data Type

A dictionary in Python is a collection of data values, used to store information like a map. Unlike other Python data types, a dictionary holds key-value pairs, where each key maps to a value, allowing efficient access and retrieval of data. Each key-value pair in a dictionary is separated by a colon (:), and multiple pairs are separated by commas.

Create a Dictionary in Python

Values in a dictionary can be of any datatype and can be duplicated, whereas keys can't be repeated and must be immutable. The dictionary can also be created by the built-in function **dict()**.

Note: Dictionary keys are case sensitive, the same name but different cases of Key will be treated distinctly.

```
# initialize empty dictionary
d = {}
```

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d)
```

```
# creating dictionary using dict() constructor
d1 = dict({1: 'Geeks', 2: 'For', 3: 'Geeks'})
print(d1)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

Accessing Key-value in Dictionary

In order to access items of a dictionary refer to its key name. Key can be used inside square brackets. Using **get()** method we can access dictionary elements.

```
d = {1: 'Geeks', 'name': 'For', 3: 'Geeks'}
```

```
# Accessing an element using key
print(d['name'])
```

```
# Accessing a element using get
print(d.get(3))
```

Output

For
Geeks

Indentation

Indentation is used to define blocks of code. It indicates to the Python interpreter that a group of statements belongs to the same block.

- All statements with the same level of indentation are treated as part of the same code block.
- Indentation is created using tabs or spaces and the commonly accepted convention is to use four spaces.
- Python expects the indentation level to be consistent within the same block. This inconsistency causes an `IndentationError` as shown in the below code.

```
print("I have no Indentation ")
    print("I have tab Indentation ")
```

ERROR!

Traceback (most recent call last):

File "<main.py>", line 2

```
print("I have tab Indentation ")
```

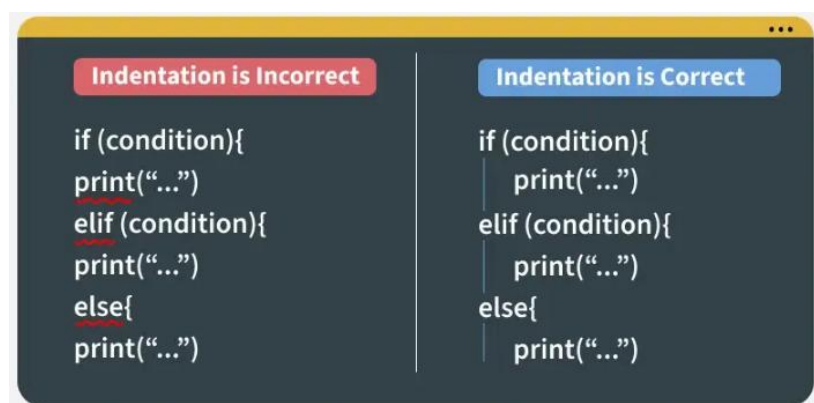
`IndentationError: unexpected indent`

Explanation:

- The first print statement has no indentation, so it is correctly executed.
- The second print statement has tab indentation, but it doesn't belong to a new block of code, that's why it throws `IndentationError`.

Indentation in Conditional Statements

All statements in a conditional block should have same alignment.



The code below demonstrate how we use indentation to define seperate scopes of if-else statements:

```
a = 20
```

```

if a >= 18:
    print('GeeksforGeeks...')
else:
    print('retype the URL.')
print('All set !')

```

Output

```

GeeksforGeeks...
All set !

```

Explanation: The statements under if and else are indented, forming two distinct blocks. Only one block executes based on the condition. The final print("Done") is outside the conditional structure, so it runs regardless of the condition

Indentation in Loops

Indentation defines the set of statements that are executed repeatedly inside a loop.

```

j = 1

while(j<= 5):
    print(j)
    j = j + 1

```

Output

```

1
2
3
4
5

```

Explanation: Both **print(j)** and **j += 1** are indented and belong to the loop block which executes in each iteration until the condition becomes false.

Comments

Comments in Python are the lines in the code that are ignored by the interpreter during the execution of the program.

It enhance the readability of the code.

It can be used to identify functionality or structure the code-base.

It can help understanding unusual or tricky scenarios handled by the code to prevent accidental removal or changes.

It can be used to prevent executing any specific part of your code, while making changes or testing.

```
# I am single line comment
```

```
""" Multi-line comment used
print("Python Comments") """
```

Single Line Comments

Single line comments starts with hashtag symbol #.

```
# sample comment
name = "geeksforgeeks"
print(name)
```

Output

```
geeksforgeeks
```

Multi-Line Comments

Python does not provide the option for multiline comments. However, there are different ways through which we can write multiline comments.

1. Using multiple hashtags (#)

We can use multiple hashtags (#) to write multiline comments. Each and every line will be considered as a single-line comment.

```
# Python program to demonstrate
# multiline comments
print("Multiline comments")
```

2. Using String Literals

Python ignores the string literals that are not assigned to a variable. So, we can use these string literals as comments.

'Single-line comments using string literals'

```
""" Python program to demonstrate
multiline comments """
print("Multiline comments")
```

Built-in Functions

Python is the most popular programming language created by Guido van Rossum in 1991. It is used for system scripting, software development, and web development (server-side). Web applications can be developed on a server using Python. Workflows can be made with Python and other technologies. Database systems are connectable with Python. Files can also be read and changed by it. Big data management and advanced mathematical operations can both be done with Python.

The syntax of Python is straightforward and resembles that of English. Python's syntax enables programmers to create programmes with fewer lines of code than they would be able to with

certain other languages. Python operates on an interpreter system, allowing for the immediate execution of written code.

Python provides a lot of built-in functions that ease the writing of code. In this article, you will learn about **Python's built-in functions**, exploring their various applications and highlighting some of the most commonly used ones.

Python Built-in Functions List

Here is a comprehensive list of Python built-in functions:

Function Name	Description
Python abs() Function	Return the absolute value of a number
Python iter() Function	It takes an asynchronous iterable as an argument and returns an asynchronous iterator for that iterable
Python all() Function	Return true if all the elements of a given iterable(List, Dictionary, Tuple, set, etc) are True else it returns False
Python any() Function	Returns true if any of the elements of a given iterable(List, Dictionary, Tuple, set, etc) are True else it returns False
Python anext() Function	used for getting the next item from an asynchronous iterator
Python ascii() Function	Returns a string containing a printable representation of an object
Python bin() Function	Convert integer to a binary string
Python bool() Function	Return or convert a value to a Boolean value i.e., True or False
Python breakpoint() Function	It is used for dropping into the debugger at the call site during runtime for debugging purposes
Python bytearray() Function	Returns a byte array object which is an array of given bytes
Python bytes() Function	Converts an object to an immutable byte-represented object of a given size and data
Python callable() Function	Returns True if the object passed appears to be callable
Python chr() Function	Returns a string representing a character whose Unicode code point is an integer
Python classmethod() Function	Returns a class method for a given function
Python compile() Function	Returns a Python code object
Python complex() Function	Creates Complex Number
Python delattr() Function	Delete the named attribute from the object
Python dict() Function	Creates a Python Dictionary
Python dir() Function	Returns a list of the attributes and methods of any object

Python <code>divmod()</code> Function	Takes two numbers and returns a pair of numbers consisting of their quotient and remainder
Python <code>enumerate()</code> Function	Adds a counter to an iterable and returns it in a form of enumerating object
Python <code>eval()</code> Function	Parses the expression passed to it and runs Python expression(code) within the program
Python <code>exec()</code> Function	Used for the dynamic execution of the program
Python <code>filter()</code> Function	Filters the given sequence with the help of a function that tests each element in the sequence to be true or not
Python <code>float()</code> Function	Return a floating-point number from a number or a string
Python <code>format()</code> Function	Formats a specified value
Python <code>frozenset()</code> Function	Returns immutable frozenset
Python <code>getattr()</code> Function	Access the attribute value of an object
Python <code>globals()</code> Function	Returns the dictionary of the current global symbol table
Python <code>hasattr()</code> Function	Check if an object has the given named attribute and return true if present
Python <code>hash()</code> Function	Encode the data into an unrecognizable value
Python <code>help()</code> Function	Display the documentation of modules, functions, classes, keywords, etc
Python <code>hex()</code> Function	Convert an integer number into its corresponding hexadecimal form
Python <code>id()</code> Function	Return the identity of an object
Python <code>input()</code> Function	Take input from the user as a string
Python <code>int()</code> Function	Converts a number in a given base to decimal
Python <code>isinstance()</code> Function	Checks if the objects belong to a certain class or not
Python <code>issubclass()</code> Function	Check if a class is a subclass of another class or not
Python <code>iter()</code> Function	Convert an iterable to an iterator
Python <code>len()</code> Function	Returns the length of the object
Python <code>list()</code> Function	Creates a list in Python
Python <code>locals()</code> Function	Returns the dictionary of the current local symbol table
Python <code>map()</code> Function	Returns a map object(which is an iterator) of the results after applying the given function to each item of a given iterable

Python max() Function	Returns the largest item in an iterable or the largest of two or more arguments
Python memoryview() Function	Returns memory view of an argument
Python min() Function	Returns the smallest item in an iterable or the smallest of two or more arguments
Python next() Function	Receives the next item from the iterator
Python object() Function	Returns a new object
Python oct() Function	returns an octal representation of an integer in a string format.
Python open() Function	Open a file and return its object
Python ord() Function	Returns the Unicode equivalence of the passed argument
Python pow() Function	Compute the power of a number
Python print() Function	Print output to the console
Python property() Function	Create a property of a class
Python range() Function	Generate a sequence of numbers
Python repr() Function	Return the printable version of the object
Python reversed() Function	Returns an iterator that accesses the given sequence in the reverse order
Python round() Function	Rounds off to the given number of digits and returns the floating-point number
Python set() Function	Convert any of the iterable to a sequence of iterable elements with distinct elements
Python setattr() Function	Assign the object attribute its value
Python slice() Function	Returns a slice object
Python sorted() Function	Returns a list with the elements in a sorted manner, without modifying the original sequence
Python staticmethod() Function	Converts a message into the static message
Python str() Function	Returns the string version of the object
Python sum() Function	Sums up the numbers in the list
Python super() Function	Returns a temporary object of the superclass
Python tuple() Function	Creates a tuple in Python
Python type() Function	Returns the type of the object

Python vars() Function	Returns the <code>__dict__</code> attribute for a module, class, instance, or any other object
Python zip() Function	Maps the similar index of multiple containers
Python <code>__import__()</code> Function	Imports the module during runtime

Console Input and Console Output

The `print()` function is used for output in various formats and the `input()` function enables interaction with users.

Taking Input using `input()`

Python's `input()` function is used to take user input. By default, it returns the user input in form of a string.

```
name = input("Enter your name: ")
print("Hello,", name, "! Welcome!")
```

Output

Enter your name: GeeksforGeeks

Hello, GeeksforGeeks ! Welcome!

The code prompts the user to input their name, stores it in the variable "name" and then prints a greeting message addressing the user by their entered name.

Printing Output using `print()`

The `print()` function allows us to display text, variables and expressions on the console. In the below example, "Hello, World!" is a string literal enclosed within double quotes. When executed, this statement will output the text to the console.

```
print("Hello, World!")
```

Output

Hello, World!

Printing Variables

We can use the `print()` function to print single and multiple variables. We can print multiple variables by separating them with commas. **Example:**

```
s = "Brad"
print(s)
```

```
s = "Anjelina"
age = 25
city = "New York"
print(s, age, city)
```

Output

Brad

Anjelina 25 New York

We can also take multiple inputs at once from the user in a single line, splitting the values entered by the user into separate variables for each value using the `split()` method:

```
x, y = input("Enter two numbers: ").split()
print(x, y)
```

Output

```
Enter two numbers: 10 20
10 20
```

Taking Different types of User Input

We can change the user input from default string type to any other type (int, float, etc) by typecasting:

```
I = int(input("How old are you?: "))
f = float(input("Evaluate 7/2: "))
```

```
print(I, f)
```

Output

```
How old are you?: 23
Evaluate 7/2: 3.5
23 3.5
```

Type Conversions

Type conversion in Python is the process of changing a value from one data type to another.

- Helps ensure correct operations and calculations.
- Examples include converting an integer to a float or a numeric string to an integer.
- Python supports two types of type conversion: **Implicit Conversion** and **Explicit Conversion**.

Implicit Type Conversion

Implicit conversion in Python happens automatically when different data types are used together in an expression.

- Python converts a smaller data type to a larger one when needed.
- Commonly occurs when integers and floats are combined.
- Conversion happens at runtime to keep results accurate.

```
x = 10      # Integer
y = 10.6    # Float
z = x + y
```

```
print("x:", type(x))
print("y:", type(y))
print("z =", z)
print("z :", type(z))
```

Output

```
x: <class 'int'>
y: <class 'float'>
z = 20.6
z : <class 'float'>
```

In the above example:

- "x" is an integer and "y" is a float.
- During "z = x + y", Python implicitly converts x to a float to avoid data loss.
- As a result, "z" becomes a float with the value **20.6**.

Explicit Type Conversion

Explicit conversion, also called type casting, is when a programmer manually changes a value from one data type to another.

- Done using Python's built-in functions like **int()**, **float()**, and **str()**.
- Gives full control over how data is interpreted or processed.
- Used when automatic conversion is not suitable.

Common type casting functions

- **int()** converts a value to an integer
- **float()** converts a value to a floating point number
- **str()** converts a value to a string
- **bool()** converts a value to a Boolean (True/False)

```
s = "100" # String
a = int(s)
print(a)
print(type(a))
```

Output

```
100
<class 'int'>
```

In the above example:

- s is a string with the value "100".
- Using int(s), the string is explicitly converted into an integer.
- This manual conversion is called explicit type conversion, and a becomes the integer 100 with type <class 'int'>.

Examples of Common Type Conversion Functions**Example 1:** Converting a binary string

```
s = "10010"
a = int(s, 2)
print(a)
```

```
b= float(s)
```

```
print(b)
```

Output

```
18
10010.0
```

In the above example:

- **int(s, 2)** interprets the binary string '10010' as the integer 18.
- **float(s)** converts the string to a floating-point number.

Example 2: ASCII, Hexadecimal and Octal Conversion

```
c = '4'
print("ASCII of '4':", ord(c))
print("56 in Hex:", hex(56))
print("56 in Octal:", oct(56))
```

Output

```
ASCII of '4': 52
56 in Hex: 0x38
56 in Octal: 0o70
```

In the above example:

- **ord()** returns the ASCII code of the character '4'.
- **hex()** and **oct()** convert the integer 56 to its hexadecimal and octal representations, respectively.

Example 3: String to Tuple, Set and List

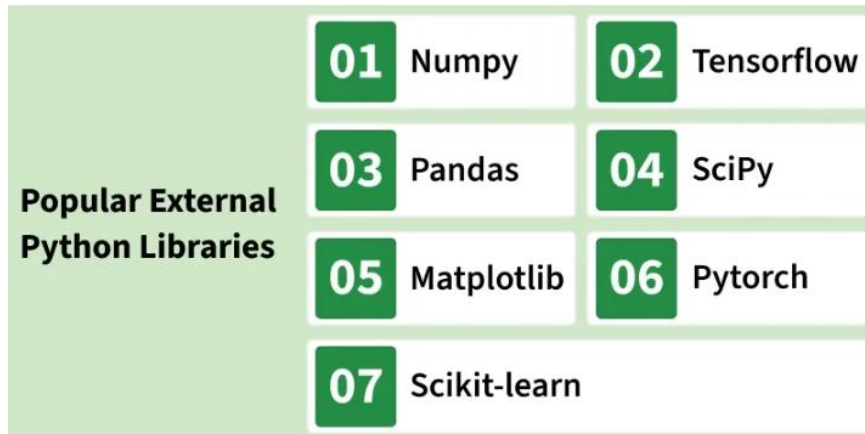
```
s = 'geeks'
print("To tuple:", tuple(s))
print("To set:", set(s))
print("To list:", list(s))
```

Output

```
To tuple: ('g', 'e', 'e', 'k', 's')
To set: {'e', 'g', 'k', 's'}
To list: ['g', 'e', 'e', 'k', 's']
```

Python Libraries

In Python, a library is a group of modules that contain functions, classes and methods to perform common tasks like data manipulation, math operations, web scraping and more. Python libraries make coding faster, cleaner and more efficient by providing ready-to-use solutions for different domains such as data science, web development, machine learning and automation.



Working of Python Library

When you import a library in Python, it gives access to pre-written code stored in separate modules. In simple terms instead of writing the logic for a task, you import the library that already has it.

For example, on Windows, libraries are stored as .dll (Dynamic Link Libraries) and on Linux/macOS as .so files. When you run your code, Python automatically loads these modules and makes their functions available to use.

Types of Python Library

Python libraries are divided into two main types:

1. Built-in Python Standard Library

It is a collection of modules that come bundled with every Python installation, we don't need to install anything separately. Most of these modules are written in C for better performance.

Examples of built-in modules:

- **math:** Mathematical operations
- **os:** Interact with the operating system
- **datetime:** Date and time operations
- **random:** Generate random numbers
- **json:** Handles JSON data encoding and decoding.

2. External Python Libraries

External (third-party) libraries are not included with Python by default. You can install them easily using the pip package manager. Popular External Python Libraries:

1. **NumPy:** It, short for Numerical Python, is the core library for numerical and scientific computing in Python. It provides powerful tools for creating and manipulating arrays, matrices and multidimensional data.
2. **Pandas:** It is a data analysis and manipulation library built on top of NumPy. It introduces data structures like DataFrame and Series that make it easy to handle structured data efficiently.
3. **Matplotlib:** It is a data visualization library that helps you create a wide variety of static, animated and interactive plots. It supports charts such as bar graphs, line charts, histograms and scatter plots.

4. **SciPy:** It, short for Scientific Python, extends the functionality of NumPy by providing tools for advanced mathematical, scientific and engineering computations. It includes modules for optimization, integration, signal processing and linear algebra.
5. **TensorFlow:** It is an open-source machine learning and deep learning framework developed by Google. It allows developers to build and train neural networks for tasks such as image recognition, natural language processing and predictive modeling.
6. **Scikit-learn:** It is a popular machine learning library built on top of NumPy and SciPy. It offers simple and efficient tools for classification, regression, clustering and dimensionality reduction.
7. **Scrapy:** It is a web scraping and data extraction library designed to efficiently crawl websites and gather structured information. It allows developers to create spiders that automatically navigate web pages and collect data.
8. **PyTorch:** It, developed by Facebook's AI Research Lab, is a deep learning framework that provides dynamic computation graphs and easy debugging. It supports GPU acceleration, making it suitable for high-performance training of neural networks.
9. **PyGame:** It is a cross-platform library used for developing 2D games and multimedia applications. It provides modules for handling graphics, sound and user input with ease.
10. **PyBrain:** It, short for Python-Based Reinforcement Learning, Artificial Intelligence and Neural Network, is a beginner-friendly machine learning library. It offers pre-built algorithms and flexible tools for training neural networks and reinforcement learning models.

Using Libraries in Python Programs

To use any library, it first needs to be imported into your Python program using the import statement. Once imported, you can directly call the functions or methods defined inside that library. You can import libraries in three main ways:

1. **Import the entire library:** import library_name for example, import math
2. **Import a specific function or class:** from library_name import function_name for example, from math import sqrt
3. **Import a library with an alias:** import library_name as alias for example, import pandas as pd

Example 1: This program imports the entire math library and uses one of its functions.

```

import math
A = 16
print(math.sqrt(A))

```

Output

4.0

Explanation:

- Here, the complete math library is imported and we use math.sqrt() to calculate the square root of 16.
- Since the full library is imported, we must prefix the function with the library name (math.).

Example 2: This program imports only selected functions from an external library to simplify usage.

```
From numpy import array, mean
a = array([10, 20, 30, 40, 50])
print(mean(a))
```

Output

30.0

Explanation:

- Here, only the `array()` and `mean()` functions are imported from the NumPy library.
- `array()` is used to create a NumPy array from a list and `mean()` calculates the average value of all elements in the array.
- Since these functions are imported directly, we don't need to use the `numpy`. Prefix each time we call them.

Importing Libraries with Examples

A **library** is a collection of functions that can be added to your Python code and called as necessary, just like any other function. There is no reason to rewrite code that will perform a standard task. With libraries, you can import pre-existing functions and efficiently expand the functionality of your code.

Some examples of popular libraries and their functionalities:

- **Requests:** an elegant and simple HTTP library for Python, commonly used for REST API calls. (See my other course, *Build Your Web Projects With REST APIs*, to learn more!)
- **Beautiful Soup:** a library for pulling data out of HTML and XML files.
- **Pandas:** a fast, powerful, and easy-to-use open-source data analysis and manipulation tool.

There are thousands of Python libraries that you can use in your Python code!

Install Libraries Using Pip

Pip is a Python package manager.

What's a package manager?

A **package manager** is a tool that allows you to install and manage additional libraries through your terminal. Pip comes ready to go with a Python installation, so you're already good to go!

To install a package using pip in your terminal, we use the following pattern:

```
pip install <name-of-package>
```

To view the packages already installed, you can type the following:

```
pip freeze
```

That will print a list of all the existing libraries, or dependencies, to your terminal.

To import library functionality into your code, use the keyword `import`. (Shocker, right?)

Using the `import` keyword at the top of your code file, you can import certain library functions or the entire library at once. For example, to import the entire **requests** library, you must first

install **requests** using your terminal with `pip install requests`. Then you can write the following code in your text editor:

```
import requests
```

Now to call the `get()` function in the requests library, write:

```
requests.get()
```

If you want to import just one function named `get` from the requests library, write the following:

```
from requests import get
```

Now you don't need to type `requests` before `get()` because the method itself has already been imported.

Instructions:

- In the module named `operations.py`, add the following functions:
 - `addition(a, b)`: which returns the sum of two numbers `a` and `b`;
 - `multiplication(a, b)`: which returns the product of two numbers `a` and `b`.
- In the main file, import the `operations.py` module and use its functions to perform the following mathematical operations:
 - Addition of 10 and 5;
 - Multiplication of 8 and 4.
- Display the results of the operations in the console.

Once you have completed the exercise, you can run the following command in the VS code terminal `pytest tests.py`.

Control Flow Statements

Control flow statements are fundamental components of programming languages that allow developers to control the order in which instructions are executed in a program. They enable execution of a block of code multiple times, execute a block of code based on conditions, terminate or skip the execution of certain lines of code, etc.

Types of Control Flow statements in Programming:

Control Flow Statements Type	Control Flow Statement	Description
Conditional Statements	if-else	Executes a block of code if a specified condition is true, and another block if the condition is false.
	switch-case	Evaluates a variable or expression and executes code based on matching cases.
Looping Statements	for	Executes a block of code a specified number of times, typically iterating over a range of values.
	while	Executes a block of code as long as a specified condition is true.

	do-while	Executes a block of code once and then repeats the execution as long as a specified condition is true.
Jump Statements	break	Terminates the loop or switch statement and transfers control to the statement immediately following the loop or switch.
	continue	Skips the current iteration of a loop and continues with the next iteration.
	return	Exits a function and returns a value to the caller.
	goto	Transfers control to a labeled statement within the same function. (Note: goto is generally discouraged due to its potential for creating unreadable and error-prone code.)

Conditional Statements in Programming:

Conditional statements in programming are used to execute certain blocks of code based on specified conditions. They are fundamental to decision-making in programs. Here are some common types of conditional statements:

1. If Statement in Programming:

The if statement is used to execute a block of code if a specified condition is true.

```
#include <iostream>
using namespace std;
int main()
{
    int a = 5;
    if (a == 5) {
        cout << "a is equal to 5";
    }
    return 0;
}
```

Output

a is equal to 5

2. if-else Statement in Programming:

The if-else statement is used to execute one block of code if a specified condition is true, and another block of code if the condition is false.

```
#include <iostream>
using namespace std;
int main()
{
    int a = 10;
    if (a == 5) {
        cout << "a is equal to 5";
    }
}
```

```

else {
    cout << "a is not equal to 5";
}
return 0;
}

```

Output

a is not equal to 5

3. if-else-if Statement in Programming:

The if-else-if statement is used to execute one block of code if a specified condition is true, another block of code if another condition is true, and a default block of code if none of the conditions are true.

```

#include <iostream>
using namespace std;
int main()
{
    int a = 15;
    if (a == 5) {
        cout << "a is equal to 5";
    }
    else if (a == 10) {
        cout << "a is equal to 10";
    }
    else {
        cout << "a is not equal to 5 or 10";
    }
    return 0;
}

```

Output

a is not equal to 5 or 10

4. Ternary Operator or Conditional Operator in Programming:

In some programming languages, a ternary operator is used to assign a value to a variable based on a condition.

```

#include <iostream>
using namespace std;

int main()
{
    int a = 10;
    cout << (a == 5 ? "a is equal to 5"
              : "a is not equal to 5");
}

```

```
    return 0;
}
```

Output

a is not equal to 5

5. Switch Statement in Programming:

In languages like C, C++, and Java, a switch statement is used to execute one block of code from multiple options based on the value of an expression.

```
#include <iostream>
using namespace std;
int main()
{
    int a = 15;
    switch (a) {
        case 5:
            cout << "a is equal to 5";
            break;
        case 10:
            cout << "a is equal to 10";
            break;
        default:
            cout << "a is not equal to 5 or 10";
    }
    return 0;
}
```

Output

a is not equal to 5 or 10

Each programming language may have its own syntax and specific variations of these conditional statements.

Looping Statements in Programming:

Looping statements, also known as iteration or repetition statements, are used in programming to repeatedly execute a block of code. They are essential for performing tasks such as iterating over elements in a list, reading data from a file, or executing a set of instructions a specific number of times. Here are some common types of looping statements:

1. For Loop in Programming:

The for loop is used to iterate over a sequence (e.g., a list, tuple, string, or range) and execute a block of code for each item in the sequence.

```
#include <iostream>
using namespace std;
int main()
{
```

```

    for (int i = 0; i < 5; i++) {
        cout << i << endl;
    }
    return 0;
}

```

Output

```

0
1
2
3
4

```

2. While Loop in Programming:

The while loop is used to repeatedly execute a block of code as long as a specified condition is true.

```

#include <iostream>
using namespace std;
int main()
{
    int count = 0;
    while (count < 5) {
        cout << count << endl;
        count++;
    }
    return 0;
}

```

Output

```

0
1
2
3
4

```

3. Do-while Loop in Programming:

In some programming languages, such as C and Java, a do-while loop is used to execute a block of code at least once, and then repeatedly execute the block as long as a specified condition is true.

```

#include <iostream>
using namespace std;
int main()
{
    int count = 0;

```

```

do {
    cout << count << endl;
    count++;
} while (count < 5);
return 0;
}

```

Output

```

0
1
2
3
4

```

4. Nested Loops in Programming:

Loops can be nested within one another to perform more complex iterations. For example, a for loop can be nested inside another for loop to create a two-dimensional iteration.

```

#include <iostream>
using namespace std;
int main()
{
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cout << "i=" << i << " j=" << j << "\n";
        }
    }
}

```

Output

```

i=0 j=0
i=0 j=1
i=1 j=0
i=1 j=1

```

Each programming language may have its own syntax and specific variations of these looping statements.

Jump Statements in Programming:

Jump statements in programming are used to change the flow of control within a program. They allow the programmer to transfer program control to different parts of the code based on certain conditions or requirements. Here are common types of jump statements:

1. Break Statement in Programming:

The break statement is primarily used to exit from loops prematurely. When encountered inside a loop, it terminates the loop's execution and transfers control to the statement immediately following the loop.


```
#include <iostream>
using namespace std;
int main()
{
    for (int i = 0; i < 10; i++) {
        if (i == 5)
            break;
        cout << i << " ";
    }
    return 0;
}
```

Output

0 1 2 3 4

2. Continue Statement in Programming:

The continue statement is used to skip the current iteration of a loop and proceed to the next iteration.

```
#include <iostream>
using namespace std;
int main()
{
    for (int i = 0; i < 10; i++) {
        if (i % 2 == 1)
            continue;
        cout << i << " ";
    }
    return 0;
}
```

Output

0 2 4 6 8

3. Return Statement in Programming:

The return statement is used to exit a function and optionally return a value to the caller.

```
#include <iostream>
using namespace std;
bool isEven(int N) { return N % 2 == 0; }
int main()
{
    int N = 5;
    if (isEven(N)) {
        cout << "N is even";
    }
}
```

```

    else {
        cout << "N is odd";
    }
    return 0;
}

```

Output

N is odd

4. Goto Statement in Programming:

Some programming languages support the goto statement, which allows transferring control to a labeled statement within the same function or block of code. However, the use of goto is generally discouraged due to its potential for creating unreadable and unmaintainable code.

```

#include <iostream>
using namespace std;
int main()
{
    int i = 0;
loopStart:
    if (i < 5) {
        cout << i << " ";
        i++;
        goto loopStart;
    }
    return 0;
}

```

Output

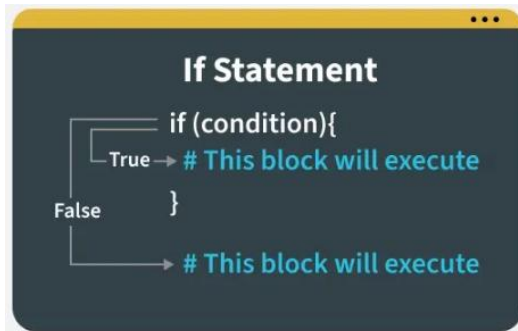
0 1 2 3 4

if, else, elif, while loop, break,

In Python, If-Else is a fundamental conditional statement used for decision-making in programming. If...Else statement allows to execution of specific blocks of code depending on the condition is True or False.

if Statement

if statement is the most simple decision-making statement. If the condition evaluates to True, the block of code inside the if statement is executed.



If Statement

Example of If Statement:

```
i = 10
```

```
# Checking if i is greater than 15
```

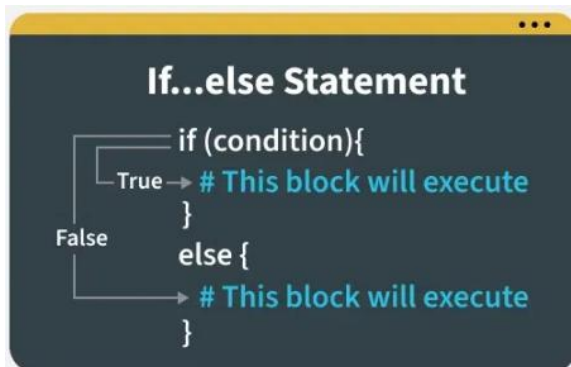
```
if i > 15:
```

```
    print("10 is less than 15")
```

```
print("I am Not in if")
```

if...else Statement

if...else statement is a control statement that helps in decision-making based on specific conditions. When the if condition is **False**. If the condition in the if statement is not true, the else block will be executed.



If...else Statement

Let's look at some examples of if-else statements.

Simple if-else

```
i = 20
```

```
# Checking if i is greater than 0
```

```
if i > 0:
```

```
    print("i is positive")
```

```
else:
```

```
    print("i is 0 or Negative")
```

If Else in One-line

If we need to execute a single statement inside the **if** or **else** block then one-line shorthand can be used.

```
a = -2
```

```
# Ternary conditional to check if number is positive or negative
res = "Positive" if a >= 0 else "Negative"
print(res)
```

Output

Negative

Logical Operators with If..Else

We can combine multiple conditions using logical operators such as **and**, **or**, and **not**.

```
age = 25
exp = 10
```

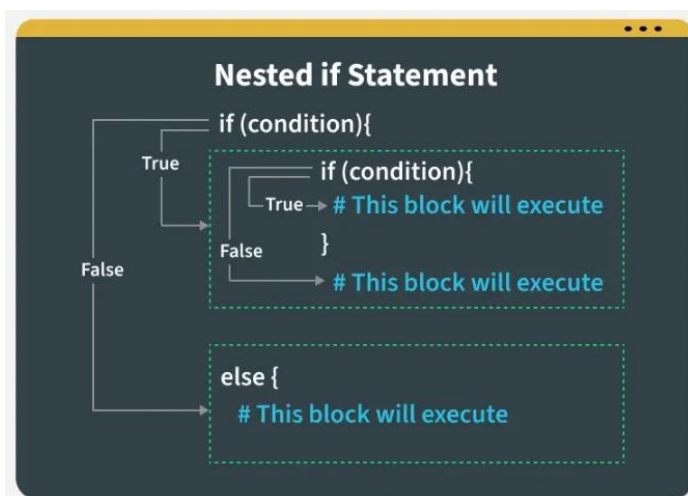
```
# Using '>' operator & 'and' with if-else
if age > 23 and exp > 8:
    print("Eligible.")
else:
    print("Not eligible.")
```

Output

Eligible.

Nested If Else Statement

Nested if...else statement occurs when **if...else** structure is placed inside another **if** or **else** block. Nested If..else allows the execution of specific code blocks based on a series of conditional checks.



Nested if Statement

Example of Nested If Else Statement:

```

i = 10
if i == 10:

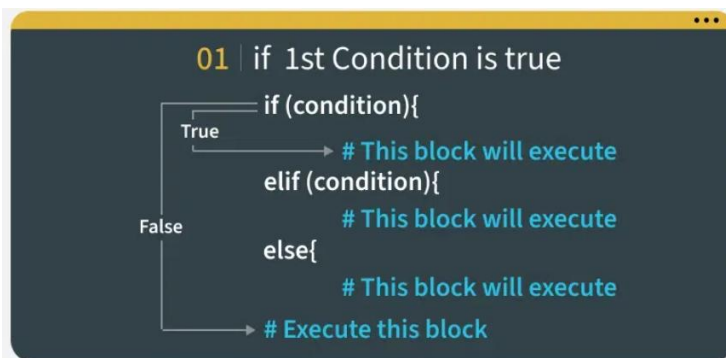
    # First if statement
    if i < 15:
        print("i is smaller than 15")

    # Nested - if statement
    # Will only be executed if statement above
    # it is true
    if i < 12:
        print("i is smaller than 12 too")
    else:
        print("i is greater than 15")
else:
    print("i is not equal to 10")

```

if...elif...else Statement

if-elif-else statement in Python is used for multi-way decision-making. This allows us to check multiple conditions sequentially and execute a specific block of code when a condition is True. If none of the conditions are true, the **else** block is executed.



Example:

```

i = 25

# Checking if i is equal to 10
if i == 10:
    print("i is 10")

# Checking if i is equal to 15
elif i == 15:
    print("i is 15")

# Checking if i is equal to 20
elif i == 20:

```

```

print("i is 20")

# If none of the above conditions are true
else:
    print("i is not present")

```

continue statements,

The continue statement in Python is a loop control statement that skips the rest of the code inside the loop for the current iteration and moves to the next iteration immediately.

When executed, the code following continue in the loop is not executed for that iteration. **Example:**

```

for i in range(1, 11):
    if i == 6:
        continue
    print(i, end=" ")

```

Output

1 2 3 4 5 7 8 9 10

Explanation: When `i == 6`, the continue statement executes, skipping the print operation for 6.

Syntax

```

while True:
...
if x == 10:
    continue
print(x)

```

Examples

Example 1: Skipping specific characters in a string

```

for char in "GeeksforGeeks":

```

```

    if char == "e":
        continue
    print(char, end=" ")

```

Output

G k s f o r G k s

Explanation: Whenever `char == 'e'`, the continue statement executes, skipping the print function for that iteration.

Example 2. Using continue in nested loops

```

a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

```

```

for row in a:
    for num in row:

```

```

if num == 3:
    continue
print(num, end=" ")

```

Output

```
1 2 4 5 6 7 8 9
```

Explanation: continue statement skips printing 3 and moves to the next iteration, so all numbers except 3 are printed in a single line.

Example 3. Using continue with a while loop

```

I = 0
while I < 10:
    if I == 5:
        I += 1 # ensure the loop variable is incremented to avoid infinite loop
        continue
    print(i)
    I += 1

```

Output

```

0
1
2
3
4
6
7
8
9

```

Explanation: When $I == 5$, the continue statement skips printing and jumps to the next iteration.

When to Use the Continue

The continue statement is useful when you need to skip specific iterations while still executing the rest of the loop:

- **Skipping Specific Values:** Ignore certain values without breaking the loop.
- **Filtering Data Dynamically:** Exclude elements that do not meet a condition.
- **Optimizing Loop Performance:** Avoid unnecessary computations for some iterations.

For loop Statement;

Python for loops are used for iterating over sequences like lists, tuples, strings and ranges.

- A for loop allows you to apply the same operation to every item within the loop.
- Using a for loop avoids the need to manually manage the index.
- A for loop can iterate over any iterable object, such as a dictionary, list or custom iterator.

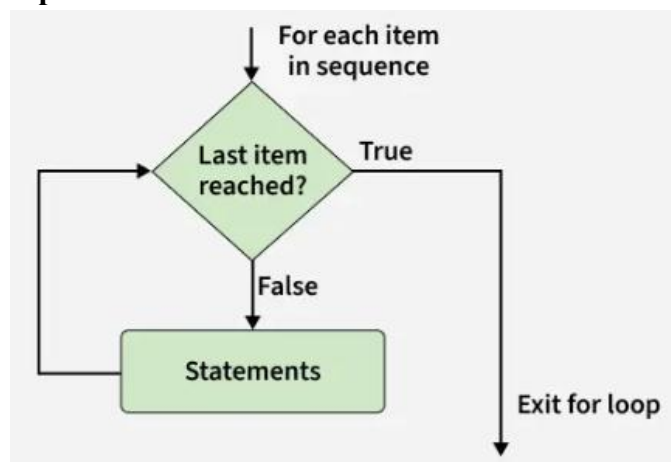
```
s = ["Geeks", "for", "Geeks"]
```

```
# using for loop with list of strings
for i in s:
    print(i)
```

Output

```
Geeks
for
Geeks
```

Flowchart of For Loop



For Loop Flowchart

Syntax

```
for var in iterable:
# statements
pass
```

Examples

Iterating over characters of strings

This code uses a for loop to iterate over a string and print each character on a new line. The loop assigns each character to the variable `i` and continues until all characters in the string have been processed.

```
s = "Geeks"
for i in s:
    print(i)
```

Output

```
G
e
e
k
s
```


Using range() with For Loop

The range() function is commonly used with for loops to generate a sequence of numbers. It can take one, two or three arguments:

- **range(stop):** Generates numbers from 0 to stop-1.
- **range(start, stop):** Generates numbers from start to stop-1.
- **range(start, stop, step):** Generates numbers from start to stop-1, incrementing by step.

```
for i in range(0, 10, 2):
    print(i)
```

Output

```
0
2
4
6
8
```

Control Statements with For Loop

Loop control statements change execution from their normal sequence. When execution leaves a scope, all automatic objects that were created in that scope are destroyed. Python supports the following control statements.

Continue with For Loop

Python continue Statement returns the control to the beginning of the loop.

Prints all letters except 'e' and 's'

```
for i in 'geeksforgeeks':
    if i == 'e' or i == 's':
        continue
    print(i)
```

Output

```
g
k
f
o
r
g
k
```

Break with For Loop

Python break statement brings control out of the loop.

```
for i in 'geeksforgeeks':
```

```

    # break the loop as soon it sees 'e'
    # or 's'
    if i == 'e' or i == 's':
        break

print(i)

```

Output

e

Pass Statement with For Loop

The pass statement to write empty loops. Pass is also used for empty control statements, functions, and classes.

```

# An empty loop
for i in 'geeksforgeeks':
    pass
print(i)

```

Output

s

Else Statement with For Loops

Python also allows us to use the else condition for loops. The else block just after for/while is executed only when the loop is NOT terminated by a break statement.

```

for i in range(1, 4):
    print(i)
else: # Executed because no break in for
    print("No Break\n")

```

Output

1
2
3
No Break

Using Enumerate with for loop

In Python, enumerate() function is used with the for loop to iterate over an iterable while also keeping track of index of each item.

```

li = ["eat", "sleep", "repeat"]

for i, j in enumerate(li):
    print (i, j)

```

Output

```
0 eat
1 sleep
2 repeat
```

Nested For Loops

This code uses nested for loops to iterate over two ranges of numbers (1 to 3 inclusive) and prints the value of I and j for each combination of these two loops.

The inner loop is executed for each value of I in outer loop. The output of this code will print the numbers from 1 to 3 three times, as each value of I is combined with each value of j.

```
for I in range(1, 4):
    for j in range(1, 4):
        print(I, j)
```

range () and exit () functions.

The range() function in Python is used to generate a sequence of integers within a specified range. It is most commonly used in loops to control how many times a block of code runs.

Note: range() returns a lazy iterable, not a full list. It generates numbers dynamically instead of storing them all in memory.. To access elements like a list, convert it using list(range(...)).

Example: This example shows the use of range() to generate numbers starting from 0 up to (but not including) a given value.

```
for i in range(5):
    print(i, end=" ")
```

Output

```
0 1 2 3 4
```

Explanation:

- range(5) generates numbers from 0 to 4
- i takes one value at a time from the generated sequence

Syntax

```
range(start, stop, step)
```

Parameters:

- **start (optional):** Starting number of the sequence (default is 0)
- **stop:** Number at which the sequence stops (not included)
- **step (optional):** Difference between consecutive numbers (default is 1)

Return: A range object representing the sequence

Examples

Example 1: This example generates numbers starting from a custom value and ending before another value.

```
for n in range(5, 10):
```

```
print(n, end=" ")
```

Output

5 6 7 8 9

Explanation:

- `range(5, 10)` starts at 5 and stops before 10
- Numbers increase by the default step of 1

Example 2: This example generates even numbers by skipping values using a custom step size.

```
for v in range(0, 10, 2):
```

```
    print(v, end=" ")
```

Output

0 2 4 6 8

Explanation:

- `range(0, 10, 2)` increases the value by 2 each time
- The step controls how much the number changes per iteration

Example 3: This example demonstrates how `range()` can be used to generate numbers in reverse order by providing a negative step value.

```
for i in range(10, 0, -2):
```

```
    print(i, end=" ")
```

Output

10 8 6 4 2

Explanation:

- `range(10, 0, -2)` starts from 10 and decreases by 2 each time
- A negative step allows backward iteration through numbers

CHAPTER 2

FUNCTIONS IN PYTHON PROGRAMMING

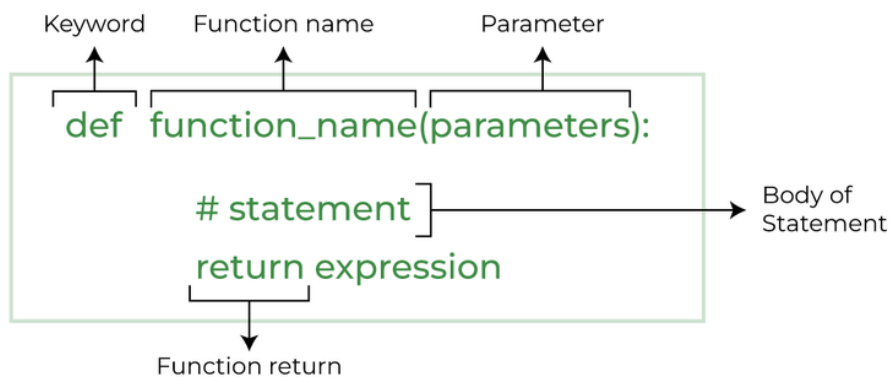
Python functions:

Python functions are reusable blocks of code used to perform a specific task. They help organize programs into smaller sections and execute the same logic whenever needed by calling the function.



Defining a Function

A function can be defined using def keyword. Below is the syntax to define a function:



Here, we define a function using def that prints a welcome message when called.

Here, we define a function using def that prints a welcome message when called.

```
def fun():
    print("Welcome to GFG")
```

Calling a Function

After creating a function, call it by using the name of the functions followed by parenthesis containing parameters of that particular function.

```
def fun():
    print("Welcome to GFG")
```

```
fun()
```

Output

Welcome to GFG

Function Arguments

Arguments are values passed to a function when it is called. They allow functions to receive input data and perform operations using those values.

Syntax

```
def function_name(arguments):
# function body
return value
```

- **def function_name(arguments):** defines a function with optional arguments.
- **# function body** contains the statements to be executed.
- **return value** returns a result from the function. If no return statement is used, it returns None by default.

Example: In this example, function checks whether the number passed as an argument is even or odd.

```
def evenOdd(x):
    if (x % 2 == 0):
        return "Even"
    else:
        return "Odd"

print(evenOdd(16))
print(evenOdd(7))
```

Output

Even
Odd

Types of Function Arguments

Python supports different types of arguments that can be passed during a function call.

1. Default argument: Default argument use a predefined value when no value is passed during the function call.

```
def myFun(x, y=50):
    print("x: ", x)
    print("y: ", y)
```

```
myFun(10)
```

Output

x: 10
y: 50

Explanation:

- `y=50` sets a default value for parameter `y` and `myFun(10)` passes only one argument.
- Since `y` is not provided, it uses the default value 50.

2. Keyword Arguments: pass values using parameter names, so argument order does not matter.

```
def student(fname, lname):
    print(fname, lname)
```

```
student(fname='Geeks', lname='Practice')
student(lname='Practice', fname='Geeks')
```

Output

```
Geeks Practice
Geeks Practice
```

Explanation: `fname` and `lname` are passed using parameter names and arguments can be provided in any order.

3. Positional Arguments: values are assigned to parameters based on their order in the function call.

```
def nameAge(name, age):
    print("Hi, I am", name)
    print("My age is ", age)
```

```
print("Case-1:")
nameAge("Olivia", 27)
```

```
print("Case-2:")
nameAge(27, "Olivia")
```

Output

```
Case-1:
Hi, I am Olivia
My age is 27
Case-2:
Hi, I am 27
My age is Olivia
```

Explanation:

- In Case-1, values match the correct parameters.
- In Case-2, values are swapped because the order changed.

4. Arbitrary Arguments: allow functions to accept multiple values. This is done using two special symbols:

- *args collects extra positional arguments as a tuple.
- **kwargs collects extra keyword arguments as a dictionary.

```
def myFun(*args, **kwargs):
    print("Non-Keyword Arguments (*args):")
    for arg in args:
        print(arg)

    print("Keyword Arguments (**kwargs):")
    for key, value in kwargs.items():
        print(f'{key} == {value}')
```

```
myFun('Hey', 'Welcome', first='Geeks', mid='for', last='Geeks')
```

Output

```
Non-Keyword Arguments (*args):
Hey
Welcome
Keyword Arguments (**kwargs):
first == Geeks
mid == for
last == Geeks
```

Explanation:

- *args stores extra positional arguments.
- **kwargs stores extra keyword arguments.
- loop prints all positional and keyword values separately.

Function within Functions

A function defined inside another function is called an inner function (or nested function). It is used to organize related logic and access variables from the outer function.

```
def f1():
    s = 'I love GeeksforGeeks'
    def f2():
        print(s)

    f2()
f1()
```

Output

```
I love GeeksforGeeks
```

Return Statement

Return is used to end a function and send a value back to the caller. It can return any data type, multiple values (packed into a tuple), or None if no value is given.

Syntax


```
return [expression]
```

Parameters: expression is the value returned by the function. If no value is returned, it returns None by default.

```
def sq_value(num):
    return num**2
```

```
print(sq_value(2))
print(sq_value(-4))
```

Output

```
4
16
```

Explanation:

- sq_value(num) accepts a number as input.
- num**2 calculates the square of the number.
- return sends the result back to the caller.

Pass by Reference and Pass by Value

Variables refer to objects. Function behavior depends on whether the object is mutable or immutable.

- Mutable objects like lists can be modified inside functions.
- Immutable objects like integers and strings remain unchanged.

```
def myFun(x):
    x[0] = 20
```

```
b = [10, 11, 12, 13]
myFun(b)
print(b)
```

```
def myFun2(x):
    x = 20
```

```
a = 10
myFun2(a)
print(a)
```

Output

```
[20, 11, 12, 13]
10
```

Explanation:

- myFun(x) modifies the first element of the list and lists are mutable, so the original list changes.

- myFun2(x) assigns a new value to x. Integers are immutable, so the original value of a remains unchanged.

Function return Types

The return statement is used inside a function to send a value back to the place where the function was called. Once return is executed, the function stops running, and any code written after it is ignored. If no value is returned, Python automatically returns None.

This example shows a function that returns a value.

```
def square(n):  
    return n * n
```

```
res = square(4)  
print(res)
```

Output

16

Explanation:

- square(n) takes a number as input
- return n * n sends the result back to the caller
- square(4) returns 16, which is stored in res

Syntax

```
def function_name(parameters):  
    # function body  
    return value
```

When the return statement is executed, the function immediately stops running and sends the specified value back to the caller. If no value is mentioned after return, Python automatically returns None.

Note: The return statement can only be used inside a function. Using it outside a function will result in an error.

Returning Multiple Values

In Python, a function can return more than one value at a time. These values are automatically grouped into a tuple and can be unpacked into separate variables.

```
def get_vals():  
    x = 10  
    y = 20  
    return x, y
```

```
a, b = get_vals()  
print(a)
```

```
print(b)
```

Output

```
10
```

```
20
```

Explanation:

- `return x, y` sends two values together as a tuple
- `a, b = get_vals()` unpacks the returned values into separate variables

Returning a List from a Function

A function can also return collections like lists, which is useful when you want to return multiple related values together.

```
def calc(n):
    return [n * n, n * n * n]
```

```
print(calc(3))
```

Output

```
[9, 27]
```

Explanation:

- The function returns a list containing calculated values
- The returned list can be used directly or stored in a variable

Function Returning Another Function

In Python, functions are first-class citizens, meaning you can return a function from another function. This is useful for creating higher-order functions.

```
def outer(msg):
    def inner():
        return msg
    return inner
```

```
f = outer("Hello")
print(f())
```

Output

```
Hello
```

Explanation:

- `outer()` returns the `inner()` function
- The returned function remembers the value of `msg`
- Calling `f()` executes the returned function

Function with parameters

Parameters are variables defined in a function declaration. This act as placeholders for the values (arguments) that will be passed to the function.

Arguments are the actual values that you pass to the function when you call it. These values replace the parameters defined in the function.

Although these terms are often used interchangeably, they have distinct roles within a function. This article focuses to clarify them and help us to use Parameters and Arguments effectively.

Table of Content

- Parameters
- Arguments
- Types of arguments in python

Parameters

A parameter is the variable defined within the parentheses when we declare a function.

Example:

```
# Here a,b are the parameters
def sum(a,b):
    print(a+b)
```

```
sum(1,2)
```

Output

```
3
```

Arguments

An argument is a value that is passed to a function when it is called. It might be a variable, value or object passed to a function or method as input.

Example:

```
def sum(a,b):
    print(a+b)
```

```
# Here the values 1,2 are arguments
sum(1,2)
```

Output

```
3
```

Types of arguments in python

Python functions can contain two types of arguments:

- Positional Arguments
- Keyword Arguments

Positional Arguments

Positional Arguments are needed to be included in proper order i.e the first argument is always listed first when the function is called, second argument needs to be called second and so on.

Example:

```
def fun(s1,s2):
    print(s1+s2)
```

```
fun("Geeks","forGeeks")
```

Output

GeeksforGeeks

Note: The order of arguments must match the order of parameters in the function definition.

Variable-Length Positional Arguments

To handle multiple arguments within the function, you can use *args.

```
def fun(*args):
    # Concatenate all arguments
    result = "".join(args)
    print(result)
```

```
# Function calls
fun("Geeks", "forGeeks")      # Two arguments
```

To read in detail, refer - *args and **kwargs in Python

Keyword Arguments

Keyword Arguments is an argument passed to a function or method which is preceded by a keyword and equal to sign (=). The order of keyword argument with respect to another keyword argument does not matter because the values are being explicitly assigned.

```
Def fun(s1, s2):
    print(s1 + s2)
```

```
# Here we are explicitly assigning the values
fun(s1="Geeks", s2="forGeeks")
```

Output

GeeksforGeeks

Function scope

In Python, variables are the containers for storing data values. Unlike other languages like C/C++/JAVA, Python is not “statically typed”. We do not need to declare variables before using them or declare their type. A variable is created the moment we first assign a value to it.

Python Scope variable

The location where we can find a variable and also access it if required is called the **scope of a variable**.

Python Local variable

Local variables are those that are initialized within a function and are unique to that function. It cannot be accessed outside of the function. Let's look at how to make a local variable.
def f():

```
# local variable
s = "I love Geeksforgeeks"
print(s)
```

```
# Driver code
```

```
f()
```

Output

```
I love Geeksforgeeks
```

If we will try to use this local variable outside the function then let's see what will happen.

```
def f():
```

```
# local variable
s = "I love Geeksforgeeks"
print("Inside Function:", s)
```

```
# Driver code
```

```
f()
```

```
print(s)
```

Output:

```
NameError: name 's' is not defined
```

Python Global variables

Global variables are the ones that are defined and declared outside any function and are not specified to any function. They can be used by any part of the program.

Example:

```
# This function uses global variable s
```

```
def f():
```

```
    print(s)
```

```
# Global scope
```

```
s = "I love Geeksforgeeks"
```

```
f()
```

Output:

I love Geeksforgeeks

Global and Local Variables with the Same Name

Now suppose a variable with the same name is defined inside the scope of the function as well then it will print the value given inside the function only and not the global value.

This function has a variable with

name same as s.

def f():

 s = "Me too."

 print(s)

Global scope

s = "I love Geeksforgeeks"

f()

print(s)

Output:

Me too.

I love Geeksforgeeks

The variable s is defined as the string "I love Geeksforgeeks", before we call the function f().

The only statement in f() is the print(s) statement. As there are no locals, the value from the global s will be used.

The question is, what will happen if we change the value of s inside of the function f()? Will it affect the global s as well? We test it in the following piece of code:

def f():

 print(s)

 # This program will NOT show error

 # if we comment below line.

 s = "Me too."

 print(s)

Global scope

s = "I love Geeksforgeeks"

f()

print(s)

Output:

Traceback (most recent call last):

File "/home/370cac45bae7f1e6096520b7a0edb604.py", line 13, in

 f()

File "/home/370cac45bae7f1e6096520b7a0edb604.py", line 3, in f

 print(s)

UnboundLocalError: local variable 's' referenced before assignment

To make the above program work, we need to use **global** keyword. We only need to use global keyword in a function if we want to do assignments / change them. global is not needed for printing and accessing. Why? Python “assumes” that we want a local variable due to the assignment to s inside of f(), so the first print statement throws this error message. Any variable which is changed or created inside of a function is local, if it hasn't been declared as a global variable.

To tell Python, that we want to use the global variable, we have to use the keyword global, as can be seen in the following

Example:

This function modifies global variable 's'

def f():

 global s

 print(s)

 s = "Look for Geeksforgeeks Python Section"

 print(s)

Global Scope

s = "Python is great !"

f()

print(s)

Output:

Python is great!

Look for Geeksforgeeks Python Section

Look for Geeksforgeeks Python Section

Consider the below example for a better understanding of the topic.

Python program to demonstrate

scope of variable

a = 1

Uses global because there is no local 'a'

def f():

 print('Inside f() : ', a)

Variable 'a' is redefined as a local

def g():

 a = 2

 print('Inside g() : ', a)

Uses global keyword to modify global 'a'

def h():

 global a

 a = 3


```
print('Inside h() : ', a)
```

```
# Global scope
print('global : ', a)
f()
print('global : ', a)
g()
print('global : ', a)
h()
print('global : ', a)
```

Output:

```
global : 1
Inside f() : 1
global : 1
Inside g() : 2
global : 1
Inside h() : 3
global : 3
```

Python Nonlocal keyword

In Python, the nonlocal keyword is used in the case of nested functions. This keyword works similarly to the global, but rather than global, this keyword declares a variable to point to the variable of an outside enclosing function, in case of nested functions.

Example:

```
# Python program to demonstrate
# nonlocal keyword
```

```
print("Value of a using nonlocal is : ", end="")
```

```
def outer():
    a = 5

    def inner():
        nonlocal a
        a = 10
    inner()
    print(a)
```

```
outer()
```

```
# demonstrating without non local
# inner loop not changing the value of outer a
# prints 5
print("Value of a without using nonlocal is : ", end="")
```

```
def outer():
```

```
    a = 5
```

```
    def inner():
```

```
        a = 10
```

```
    inner()
```

```
    print(a)
```

```
outer()
```

Output:

Value of a using nonlocal is : 10

Value of a without using nonlocal is : 5

Local and Global functions

Variables store data in memory and scope defines the specific region of a program where a variable is accessible. It dictates the visibility and lifetime of the variable within the source code. Variables are categorized into two primary scopes: Global and Local.

Local Variables

Local variables are defined inside a function and exist only during its execution. They cannot be accessed from outside the function.

Example 1: In this example, we are creating and accessing a local variable inside a function.

```
def greet():
```

```
    msg = "Hello from inside the function!"
```

```
    print(msg)
```

```
greet()
```

Output

Hello from inside the function!

Explanation: We define greet() with a local variable msg and print it. Since msg exists only during the function's execution, it's accessed within the function. Calling greet() displays the message.

Example 2: In this example, we are creating a local variable inside a function and then trying to access it outside the function, which causes an error.

```
def greet():
```

```
    msg = "Hello!"
```

```
    print("Inside function:", msg)
```

```
greet()
```

```
print("Outside function:", msg)
```

Output

Traceback (most recent call last):

File "/home/guest/sandbox/Solution.py", line 6, in <module>

print("Outside function:", msg)

NameError: name 'msg' is not defined

Explanation: msg is a local variable inside greet() and can only be accessed there. Printing it outside causes an error because it doesn't exist globally.

Global Variables

Global variables are declared outside all functions and can be accessed anywhere in the program, including inside functions.

Example: In this example, we are creating a global variable and then accessing it both inside and outside a function.

```
msg = "Python is awesome!"
```

```
def display():
```

```
    print("Inside function:", msg)
```

```
display()
```

```
print("Outside function:", msg)
```

Output

Inside function: Python is awesome!

Outside function: Python is awesome!

Explanation: msg is a global variable accessible both inside and outside the display() function. Calling display() prints the global msg and printing msg outside the function works as expected.

Note: If a variable is not found inside a function (local scope), Python automatically looks for it in global scope. However, if a local variable has same name as a global one, it will shadow global variable inside that function.

Use of Local and Global variables

If a variable is defined both globally and locally with the same name, local variable shadows the global variable inside the function. Changes to the local variable do not affect the global variable unless explicitly declare variable as global.

```
def fun():
```

```
    s = "Me too."
```

```
    print(s)
```

```
s = "I love Geeksforgeeks"
```

```
fun()
```

```
print(s)
```

Output

Me too.

I love Geeksforgeeks

Explanation: Inside fun(), 's' is a local variable set to "Me too." and prints that value. Outside, the global s remains "I love Geeksforgeeks", so printing s afterward shows the global value.

Modifying Global Variables Inside a Function

By default, one cannot modify a global variable inside a function without declaring it as global. If you try, Python will raise an error because it treats variable as local. To modify a global variable use the global keyword.

Without global (causes error)

```
def fun():
    s += ' GFG' # Error: Python thinks s is local
    print(s)
```

```
s = "I love GeeksforGeeks"
```

```
fun()
```

Output

UnboundLocalError: local variable 's' referenced before assignment

Explanation: Inside fun(), Python assumes 's' is local since we try to modify it. But no local 's' exists before that line, so an error occurs.

With global (works correctly)

```
s = "Python is great!"
```

```
def fun():
    global s
    s += " GFG" # Modify global variable
    print(s)
    s = "Look for GeeksforGeeks Python Section" # Reassign global
    print(s)
```

```
fun()
```

```
print(s)
```

Output

Python is great! GFG

Look for GeeksforGeeks Python Section

Look for GeeksforGeeks Python Section

Explanation:

- Declaring 's' as global tells Python to use the variable from the global scope.
- The function first appends " GFG", then reassigns 's'.
- Changes persist outside the function.

Global vs Local with Same Name

```

a = 1 # Global variable

def f():
    print("f():", a) # Uses global a

def g():
    a = 2 # Local shadows global
    print("g():", a)

def h():
    global a
    a = 3 # Modifies global a
    print("h():", a)

print("global:", a)
f()
print("global:", a)
g()
print("global:", a)
h()
print("global:", a)

```

Output

```

global: 1
f(): 1
global: 1
g(): 2
global: 1
h(): 3
global: 3

```

Explanation:

- f() prints global 'a' without changing it.
- g() creates a local 'a', leaving global one untouched.
- h() declares 'a' as global and modifies it.

Local Variable vs Global Variable

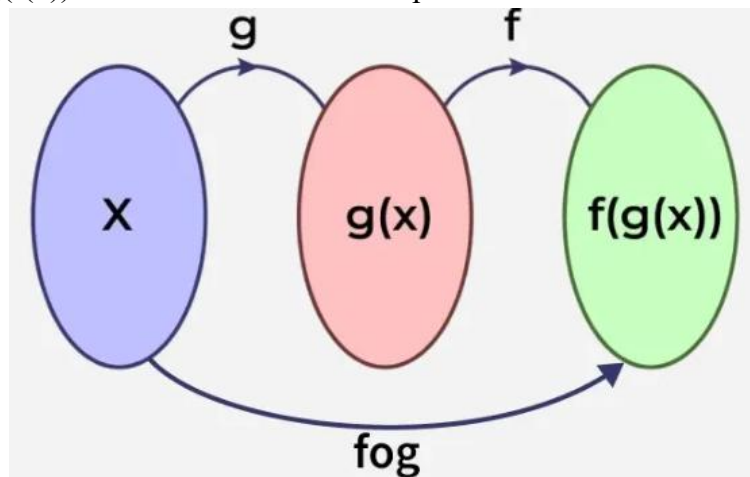
Comparison basis	Global Variable	Local Variable
Definition	Declared outside functions	Declared inside a function
Lifetime	Created when defined and exists until program ends or is deleted	Created when the function is called and destroyed when it returns
Data Sharing	Shared across all functions	Not shared, exists only within its function

Scope	Can be accessed anywhere in the program	Accessible only inside the function
Parameters needed	No parameter passing needed	May be created through parameters or assignments inside the function
Storage	Managed in the global namespace (not fixed memory location)	Stored in the function's local namespace (stack frame)
Value	Changes affect the entire program	Changes are local and don't affect other functions

Function composition

The composition of functions is a process where you combine two functions into a new function. Specifically, it involves applying one function to the result of another function. In simpler terms, the output of one function becomes the input for the other function.

For example, if two functions f and g are given, then applying g to the output of f gives a new function $h(x) = g(f(x))$. This is known as the composition of functions.



Breakdown of the diagram:

1. x : The initial input value.
2. $g(x)$: The first function g is applied to the input x , producing the output $g(x)$.
3. $f(g(x))$: The second function f is then applied to the result of $g(x)$, leading to the final output $f(g(x))$.

This illustrates the composition $(f \circ g)(x) = f(g(x))$, where the output of the first function $g(x)$ is fed into the second function $f(x)$.

Mathematically, the composition of two functions, f and g , is denoted as:

$$(f \circ g)(x) = f(g(x))$$

- First, apply the function g to the input x , resulting in $g(x)$
- Then, apply the function f to the result $g(x)$, resulting in $f(g(x))$

So, the output of g becomes the input of f .

Example: If $f(x) = x^2$ and $g(x) = x + 3$. Then calculate the composition $g(f(x))$ and $f(g(x))$.

Solution:

$$g(f(x)) = g(x^2) = x^2 + 3$$

Similarly, we can easily calculate the $f(g(x))$ or $(f \circ g)(x)$ where $g(x)$ is computed first.

$$f(g(x)) = f(x + 3) = (x + 3)^2$$

Notice that $g(f(x))$ is not equal to $f(g(x))$; they can or cannot be equal depending on the functions $f(x)$ and $g(x)$. The composition of a function is also called the function of a function.

Thus, we can say that,

- For $f(g(x))$, $g(x)$ is the input of the function $f(x)$.
- For $g(f(x))$, $f(x)$ is the input of the function $g(x)$.

Symbol

The composition of functions is represented using the symbol $\circ$. We can also represent the composition of functions by simply using the parenthesis $()$. For any two given functions $f(x)$ and $g(x)$, we can find the composition of the function by using the formula,

$$(f \circ g)(x) = f(g(x))$$

The above function is read as "f of g of x." Here, first x is passed to $g(x)$, which gives the answer in x , and then the answer is passed to $f(x)$ to find the desired composition of the function.

$$(g \circ f)(x) = g(f(x))$$

The above function is read as "g of f of x." Here, first x is passed to $f(x)$, which gives the answer in x , and then the answer is passed to $g(x)$ to find the desired composition of the function.

Solving Composite Function

The composition of functions means applying one function to the result of another. It is written as $(f \circ g)(x) = f(g(x))$, which means you first find the value of $g(x)$, then use that result as the input for f . Similarly, $g(f(x))$ means you solve $f(x)$ first and then apply g to it.

It's just like solving an expression using the BODMAS rule—work from the inside out. The correct order matters a lot; if you reverse it, you'll get a completely different answer.

This idea helps solve many math problems more clearly and step-by-step.

For $f(g(x))$:

- First solve $g(x)$
- Then plug that into $f(x)$ to get $f(g(x))$
- For $g(f(x))$:
- First solve $f(x)$
- Then plug that into $g(x)$ to get $g(f(x))$

This can be easily understood with the help of an example

Example: If $f(x) = x + 2$ and $g(x) = (2x)^2$, find $f(g(x))$ and $g(f(x))$

Solution:

For $f(g(x))$

$$\text{Given, } g(x) = (2x)^2 = 4x^2$$

$$\text{Thus, } f(g(x)) = f(4x^2) = 4x^2 + 2$$

Now for $g(f(x))$

Given, $f(x) = x + 2$

Thus, $g(f(x)) = (2(x+2))^2 = (2x + 4)^2$

$\Rightarrow g(f(x)) = 4x^2 + 16 + 16x$

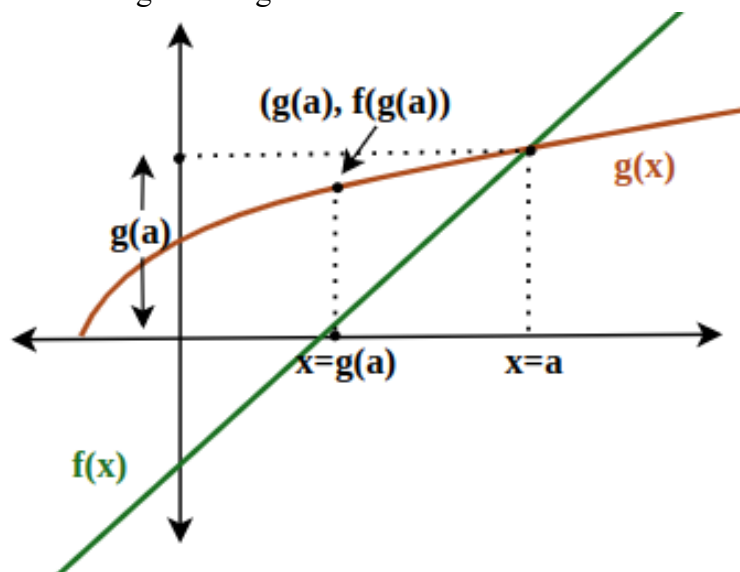
Finding Composite Function From Graph

If the functions are not given algebraically and only their graphical value is given, then also they can be easily calculated using the graphs given.

To find the composite function of two functions $f(x)$ and $g(x)$ whose graphs are given, if (x, y) is a point on a function $f(x)$, then $f(x) = y$. We can find $f(g(a))$ (i.e., $f(g(x))$ at $x = a$) using the above information.

- **Step 1:** Firstly find $g(a)$, i.e., the y-coordinate on the graph of $g(x)$ at $x = a$
- **Step 2:** Now find $f(g(a))$, i.e., the y-coordinate on the graph of $f(x)$ at $x = g(a)$.

This can be understood using the image discussed below.



Finding Composite Function From Table

A composite function, or composition of a function, can also be easily calculated using the table in which the values of the function corresponding to a given input value are given. Let's find the composite function using the table given below.

Example: From the following tables of $f(x)$ and $g(x)$, find $g(f(1))$.

For $f(x)$, the table is,

x	1	2	3
f(x)	3	4	5

For $g(x)$, the table is,

x	1	2	3
g(x)	1	4	9

Solution:

$f(1) = 3$ (from the first table)

$g(f(1)) = g(3)$

Now using the second table we can get the above value.

$g(3) = 9$

Thus, the required solution for $g(f(1))$ is 9

Self-Composition

We can also compose a function with itself, and it is called a self-composite function. For any given function $f(x)$, the function composition with itself is $f(f(x))$; it is also defined as $(f \circ f)(x)$.

Now,

$$(f \circ f)(x) = f(f(x))$$

It can be better understood with the help of an example

Example: If $f(x) = x^3$, then find $(f \circ f)(x)$.

Solution:

$$\text{Given: } f(x) = x^3$$

$$(f \circ f)(x) = f(f(x))$$

$$= f(x^3)$$

$$= (x^3)^3$$

$$(f \circ f)(x) = x^9$$

Domain and Range of Composition of Functions

It is not possible to compose any two functions; some functions cannot be composed together. For example, let's say $f(x) = \ln(x)$ and $g(x) = -x$. If we try to compose $f(g(x))$, it is not possible for the positive value of x , as the logarithmic function cannot take negative input values, so $f(g(x))$ is not possible. So, there are certain things that should be kept in mind while deciding on composing the function.

So before composing any two functions, first we have to find the domain and range of the function.

Domain of Composite Functions

For any function $f(x)$ and $g(x)$ defined as $g: X \rightarrow Y$ and $f: Y \rightarrow Z$, then $f(g(x))$ is defined as $f \circ g: X \rightarrow Z$, i.e., the domain of $f \circ g$ is X and the range is Z .

If the functions are defined algebraically, then also we can easily define their domain. To find the domain of the composite function, use the following steps. If we have to find the domain of $f(g(x))$

Step 1: Firstly we will find the domain of the inner function $g(x)$

Step 2: Then we find the domain of the function obtained by finding $f(g(x))$

Step 3: Now the intersection of the domain of $g(x)$ and the domain of $f(g(x))$ is the domain of $f(g(x))$

Range of Composite Functions

The range of a composite function is the set of all output values obtained when one function is applied to the result of another.

Example: Find the domain and range of $f(g(x))$ when $f(x) = x + 2$ and $g(x) = x^2$.

Solution:

For $f(g(x))$,

The inner function is $g(x)$ and its domain is $A = \mathbb{R}$

$$\text{Now } f(g(x)) = f(x^2)$$

$$f(g(x)) = x^2 + 2$$

The domain of $x^2 + 2$ is $\mathbb{R}$ (say B)

Now the intersection of A and B is the domain of $f(g(x))$

Domain of $f(g(x)) = A \cap B = \mathbb{R}$

The range of $f(g(x))$ is the range of $x^2 + 2$

Now the range is $[2, \infty)$

Properties of Composition of Function

There are various properties of the composition of a function:

- **Associativity Property:** For functions f , g , and h , the composition of these functions is associative, if and only if
- $(f \circ g) \circ h = f \circ (g \circ h)$
- **Identity Property:** For any function f , the identity function $I(x) = x$ acts as the identity element for composition, meaning
- $f \circ I = I \circ f = f$
- **Inverse Property:** If a function f has an inverse function f^{-1} , then
- $(f \circ f^{-1}) = I = (f^{-1} \circ f)$
- Some other properties of the composition of functions are
- The composition of two or more one-to-one functions (injective) is always one-to-one.
- The composition of onto functions (surjective) is always onto.
- Also, the composition of two or more bijections (one-to-one and onto) is always a bijection.
- The inverse function of the composition of two invertible functions has the property that $(f \circ g)^{-1} = g^{-1} \circ f^{-1}$.

Solved Examples

Example 1: For the given functions $f(x) = e^x$ and $g(x) = x^2 + 1$. Find out the values of $f(g(x))$ and $g(f(x))$.

Solution:

The domain of both the functions are real numbers, so there is no need to modify the domain for the first function in any case.

For $f \circ g(x)$,

$$f \circ g(x) = f(g(x))$$

$$\Rightarrow f \circ g(x) = f(x^2 + 1)$$

$$\Rightarrow f \circ g(x) = e^{x^2+1}$$

For $g \circ f(x)$

$$g \circ f(x) = g(f(x))$$

$$\Rightarrow g \circ f(x) = g(e^x)$$

$$\Rightarrow g \circ f(x) = (e^x)^2 + 1$$

$$\Rightarrow g \circ f(x) = e^{2x} + 1$$

Example 2: For the given functions $f(x) = 2x$ and $g(x) = x^2 + 1$. Find out the values of $f(g(x))$ and $g(f(x))$ at $x = 2$.

Solution:

The domain of both the functions are real numbers, so there is no need to modify the domain for the first function in any case.

$$F \circ g(x) = f(g(x))$$

$$\Rightarrow f \circ g(x) = f(x^2 + 1)$$

$$\Rightarrow f \circ g(x) = 2(x^2 + 1)$$

At $x = 2$,

$$\Rightarrow f(g(2)) = 2(4 + 1)$$

$$\Rightarrow f(g(2)) = 10$$

$$g \circ f(x) = g(f(x))$$

$$\Rightarrow g \circ f(x) = g(2x)$$

$$\Rightarrow g \circ f(x) = (2x)^2 + 1$$

$$\Rightarrow g \circ f(x) = 4x^2 + 1$$

At $x = 2$

$$g(f(2)) = 4(2^2) + 1$$

$$\Rightarrow g(f(2)) = 4(4) + 1$$

$$\Rightarrow g(f(2)) = 17$$

Example 3: For the given functions $f(x) = \sin(x)$ and $g(x) = x^2$. Find out the domain and range for $f \circ g(x)$ and $g \circ f(x)$.

Solution:

$f(x) = \sin x$ has domain as all real numbers and range $[-1, 1]$.

While $g(x) = x^2$ has domain all the real numbers and range $\mathbb{R}^+$.

$$f \circ g(x) = f(g(x)) = f(x^2) = \sin(x^2)$$

Domain is all real numbers, the range is $[-1, 1]$

$$g \circ f(x) = g(f(x)) = g(\sin x) = \sin^2 x$$

Domain is all real numbers, the range is between 0 and 1.

Example 4: For the given functions $f(x) = \log(x)$ and $g(x) = x + 1$, find out the values of $f(g(x))$.

Solution:

Domain of $f(x)$ is all positive numbers i.e $\mathbb{R}^+$ and range is all real numbers.

Domain and range for $g(x)$ is all real numbers.

$$F(g(x)) = f(x + 1) = \log(x + 1)$$

While doing this, the domain of $f(x)$ must be kept in mind as it is logarithmic function it only takes positive values.

$$x + 1 > 0$$

$$x > -1$$

So,

Domain is $(-1, \infty)$

Range is all real numbers $\mathbb{R}$

User Defined Functions

User-Defined Function is a function created by the user to perform specific tasks in a program. Unlike built-in functions provided by a programming language, it allow for customization and code reusability, improving program structure and efficiency. **For Example:**

```
def fun( x ):
    if (x % 2 == 0):
        print("even")
    else:
        print("odd")
```

fun(2) # function calling

Output

even

Explanation: fun(x) checks if a number is even or odd using $x \% 2$. If divisible by 2, it prints "even", otherwise "odd".

Syntax

```
def function_name(parameters):
# Function body
return result
```

Function call:

```
function_name(arguments)
```

Types of Users Defined Functions

1. Parameterized Functions

These functions accept parameters (arguments) to process and return results dynamically. Parameters allow for flexibility, enabling the function to handle different inputs each time it is called.

```
def fun(name):
    print("Hello,", name)
```

fun("shakshi")

Output

Hello, shakshi

Explanation: fun(name) prints a greeting with the provided name. When called with "shakshi", it outputs "Hello, shakshi".

2. Functions with default arguments

A function can have default values assigned to its parameters. If no argument is provided when calling the function, it takes the default value.

```
def fun(x, y=50):
    print("x:", x)
    print("y:", y)
```

fun(10)

Output

```
x: 10
```

```
y: 50
```

Explanation: `fun(x, y=50)` takes two parameters, where `y` has a default value of 50. It prints the values of `x` and `y`. When called with `fun(10)`, it uses 10 for `x` and the default value 50 for `y`.

3. Keyword argument functions

Function arguments can be passed using keywords to improve code readability. This ensures the correct mapping of values to parameters, regardless of their order.

```
def fun(name, age):
    print(name, "is", age, "years old.")
```

```
fun(age=21, name="shakshi")
```

Output

```
shakshi is 21 years old.
```

Explanation: `fun(name, age)` prints the name and age with the message "is [age] years old." It is called using keyword arguments (`age=21, name="shakshi"`), which assigns the values explicitly to `name` and `age`.

4. Variable length argument functions

When the number of arguments is unknown, a function can accept multiple arguments using `*args` (for non-keyword arguments) or `**kwargs` (for keyword arguments).

```
def fun(*args):
    for arg in args:
        print(arg)
```

```
fun("Python", "Java", "C++")
```

Output

```
Python
```

```
Java
```

```
C++
```

Explanation: `fun(*args)` uses a variable-length argument list, allowing it to accept any number of arguments. It then iterates through each argument and prints it.

5. Functions with Return value

A function can return a value using the `return` statement. This allows the function to send back a result for further computation.

```
def fun(num):
    return num * num
```

```
res = fun(5)
print(res)
```

Output

25

Explanation: fun(num) takes a number num as input and returns its square (num * num). When called with fun(5), it calculates 5 * 5 and stores the result in res.

6. Lambda functions

A lambda function is an anonymous (nameless) function that is defined in a single line using the lambda keyword. It is used for short, simple operations where defining a full function is unnecessary.

```
res = lambda x: x * x
print(res(4))
```

Output

16

Explanation: lambda x: x * x takes one input x and returns its square (x * x). When res(4) is called, it computes 4 * 4, resulting in 16 .

Pass by Reference or Pass by Value

Python does not follow strictly pass by value or pass by reference. Instead, it uses pass by object reference (call by object sharing). This means that references to objects are passed to functions.

- If the object is immutable (e.g., integers, strings, tuples), modifying it inside the function creates a new object, so the original remains unchanged.
- If the object is mutable (e.g., lists, dictionaries), changes made inside the function affect the original object.

Example: Immutable Objects

```
def fun(x):
    print("Value received:", x, "id:", id(x))
```

```
x = 12
print("Value passed:", x, "id:", id(x))
```

```
fun(x)
```

Output

```
Value passed: 12 id: 140027364606256
Value received: 12 id: 140027364606256
```

Explanation:

- fun(x) prints the value of x and its memory identifier using id() and x is assigned the value 12 and before calling the function, both the value and id of x are printed.
- When the function is called, it prints the same information for x. Since integers are immutable, no modification happens to the original object.

Example: Mutable Objects

```
def fun(a):
    a[0] = 100 # Changing the first element of the list
    print("Inside function - lst:", a)

a = [1, 2, 3] # List is mutable
print("Before function call - lst:", a)

fun(a)
# List is modified outside the function
print("After function call - lst:", a)
```

Output

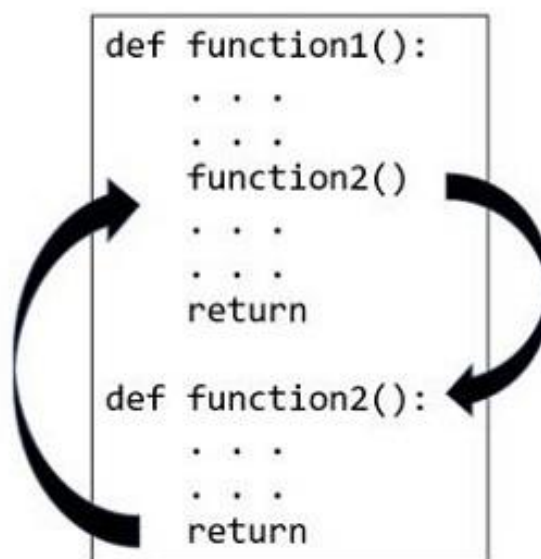
```
Before function call - lst: [1, 2, 3]
Inside function - lst: [100, 2, 3]
After function call - lst: [100, 2, 3]
```

Explanation: fun(a) modifies the first element of the passed list ([0] = 100). Since lists are mutable in Python, the change is reflected outside the function. Before the call, a is [1, 2, 3], and after the call, it becomes [100, 2, 3].

Function Defining

A Python function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

A top-to-down approach towards building the processing logic involves defining blocks of independent reusable functions. A Python function may be invoked from any other function by passing required data (called **parameters** or **arguments**). The called function returns its result back to the calling environment.



Types of Python Functions

Python provides the following types of functions –

S.No	Type & Description
1	Built-in functions Python's standard library includes number of built-in functions. Some of Python's built-in functions are print(), int(), len(), sum(), etc. These functions are always available, as they are loaded into computer's memory as soon as you start Python interpreter.
2	Functions defined in built-in modules The standard library also bundles a number of modules. Each module defines a group of functions. These functions are not readily available. You need to import them into the memory from their respective modules.
3	User-defined functions In addition to the built-in functions and functions in the built-in modules, you can also create your own functions. These functions are called user-defined functions.

Defining a Python Function

You can define custom functions to provide the required functionality. Here are simple rules to define a function in Python –

- Function blocks begin with the keyword `def` followed by the function name and parentheses `()`.
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement; the documentation string of the function or docstring.
- The code block within every function starts with a colon `:` and is indented.
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

Syntax to Define a Python Function

```
def function_name( parameters ):
    "function_docstring"
    function_suite
    return [expression]
```

By default, parameters have a positional behavior and you need to inform them in the same order that they were defined.

Once the function is defined, you can execute it by calling it from another function or directly from the Python prompt.

Example to Define a Python Function

The following example shows how to define a function `greetings()`. The bracket is empty so there aren't any parameters. Here, the first line is a docstring and the function block ends with return statement.


```
def greetings():
    "This is docstring of greetings function"
    print ("Hello World")
    return
```

When this function is called, Hello world message will be printed.

Function Calling

Function Calling in programming refers to the process of invoking or executing a function within a program. Functions are blocks of code that perform a specific task, and they allow programmers to organize their code into reusable units, making it easier to manage and maintain.

What is Function Calling?

Function Calling involves specifying the function's name followed by **parentheses()**. If the function requires input values, known as **parameters**, then they are passed inside the parentheses as **arguments**. When the function is called, the program execution jumps to the function's code block, executes it, and then returns control to the point in the program where the function was called.

Syntax:

Below is the most common syntax to call a function:

```
function_name(argument_1, argument_2, argument_3, ...)
```

If a function returns some value and we need to store the value in a variable, then we can write it as:

```
variable_name = function_name(argument_1, argument_2, argument_3, ...)
```

Function Calling in C:

Here are the implementation of Function Calling in C language:

```
#include <stdio.h>
```

```
// sum function which takes two parameters and return their
```

```
// sum
```

```
int sum(int a, int b) { return a + b; }
```

```
int main()
```

```
{
```

```
    int x = 5, y = 10;
```

```
    // Calling the function sum with x and y as arguments
```

```
    int s = sum(x, y);
```

```

    printf("%d + %d = %d", x, y, s);
    return 0;
}

```

Output

5 + 10 = 15

Function Calling in C++:

Here are the implementation of Function Calling in C++ language:

```

#include <iostream>
using namespace std;

// sum function which takes two parameters and returns their
// sum
int sum(int a, int b) { return a + b; }

int main()
{
    int x = 5, y = 10;
    // Calling the function sum with x and y as arguments
    int s = sum(x, y);

    cout << x << " + " << y << " = " << s;
    return 0;
}

```

Output

5 + 10 = 15

Function Calling in Java:

Here are the implementation of Function Calling in java language:

```

/*package whatever //do not write package name here */

import java.io.*;

class GFG {
    // sum function which takes two parameters and returns
    // their sum
    public static int sum(int a, int b) { return a + b; }

    public static void main(String[] args)
    {
        int x = 5, y = 10;
        // Calling the function sum with x and y as

```

```

    // arguments
    int s = sum(x, y);

    System.out.println(x + " + " + y + " = " + s);
}
}

```

Output

5 + 10 = 15

Function Calling in Python:

Here are the implementation of Function Calling in python language:

sum function which takes two parameters and returns their sum

```

def sum(a, b):
    return a + b

```

```

x, y = 5, 10
# Calling the function sum with x and y as arguments
s = sum(x, y)
print(f'{x} + {y} = {s}')

```

Output

5 + 10 = 15

Function Calling in C#:

Here are the implementation of Function Calling in C# language:

using System;

```

public class GFG {

    // sum function which takes two parameters and returns
    // their sum
    static int Sum(int a, int b) { return a + b; }

    static void Main()
    {
        int x = 5, y = 10;
        // Calling the function sum with x and y as
        // arguments
        int s = Sum(x, y);
    }
}

```

```

        Console.WriteLine($"{x} + {y} = {s}");
    }
}

```

Output

5 + 10 = 15

Function Calling in JavaScript:

Here are the implementation of Function Calling in javascript language:

// sum function which takes two parameters and returns their sum

```

function sum(a, b) {
    return a + b;
}

```

let x = 5, y = 10;

// Calling the function sum with x and y as arguments

let s = sum(x, y);

```

console.log(x + " + " + y + " = " + s);

```

Output

5 + 10 = 15

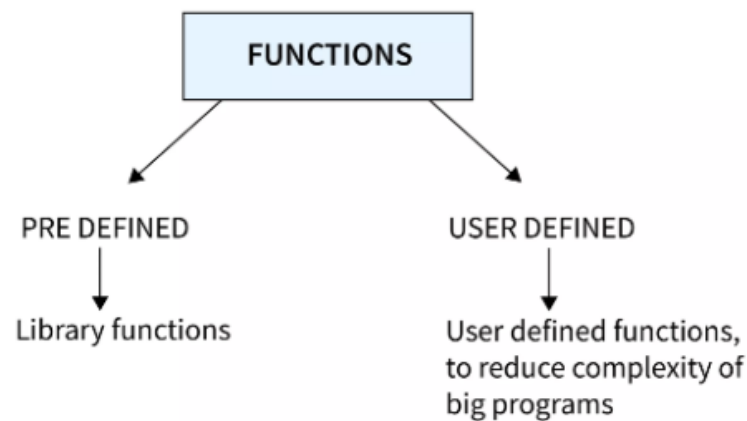
Types of Functions,**What are the types of Functions in Python?**

Functions are the basic building block of any Python program, defined as the organized block of reusable code, which can be called whenever required.

A function is used to carry out a specific task. The function might require multiple inputs. When the task is done executing, the function can or can not return one or more values.

There are two types of functions in python:

- **User-Defined Functions** - these types of functions are defined by the user to perform any specific task
- **Built-in Functions** - These are pre-defined functions in python.



Built-in Functions

Built-in functions are already defined in python. A user has to remember the name and parameters of a particular function. Since these functions are pre-defined, there is no need to define them again.

Some of the widely used built-in functions are given below:

Example:

```
l = [2, 5, 19, 7, 43]
```

```
print("Length of string is",len(l))
print("Maximum number in list is ",max(l))
print("Type is",type(l))
```

Output:

```
Length of string is 5
Maximum number in list is 43
```

```
Type is <class 'list'>
```

In the above program, we created a list and stored a few numbers in it, then called various in-built functions on the list and displayed the output.

Some of the widely used python built-in functions are:

Function	Description
len()	Returns the length of a python object
abs()	Returns the absolute value of a number
max()	Returns the largest item in a python iterable
min()	Returns the largest item in a python iterable
sum()	Sum() in Python returns the sum of all the items in an iterator
type()	The type() in Python returns the type of a python object
help()	Executes the python built-in interactive help console
input()	Allows the user to give input
format()	Formats a specified value
bool()	Returns the boolean value of an object

Quiz Pop

Quiz Type

SCQ

100

Success Rate:35%

What is the purpose of using functions in Python?

To increase code complexity

To minimize code modularity

To maximize code redundancy

To organize and reuse code

Submit

User-Defined Functions

These functions are defined by a programmer to perform any specific task or to reduce the complexity of big problems and use that function according to their need.

Example:

```
def sub(x, y):
    return x-y
```

```
print(sub(5,2))
```

Output:

3

In the above program, we have created our program that subtracts two numbers passed as arguments and displays the result. Such types of functions are known as user-defined functions. Use this Python Online Compiler to compile your code.

Before moving ahead, do check out our Python course for beginners and learn Python from scratch!

Advantages of functions in Python

- **Helps in increasing modularity of code** – Functions in python help the user to divide the program into smaller parts and solve them individually, thus making it easier to implement.
- **Minimizes Redundancy** – Python functions help us to save the effort of rewriting the whole code. All we got to do is call the function once it is defined.
- **Maximizes Code Reusability** – Once a function is defined in python, it can be called as many times as needed, thus enhancing code reusability.
- **Improves Clarity of Code** – Since a large program is divided into sections with the help of functions, it helps increase the readability of code while ensuring easy debugging.

Function Arguments

Function Arguments are values or variables passed into a function when it is called. The arguments to a function play a significant role as they provide the necessary input for the function to perform its intended task. In this article, we will discuss what are Function Arguments in Programming across various programming languages.

What are Function Arguments?

Function Arguments are the input values passed to a function when it is called so that the function can use them while performing its task. The number of arguments that can be passed into a function is determined by the number of parameters in the function definition. Below is the syntax for Function Arguments:

function_name(arg1, arg2, ...), where arg1 and arg2 are arguments passed while calling the function.

Function Arguments in C:

Here is the implementation of the Function Arguments in C language:

```
#include <stdio.h>

// Function declaration with parameters
void printSum(int X, int Y) { printf("%d\n", (X + Y)); }

int main()
{
    // Function call with arguments 4 and 5
    printSum(4, 5);
    return 0;
}
```

Output

9

Function Arguments in C++:

Here is the implementation of the Function Arguments in C++ language:

```
#include <iostream>
using namespace std;

// Function declaration with parameters
void printSum(int X, int Y) { cout << (X + Y) << endl; }

int main()
{
    // Function call with arguments 4 and 5
    printSum(4, 5);
}
```

```
    return 0;
}
```

Output

9

Function Arguments in Java:

Here is the implementation of the Function Arguments in java language:

```
/*package whatever //do not write package name here */
```

```
import java.io.*;
```

```
class GFG {
    // Method definition with parameters
    static void printSum(int X, int Y)
    {
        System.out.println(X + Y);
    }

    public static void main(String[] args)
    {
        // Method call with arguments 4 and 5
        printSum(4, 5);
    }
}
```

Output

9

Function Arguments in Python:

Here is the implementation of the Function Arguments in python language:

```
# Function definition with parameters
```

```
def printSum(X, Y):
    print(X + Y)
```

```
# Function call with arguments 4 and 5
printSum(4, 5)
```

Output

9

Function Arguments in C#:

Here is the implementation of the Function Arguments in C# language:

```
using System;
```



```

public class GFG {

    // Method declaration with parameters
    static void PrintSum(int X, int Y)
    {
        Console.WriteLine(X + Y);
    }

    static void Main(string[] args)
    {
        // Method call with arguments 4 and 5
        PrintSum(4, 5);
    }
}

```

Output

9

Function Arguments in Javascript:

Here is the implementation of the Function Arguments in javascript language:

```

// Function definition with parameters
function printSum(X, Y) {
    console.log(X + Y);
}

// Function call with arguments 4 and 5
printSum(4, 5);

```

Output

9

Lambda Function

Lambda functions are small anonymous functions, meaning they do not have a defined name. These are small, short-lived functions used to pass simple logic to another function.

- Contain only one expression.
- Result of that expression is returned automatically (no return keyword needed).

In this example, a lambda function is defined to convert a string to its upper case using upper().

```

a = 'GeeksforGeeks'
upper = lambda x: x.upper()
print(upper(a))

```

Output

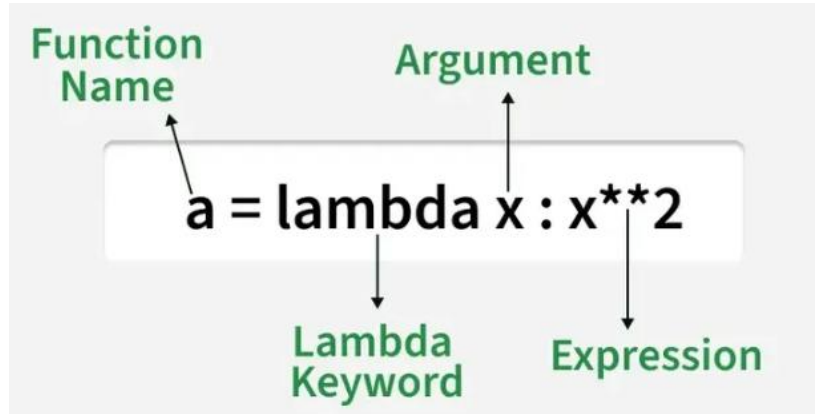
GEEKSFORGEEKS

Explanation:

- 'a' store the string 'GeeksforGeeks'.
- upper is a lambda function that takes an argument x and returns x.upper().
- upper(a) applies the lambda to a, converting it to uppercase.

Syntax

Lambda functions are created using the lambda keyword. Below is the syntax:

**Python Lambda Expression**

- **Function name (a):** Stores the lambda function so it can be reused later.
- **Lambda keyword (lambda):** Defines an anonymous (inline) function in Python.
- **Argument (x):** The input value passed to the lambda function.
- **Expression (x**2):** The operation performed on the argument and returned as the result.

Use Cases of Lambda Functions**1. Using with Condition Checking**

A lambda function can use conditional expressions (if-else) to return different results based on a condition.

```
check = lambda x: "Positive" if x > 0 else "Negative" if x < 0 else "Zero"
print(check(5))
print(check(-3))
print(check(0))
```

Output

```
Positive
Negative
Zero
```

Explanation:

- lambda function takes x as input.
 - It uses nested if-else statements to return "Positive," "Negative," or "Zero."
- Below example checks divisibility by 2 and returns "Even" or "Odd" accordingly.
- ```
check = lambda x: "Even" if x % 2 == 0 else "Odd"
print(check(4))
print(check(7))
```

**Output**

Even

Odd

**Explanation:**

- lambda checks if a number is divisible by 2 ( $x \% 2 == 0$ ).
- Returns "Even" for true and "Odd" otherwise.

**2. Using with List Comprehension**

Lambda functions can be combined with list comprehensions to apply the same operation to multiple values in a compact way.

```
func = [lambda arg=x: arg * 10 for x in range(1, 5)]
```

```
for i in func:
```

```
 print(i())
```

**Output**

10

20

30

40

**Explanation:**

- lambda function multiplies each element by 10.
- list comprehension iterates through range(1, 5) and applies the lambda to each value.

**3. Using for Returning Multiple Results**

Although a lambda can contain only one expression, it can still return multiple results by combining them into a tuple.

```
calc = lambda x, y: (x + y, x * y)
```

```
res = calc(3, 4)
```

```
print(res)
```

**Output**

(7, 12)

**Explanation:** lambda function performs both addition and multiplication and returns a tuple with both results.

**6. Using with filter()**

filter() function uses a lambda expression to select elements from a list that satisfy a given condition, such as keeping only even numbers.

```
C = [1, 2, 3, 4, 5, 6]
```

```
even = filter(lambda x: x \% 2 == 0, c)
```

```
print(list(even))
```

**Output**

```
[2, 4, 6]
```

**Explanation:**

- lambda function checks if a number is even ( $x \% 2 == 0$ ).
- filter() applies this condition to each element in the list c.

**6. Using with map()**

map() function applies a lambda expression to each element and returns a map object. It can be converted to a list using list().

```
A = [1, 2, 3, 4]
double = map(lambda x: x * 2, a)
print(list(double))
```

**Output**

```
[2, 4, 6, 8]
```

**Explanation:**

- lambda function doubles each number.
- map() iterates through a and applies the transformation.

**6. Using with reduce()**

reduce() function repeatedly applies a lambda expression to elements of a list to combine them into a single result.

```
From functools import reduce
a = [1, 2, 3, 4]
mul = reduce(lambda x, y: x * y, a)
print(mul)
```

**Output**

```
24
```

**Explanation:**

- lambda multiplies two numbers at a time.
- reduce() applies this operation across the list.

**Importing Modules**

In Python, modules help organize code into reusable files. They allow you to import and use functions, classes and variables from other scripts. The import statement is the most common way to bring external functionality into your Python program.

Python modules are of two types: **built-in** (come with Python) module and **external** modules like pandas and numpy, which need to be installed separately.

### Importing built-in Module

Built-in modules can be directly imported using "import" keyword without any installation. This allows access to all the functions and variables defined in the module.

```
import math
pie = math.pi
print("Value of pi:", pie)
```

### Output

Value of pi: 3.141592653589793

### Explanation:

- math module is imported using import math.
- We access the constant pi using math.pi and then the value is printed as part of a formatted string.

### Importing External Modules

To use external modules, we need to install them first, we can easily install any external module using pip command in the terminal, for example:

```
pip install module_name
```

Make sure to replace "module\_name" with the name of the module we want to install, example: pandas, numpy, etc.

After installation, we can import the module like a regular built-in module using "import statement".

```
import pandas
```

```
Create a simple DataFrame
data = {
 "Name": ["Elon", "Trevor", "Swastik"],
 "Age": [25, 30, 35]
}
```

```
df = pandas.DataFrame(data)
print(df)
```

### Output

```
 Name Age
0 Elon 25
1 Trevor 30
2 Swastik 35
```

### Explanation:

- **import pandas:** imports the external pandas library
- **pd.DataFrame(data):** creates a table-like data structure (DataFrame) from the given dictionary.

**Importing Specific Functions**

Instead of importing the entire module, we can import only the functions or variables we need using the `from` keyword. This makes the code cleaner and avoids unnecessary imports.

```
from math import pi
print(pi)
```

**Output**

```
3.141592653589793
```

**Explanation:**

- `from math import pi` imports only the `pi` constant, so we can use it directly without `math.` prefix.
- This reduces unnecessary module overhead when only specific functions or constants are needed.

**Importing Modules with Aliases**

To make code more readable and concise, we can assign an alias to a module using `as` keyword. This is especially useful when working with long module names.

```
import math as m
result = m.sqrt(25)
print("Square root of 25:", result)
```

**Output**

```
Square root of 25: 5.0
```

**Explanation:**

- `import math as m` imports the `math` module and assigns it the alias `m`.
- `m.sqrt(25)` calls the square root function using the alias.

**Importing Everything from a Module (\*)**

Instead of importing specific functions, we can import all functions and variables from a module using the `*` symbol. This allows direct access to all module contents without prefixing them with the module name.

```
from math import *
print(pi) # Accessing the constant 'pi'
print(factorial(6)) # Using the factorial function
```

**Output**

```
3.141592653589793
720
```

**Explanation:**

- `from math import *` imports all functions and constants from the `math` module.
- `pi` and `factorial(6)` are accessed directly without using `math.` as a prefix.
- While convenient, this method is not recommended in larger programs as it can lead to conflicts with existing variables and functions.

## Handling Import Errors in Python

When importing a module that doesn't exist or isn't installed, Python raises an `ImportError`. To prevent this, we can handle such cases using `try-except` blocks.

`try:`

```
import mathematics # Incorrect module name
print(mathematics.pi)
```

`except ImportError:`

```
print("Module not found! Please check the module name or install it if necessary.")
```

## Output

Module not found! Please check the module name or install it if necessary.

## Explanation:

- `try` block attempts to import a module, if the module is missing or misspelled, Python raises an `ImportError`.
- The `except` block catches the error and displays a user-friendly message instead of crashing the program.

## Why do we need to import modules?

- **Reuse Ready-Made Tools:** Python modules let you use pre-written code like `math.sqrt()` or `random.randint()` instead of writing everything yourself.
- **Keep Code Organized:** Modules break large programs into smaller, manageable pieces across multiple files.
- **Boost Productivity:** You get access to Python's standard library or third-party packages without reinventing the wheel.
- **Write Less, Do More:** Importing a module gives you lots of functionality in a single line.
- **Avoid Clutter:** Code stays cleaner, shorter and more readable when organized into reusable modules.

## Recursion

Recursion is a programming technique where a function calls itself either directly or indirectly to solve a problem by breaking it into smaller, simpler subproblems. Recursion is especially useful for problems that can be divided into identical smaller tasks, such as mathematical calculations, tree traversals or divide-and-conquer algorithms.

## Working of Recursion

A recursive function is just like any other function except that it calls itself in its body. Let's see basic structure of recursive function:

```
def recursive_function(parameters):
 if base_case_condition:
 return base_result
 else:
 return recursive_function(modified_parameters)
```



Recursive function contains two key parts:

- **Base Case:** The stopping condition that prevents infinite recursion.
- **Recursive Case:** The part of the function where it calls itself with modified parameters.

### Examples

**Example 1:** This code defines a recursive function to calculate factorial of a number, where function repeatedly calls itself with smaller values until it reaches the base case.

```
def factorial(n):
 if n == 0: # Base case
 return 1
 else: # Recursive case
 return n * factorial(n - 1)
```

```
print(factorial(5))
```

### Output

120

### Explanation:

- **Base Case:** When  $n == 0$ , recursion stops and returns 1.
- **Recursive Case:** Multiplies  $n$  with the factorial of  $n-1$  until it reaches the base case.

**Example 2:** This code defines a recursive function to calculate  $n^{\text{th}}$  Fibonacci number, where each number is the sum of the two preceding ones, starting from 0 and 1.

```
def fibonacci(n):
 if n == 0:
 return 0
 elif n == 1:
 return 1
 else:
 return fibonacci(n-1) + fibonacci(n-2)
```

```
print(fibonacci(10))
```



**Output**

55

**Explanation:**

- **Base Cases:** If  $n == 0$ , the function returns 0. If  $n == 1$ , the function returns 1. These two cases are necessary to stop the recursion.
- **Recursive Case:** function calls itself twice with decrements of  $n$  (i.e., `fibonacci(n-1)` and `fibonacci(n-2)`), summing results of these calls.

**Types of Recursion**

Recursion can be broadly classified into two types: tail recursion and non-tail recursion. The main difference between them is related to what happens after recursive call.

- **Tail Recursion:** The recursive call is the last thing the function does, so nothing happens after it returns. Some languages can optimize this to work like a loop, saving memory.
- **Non-Tail Recursion:** The function does more work after the recursive call returns, so it can't be optimized into a loop.

**Example:** This code compares tail recursion and non-tail recursion using two versions of factorial function one with an accumulator (tail-recursive) and one with multiplication after recursive call (non-tail-recursive).

```
def tail_fact(n, acc=1):
 if n == 0:
 return acc
 else:
 return tail_fact(n-1, acc * n)

def nontail_fact(n):
 if n == 0:
 return 1
 else:
 return n * nontail_fact(n-1)
```

```
print(tail_fact(5))
print(nontail_fact(5))
```

**Output**

120

120

**Explanation:**

- **def tail\_fact(n, acc=1):** - Defines a tail-recursive factorial function with an accumulator `acc` to store intermediate results.
- **if n == 0: return acc** - Base case: when  $n$  reaches 0, return the accumulated result.
- **return tail\_fact(n-1, acc \* n)** - Tail-recursive call: multiplies `acc` by `n` before the call, so no extra work is left after recursion.

- **def nontail\_fact(n):** - Defines a non-tail-recursive factorial function.
- **if n == 0: return 1** - Base case: when n == 0, return 1 (factorial of 0 is 1).
- **return n \* nontail\_fact(n-1)** - Non-tail call: multiplication happens after the recursive call returns, so more work remains after recursion.

### When to Avoid Recursion

- When the problem can be solved easily with loops.
- When recursion depth is large enough to risk a stack overflow.
- When performance is critical and function call overhead matters.

### Recursion vs Iteration

Recursion and iteration are two common techniques used to repeat tasks in programming below table highlights the key differences between them:

| Feature       | Recursion                                          | Iteration                                             |
|---------------|----------------------------------------------------|-------------------------------------------------------|
| Method        | A function calls itself to repeat the task         | Uses loops (for, while) to repeat steps               |
| Memory Use    | Uses more memory due to function calls             | Uses less memory                                      |
| Performance   | Usually slower because of function call overhead   | Generally faster as it avoids repeated function calls |
| Best Used For | Problems like tree traversal or divide-and-conquer | Repeating steps in a sequence                         |
| Risk          | May cause stack overflow if recursion is too deep  | No stack overflow risk                                |

# CHAPTER 3

## PYTHON DATA STRUCTURES

### String, Lists, Tuples, Dictionaries, Set, Arrays

Python provides us with several in-built data structures such as **lists**, **tuples**, **sets**, and **dictionaries** that store and organize the data efficiently. In this article, we will learn the difference between them and their applications in Python.

#### Difference between List, Tuple, Set, and Dictionary

The following table shows the difference between various Python built-in data structures.

| List                                                                                                             | Tuple                                                                                                         | Set                                                                                       | Dictionary                                                                         |
|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| A list is a non-homogeneous data structure that stores the elements in columns of a single row or multiple rows. | A Tuple is a non-homogeneous data structure that stores elements in columns of a single row or multiple rows. | The set data structure is non-homogeneous but stores the elements in a single row.        | A dictionary is also a non-homogeneous data structure that stores key-value pairs. |
| The list can be represented by [ ]                                                                               | A tuple can be represented by ( )                                                                             | The set can be represented by { }                                                         | The dictionary can be represented by { }                                           |
| The list allows duplicate elements                                                                               | Tuple allows duplicate elements                                                                               | The Set will not allow duplicate elements                                                 | The dictionary doesn't allow duplicate keys.                                       |
| The list can be nested among all                                                                                 | A tuple can be nested among all                                                                               | The set can be nested among all                                                           | The dictionary can be nested among all                                             |
| Example: [1, 2, 3, 4, 5]                                                                                         | Example: (1, 2, 3, 4, 5)                                                                                      | Example: {1, 2, 3, 4, 5}                                                                  | Example: {1: "a", 2: "b", 3: "c", 4: "d", 5: "e"}                                  |
| A list can be created using the <b>list()</b> function                                                           | Tuple can be created using the <b>tuple()</b> function.                                                       | A set can be created using the <b>set()</b> function                                      | A dictionary can be created using the <b>dict()</b> function.                      |
| A list is mutable i.e we can make any changes in the list.                                                       | A tuple is immutable i.e we can not make any changes in the tuple.                                            | A set is mutable i.e we can make any changes in the set, its elements are not duplicated. | A dictionary is mutable, its Keys are not duplicated.                              |
| List is ordered                                                                                                  | Tuple is ordered                                                                                              | Set is unordered                                                                          | Dictionary is ordered (Python 3.7 and above)                                       |

|                                                |                                                 |                                                                 |                                                      |
|------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------|
| Creating an empty<br>list<br><code>l=[]</code> | Creating an empty<br>Tuple<br><code>t=()</code> | Creating a set<br><code>a=set()</code><br><code>b=set(a)</code> | Creating an empty<br>dictionary<br><code>d={}</code> |
|------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------|

## Python List

Python Lists are just like dynamic-sized arrays, declared in other languages (vector in C++ and ArrayList in Java). Lists need not be homogeneous always which makes it the most powerful tool in Python. Here are five important applications of Python lists:

### 1. Data Storage and Manipulation

Lists are great for holding collections of data, whether all of the same type (like all integers) or mixed types (e.g., integers, strings). You can **add**, **remove**, and **update** elements easily.

#### Example:

```
a = [23, 45, 12, 67, 34]
a.append(89)
a.remove(45)
print(a)
```

#### Output

```
[23, 12, 67, 34, 89]
```

### 2. Implementing Stacks and Queues

Lists can simulate stack (LIFO) and queue (FIFO) behaviors. Stack operations use `append()` and `pop()`, while queues may require `pop(0)` or `collections.deque`.

#### Example:

```
s = []
s.append('a')
s.append('b')
print(s.pop())
```

#### Output

```
b
```

**Explanation:** This behaves like a stack. Items are pushed using `append()`, and the last item 'b' is popped from the stack.

### 3. Iteration and Data Processing

Lists are iterable and perfect for processing data, especially in loops where you perform calculations or transformations.

#### Example:

```
a = [1, 2, 3, 4, 5]
t = sum(a)
print(f"Total: {t}")
```

**Output**

Total: 15

**Explanation:** We use Python's built-in `sum()` to add all numbers in the list and print the total.

**4. Dynamic Arrays**

Lists can grow or shrink in size, which is useful when collecting data dynamically such as user inputs or computation results.

**Example:**

```
s = []
for i in range(10):
 s.append(i * i)
print(s)
```

**Output**

[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

**Explanation:** We create a list of squares from 0 to 9 dynamically using a loop and store the results in the list.

**5. Storing and Processing Strings**

Lists can be used to manipulate strings for tasks like tokenization, filtering, or formatting.

**Example:**

```
s = "Subtle art of not giving a bug"
w = s.split()
for word in w:
 print(word.upper())
```

**Output**

SUBTLE  
ART  
OF  
NOT  
GIVING  
A  
BUG

**Explanation:** We split a sentence into words using `split()` and convert each word to uppercase using a loop.

**Python Tuple**

A Tuple is a collection of Python objects separated by commas. In some ways, a tuple is similar to a list in terms of indexing, nested objects, and repetition but a tuple is **immutable**, unlike lists that are mutable.

**1. Immutable Data Storage**

Tuples are used when data should not be changed, such as coordinates or constants.

**Example:**

```
p= (10, 20)
print(p)
```

**Output**  
(10, 20)

## 2. Dictionary Keys

Tuples are hashable and can be used as dictionary keys, unlike lists.

**Example:**

```
p = {
 ("Paris", "France"): "Eiffel Tower",
 ("New York", "USA"): "Statue of Liberty"
}
print(p[("Paris", "France")])
```

**Output**  
Eiffel Tower

## 3. Storing Heterogeneous Data

Tuples are ideal for grouping data of different types like user records.

**Example:**

```
p = ("Prajwal", 22, "prajwal@gfg.org")
print(p)
```

**Output**  
(('Prajwal', 22, 'prajwal@gfg.org'))

## Python Set

A Python set is an unordered collection of unique elements. It is mutable, meaning you can add or remove elements (only regular sets), and it supports mathematical operations like union, intersection, and difference.

**Important Note:**

- Regular sets (set) are mutable and therefore not hashable.
- They cannot be used as dictionary keys or stored inside other sets.
- Python provides frozenset, an immutable version of a set, which is hashable and can be used as a key or element in another set.

### 1. Removing Duplicates from a Set

When you need to eliminate duplicate elements from a list, converting the list to a set is a quick and efficient way to do so.

**Example:**

```
a = [1, 2, 2, 3, 4, 4, 5]
u = set(a)
print(u)
```

**Output**

```
{1, 2, 3, 4, 5}
```

**2. Set Operations**

Sets are ideal for performing mathematical operations like **union**, **intersection**, and **difference**, which are useful in fields like data analysis, database management, and computational biology.

**Example:**

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print("Union:", a | b)
print("Intersection:", a & b)
print("Difference:", a - b)
```

**Output**

```
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference: {1, 2}
```

**Explanation:** We perform **union** (`|`), **intersection** (`&`), and **difference** (`-`) between two sets.

**3. Membership Testing**

Checking for the existence of an element in a collection is very efficient with sets due to their underlying hash table implementation. This makes sets a good choice for membership testing in scenarios like checking if an item is in a list of prohibited items.

**Example:**

```
ban = {"knife", "gun", "drugs"}
i = "knife"
if i in ban:
 print(f'{i} is prohibited.')
```

**Output**

```
knife is prohibited.
```

**Explanation:** We check if a given item is in the banned set. Membership testing in sets is fast.

**4. Finding Common Elements**

Sets are highly efficient for finding common elements between two or more collections. This is useful in scenarios like finding common interests, mutual friends, or common values between datasets.

**Example:**

```
a_f = {"Aryan", "Anurag", "Vishakshi"}
b_f = {"Prajwal", "Brijkant", "Aryan", "Kareena"}
comm = a_f & b_f
print(comm)
```

**Output**

```
{'Aryan'}
```

**Explanation:** We use intersection (&) to find common friends between two sets.

## 5. Handling Data in Multi-Set Scenarios

Sets are useful when dealing with multiple data sources where you need to ensure the uniqueness of the combined dataset or identify overlaps between different data sources.

**Example:**

```
d1 = {"apple", "banana", "cherry"}
d2 = {"banana", "cherry", "date"}
comb = d1 | d2
over = d1 & d2
print("Combined:", comb)
print("Overlap:", over)
```

### Output

```
Combined: {'apple', 'cherry', 'banana', 'date'}
Overlap: {'cherry', 'banana'}
```

**Explanation:** We merge two sets using **union (|)** and find shared items using **intersection (&)**.

## Python Dictionary

Dictionary in Python is an ordered (since Py 3.7) collection of data values, used to store data values like a map, which, unlike other Data Types that hold only a single value as an element. Dictionaries store data in **key: value** pairs. They are ideal for fast lookups and representing structured data. Here are five important applications of Python dictionaries:

### 1. Database Record Representation

Dictionaries are often used to represent database records where each **key-value** pair corresponds to a column and its associated value. This makes it easy to access and manipulate data based on field names.

**Example:**

```
rec = {
 'id': 1,
 'name': 'Prajjwal',
 'email': 'prajjwal@example.com',
 'age': 30
}
print(rec['name'])
```

### Output

```
Prajjwal
```

### 2. Counting Frequency of Elements

Dictionaries can be used to count the frequency of elements in a list, string, or any iterable. The element itself serves as the key, and its frequency is the value.

**Example:**



```
i = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple']
freq = {}
for item in i:
 freq[item] = freq.get(item, 0) + 1
print(freq)
```

**Output**

```
{'apple': 3, 'banana': 2, 'orange': 1}
```

**Explanation:** We use a dictionary to count how often each fruit appears in the list.

**3. Fast Lookup Tables**

Dictionaries provide **O(1)** average time complexity for lookups, making them ideal for creating fast lookup tables. This can be used in applications like caching, memoization, or mapping operations.

**Example:**

```
lookup = {
 'USD': 'United States Dollar',
 'EUR': 'Euro',
 'JPY': 'Japanese Yen'
}
print(lookup.get('USD'))
```

**Output**

```
United States Dollar
```

**4. Storing and Accessing JSON Data**

JSON data, which is commonly used for APIs and web services, maps directly to Python dictionaries. This allows for easy storage, access, and manipulation of JSON data.

**Example:**

```
import json

a = '{"name": "prajjwal", "age": 23, "city": "Prayagraj"}'
data = json.loads(a)

print(data['name'])
```

**Output**

```
prajjwal
```

**Explanation:** We parse a JSON string into a Python dictionary and access a value by key.

**5. Grouping Data by Keys**

Dictionaries can be used to group data by certain keys, which is particularly useful in data analysis and aggregation tasks.

**Example:**

```
emp = [
 {'name': 'prajj', 'dept': 'HR'},
 {'name': 'brij', 'dept': 'IT'},
 {'name': 'kareena', 'dept': 'HR'},
 {'name': 'aryan', 'dept': 'IT'}
]

grp = {}
for emp in emp:
 dept = emp['dept']
 grp.setdefault(dept, []).append(emp['name'])

print(grp)
```

**Output**

```
{'HR': ['prajj', 'kareena'], 'IT': ['brij', 'aryan']}
```

**String Functions:**

Strings are sequence of characters written inside quotes. It can include letters, numbers, symbols and spaces. Python does not have a separate character type.

- A single character is treated as a string of length one.
- Strings are commonly used for text handling and manipulation.

**Creating a String**

Strings can be created using either single ('...') or double ("...") quotes. Both behave the same.

**Example:** Creating two equivalent strings one with single and other with double quotes.

```
s1 = 'GfG'
s2 = "GfG"
print(s1)
print(s2)
```

**Output**

```
GfG
GfG
```

**Multi-line Strings**

Use triple quotes ("..." ) or ( """...""") for strings that span multiple lines. Newlines are preserved.

**Example:** Define and print multi-line strings using both styles.

```
s = """I am Learning
Python String on GeeksforGeeks"""
print(s)
```

```
s = "I'm a
```

```
Geek"
print(s)
```

**Output**

```
I am Learning
Python String on GeeksforGeeks
I'm a
Geek
```

**Accessing characters in String**

Strings are indexed sequences. Positive indices start at 0 from the left, negative indices start at -1 from the right as represented in below image:

Indices of string in reverse

**Example 1:** Access specific characters through positive indexing.

```
s = "GeeksforGeeks"
print(s[0])
print(s[4])
```

**Output**

```
G
s
```

**Note:** Accessing an index out of range will cause an `IndexError`. Only integers are allowed as indices and using a float or other types will result in a `TypeError`.

**Example 2:** Read characters from the end using negative indices.

```
s = "GeeksforGeeks"
print(s[-10])
print(s[-5])
```

**Output**

```
k
G
```

**String Slicing**

Slicing is a way to extract a portion of a string by specifying the start and end indexes. The syntax for slicing is `string[start:end]`, where start starting index and end is stopping index (excluded).

**Example:** this example demonstrates slicing through range and reversing a string.

```
s = "GeeksforGeeks"
print(s[1:4])
print(s[:3])
print(s[3:])
print(s[::-1])
```

**Output**

```
eek
Gee
ksforGeeks
skeeGrofskeeG
```

### String Iteration

Strings are iterable, one can loop through characters one by one.

**Example:** Here, it prints each character on its own line.

```
s = "Python"
for char in s:
 print(char)
```

### Output

```
P
y
t
h
o
n
```

**Explanation:** for loop pulls characters in order and each iteration prints the next character.

### String Immutability

Strings are immutable, which means that they cannot be changed after they are created. If we need to manipulate strings then we can use methods like concatenation, slicing or formatting to create new strings based on original.

**Example:** In this example we are changing first character by building a new string.

```
s = "geeksforGeeks"
s = "G" + s[1:]
print(s)
```

### Output

```
GeeksforGeeks
```

### Deleting a String

It's not possible to delete individual characters from a string since strings are immutable. However, we can delete an entire string variable using the del keyword.

**Example:** Here, we are using del keyword to delete a string.

```
s = "GfG"
del s
```

**Note:** After deleting the string if we try to access s then it will result in a NameError because variable no longer exists.

### Updating a String

As strings are immutable, “updates” create new strings using slicing or methods such as replace().

**Example:** This code fixes the first letter and replace a word.

```
s = "hello geeks"
s1 = "H" + s[1:]
s2 = s.replace("geeks", "GeeksforGeeks")
print(s1)
print(s2)
```

### Output

```
Hello geeks
hello GeeksforGeeks
```

### Explanation:

- **s1:** slice from index 1 onward and prepend "H".
- **s2:** replace("geeks", "GeeksforGeeks") returns a new string.

### Common String Methods

Python provides various built-in methods to manipulate strings. Below are some of the most useful methods:

**1. len():** returns the total number of characters in a string (including spaces and punctuation).

```
s = "GeeksforGeeks"
print(len(s))
```

### Output

```
13
```

**2. upper() and lower():** upper() method converts all characters to uppercase whereas, lower() method converts all characters to lowercase.

```
s = "Hello World"
print(s.upper())
print(s.lower())
```

### Output

```
HELLO WORLD
hello world
```

**3. strip() and replace():** strip() removes leading and trailing whitespace from the string and replace() replaces all occurrences of a specified substring with another.

```
s = " Gfg "
print(s.strip())
```

```
s = "Python is fun"
print(s.replace("fun", "awesome"))
```

### Output

```
Gfg
Python is awesome
```

To learn more about string methods, please refer to Python String Methods.

### Concatenating and Repeating Strings

We can concatenate strings using + operator and repeat them using \* operator.

1. Strings can be combined by using + operator.

**Example:** Join two words with a space.

```
s1 = "Hello"
s2 = "World"
print(s1 + " " + s2)
```

#### Output

Hello World

2. We can repeat a string multiple times using \* operator.

**Example:** Repeat a greeting three times.

```
s = "Hello "
print(s * 3)
```

#### Output

Hello Hello Hello

### Formatting Strings

1. **Using f-strings:** most preferred way to format strings is by using f-strings.

**Example:** Embed variables directly using {} placeholders.

```
name = "Jake"
age = 22
print(f'Name: {name}, Age: {age}')
```

#### Output

Name: Jake, Age: 22

2. **Using format():** Another way to format strings is by using format() method.

**Example:** Use placeholders {} and pass values positionally.

```
s = "My name is {} and I am {} years old.".format("Emily", 22)
print(s)
```

#### Output

My name is Emily and I am 22 years old.

### String Membership Testing

in keyword checks if a particular substring is present in a string.

**Example:** Here, we are testing for the presence of substrings.

```
s = "GeeksforGeeks"
print("Geeks" in s)
print("GfG" in s)
```

**Output**

True

False

**String slices**

**String slicing** in Python is a way to get specific parts of a string by using start, end and step values. It's especially useful for text manipulation and data parsing.

Let's take a quick example of string slicing:

```
s = "Hello, Python!"
```

```
print(s[0:5])
```

**Output**

Hello

**Explanation:** In this example, we used the slice `s[0:5]` to obtain the substring "Hello" from the original string.

**Syntax of String Slicing in Python**

```
substring = s[start : end : step]
```

**Parameters:**

- **s:** The original string.
- **start (optional):** Starting index (inclusive). Defaults to **0** if omitted.
- **end (optional):** Stopping index (exclusive). Defaults to the end of the string if omitted.
- **step (optional):** Interval between indices. A positive value slices from left to right, while a negative value slices from right to left. If omitted, it defaults to 1 (no skipping of characters).

**Return Type:** The result of a slicing operation is always a string (**str** type) that contains a subset of the characters from the original string.

**Using Negative Indexing in Slicing**

**Negative indexing** is useful for accessing elements from the end of the String. The last element has an index of -1, the second last element -2 and so on.

**Extract Characters Using Negative Indices**

Below example shows how to use negative numbers to access elements from the string starting from the end. Negative indexing makes it easy to get items without needing to know the exact length of the string.

```
s = "abcdefghijklmno"
```

```
print(s[-4:])
```

```
print(s[:-3])
```

```
print(s[-5:-2])
```

```
print(s[-8:-1:2])
```

### Output

```
lmno
abcdefghijkl
klm
hjl
```

### Explanation:

- **s[-4:]** slices the string starting from the 4th character from the end ('m') to the end of the string.
- **s[:-3]** slices the string from the beginning up to the 3rd character from the end ('k'), excluding it.
- **s[-5:-2]** slices the string from the 5th character from the end ('l') to the 2nd character from the end ('n'), excluding the last character.
- **s[-8:-1:2]** slices the string from the 8th character from the end ('g') to the 2nd character from the end ('n'), with a step of 2, taking every second character.

### Reverse a String Using Slicing

To reverse a string, use a **negative step value of -1**, which moves from the end of the string to the beginning.

```
s = "Python"
```

```
Reverse the string
print(s[::-1])
```

### Output

```
nohtyP
```

**Explanation:** The slice **s[::-1]** starts from the end and steps backward through the string, which effectively reversing it. This method does not alter the original string.

### String Slicing Examples

Let's see how to use string slicing in Python with the examples below:

#### Example 1: Retrieve All Characters

To retrieve the entire string, use slicing without specifying any parameters.

```
s = "Hello, World!"
```

```
Get the entire string
s2 = s[:]
s3 = s[::]
```

```
print(s2)
print(s3)
```

### Output

```
Hello, World!
Hello, World!
```



**Explanation:** Using `[:]` or `::]` without specifying **start**, **end** or **step** returns the complete string.

### Example 2: Get All Characters Before or After a Specific Position

To get all the items from a specific position to the end of the string, we can specify the **start** index and leave the **end** blank. And to get all the items before a specific index, we can specify the **end** index while leaving start blank.

```
s = "Hello, World!"
```

```
Characters from index 7 to the end
```

```
print(s[7:])
```

```
Characters from the start up to index 5 (exclusive)
```

```
print(s[:5])
```

### Output

```
World!
```

```
Hello
```

**Explanation:** The slice `s[7:]` starts from index 7 and continues to the end of the string.

### Example 3: Extract Characters Between Two Positions

To extract characters between specific positions, provide both **start** and **end** indices.

```
s = "Hello, World!"
```

```
Characters from index 1 to index 5 (excluding 5)
```

```
print(s[1:5])
```

### Output

```
ello
```

**Explanation:** The code slices the string `s` to extract characters starting from index 1 up to, but not including, index 5, resulting in the substring "ello".

### Example 4: Get Characters at Specific Intervals

To retrieve characters at regular intervals, use the **step** parameter.

```
s = "abcdefghi"
```

```
Every second character
```

```
print(s[::2])
```

```
Every third character from index 1 to 8 (exclusive)
```

```
print(s[1:8:3])
```

### Output

```
acegi
```

```
beh
```

## Immutability

Strings in Python are "**immutable**" i.e. they cannot be changed after they are created. Strings are immutable by design to keep them safe, consistent, and efficient. Immutability makes strings hashable (usable as dictionary keys), memory-efficient, and thread-safe, ensuring they can be reused without unexpected changes.

### Example:

```
var="Aarun"
```

```
var[0]='T' -> Throws: TypeError: 'str' object does not support item assignment.
```

**Explanation:** We cannot update the string after declaring it means once an immutable the objects instantiated, its value cannot be changed.

### Benefits of Immutable Objects

1. **Hashability:** Immutable objects can be dictionary keys since their hash never changes.
2. **Memory Efficiency:** Reused by Python, saving memory.
3. **Thread Safety:** Safe in multithreading as they can't be modified.
4. **Predictability:** Values stay fixed, making code easier to debug.
5. **Performance:** Support faster lookups with cached hash values.

### Ways to Deal with Immutability

#### String Slicing and Reassembling

You can use slicing to extract parts of the string and then reassemble them as needed.

```
n1 = "Aarun"
n2 = "T" + n1[1:]
print(n2)
```

### Output

Tarun

### Explanation:

- **n1[1:]** gives the slice of n1 from index 1 to the end of the string.
- **"T" + n1[1:]** concatenates 'T' in front of the sliced part and creates a new string.

#### String Concatenation

Instead of modifying a string in place, you can concatenate strings to create a new one

```
v1 = "Hello"
Creates a new string with the concatenated result
v2 = v1 + ", world!"
print(v2)
```

### Output

Hello, world!

**Explanation:** String concatenation creates a new string since the original string is immutable and cannot be modified in place.

### 3. Using the join() method

We can use the .join() method if we have list of strings to concatenate into a single string with desired separator.

```
l1 = ["Hello", "world!"]
```

```
Joins the list elements with a space separator
```

```
l2 = " ".join(l1)
```

```
print(l2)
```

#### Output

Hello world!

**Explanation:** " ".join() method combines list elements into a single string.

### 4. Using String Formatting

With the help of string formatting, we can insert the value and variable into the string

```
n1 = "Geeks"
```

```
Insert the value of 'name' into the string using format()
```

```
n2 = "Hello {} For Geeks".format(n1)
```

```
print(n2)
```

#### Output

Hello Geeks For Geeks

**Explanation:** format() method is used to insert variable values into strings, creating a new string without modifying the original.

### Difference between Mutable and Immutable Objects

Below is the difference between Mutability and Immutability:

| Features                   | Mutable Objects                            | Immutable Objects                             |
|----------------------------|--------------------------------------------|-----------------------------------------------|
| <b>Definition</b>          | Objects that can be changed after creation | Objects that cannot be changed after creation |
| <b>Examples</b>            | list, dict, set, byte array                | int, float, str, tuple, bool, frozen set      |
| <b>Memory Address (id)</b> | Can remain the same after modification     | Changes when a new value is assigned          |
| <b>Use Case</b>            | Preferred when frequent updates are needed | Preferred when data should remain constant    |
| <b>Thread Safety</b>       | Less thread-safe                           | More thread-safe                              |
| <b>Performance</b>         | May be slower due to extra safety measures | Often faster for read-only operations         |
| <b>Example Code</b>        | lst = [1, 2]; lst[0] = 9 -> [9, 2]         | s = "hi"; s[0] = "H" -> TypeError             |

## String functions and methods

**Python string methods** is a collection of in-built Python functions that operates on strings.

**Note:** Every string method in Python does not change the original string instead returns a new string with the changed attributes.

**Python string** is a sequence of Unicode characters that is enclosed in quotation marks. In this article, we will discuss the in-built string functions i.e. the functions provided by Python to operate on strings.

### Case Changing of Python String Methods

The below Python functions are used to change the case of the strings. Let's look at some Python string methods with examples:

- **lower():** Converts all uppercase characters in a string into lowercase
- **upper():** Converts all lowercase characters in a string into uppercase
- **title():** Convert string to title case
- **swapcase():** Swap the cases of all characters in a string
- **capitalize():** Convert the first character of a string to uppercase

**Example:** Changing the case of Python String Methods

# Python3 program to show the

# working of upper() function

```
text = 'geEKs For geEkS'
```

```
upper() function to convert
```

```
string to upper case
```

```
print("\nConverted String:")
```

```
print(text.upper())
```

```
lower() function to convert
```

```
string to lower case
```

```
print("\nConverted String:")
```

```
print(text.lower())
```

```
converts the first character to
```

```
upper case and rest to lower case
```

```
print("\nConverted String:")
```

```
print(text.title())
```

```
swaps the case of all characters in the string
```

```
upper case character to lowercase and viceversa
```

```
print("\nConverted String:")
```

```
print(text.swapcase())
```

```
convert the first character of a string to uppercase
```

```
print("\nConverted String:")
```

```
print(text.capitalize())

original string never changes
print("\nOriginal String")
print(text)
```

### Output

```
Converted String:
GEEKS FOR GEEKS
Converted String:
geeks for geeks
Converted String:
Geeks For Geeks
Converted String:
GEEkS fOR GEeKs
```

```
Original String
geeKs For geEkS
```

**Time complexity:**  $O(n)$  where  $n$  is the length of the string 'text'

**Auxiliary space:**  $O(1)$

### List of String Methods in Python

Here is the list of in-built Python string methods, that you can use to perform actions on string:

| Function Name  | Description                                                                        |
|----------------|------------------------------------------------------------------------------------|
| capitalize()   | Converts the first character of the string to a capital (uppercase) letter         |
| casefold()     | Implements caseless string matching                                                |
| center()       | Pad the string with the specified character.                                       |
| count()        | Returns the number of occurrences of a substring in the string.                    |
| encode()       | Encodes strings with the specified encoded scheme                                  |
| endswith()     | Returns "True" if a string ends with the given suffix                              |
| expandtabs()   | Specifies the amount of space to be substituted with the “\t” symbol in the string |
| find()         | Returns the lowest index of the substring if it is found                           |
| format()       | Formats the string for printing it to console                                      |
| format_map()   | Formats specified values in a string using a dictionary                            |
| index()        | Returns the position of the first occurrence of a substring in a string            |
| isalnum()      | Checks whether all the characters in a given string is alphanumeric or not         |
| isalpha()      | Returns “True” if all characters in the string are alphabets                       |
| isdecimal()    | Returns true if all characters in a string are decimal                             |
| isdigit()      | Returns “True” if all characters in the string are digits                          |
| isidentifier() | Check whether a string is a valid identifier or not                                |
| islower()      | Checks if all characters in the string are lowercase                               |

|               |                                                                                        |
|---------------|----------------------------------------------------------------------------------------|
| isnumeric()   | Returns “True” if all characters in the string are numeric characters                  |
| isprintable() | Returns “True” if all characters in the string are printable or the string is empty    |
| isspace()     | Returns “True” if all characters in the string are whitespace characters               |
| istitle()     | Returns "True" if the string is a title cased string                                   |
| isupper()     | Checks if all characters in the string are uppercase                                   |
| join()        | Returns a concatenated String                                                          |
| ljust()       | Left aligns the string according to the width specified                                |
| lower()       | Converts all uppercase characters in a string into lowercase                           |
| lstrip()      | Returns the string with leading characters removed                                     |
| maketrans()   | Returns a translation table                                                            |
| partition()   | Splits the string at the first occurrence of the separator                             |
| replace()     | Replaces all occurrences of a substring with another substring                         |
| rfind()       | Returns the highest index of the substring                                             |
| rindex()      | Returns the highest index of the substring inside the string                           |
| rjust()       | Right aligns the string according to the width specified                               |
| rpartition()  | Split the given string into three parts                                                |
| rsplit()      | Split the string from the right by the specified separator                             |
| rstrip()      | Removes trailing characters                                                            |
| splitlines()  | Split the lines at line boundaries                                                     |
| startswith()  | Returns "True" if a string starts with the given prefix                                |
| strip()       | Returns the string with both leading and trailing characters                           |
| swapcase()    | Converts all uppercase characters to lowercase and vice versa                          |
| title()       | Convert string to title case                                                           |
| translate()   | Modify string according to given translation mappings                                  |
| upper()       | Converts all lowercase characters in a string into uppercase                           |
| zfill()       | Returns a copy of the string with ‘0’ characters padded to the left side of the string |

## String module

The **string module** is a part of Python's standard library and provides several helpful utilities for working with strings. From predefined sets of characters (such as ASCII letters, digits and punctuation) to useful functions for string formatting and manipulation, the string module streamlines various string operations that are commonly encountered in programming.

**Note:** The Python String module is a part of the standard Python library, so we do not need to explicitly install it.

### Key features:

- **Validation:** Check if characters in a string belong to specific sets.
- **Formatting:** Format strings for better output or presentation.
- **Template Handling:** Create templates with placeholders that can be dynamically filled.

To begin using the string module, we can simply import it into our Python program:

```
import string
```

## Functions in the Python String Module

In addition to constants, the string module provides several useful functions that simplify string manipulation.

### 1. capwords()

`capwords()` capitalizes the first letter of each word in a string and makes the rest lowercase, while collapsing multiple spaces between words into a single space.

#### Example:

```
import string
```

```
s = "hello, geek! this is your geeksforgeeks!"
print(string.capwords(s))
```

#### Output

Hello, Geek! This Is Your Geeksforgeeks!

**Explanation:** `string.capwords()` function splits the input string `s` into words by spaces, capitalizes the first letter of each word and then joins the words back together into a single string. It doesn't affect punctuation marks, which remain in their original position.

### 2. Formatter()

**Formatter class** allows custom string formatting. It works behind the scenes in Python's `str.format()` method but can be extended for more complex formatting needs.

```
from string import Formatter
fmt = Formatter()
print(fmt.format("Hello, {}!", "geek"))
```

#### Output

Hello, geek!

**Explanation:** `fmt.format()` insert values into a string using placeholders marked by curly braces `{}`. In this case, the string `"Hello, {}!"` contains a placeholder `{}`, which is replaced by the argument `"geek"`.

### 3. Template()

**Template class** is useful for creating string templates with placeholders that can be substituted with dynamic data. It has a `substitute()` method and a safer `safe_substitute()` method that doesn't raise an error if a placeholder is missing.

```
from string import Template
s = Template("Hello, $name!")

print(s.substitute(name="geek"))
```

```
print(s.safe_substitute(name="Python"))
print(s.safe_substitute())
```

### Output

```
Hello, geek!
Hello, Python!
Hello, $name!
```

**Explanation:** `substitute()` replaces placeholders with provided values, while `safe_substitute()` does the same but doesn't raise an error if a placeholder is missing, using a default value instead. If no value is given, `safe_substitute()` returns the original string.

### Constants in the Python String Module

The string module provides several constants that represent commonly used sets of characters. These constants can be helpful for validating, cleaning, or manipulating strings efficiently.

| Constant               | Description                                                                       | Example Output                                                                               |
|------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>ascii_letters</b>   | All ASCII letters, both lowercase and uppercase.                                  | abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ                                         |
| <b>ascii_lowercase</b> | All lowercase ASCII letters.                                                      | abcdefghijklmnopqrstuvwxyz                                                                   |
| <b>ascii_uppercase</b> | All uppercase ASCII letters.                                                      | ABCDEFGHIJKLMNOPQRSTUVWXYZ                                                                   |
| <b>digits</b>          | All decimal digits (0-9).                                                         | 0123456789                                                                                   |
| <b>hexdigits</b>       | All characters used in hexadecimal numbers (0-9 and A-F).                         | 0123456789abcdefABCDEF                                                                       |
| <b>octdigits</b>       | All characters used in octal numbers (0-7).                                       | 01234567                                                                                     |
| <b>punctuation</b>     | All punctuation marks.                                                            | !"#\$%&'()*+,-./:;<=>?@[\]^_{'                                                               |
| <b>printable</b>       | All printable characters, including letters, digits, punctuation, and whitespace. | 0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!"#\$%&'()*+,-./:;<=>?@[\]^_{' |
| <b>whitespace</b>      | All characters considered whitespace, such as spaces, tabs, and newlines.         | "\t\n\r\x0b\x0c"                                                                             |

### Real-World Use Cases of the Python String Module

Let's explore a few practical scenarios where the Python string module can simplify our tasks.

#### 1. Validate User Input Data

The string module helps with input validation, such as checking if a string consists only of digits or alphabetic characters.

```
import string
```



```
def fun(s):
 return all(char in string.digits for char in s)

print(fun("12345"))
print(fun("123a5"))
```

**Output**

True

False

**Explanation:** **fun(s)** function checks if all characters in the string **s** are digits using `string.digits` and the **all()** function. It returns True if all characters are digits, otherwise False.

**2. Custom formatting**

Using `Formatter()`, we can create custom string formats to convert numbers into specific formats or create custom messages.

```
from string import Formatter
```

```
def fun(number):
 return f"The number is: {number:.2f}"

print(fun(123.456))
```

**Output**

The number is: 123.46

**Explanation:** **fun(number)** that formats a floating-point number to display two decimal places. It uses an f-string with the format specifier `:.2f`, which ensures that the number is rounded to two decimal places.

**3. Parsing data files**

When parsing data files (like logs), `Template` can simplify the process by allowing dynamic substitution of values into predefined strings.

```
from string import Template
```

```
log_temp = Template("User: $user, Action: $action")
res = log_temp.safe_substitute(user="Geek", action="Login")
print(res)
```

**Output**

User: Geek, Action: Login

**Lists:**

List is a built-in data structure used to store an ordered collection of items. They are dynamic, resizable and capable of storing multiple data types.

- **Mutable:** list elements can be changed, updated, added, or removed after the list is created.
- **Ordered:** elements maintain the order in which they are inserted.
- **Index-based:** elements are accessed using their position, starting from index 0.

### Creating a List

Lists can be created in several ways, such as using square brackets `[]`, the `list()` constructor or by repeating elements.

**1. Using Square Brackets:** Square brackets `[]` are used to create a list directly.

```
a = [1, 2, 3]
print(a)
```

```
b = ["apple", "banana"]
print(b)
```

### Output

```
[1, 2, 3]
['apple', 'banana']
```

**2. Using list() Constructor:** A list can also be created by passing an iterable (such as tuple, string or another list) to the `list()` constructor.

```
a = list((1, 2, 3, 'apple', 4.5))
print(a)
```

```
b = list("GFG")
print(b)
```

### Output

```
[1, 2, 3, 'apple', 4.5]
['G', 'F', 'G']
```

**3. Creating List with Repeated Elements:** A list with repeated elements can be created using the multiplication `(*)` operator.

```
a = [2] * 5
b = [0] * 7
```

```
print(a)
print(b)
```

### Output

```
[2, 2, 2, 2, 2]
[0, 0, 0, 0, 0, 0, 0]
```

### Internal Representation of Lists

Python list stores references to objects, not the actual values directly.

- The list keeps memory addresses of objects like integers, strings or booleans.

- Actual objects exist separately in memory.
- Modifying a mutable object inside a list changes the original object.
- Reassigning an immutable object creates a new object instead of changing the old one.

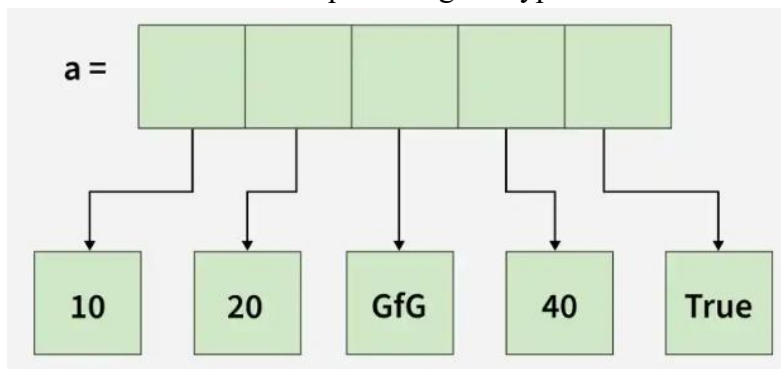
```
a = [1, 2, 2, "Python"]
print(a[0]) # index-based
print(a)
```

### Output

```
1
[1, 10, 2, 'Python']
```

### Explanation:

- 
- The list `a` contains an integer (10, 20 and 40), a string ("GfG") and a boolean (True).
- Elements are accessed using indexing (`a[0]`, `a[1]`, etc.).
- Each element keeps its original type.



### Accessing List Elements

Elements in a list are accessed using indexing. Python uses zero-based indexing, meaning `a[0]` represents the first element. Negative indexing is also supported, where `-1` accesses the last element.

```
a = [10, 20, 30]
print(a[0])
print(a[-1])
```

### Output

```
10
30
```

### Adding Elements into List

Elements can be added to a list using the following methods:

1. **append():** Adds an element at the end of the list.

```
a = [1, 2]
a.append(3)
print(a)
```

**Output**

```
[1, 2, 3]
```

2. **insert()**: Adds an element at a specific position.

```
a = [1, 3]
a.insert(1, 2)
print(a)
```

**Output**

```
[1, 2, 3]
```

3. **extend()**: Adds multiple elements to the end of the list.

```
a = [1, 2]
a.extend([3, 4])
print(a)
```

**Output**

```
[1, 2, 3, 4]
```

**Updating Elements into List**

Since lists are mutable, elements can be updated by assigning new values using their index.

```
a = [10, 20, 30, 40, 50]
a[1] = 25
print(a)
```

**Output**

```
[10, 25, 30, 40, 50]
```

**Removing Elements from List**

Elements can be removed from a list using the following methods:

1. **remove()**: Removes the first occurrence of an element.

```
a = [1, 2, 3]
a.remove(2)
print(a)
```

**Output**

```
[1, 3]
```

2. **pop()**: Removes the element at a specific index or the last element if no index is specified.

```
a = [1, 2, 3]
a.pop()
print(a)
```

**Output**

```
[1, 2]
```

3. **del statement**: Deletes an element at a specified index.

```
a = [1, 2, 3]
del a[1]
print(a)
4. clear(): removes all items.
a = [1, 2, 3]
a.clear()
print(a)
```

**Output**

```
[]
```

**Iterating Over Lists**

Lists can be iterated using loops, allowing operations to be performed on each element.

```
A = ['apple', 'banana', 'cherry']
for item in a:
 print(item)
```

**Output**

```
apple
banana
cherry
```

**Nested Lists**

A nested list is a list that contains another list as its element. It is commonly used to represent matrices or tabular data. Nested elements can be accessed by chaining multiple indexes.

```
A = [[1, 2], [3, 4]]
print(a[0])
print(a[1][0])
```

**Output**

```
[1, 2]
3
```

**list operations,**

Lists are sequence objects in Python that support the same fundamental operations as strings. They respond to the + and \* operators for concatenation and repetition, except the result is a new list rather than a string.

Lists support all general sequence operations, making them versatile for data manipulation and processing tasks.

Length Operation

Use len() to get the number of elements in a list ?

```
numbers = [1, 2, 3]
print("Length of list:", len(numbers))
```

Length of list: 3

Concatenation with + Operator

Combine two lists using the + operator ?

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
combined = list1 + list2
print("Concatenated list:", combined)
```

Concatenated list: [1, 2, 3, 4, 5, 6]

Repetition with \* Operator

Repeat list elements using the \* operator ?

```
greeting = ['Hi!']
repeated = greeting * 4
print("Repeated list:", repeated)
```

Repeated list: ['Hi!', 'Hi!', 'Hi!', 'Hi!']

Membership Testing with in Operator

Check if an element exists in a list using the in operator ?

```
numbers = [1, 2, 3]
result = 3 in numbers
print("Is 3 in the list?", result)
print("Is 5 in the list?", 5 in numbers)
```

Is 3 in the list? True

Is 5 in the list? False

Iteration with for Loop

Loop through list elements using a for loop ?

```
numbers = [1, 2, 3]
print("Iterating through list:")
for x in numbers:
 print(x, end=" ")
```

Iterating through list:

1 2 3

## Summary of Basic Operations

| Python Expression                         | Results                                   | Description   |
|-------------------------------------------|-------------------------------------------|---------------|
| <code>len([1, 2, 3])</code>               | 3                                         | Length        |
| <code>[1, 2, 3] + [4, 5, 6]</code>        | <code>[1, 2, 3, 4, 5, 6]</code>           | Concatenation |
| <code>['Hi!'] * 4</code>                  | <code>['Hi!', 'Hi!', 'Hi!', 'Hi!']</code> | Repetition    |
| <code>3 in [1, 2, 3]</code>               | True                                      | Membership    |
| <code>for x in [1, 2, 3]: print(x)</code> | 1 2 3                                     | Iteration     |

## List slices

Python **list slicing** is fundamental concept that let us easily access specific elements in a list. In this article, we'll learn the syntax and how to use both positive and negative indexing for slicing with examples.

**Example:** Get the items from a list starting at position 1 and ending at position 4 (exclusive).

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Get elements from index 1 to 4 (excluded)
```

```
print(a[1:4])
```

**Output**

```
[2, 3, 4]
```

**Python List Slicing Syntax**

```
list_name[start : end : step]
```

**Parameters:**

- **start (optional):** Index to begin the slice (inclusive). Defaults to 0 if omitted.
- **end (optional):** Index to end the slice (exclusive). Defaults to the length of list if omitted.
- **step (optional):** Step size, specifying the interval between elements. Defaults to 1 if omitted

**List Slicing Examples**

Let's see how to use list slicing in Python with the examples below.

**Get all the items from a list**

To retrieve all items from a list, we can use slicing without specifying any parameters.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Get all elements in the list
```

```
print(a[:])
```

```
print(a[:])
```

**Output**

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

**Explanation:** Using `[:]` & `::` without specifying any **start**, **end**, or **step** returns all elements of the list.

### Get all items before/after a specific position

To get all the items from a specific position to the **end** of the list, we can specify the **start** index and leave the end blank. And to get all the items before a specific index, we can specify the **end** index while leaving **start** blank.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Get elements starting from index 2
to the end of the list
b = a[2:]
print(b)
```

```
Get elements starting from index 0
to index 3 (excluding 3th index)
c = a[:3]
print(c)
```

### Output

```
[3, 4, 5, 6, 7, 8, 9]
[1, 2, 3]
```

### Get all items between two positions

To extract elements between two specific positions, specify both the start and end indices

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Get elements from index 1
to index 4 (excluding index 4)
b = a[1:4]
print(b)
```

### Output

```
[2, 3, 4]
```

### Get items at specified intervals

To extract elements at specific intervals, use the **step** parameter.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Get every second element from the list
starting from the beginning
b = a[::2]
print(b)
```

```
Get every third element from the list
```



```
starting from index 1 to 8(exclusive)
c = a[1:8:3]
print(c)
```

### Output

```
[1, 3, 5, 7, 9]
[2, 5, 8]
```

### Out-of-bound slicing

In Python, **list slicing allows out-of-bound indexing without raising errors**. If we specify indices beyond the list length then it will simply return the available items.

**Example:** The slice **a[7:15]** starts at index 7 and attempts to reach index 15, but since the list ends at index 8, so it will return only the available elements (i.e. [8,9]).

### Negative Indexing

**Negative indexing** is useful for **accessing elements from the end of the list**. The last element has an index of **-1**, the second last element **-2**, and so on.

### Extract elements using negative indices

This example shows how to use negative numbers to access elements from the list starting from the end. **Negative indexing** makes it easy to get items without needing to know the exact length of the list.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Get elements starting from index -2
to end of list
b = a[-2:]
print(b)
```

```
Get elements starting from index 0
to index -3 (excluding 3th last index)
c = a[: -3]
print(c)
```

```
Get elements from index -4
to -1 (excluding index -1)
d = a[-4:-1]
print(d)
```

```
Get every 2nd elements from index -8
to -1 (excluding index -1)
e = a[-8:-1:2]
print(e)
```

### Output

```
[8, 9]
[1, 2, 3, 4, 5, 6]
[6, 7, 8]
[2, 4, 6, 8]
```

### Reverse a list using slicing

In this example, we'll reverse the entire list using a **slicing** trick. By using a negative step value, we can move through the list in reverse order.

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
Get the entire list using negative step
b = a[::-1]
print(b)
```

### Output

```
[9, 8, 7, 6, 5, 4, 3, 2, 1]
```

**Explanation:** The negative **step (-1)** indicates that Python should traverse the list in reverse order, starting from the end. The slice **a[::-1]** starts from the end of the list and moves to the beginning which result in reversing list. It's a quick and easy way to get the list in reverse without changing the original list.

## List methods

Python list methods are built-in functions that allow us to perform various operations on lists, such as adding, removing, or modifying elements. In this article, we'll explore all Python list methods with a simple example.

### append()

Adds an element to the end of the list.

**Syntax:** list\_name.append(element)

In the code below, we will add an element to the list.

```
a = [1, 2, 3]

Add 4 to the end of the list
a.append(4)
print(a)
```

### Output

```
[1, 2, 3, 4]
```

### copy()

Returns a shallow copy of the list.

**Syntax:** list\_name.copy()

In the code below, we will create a copy of a list.

```
a = [1, 2, 3]
```

```
Create a copy of the list
b = a.copy()
print(b)
```

### Output

```
[1, 2, 3]
```

### clear()

Removes all elements from the list.

**Syntax:** list\_name.clear()

In the code below, we will clear all elements from the list.

```
a = [1, 2, 3]
```

```
Remove all elements from the list
a.clear()
print(a)
```

### Output

```
[]
```

### count()

Returns the number of times a specified element appears in the list.

**Syntax:** list\_name.count(element)

In the code below, we will count the occurrences of a specific element in the list.

```
a = [1, 2, 3, 2]
```

```
Count occurrences of 2 in the list
print(a.count(2))
```

### Output

```
2
```

### extend()

Adds elements from another list to the end of the current list.

**Syntax:** list\_name.extend(iterable)

In the code below, we will extend the list by adding elements from another list.

```
A = [1, 2]
```

```
Extend list a by adding elements from list [3, 4]
a.extend([3, 4])
print(a)
```

### Output

```
[1, 2, 3, 4]
```

### **index()**

Returns the index of the first occurrence of a specified element.

**Syntax:** list\_name.index(element)

In the code below, we will find the index of a specific element in the list.

```
A = [1, 2, 3]
```

```
Find the index of 2 in the list
```

```
print(a.index(2))
```

### **Output**

```
1
```

### **insert()**

Inserts an element at a specified position.

**Syntax:** list\_name.insert(index, element)

In the code below, we will insert an element at a specific position in the list.

```
A = [1, 3]
```

```
Insert 2 at index 1
```

```
a.insert(1, 2)
```

```
print(a)
```

### **Output**

```
[1, 2, 3]
```

### **pop()**

Removes and returns the element at the specified position (or the last element if no index is specified).

**Syntax:** list\_name.pop(index)

In the code below, we will remove the last element from the list.

```
A = [1, 2, 3]
```

```
Remove and return the last element in the list
```

```
a.pop()
```

```
print(a)
```

### **Output**

```
[1, 2]
```

### **remove()**

Removes the first occurrence of a specified element.

**Syntax:** list\_name.remove(element)

In the code below, we will remove the first occurrence of a specified element from the list.

```
A = [1, 2, 3]
```

```
Remove the first occurrence of 2
a.remove(2)
print(a)
```

### Output

```
[1, 3]
```

### reverse()

Reverses the order of the elements in the list.

**Syntax:** list\_name.reverse()

In the code below, we will reverse the order of the elements in the list.

```
A = [1, 2, 3]
```

```
Reverse the list order
a.reverse()
print(a)
```

### Output

```
[3, 2, 1]
```

### sort()

Sorts the list in ascending order (by default).

**Syntax:** list\_name.sort(key=None, reverse=False)

In the code below, we will sort the elements of the list in ascending order

```
a = [3, 1, 2]
```

```
Sort the list in ascending order
a.sort()
print(a)
```

### Output

```
[1, 2, 3]
```

## List loop

### Loop Through a List

You can loop through the list items by using a for loop:

Example

Print all items in the list, one by one:

```
thislist = ["apple", "banana", "cherry"]
for x in thislist:
 print(x)
```

### Loop Through the Index Numbers

You can also loop through the list items by referring to their index number.

Use the range() and len() functions to create a suitable iterable.

### Example

Print all items by referring to their index number:

```
thislist = ["apple", "banana", "cherry"]
for i in range(len(thislist)):
 print(thislist[i])
```

### Using a While Loop

You can loop through the list items by using a while loop.

Use the len() function to determine the length of the list, then start at 0 and loop your way through the list items by referring to their indexes.

Remember to increase the index by 1 after each iteration.

### Example

Print all items, using a while loop to go through all the index numbers

```
thislist = ["apple", "banana", "cherry"]
i = 0
while i < len(thislist):
 print(thislist[i])
 i = i + 1
```

### Looping Using List Comprehension

List Comprehension offers the shortest syntax for looping through lists:

#### Example

A short hand for loop that will print all items in a list:

```
thislist = ["apple", "banana", "cherry"]
[print(x) for x in thislist]
```

## Mutability

In Python, we can use mutable lists which are lists that we can change after creating them. We can add, remove, or modify items in the list without creating a new one. In this article, we will check the concept of **Mutable lists in Python**.

### Modifying Elements of a List

We can modify an element in a list by accessing it through its index and assigning a new value.

```
A = [1, 2, 3, 4, 5]
```

```
Modifying an element
```

```
a[2] = 10
```

```
print(a)
```

### Output

```
[1, 2, 10, 4, 5]
```

### **Adding Elements in a list**

We can add a single element to the end of a list using the `append()` method.

```
A = [1, 2, 3, 4, 5]
```

```
Appending 6 to the end of the list
```

```
a.append(6)
```

```
print(a)
```

### **Output**

```
[1, 2, 3, 4, 5, 6]
```

### **Removing Elements in a List**

We can remove the first occurrence of a specific element from a list using the `remove()` method.

```
A = [1, 2, 3, 4, 5]
```

```
Removing element with value 3 from the list
```

```
a.remove(3)
```

```
print(a)
```

### **Output**

```
[1, 2, 4, 5]
```

### **Extending list with another list**

We can add all elements from another list to the end of our list using the `extend()` method.

```
A = [1, 2, 3]
```

```
Add elements of another list
```

```
a.extend([4, 5, 6])
```

```
print(a)
```

### **Output**

```
[1, 2, 3, 4, 5, 6]
```

### **Slicing and Modifying a Sublist**

We can slice a list to create a sublist and modify elements within that sublist.

```
A = [1, 2, 3, 4, 5]
```

```
Slicing and modifying a sublist
```

```
a[1:4] = [20, 30, 40]
```

```
print(a)
```

### **Output**

```
[1, 20, 30, 40, 5]
```

### **Various Methods used in Lists**

Let's understand the various methods of the mutable lists.

| Method                              | Method's description                                                                          |
|-------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>append()</b>                     | Add a single element to the end of the list                                                   |
| pop()                               | Removes and returns the element at index I (default is the last item).                        |
| clear()                             | Removes all elements from the list, leaving it empty.                                         |
| reduce()                            | Repeatedly applies a function to an iterable, reducing it to a single value.                  |
| sum()                               | Returns the sum of all elements in an iterable.                                               |
| ord()                               | Returns the character's numeric code point.                                                   |
| max()                               | Returns the largest element from the list.                                                    |
| min()                               | Returns the smallest element from the list.                                                   |
| extend()                            | Appends all elements from the another list to the end of the list.                            |
| remove()                            | Removes the first occurrence of element from the list.                                        |
| insert()                            | Inserts element at the specified index in the list.                                           |
| reverse()                           | Reverses the elements of the list in place.                                                   |
| <b>sort(key=None,reverse=False)</b> | Sorts the list in ascending order, optionally using a custom sort key or in reverse order.    |
| index()                             | Returns the index of the first occurrence of element within the optional start and end range. |
| count()                             | Returns the number of times element appears in the list.                                      |
| all()                               | Returns True if all elements are true or the list is empty.                                   |
| <b>copy()</b>                       | Returns shallow copy of the list.                                                             |
| <b>del list[i]</b>                  | Deletes the element at index I from the list.                                                 |

## Aliasing

In Python, we can give another name of the function. For the existing function, we can give another name, which is nothing but function aliasing.

### Function aliasing in Python

In function aliasing, we create a new variable and assign the function reference to one existing function to the variable. We can verify that both the objects pointing to the same memory location using id() built-in function in Python.

#### Example 1: Basic usage of function aliasing in Python

```
def fun(name):
 print(f'Hello {name}, welcome to GeeksForGeeks!!!')
```

```
cheer = fun
print(f'The id of fun() : {id(fun)}')
print(f'The id of cheer() : {id(cheer)}')
```

```
fun('Geeks')
cheer('Geeks')
```



**Output:**

The id of fun() : 140716797521432

The id of cheer() : 140716797521432

Hello Geeks, welcome to GeeksForGeeks!!!

Hello Geeks, welcome to GeeksForGeeks!!!

**Explanation:** In the above example, only one function is available, but we can call that function by using either fun or cheer name.

**Example 2: Function aliasing of custom objects**

Here, we have defined a class Test and a method name() to return the u\_name attribute of the class. Here, we are demonstrating the usage of function aliasing on the name() method of the test object.

Class Test:

```
def __init__(self):
 self.u_name = "user-origin"
```

```
def name(self):
 return self.u_name
```

```
create object of Test class
test = Test()
```

```
create function reference and it's alias
name_fn = test.name
name_fn_ref = name_fn
```

```
this will print user-origin
print(name_fn())
```

```
test.u_name = "user-mod"
```

```
this will print user-mod
print(name_fn_ref())
```

**Output:**

user-origin

user-mod

**Explanation:** `name_fn` is the function reference to the `name()` method of the test object. `name_fn_ref` is the function alias of the function `name_fn`. After changing the attribute value of `u_name` the function alias also reflects the change that establishes the fact that, `name_fn` and `name_fn_ref` points to the same function.

## Cloning lists

Given a list of elements, the task is to create a copy of it. Copying a list ensures that the original list remains unchanged while we perform operations on the duplicate. This is useful when working with mutable lists, especially nested ones.

### For example:

```
a = [1, 2, 3, 4, 5]
```

```
b = a.copy()
```

```
Output: [1, 2, 3, 4, 5]
```

Let's explore different methods to clone or copy a list in Python.

### Using copy()

copy() method is a built-in method in Python that creates a shallow copy of a list. This method is simple and highly efficient for cloning a list.

```
a = [1, 2, 3, 4, 5]
```

```
b = a.copy()
```

```
print(b)
```

### Output

```
[1, 2, 3, 4, 5]
```

**Explanation:** copy() creates a new list with the same elements. Changes to b do not affect a. For nested lists, it still copies references only (shallow copy).

### Using List Slicing

This method creates a new list by slicing all elements from the original. It's concise and performs well.

```
a = [1, 2, 3, 4, 5]
```

```
b = a[:]
```

```
print(b)
```

### Output

```
[1, 2, 3, 4, 5]
```

**Explanation:** The [:] operator selects all elements and creates a new list, independent of the original.

### Using list()

This method uses Python's list() constructor to generate a copy of the original list.

```
a = [1, 2, 3, 4, 5]
```

```
b = list(a)
```

```
print(b)
```

### Output

```
[1, 2, 3, 4, 5]
```

**Explanation:** list(a) converts the iterable a into a new list. This is also a shallow copy.

### Using List Comprehension

A list comprehension can also be used to copy the elements of a list into a new list.

```
a = [1, 2, 3, 4, 5]
b = [item for item in a]
print(b)
```

### Output

```
[1, 2, 3, 4, 5]
```

**Explanation:** Iterates through a and appends each element to a new list b. Useful if additional processing is needed during copy.

### Using copy.deepcopy() for Nested Lists

For nested lists, a deep copy is required to ensure that changes to the cloned list do not affect the original. The copy module provides the deepcopy() method for this purpose.

```
import copy
a = [[1, 2], [3, 4], [5, 6]]
b = copy.deepcopy(a)
print(b)
```

### Output

```
[[1, 2], [3, 4], [5, 6]]
```

## list parameters

In Python, we can pass a list to a function, allowing to access or update the list's items. This makes the function more versatile and allows us to work with the list in many ways.

### Passing list by Reference

When we pass a list to a function by reference, it refers to the original list. If we make any modifications to the list, the changes will reflect in the original list.

#### Example:

```
def fun(l):
 for i in l:
 print(i,end=" ") # Iterates
l = [1, 2, 3, 4]
fun(l)
```

### Output

```
1 2 3 4
```

#### Explanation:

- This function prints each element of the list l.
- list l is passed, printing the numbers one by one.

### By using \*args (Variable-Length Arguments)

This allows us to pass a list to a function and unpack it into separate arguments. It is useful when we don't know the exact number of arguments the function will receive.

**Example:**

```
def fun(*args):
 for i in args:
 print(i,end=" ")
l = [1, 2, 3, 4, 5]
fun(*l)
```

**Output**

```
1 2 3 4 5
```

**Explanation:**

- This function unpacks the list **l** into separate arguments.
- It prints each element on the same line, separated by spaces.

**Passing a copy**

Shallow copy of a list creates a new list with the same elements, ensuring the original list remains unchanged. This is useful when we want to work with a duplicate without modifying the original.

Example:

```
def fun(l):
 l.append(6)
a = [1, 2, 3, 4, 5]
b = a.copy() # shallow copy of list
fun(b)
print(a) # Original list
print(b) # Modified copy
```

**Output**

```
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 6]
```

**Explanation:**

- Shallow copy of **l** is created, keeping the original list unchanged.
- Copied list is modified by appending 6, while the original list remains the same.

**Tuples**

A tuple is an immutable ordered collection of elements.

- Tuples are similar to lists, but unlike lists, they cannot be changed after their creation.
- Can hold elements of different data types.
- These are ordered, heterogeneous and immutable.

**Creating a Tuple**

A tuple is created by placing all the items inside parentheses (), separated by commas. A tuple can have any number of items.

```
tup = ()
print(tup)

Using String
tup = ('Geeks', 'For')
print(tup)

Using List
li = [1, 2, 4, 5, 6]
print(tuple(li))

Using Built-in Function
tup = tuple('Geeks')
print(tup)
```

**Output**

```
()
('Geeks', 'For')
(1, 2, 4, 5, 6)
('G', 'e', 'e', 'k', 's')
```

**Creating a Tuple with Mixed Datatypes**

Tuples can store elements of different data types, such as integers, strings, lists and dictionaries, within a single structure.

```
tup = (5, 'Welcome', 7.5, True, [1, 2, 3], {'key': 'value'})
print(tup)
```

**Output**

```
(5, 'Welcome', 7.5, True, [1, 2, 3], {'key': 'value'})
```

**Tuple Basic Operations****Accessing of Tuples**

We can access the elements of a tuple by using indexing and slicing, similar to how we access elements in a list. Indexing starts at 0 for the first element and goes up to n-1, where n is the number of elements in the tuple. Negative indexing starts from -1 for the last element and goes backward.

```
tup = tuple("Geeks")
print(tup[0])
print(tup[1:4])
print(tup[:3])

Tuple unpacking
tup = ("Geeks", "For", "Geeks")
```

```
This line unpack values of Tuple1
a, b, c = tup
print(a)
print(b)
print(c)
```

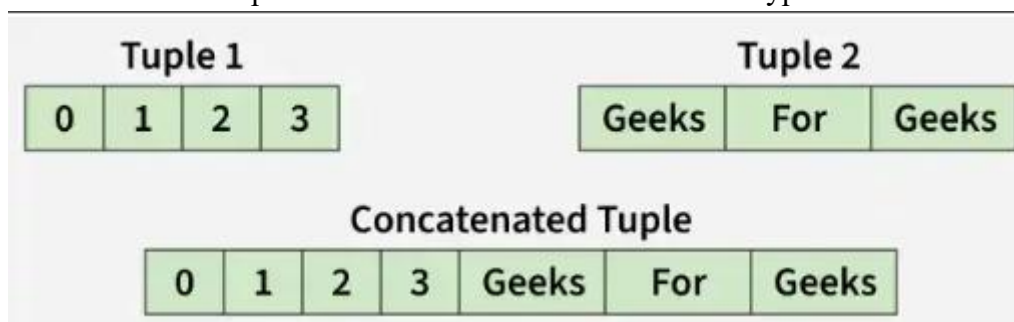
**Output**

```
G
('e', 'e', 'k')
('G', 'e', 'e')
Geeks
For
Geeks
```

**Concatenation of Tuples**

Tuples can be concatenated using the + operator. This operation combines two or more tuples to create a new tuple.

**Note:** Only tuples can be concatenated with tuples. Combining a tuple with other types like lists will raise an error. Tuples themselves can contain mixed datatypes.



concatenation of two tuples

```
tup1 = (0, 1, 2, 3)
tup2 = ('Geeks', 'For', 'Geeks')
tup3 = tup1 + tup2
print(tup3)
```

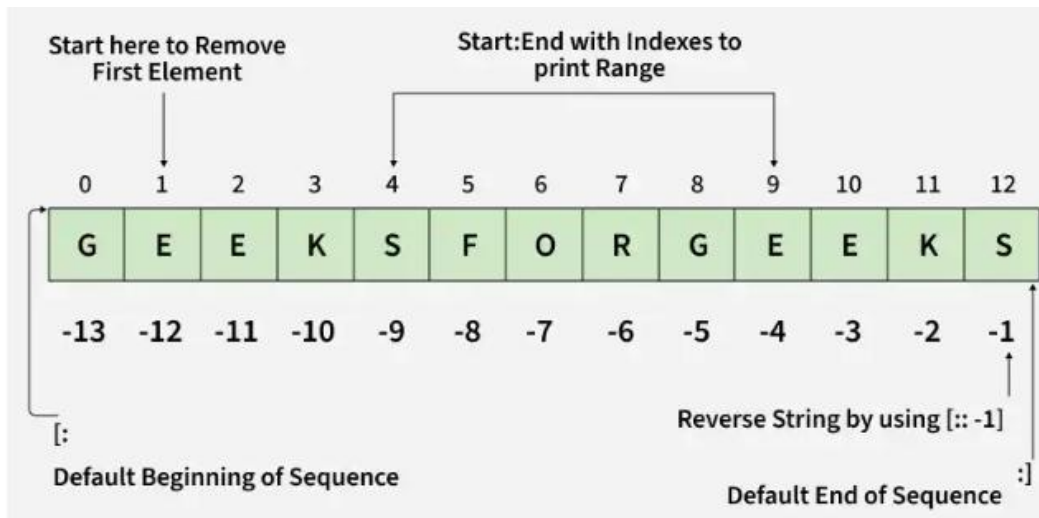
**Output**

```
(0, 1, 2, 3, 'Geeks', 'For', 'Geeks')
```

**Slicing of Tuple**

Slicing a tuple means creating a new tuple from a subset of elements of the original tuple. The slicing syntax is tuple[start:stop:step].

**Note:** Negative Increment values can also be used to reverse the sequence of Tuples.



tuple slicing

```
tup = tuple('GEEKSFORGEEKS')
print(tup[1:])
print(tup[::-1])
print(tup[4:9])
```

### Output

```
('E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S')
('S', 'K', 'E', 'E', 'G', 'R', 'O', 'F', 'S', 'K', 'E', 'E', 'G')
('S', 'F', 'O', 'R', 'G')
```

**Note:** [:] returns a shallow copy of the tuple, while [::step] allows stepping through elements. Using [::-1] reverses the sequence.

### Deleting a Tuple

Since tuples are immutable, we cannot delete individual elements of a tuple. However, we can delete an entire tuple using del statement.

**Note:** Printing of Tuple after deletion results in an Error.

```
tup = (0, 1, 2, 3, 4)
del tup
print(tup)
```

### Output

ERROR!

Traceback (most recent call last):

File "<main.py>", line 6, in <module>

NameError: name 'tup' is not defined

### Tuple Unpacking with Asterisk (\*)

\*operator is used in tuple unpacking to grab multiple items into a list. This is useful to extract just a few specific elements and collect the rest together.

```
Tup = (1, 2, 3, 4, 5)
```

```
a, *b, c = tup
```

```
print(a)
print(b)
print(c)
```

### Output

```
1
[2, 3, 4]
5
```

**Explanation:** **a** gets the first item, **c** gets the last item and **\*b** collects everything in between into a list.

## Operations and methods

In Python programming, Operators in general are used to perform operations on values and variables.

- **Operators:** Special symbols like -, +, \*, /, etc.
- **Operands:** Value on which the operator is applied.

### Types of Operators in Python

| Type                 | Operators             |
|----------------------|-----------------------|
| Arithmetic Operators | +, -, *, /, %         |
| Relational Operators | <, <=, >, >=, ==, !=  |
| Logical Operators    | AND, OR, NOT          |
| Bitwise operator     | &,  , ^, ~, <<, >>    |
| Assignment Operator  | =, +=, -=, *=, %=     |
| Ternary Operator     | x if condition else y |
| Identity Operators   | is, is not            |

### Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication and division.

In Python, the division operator (/) returns a floating-point result, while floor division (//) returns an integer result.

```
a = 15
b = 4
```

```
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
```



```
print("Modulus:", a % b)
print("Exponentiation:", a ** b)
```

### Output

Addition: 19

Subtraction: 11

Multiplication: 60

Division: 3.75

Floor Division: 3

Modulus: 3

Exponentiation: 50625

**Note:** Refer to Differences between / and //.

### Comparison Operators

Comparison(or Relational) operators compares values. It either returns True or False according to the condition.

```
a = 13
```

```
b = 33
```

```
print(a > b)
```

```
print(a < b)
```

```
print(a == b)
```

```
print(a != b)
```

```
print(a >= b)
```

```
print(a <= b)
```

### Output

False

True

False

True

False

True

### Logical Operators

Logical operators perform **Logical AND**, **Logical OR** and **Logical NOT** operations. It is used to combine conditional statements.

The precedence of Logical Operators in Python is as follows:

1. Logical not
2. logical and
3. logical or

```
a = True
```

```
b = False
print(a and b)
print(a or b)
print(not a)
```

### Output

```
False
True
False
```

### Bitwise Operators

Bitwise operators act on bits and perform bit-by-bit operations. These are used to operate on binary numbers.

Bitwise Operators in Python are as follows:

1. Bitwise NOT
2. Bitwise Shift
3. Bitwise AND
4. Bitwise XOR
5. Bitwise OR

```
a = 10
b = 4
```

```
print(a & b)
print(a | b)
print(~a)
print(a ^ b)
print(a >> 2)
print(a << 2)
```

### Output

```
0
14
-11
14
2
40
```

### Assignment Operators

Assignment operators are used to assign values to the variables. This operator is used to assign the value of the right side of the expression to the left side operand. **Example:**

```
a = 10
b = a
print(b)
b += a
print(b)
```

```

b -= a
print(b)
b *= a
print(b)
b <<= a
print(b)

```

### Output

```

10
20
10
100
102400

```

### Identity Operators

"is" and "is not" are the identity operators and both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

**is** True if the operands are identical

**is not** True if the operands are not identical

```

a = 10
b = 20
c = a

```

```

print(a is not b)
print(a is c)

```

### Output

```

True
True

```

### Membership Operators

"in" and "not in" are the membership operators that are used to test whether a value or variable is in a sequence.

**In** True if value is found in the sequence

**not in** True if value is not found in the sequence

```

x = 24
y = 20
my_list = [10, 20, 30, 40, 50]

```

```

if (x not in my_list):
 print("x is NOT present in given list")
else:
 print("x is present in given list")

```

```

if (y in my_list):
 print("y is present in given list")
else:
 print("y is NOT present in given list")

```

**Output**

```

x is NOT present in given list
y is present in given list

```

**Ternary Operator**

Ternary operators also known as conditional expressions are operators that evaluate something based on a condition being true or false. It was added to Python in version 2.5.

It simply allows testing a condition in a single line replacing the multiline if-else, making the code compact.

**Syntax :** [on\_true] if [expression] else [on\_false]

```
a, b = 10, 20
```

```
min = a if a < b else b
```

```
print(min)
```

**Output**

```
10
```

**Precedence and Associativity of Operators**

Operator precedence and associativity determine the priorities of the operator.

**Operator Precedence**

This is used in an expression with more than one operator with different precedence to determine which operation to perform first.

```
Expr = 10 + 20 * 30
```

```
print(expr)
```

```
name = "Alex"
```

```
age = 0
```

```
if name == "Alex" or name == "John" and age >= 2:
```

```
 print("Hello! Welcome.")
```

```
else:
```

```
 print("Good Bye!!")
```

**Output**

```
610
```

```
Hello! Welcome.
```

**Operator Associativity**

If an expression contains two or more operators with the same precedence then Operator Associativity is used to determine. It can either be Left to Right or from Right to Left.

```
Print(100 / 10 * 10)
print(5 - 2 + 3)
print(5 - (2 + 3))
print(2 ** 3 ** 2)
```

### Output

```
100.0
6
0
512
```

## Tuple assignment

Python Tuple is a collection of objects separated by commas. A tuple is similar to a Python list in terms of indexing, nested objects, and repetition but the main difference between both is Python tuple is immutable, unlike the Python list which is mutable.

```
Note : In case of list, we use square
brackets []. Here we use round brackets ()
tup = (10, 20, 30)
```

```
print(tup)
print(type(tup))
```

### Output

```
(10, 20, 30)
<class 'tuple'>
```

### What is Immutable in Tuples?

Unlike Python lists, tuples are immutable. Some Characteristics of Tuples in Python.

- Like Lists, tuples are ordered and we can access their elements using their index values
- We cannot update items to a tuple once it is created.
- Tuples cannot be appended or extended.
- We cannot remove items from a tuple once it is created.

Let us see this with an example.

```
tup = (1, 2, 3, 4, 5)
```

```
tuples are indexed
print(tup[1])
print(tup[4])
```

```
tuples contain duplicate elements
tup = (1, 2, 3, 4, 2, 3)
print(t)
```

```
updating an element
```

```
tup[1] = 100
```

```
print(tup)
```

**Output:**

```
2
```

```
5
```

```
(1, 2, 3, 4, 2, 3)
```

Traceback (most recent call last):

```
File "Solution.py", line 12, in <module>
```

```
t[1] = 100
```

TypeError: 'tuple' object does not support item assignment

### Accessing Values in Python Tuples

Tuples in Python provide two ways by which we can access the elements of a tuple.

#### Python Access Tuple using a Positive Index

Using square brackets we can get the values from tuples in Python.

```
tup = (10, 5, 20)
```

```
print("Value in tup[0] = ", tup[0])
```

```
print("Value in tup[1] = ", tup[1])
```

```
print("Value in tup[2] = ", tup[2])
```

#### Output

```
Value in tup[0] = 10
```

```
Value in tup[1] = 5
```

```
Value in tup[2] = 20
```

#### Access Tuple using Negative Index

In the above methods, we use the positive index to access the value in Python, and here we will use the negative index within [].

```
tup = (10, 5, 20)
```

```
print("Value in tup[-1] = ", tup[-1])
```

```
print("Value in tup[-2] = ", tup[-2])
```

```
print("Value in tup[-3] = ", tup[-3])
```

#### Output

```
Value in tup[-1] = 20
```

```
Value in tup[-2] = 5
```

```
Value in tup[-3] = 10
```

### Different Operations Related to Tuples

Below are the different operations related to tuples in Python:

#### Traversing Items of Python Tuples

Like List Traversal, we can traverse through a tuple using for loop.

```
Define a tuple
tup = (1, 2, 3, 4, 5)

Traverse through each item in the tuple
for x in tup:
 print(x, end=" ")
```

### Output

1 2 3 4 5

### Concatenation of Python Tuples

To Concatenation of Python Tuples, we will use plus operators(+).

```
Code for concatenating 2 tuples
tup1 = (0, 1, 2, 3)
tup2 = ('python', 'geek')
```

```
Concatenating above two
print(tup1 + tup2)
```

### Output

(0, 1, 2, 3, 'python', 'geek')

### Nesting of Python Tuples

A nested tuple in Python means a tuple inside another tuple.

```
Code for creating nested tuples
tup1 = (0, 1, 2, 3)
tup2 = ('python', 'geek')
```

```
tup3 = (tup1, tup2)
print(tup3)
```

### Output

((0, 1, 2, 3), ('python', 'geek'))

### Repetition Python Tuples

We can create a tuple of multiple same elements from a single element in that tuple.

```
Code to create a tuple with repetition
tup = ('python',)*3
print(tup)
```

### Output

('python', 'python', 'python')

Try the above without a comma and check. You will get tup3 as a string 'pythonpythonpython'.

### Slicing Tuples in Python

Slicing a Python tuple means dividing a tuple into small tuples using the indexing method. In this example, we slice the tuple from index 1 to the last element. In the second print statement, we printed the tuple using reverse indexing. And in the third print statement, we printed the elements from index 2 to 4.

```
code to test slicing
```

```
tup = (0 ,1, 2, 3)
```

```
print(tup[1:])
```

```
print(tup[::-1])
```

```
print(tup[2:4])
```

### Output

```
(1, 2, 3)
```

```
(3, 2, 1, 0)
```

```
(2, 3)
```

**Note:** In Python slicing, the end index provided is not included.

### Deleting a Tuple in Python

In this example, we are deleting a tuple using 'del' keyword. The output will be in the form of error because after deleting the tuple, it will give a NameError.

**Note:** Remove individual tuple elements is not possible, but we can delete the whole Tuple using Del keyword.

```
Code for deleting a tuple
```

```
tup = (0, 1)
```

```
del tup
```

```
print(tup)
```

### Output:

```
Hangup (SIGHUP)
```

```
Traceback (most recent call last):
```

```
File "Solution.py", line 5, in <module>
```

```
 print(t)
```

```
NameError: name 't' is not defined
```

### Finding the Length of a Python Tuple

To find the length of a tuple, we can use Python's len() function and pass the tuple as the parameter.

```
Code for printing the length of a tuple
```

```
tup = ('python', 'geek')
```

```
print(len(tup))
```

### Output



2

**Multiple Data Types With Tuple**

Tuples in Python are heterogeneous in nature. This means tuples support elements with multiple datatypes.

```
tuple with different datatypes
tup = ("immutable", True, 23)
print(tup)
```

**Output**

```
('immutable', True, 23)
```

**Converting a List to a Tuple**

We can convert a list in Python to a tuple by using the tuple() constructor and passing the list as its parameters.

```
Code for converting a list and a string into a tuple
a = [0, 1, 2]
tup = tuple(a)

print(tup)
```

**Output**

```
(0, 1, 2)
```

**Output:**

Tuples take a single parameter which may be a list, string, set, or even a dictionary(only keys are taken as elements), and converts them to a tuple.

**Tuples in a Loop**

We can also create a tuple with a single element in it using loops.

```
python code for creating tuples in a loop
tup = ('gfg',)

Number of time loop runs
n = 5
for i in range(int(n)):
 tup = (tup,)
 print(tup)
```

**Output**

```
(('gfg',),)
(((('gfg',),),),)
((((('gfg',),),),),)
((((('gfg',),),),),)
((((('gfg',),),),),),)
```

**Different Ways of Creating a Tuple**

- Using round brackets
- Without Brackets
- Tuple Constructor
- Empty Tuple
- Single Element Tuple
- Using Tuple Packing

**Using Round Brackets**

```
tup = ("gfg", "Python")
print(tup)
```

**Output**

```
('gfg', 'Python')
```

**Using Comma Separated**

```
Creating a tuple without brackets
tup = 4, 5, 6
print(tup)
```

**Output**

```
(4, 5, 6)
```

**Using Tuple Constructor**

```
Creating a tuple using the tuple() constructor
tup = tuple([7, 8, 9])
print(tup)
```

**Output**

```
(7, 8, 9)
```

**Creating an Empty Tuple**

```
Creating an empty tuple
tup = ()
print(tup)
```

**Output**

```
()
```

**Single Element Tuple**

```
Creating a single-element tuple
tup = (10,) # Comma is important here
print(tup) # Output: (10,)
```

```
print(type(tup))
What if we do not use comma
tup = (10) # This an integer (not a tuple)
print(tup)
print(type(tup))
```

**Output**

```
(10,)
<class 'tuple'>
10
<class 'int'>
```

**Tuple Packing**

```
Tuple packing
a, b, c = 11, 12, 13
tup = (a, b, c)
print(tup)
```

**Output**

```
(11, 12, 13)
```

**Tuple Built-In Methods**

Tuples support only a few methods due to their immutable nature. The two most commonly used methods are `count()` and `index()`

| Built-in-Method | Description                                                                     |
|-----------------|---------------------------------------------------------------------------------|
| <b>index( )</b> | Find in the tuple and returns the index of the given value where it's available |
| <b>count( )</b> | Returns the frequency of occurrence of a specified value                        |

**Tuple Built-In Functions**

| Built-in Function  | Description                                                                      |
|--------------------|----------------------------------------------------------------------------------|
| <b>all()</b>       | Returns true if all element are true or if tuple is empty                        |
| <b>any()</b>       | return true if any element of the tuple is true. if tuple is empty, return false |
| <b>len()</b>       | Returns length of the tuple or size of the tuple                                 |
| <b>enumerate()</b> | Returns enumerate object of tuple                                                |
| <b>max()</b>       | return maximum element of given tuple                                            |
| <b>min()</b>       | return minimum element of given tuple                                            |
| <b>sum()</b>       | Sums up the numbers in the tuple                                                 |
| <b>sorted()</b>    | input elements in the tuple and return a new sorted list                         |
| <b>tuple()</b>     | Convert an iterable to a tuple.                                                  |

**Tuples VS Lists**

| Similarities                                                                                                | Differences                                                                                                          |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Functions that can be used for both lists and tuples:<br>len(), max(), min(), sum(), any(), all(), sorted() | Methods that cannot be used for tuples:<br>append(), insert(), remove(), pop(), clear(), sort(), reverse()           |
| Methods that can be used for both lists and tuples:<br>count(), Index()                                     | we generally use 'tuples' for heterogeneous (different) data types and 'lists' for homogeneous (similar) data types. |
| Tuples can be stored in lists.                                                                              | Iterating through a 'tuple' is faster than in a 'list'.                                                              |
| Lists can be stored in tuples.                                                                              | 'Lists' are mutable whereas 'tuples' are immutable.                                                                  |

**Tuple as return value**

In Python, a function can return more than one value at a time using commas. These values are usually returned as a tuple. This is useful when a function needs to give back several related results together.

Let's explore different ways to do it.

**Using Tuple**

Tuple is a group of values separated by commas. Python automatically packs the values into a tuple, which can then be unpacked into variables.

```
def fun():
 return "geeksforgeeks", 20
s, x = fun()
print(s)
print(x)
```

**Output**

```
geeksforgeeks
20
```

**Explanation:** `fun()` returns two values as a tuple, which are unpacked into 's' and 'x' and then printed.

**Using Dictionary**

Dictionary stores data in key-value pairs. It can be used to return multiple values from a function by assigning each value a key, making the result easy to read and access.

```
def fun():
 d = {"str": "GeeksforGeeks", "x": 20}
 return d
```

```
d = fun()
print(d)
```

### Output

```
{'x': 20, 'str': 'GeeksforGeeks'}
```

**Explanation:** `fun()` creates a dictionary with two key-value pairs and returns it, which is then printed.

### Using Object

An object created from a class can hold multiple values as attributes. A function can return an object to group and return several related values together.

```
class Info:
 def __init__(self):
 self.name = "Henry"
 self.age = 25

def get_info():
 return Info()
```

```
person = get_info()
print(person.name)
print(person.age)
```

### Output

```
Henry
25
```

### Explanation:

- **get\_info()** returns an object of the `Info` class containing two values `name` and `age`.
- values are accessed using dot notation `person.name` and `person.age`.

### Using List

List is an ordered collection of items, created using square brackets `[]`. A list can be used to return multiple values from a function, especially when the number of values is fixed.

```
def fun():
 l = ["geeksforgeeks", 20]
 return l
```

```
d = fun()
print(d)
```

### Output

```
['geeksforgeeks', 20]
```

**Explanation:** `fun()` returns a list with two values, which is stored in `data` and then printed.

### Using Data Class

Data class is a special type of class used to store multiple related values. It automatically creates useful methods like `__init__()` and `__repr__()`, so it's easy to create and return multiple values in a clean, readable way.

From dataclasses import dataclass

```
@dataclass
class Student:
 name: str
 marks: int

def get_student():
 return Student("Olivia", 85)
```

```
student = get_student()
print(student.name)
print(student.marks)
```

### Output

```
Olivia
85
```

**Explanation:** `get_student()` returns a Student dataclass object with multiple values, which can be accessed using dot notation like `student.name` and `student.marks`.

## Dictionaries

Dictionary is a data structure that stores information in key-value pairs. While keys must be unique and immutable (like strings or numbers), values can be of any data type, whether mutable or immutable. This makes dictionaries ideal for accessing data by a specific name rather than a numeric position like in list.

**Example:** This example shows how a dictionary stores data using keys and values.

```
data = { "name": "Jake", "age": 22 }
print(data)
```

### Output

```
{'name': 'Jake', 'age': 22}
```

**Explanation:** "name" and "age" are keys, "Jake" and 22 are their values and dictionary stores data in key:value format

### Creating a Dictionary

A dictionary is created by writing key-value pairs inside `{ }`, where each key is connected to a value using colon (:). A dictionary can also be created using `dict()` function.

```
a = {"x": 1, "y": 2}
print(a)
```

```
b = dict(name="Sam", age=20)
print(b)
```

### Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
{'a': 'Geeks', 'b': 'for', 'c': 'Geeks'}
```

### Accessing Dictionary Items

A value in a dictionary is accessed by using its key. This can be done either with square brackets [ ] or with the get() method. Both return the value linked to the given key.

```
d = {"name": "Kat", "age": 21}
```

```
print(d["name"]) # Access using key
print(d.get("age")) # Access using get()
```

### Output

```
Kat
21
```

**Note:** Accessing a missing key with [ ] raises a KeyError, while get() is safer because it returns None (or a default value) instead of an error.

### Adding and Updating Dictionary Items

New items are added to a dictionary using the assignment operator (=) by giving a new key a value. If an existing key is used with the assignment operator, its value is updated with the new one.

```
d = {"name": "Sam"}
```

```
d["age"] = 21 # Adding a new key-value pair
d["name"] = "Alex" # Updating an existing value
print(d)
```

### Output

```
{'name': 'Alex', 'age': 21}
```

### Removing Dictionary Items

Dictionary items can be removed using built-in deletion methods that work on keys:

1. **del:** removes an item using its key

```
d = {"a": 1, "b": 2}
del d["a"]
print(d)
```

### Output

```
{'b': 2}
```

2. **pop():** removes the item with the given key and returns its value

```
d = {"a": 1, "b": 2}
```

```
val = d.pop("a")
print(val)
print(d)
```

### Output

```
1
```

```
{'b': 2}
```

3. **popitem():** removes and returns the last inserted key-value pair

```
d = {"a": 1, "b": 2}
```

```
print(d.popitem())
```

### Output

```
('b', 2)
```

4. **clear():** removes all items from the dictionary

```
d = {"a": 1, "b": 2}
```

```
d.clear()
```

```
print(d)
```

### Output

```
{}
```

## Iterating Through a Dictionary

A dictionary can be traversed using a for loop to access its keys, values or both key-value pairs by using the built-in methods `keys()`, `values()` and `items()`.

**1. Iterate keys:** Returns all keys from the dictionary.

```
d = {"a": 1, "b": 2}
```

```
for key in d:
```

```
 print(key)
```

### Output

```
a
```

```
b
```

**2. Iterate values:** Returns all values from the dictionary.

```
d = {"a": 1, "b": 2}
```

```
for value in d.values():
```

```
 print(value)
```

### Output

```
1
```

```
2
```

**3. Iterate key-value pairs:** Returns all key-value pairs as tuples.



```
d = {"a": 1, "b": 2}
for key, value in d.items():
 print(key, value)
```

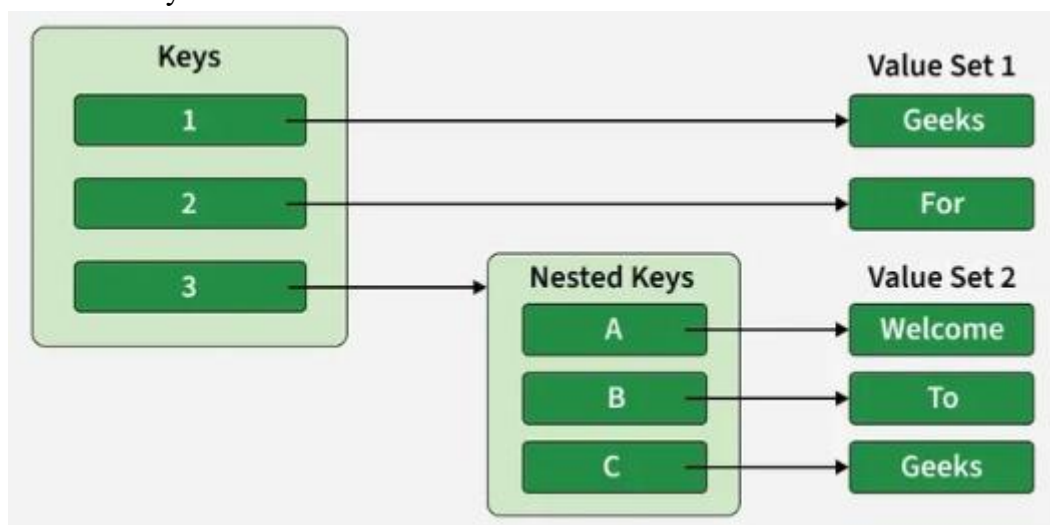
**Output**

```
a 1
b 2
```

Read in detail: [Ways to Iterate Over a Dictionary](#)

**Nested Dictionaries**

A nested dictionary is a dictionary that contains another dictionary as one of its values. Below diagram shows how a nested dictionary works, where key 3 points to another dictionary inside the main dictionary.

**Representation of Nested Dictionary**

The arrows show how each key is connected to its corresponding value.

```
D = {
 "student": {
 "name": "Sam",
 "age": 20
 }
}
```

```
print(d["student"]["name"])
```

**Dictionaries assignment**

Sometimes, while working with python dictionaries, we can have a problem in which we need to initialize dictionary keys with values. We save a mesh of keys to be initialized. This usually happens during web development while working with JSON data. Lets discuss certain ways in which this task can be performed.

**Method #1 : Using dict() + zip()** The combination of above methods can be used to perform this task. In this, we allot the list values to already constructed mesh and zip() helps in mapping values as per list index.

```
Python3 code to demonstrate working of
Assign values to initialized dictionary keys
using dict() + zip()

initializing dictionary
test_dict = {'gfg': '', 'is': '', 'best': ''}

Initializing list
test_list = ['A', 'B', 'C']

printing original dictionary
print("The original dictionary is : " + str(test_dict))

Assign values to initialized dictionary keys
using dict() + zip()
res = dict(zip(test_dict, test_list))

printing result
print("The assigned value dictionary is : " + str(res))
```

**Output :**

The original dictionary is : {'is': '', 'best': '', 'gfg': ''}  
 The assigned value dictionary is : {'gfg': 'C', 'best': 'B', 'is': 'A'}

**Time Complexity:**  $O(n)$ , where  $n$  is the number of elements in the dictionary.

**Auxiliary Space:**  $O(n)$ , as we are using a dictionary and a list to store the values.

**Method #2 : Using loop + zip()** This is extended way in which this task can be performed. In this, we iterate through the zipped list and assign value to dictionary.

```
Python3 code to demonstrate working of
Assign values to initialized dictionary keys
using loop + zip()

initializing dictionary
test_dict = {'gfg': '', 'is': '', 'best': ''}

Initializing list
test_list = ['A', 'B', 'C']

printing original dictionary
print("The original dictionary is : " + str(test_dict))

Assign values to initialized dictionary keys
using loop + zip()
```

```
for key, val in zip(test_dict, test_list):
 test_dict[key] = val
```

# printing result

```
print("The assigned value dictionary is : " + str(test_dict))
```

### Output :

The original dictionary is : {'is': '', 'best': '', 'gfg': ''}

The assigned value dictionary is : {'gfg': 'C', 'best': 'B', 'is': 'A'}

Time Complexity:

O(n)

Auxiliary Space: O(n)

### Method #3: Using keys() and for loop

# Python3 code to demonstrate working of

# Assign values to initialized dictionary keys

# initializing dictionary

```
test_dict = {'gfg': '', 'is': '', 'best': ''}
```

# Initializing list

```
test_list = ['A', 'B', 'C']
```

# printing original dictionary

```
print("The original dictionary is : " + str(test_dict))
```

# Assign values to initialized dictionary keys

```
x = list(test_dict.keys())
```

```
for i in range(0, len(x)):
```

```
 test_dict[x[i]] = test_list[i]
```

# printing result

```
print("The assigned value dictionary is : " + str(test_dict))
```

### Output

The original dictionary is : {'gfg': '', 'is': '', 'best': ''}

The assigned value dictionary is : {'gfg': 'A', 'is': 'B', 'best': 'C'}

Time Complexity: O(n), where n is the length of the list test\_dict

Auxiliary Space: O(n) additional space of size n is created where n is the number of elements in the res list

### Method #4: Using dictionary comprehension

Use dictionary comprehension to create a new dictionary where the keys are the same as the original dictionary and the values are taken from the test\_list. The enumerate() function is used to get the index of each key in the original dictionary, which is used to get the corresponding value from the test\_list.

- Initialize a dictionary with some keys and empty values.

- Create a list of values that need to be assigned to the keys of the dictionary.
- Print the original dictionary to verify the initial state.
- Use any of the provided methods (or the new method) to assign the values to the keys of the dictionary.
- Print the updated dictionary to verify the assigned values.

# Python3 code to demonstrate working of

# Assign values to initialized dictionary keys

# initializing dictionary

```
test_dict = {'gfg': '', 'is': '', 'best': ''}
```

# Initializing list

```
test_list = ['A', 'B', 'C']
```

# printing original dictionary

```
print("The original dictionary is : " + str(test_dict))
```

# Assign values to initialized dictionary keys

```
test_dict = {key: test_list[i] for i, key in enumerate(test_dict)}
```

# printing result

```
print("The assigned value dictionary is : " + str(test_dict))
```

## Output

The original dictionary is : {'gfg': '', 'is': '', 'best': ''}

The assigned value dictionary is : {'gfg': 'A', 'is': 'B', 'best': 'C'}

Time complexity:  $O(n)$ , where  $n$  is the number of keys in the dictionary

Auxiliary space:  $O(n)$ , where  $n$  is the number of keys in the dictionary

## Method #5: Using dictionary.update() method

Steps:

- Initialize the dictionary test\_dict with 3 keys and empty values.
- Initialize the list test\_list with 3 elements.
- Print the original dictionary test\_dict.
- Loop through the keys in test\_dict using enumerate() to get the index I and the key itself key.
- Update the value of the key in test\_dict using update() method with a new dictionary containing only the key-value pair key: test\_list[i].
- Print the resulting dictionary test\_dict with assigned values.

# Python3 code to demonstrate working of

# Assign values to initialized dictionary keys

# initializing dictionary

```
test_dict = {'gfg': '', 'is': '', 'best': ''}
```

```
Initializing list
test_list = ['A', 'B', 'C']

printing original dictionary
print("The original dictionary is : " + str(test_dict))

Assign values to initialized dictionary keys using update() method
for I, key in enumerate(test_dict):
 test_dict.update({key: test_list[i]})

printing result
print("The assigned value dictionary is : " + str(test_dict))
```

**Output**

The original dictionary is : {'gfg': '', 'is': '', 'best': ''}

The assigned value dictionary is : {'gfg': 'A', 'is': 'B', 'best': 'C'}

Time complexity:  $O(n)$ , where  $n$  is the number of keys in the dictionary

Auxiliary space:  $O(n)$ , where  $n$  is the number of keys in the dictionary

**Dictionaries as return value**

`dict.values()` method in Python returns a view object that contains all the values of the dictionary. This object updates automatically when the dictionary changes and is useful when only values are needed.

**Example:** In this example, `values()` is used to get all values from a dictionary.

```
d = {'A': 'Python', 'B': 'Java', 'C': 'C++'}
print(d.values())
```

**Output**

```
dict_values(['Python', 'Java', 'C++'])
```

**Explanation:** `d.values()` returns a view object containing all values of dictionary `d`.

**Syntax**

```
dict_name.values()
```

- **Parameters:** No parameters are required.
- **Returns:** Returns a dynamic view object containing all dictionary values.

**Examples**

**Example 1:** Here, the code retrieves all dictionary values and prints them using a loop. This helps access each value individually.

```
d = {'x': 10, 'y': 20, 'z': 30}
for v in d.values():
 print(v)
```

**Output**

```
10
20
30
```

**Explanation:** `d.values()` returns all values and for `v` in `d.values()` accesses each value one by one.

**Example 2:** In this example, the values view object is converted into a list. This allows indexing and other list operations.

```
d = {'a': 1, 'b': 2, 'c': 3}
v = list(d.values())
print(v)
```

**Output**

```
[1, 2, 3]
```

**Explanation:** `list(d.values())` converts the view object into a list.

**Example 3:** Here, the code checks whether a specific value exists in the dictionary.

```
d = {'p': 100, 'q': 200}
print(200 in d.values())
```

**Output**

```
True
```

**Explanation:** `d.values()` provides all values and `200 in d.values()` checks if value exists.

**Advanced list processing**

Lists are just like dynamically sized arrays, declared in other languages (vector in C++ and ArrayList in Java). Lists need not be homogeneous always which makes it a most powerful tool in Python. A single list may contain DataTypes like Integers, Strings, as well as Objects. Lists are mutable, and hence, they can be altered even after their creation.

Lists are one of the most powerful tools in python. They have a lot of hidden tricks which makes them extremely versatile. Let's explore some of these useful techniques which make our lives much easier !!!

**Sorting a list**

To sort a list in ascending or descending order, we use the `sort()` function with the following syntax:

**For ascending order:**

```
list.sort()
```

**For descending order:**

```
list.sort(reverse=True)
```

**Example:**

```
sorting a list using sort() function
```

```
my_list = [5, 2, 90, 24, 10]
```

```
sorting in ascending order permanently
changes the order of the list
my_list.sort()
print(my_list)
```

```
sorting in descending order permanently
changes the order of the list
my_list.sort(reverse=True)
print(my_list)
```

**Output:**

```
[2, 5, 10, 24, 90]
```

```
[90, 24, 10, 5, 2]
```

To temporarily change the order of the list use the `sorted()` function with the syntax:

```
list.sorted()
temporary sorting using sorted() method
```

```
my_list = [5, 2, 90, 24, 10]
```

```
ascending order
print(sorted(my_list))
descending order
my_list_2 = sorted(my_list)
print(my_list)
```

**Output:**

```
[2, 5, 10, 24, 90]
```

```
[5, 2, 90, 24, 10]
```

**Reversing a list**

To reverse the order of a list we use the `reverse()` function. Its syntax is:

```
list.reverse()
```

**Example:**

```
reverse a list using reverse()
```

```
my_list = [5, 2, 90, 24, 10]
```

```
reverse
my_list.reverse()
print(my_list)
```

**Output:**

```
[10, 24, 90, 2, 5]
```

Or we could apply list comprehension to reverse a list:

```
list = list[::-1]
```

**Example:**

```
reverse using list comprehension
```

```
my_list = [5, 2, 90, 24, 10]
```

```
reverse
```

```
print(my_list[::-1])
```

**Output:**

```
[10, 24, 90, 2, 5]
```

### Removing duplicates

As dictionaries in python do not contain duplicate keys. We use the dict.fromkeys() function to convert our list into a dictionary with list elements as keys. And then we convert the dictionary back to list.

This is a powerful trick to automatically remove all the duplicates. Its syntax is:

```
My_list=['a', 'b', 'c', 'b', 'a']
```

```
Mylist = list(dict.fromkeys(My_List))
```

**Example:**

```
removing duplicates from a list using dictionaries
```

```
my_list_1 = [5, 2, 90, 24, 10, 2, 90, 34]
```

```
my_list_2 = ['a', 'a', 'a', 'b', 'c', 'd', 'd', 'e']
```

```
removing duplicates from list 1
```

```
my_list_1 = list(dict.fromkeys(my_list_1))
```

```
print(my_list_1)
```

```
removing duplicates from list 2
```

```
my_list_2 = list(dict.fromkeys(my_list_2))
```

```
print(my_list_2)
```

**Output:**

```
[5, 2, 90, 24, 10, 34]
```

```
['a', 'b', 'c', 'd', 'e']
```

### Filtering a list

Python lists can be filtered with the help of filter() function or with the help of list comprehension. Below is the syntax:

```
My_list = list(filter(filter_function , iterable_item))
```

The Filter function returns an iterator object which we must convert back into the list.

**Example:**

```
filtering with the help of filter() function
```

```
creating a filter function filter all the values less than 20
```



```
filter function
def my_filter(n):
 if n > 20:
 return n

driver code
if __name__ == "__main__":
 my_list = [5, 2, 90, 24, 10, 2, 95, 36]
 my_filtered_list = list(filter(my_filter, my_list))
 print(my_filtered_list)
```

**Output:**

```
[90, 24, 95, 36]
```

We can also filter using list comprehension. It is a much easier and elegant way to filter lists, below is the syntax:

```
My_list = [item for item in my_list if (condition)]
```

**Example:**

```
filtering with the help of list comprehension
```

```
my_list = [5, 2, 90, 24, 10, 2, 95, 36]
```

```
an elegant way to sort the list
```

```
my_list = [item for item in my_list if item > 20]
```

```
print(my_list)
```

**Output:**

```
[90, 24, 95, 36]
```

**Modifying list**

To modify the values in the list with the help of an external function, we make use of the map() function. Map() function returns a map object (iterator) of the results after applying the given function to each element of a given iterable(list, tuple etc.). Below is the syntax:

```
My_list = list(map(function,iterable))
```

**Example:**

```
using map() function to modify the text
```

```
def squaring(n):
```

```
 return n**2
```

```
driver code
```

```
if __name__ == "__main__":
```

```
 my_list = [5, 2, 90, 24, 10, 2, 95, 36]
```

```
 my_squared_list = list(map(squaring, my_list))
```

```
 print(my_squared_list)
```

**Output:**

```
[25, 4, 8100, 576, 100, 4, 9025, 1296]
```

A much cleaner approach is using list comprehension.

**Example:**

```
the same result can be obtained by a much pythonic approach
```

```
i.e., by using list comprehension
```

```
my_list = [5, 2, 90, 24, 10, 2, 95, 36]
```

```
print([i**2 for i in my_list])
```

**Output:**

```
[25, 4, 8100, 576, 100, 4, 9025, 1296]
```

**Combining lists**

We can even combine lists with the help of the `zip()` function which results in a list of tuples. Here each item from list A is combined with corresponding elements from list B in the form of a tuple. Below is the syntax:

```
My_list = zip(list_1, list_2)
```

**Example:**

```
combining lists with the help of zip() function
```

```
my_list_1 = [5, 2, 90, 24, 10]
```

```
my_list_2 = [6, 3, 91, 25, 12]
```

```
combined
```

```
my_combined_list = list(zip(my_list_1, my_list_2))
```

```
print(my_combined_list)
```

**Output:**

```
[(5, 6), (2, 3), (90, 91), (24, 25), (10, 12)]
```

**Finding the most common item**

To find the most frequent element we make use of the `set()` function. The `set()` function removes all the duplicates from the list, and the `max()` function returns the most frequent element (which is found with the help of 'key'). The key is an optional single argument function. Below is the syntax:

```
Most_frequent_value = max(set(my_list), key=mylist.count)
```

**Example :**

```
to find the most frequent element from the list
```

```
my_list = ['a', 'a', 'a', 'b', 'c', 'd', 'd', 'e']
```

```
most_frequent_value = max(set(my_list), key=my_list.count)
```

```
print("The most common element is:", most_frequent_value)
```

**Output:**

```
The most common element is: a
```

**Checking for membership in a list**

To check whether an item exists in a list, we use the `in` statement.

**Example:**

```
my_list = ['a', 'a', 'a', 'b', 'c', 'd', 'd', 'e']
```

```
to check whether 'c' is a member of my_list
returns true if present
print('c' in my_list)
```

```
to check whether 'f' is a member of my_list
returns false if not present
print('f' in my_list)
```

**Output:**

```
True
```

```
False
```

**Flatten a list of lists**

Sometimes we encounter a list where each element in itself is a list. To convert a list of lists into a single list, we use list comprehension.

```
My_list = [item for List in list_of_lists for item in List]
```

**Example:**

```
to flatten a list_of_lists by using list comprehension
list_of_lists = [[1, 2],
 [3, 4],
 [5, 6],
 [7, 8]]
```

```
using list comprehension
my_list = [item for List in list_of_lists for item in List]
print(my_list)
```

**Output:**

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

**list comprehension.**

List comprehension is a concise way to create new lists by applying an expression to each item in an existing iterable like a list, tuple or range. It helps to write clean, readable and efficient code compared to traditional loops.

Suppose you want to square every number in a list:

```
a = [2, 3, 4, 5]
res = [val ** 2 for val in a]
print(res)
```

**Output**

```
[4, 9, 16, 25]
```

**Explanation:** `res = [val ** 2 for val in a]` use list comprehension to create a new list by squaring each number in `a`.

**Syntax**

```
[expression for item in iterable if condition]
```

**Parameters:**

- **expression:** operation or value to include in the new list.
- **item:** current element from the iterable.
- **iterable:** sequence like a list, tuple or range.
- **if condition (optional):** filter to include only items that satisfy the condition.

**Conditional Statements in List Comprehension**

List comprehensions can use conditions to select or transform items based on specific rules. This allows creating customized lists more concisely and improves code readability and efficiency.

**Example 1:** This code uses a list comprehension with a condition to create a new list with only even numbers from list `a`.

```
a = [1, 2, 3, 4, 5]
res = [val for val in a if val % 2 == 0]
print(res)
```

**Output**

```
[2, 4]
```

**Example 2:** Here, list comprehension is used with a condition to create a new list with numbers greater than 10 from list `b`.

```
b = [5, 12, 7, 18, 3, 20]
res = [val for val in b if val > 10]
print(res)
```

**Output**

```
[12, 18, 20]
```

**Examples**

**1. Creating a list from a range:** One can quickly create a list of numbers within a specific range using list comprehension.

```
a = [i for i in range(10)]
print(a)
```

**Output**

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

**2. Using nested loops:** A list of all coordinate pairs in a 3x3 grid can be generated by combining two loops inside a list comprehension.

```
c = [(x, y) for x in range(3) for y in range(3)]
print(c)
```

#### Output

```
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
```

**3. Flattening a list of lists:** A nested list (matrix) can be transformed into a single flat list by iterating through each sublist and its elements.

```
mat = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
res = [val for row in mat for val in row]
print(res)
```

#### Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

### For Loop vs List Comprehension

| For Loop                                | List Comprehension                       |
|-----------------------------------------|------------------------------------------|
| Uses multiple lines of code             | Uses a single line of code               |
| Requires manual appending of elements   | Creates the list directly                |
| Better for complex logic and conditions | Better for simple and concise operations |
| More readable for long operations       | More compact and faster to write         |

### Lists as arrays

We can use a number of approaches to convert a list into a Python array based on the requirements. One option is to use the **array module**, which allows us to create arrays with a specified data type. Another option is to use the **numpy.array()** method, which provides more features for working with arrays

#### Using array() with data type indicator

This task can be easily performed using the `array()` function. This is an inbuilt function in Python to convert to an array. The data type indicator "i" is used in the case of integers, which restricts data type.

```
from array import array
```

```
a = [6, 4, 8, 9, 10]
print(str(a))
```

```
res = array("i", a)
print(str(res))
```

**Output**

```
[6, 4, 8, 9, 10]
```

```
array('i', [6, 4, 8, 9, 10])
```

**Explanation:** In `array("i", a)` "i" indicates that the data type of the array elements will be integers (**int**). `res` is the array created from the list `a`.

**Using `numpy.array()`**

This method uses NumPy's `array()` function to convert a list into a NumPy array, offering more flexibility and additional functionality for numerical operations.

```
import numpy as np
```

```
a = [6, 4, 8, 9, 10]
```

```
print(str(a))
```

```
res = np.array(a)
```

```
print(str(res))
```

**Output**

```
[6, 4, 8, 9, 10]
```

```
[6 4 8 9 10]
```

**Explanation:** The `numpy.array()` function converts the list `a` into a NumPy array. By default, NumPy will infer the data type (**int** in this case), but you can specify it explicitly using the `dtype` argument if needed.

# CHAPTER 4

## REGULAR EXPRESSION

---

### Regular Expression

A Regular Expression or RegEx is a special sequence of characters that uses a search pattern to find a string or set of strings.

It can detect the presence or absence of a text by matching it with a particular pattern and also can split a pattern into one or more sub-patterns.

### Regex Module in Python

Python has a built-in module named "**re**" that is used for regular expressions in Python. We can import this module by using import statement.

Importing re module in Python using following command:

```
import re
```

### How to Use RegEx in Python?

You can use RegEx in Python after importing re module.

#### Example:

This Python code uses regular expressions to search for the word "**portal**" in the given string and then prints the start and end indices of the matched word within the string.

```
import re
```

```
s = 'GeeksforGeeks: A computer science portal for geeks'
```

```
match = re.search(r'portal', s)
```

```
print('Start Index:', match.start())
```

```
print('End Index:', match.end())
```

### Output

```
Start Index: 34
```

```
End Index: 40
```

**Note:** Here r character (r'portal') stands for raw, not regex. The raw string is slightly different from a regular string, it won't interpret the \ character as an escape character. This is because the regular expression engine uses \ character for its own escaping purpose.

Before starting with the Python regex module let's see how to actually write regex using metacharacters or special sequences.

### RegEx Functions

The re module in Python provides various functions that help search, match, and manipulate strings using regular expressions.

Below are main functions available in the re module:

| Function     | Description                                                                                   |
|--------------|-----------------------------------------------------------------------------------------------|
| re.findall() | finds and returns all matching occurrences in a list                                          |
| re.compile() | Regular expressions are compiled into pattern objects                                         |
| re.split()   | Split string by the occurrences of a character or a pattern.                                  |
| re.sub()     | Replaces all occurrences of a character or patten with a replacement string.                  |
| resubn       | It's similar to re.sub() method but it returns a tuple: (new_string, number_of_substitutions) |
| re.escape()  | Escapes special character                                                                     |
| re.search()  | Searches for first occurrence of character or pattern                                         |

## Python RegEx

A Regular Expression or RegEx is a special sequence of characters that uses a search pattern to find a string or set of strings.

It can detect the presence or absence of a text by matching it with a particular pattern and also can split a pattern into one or more sub-patterns.

### Regex Module in Python

Python has a built-in module named "**re**" that is used for regular expressions in Python. We can import this module by using import statement.

Importing re module in Python using following command:

```
import re
```

### How to Use RegEx in Python?

You can use RegEx in Python after importing re module.

#### Example:

This Python code uses regular expressions to search for the word "**portal**" in the given string and then prints the start and end indices of the matched word within the string.

```
import re
```

```
s = 'GeeksforGeeks: A computer science portal for geeks'
match = re.search(r'portal', s)
```

```
print('Start Index:', match.start())
print('End Index:', match.end())
```

### Output

Start Index: 34

End Index: 40



**Note:** Here r character (r'portal') stands for raw, not regex. The raw string is slightly different from a regular string, it won't interpret the \ character as an escape character. This is because the regular expression engine uses \ character for its own escaping purpose.

Before starting with the Python regex module let's see how to actually write regex using metacharacters or special sequences.

## RegEx Functions

The re module in Python provides various functions that help search, match, and manipulate strings using regular expressions.

Below are main functions available in the re module:

| Function     | Description                                                                                   |
|--------------|-----------------------------------------------------------------------------------------------|
| re.findall() | finds and returns all matching occurrences in a list                                          |
| re.compile() | Regular expressions are compiled into pattern objects                                         |
| re.split()   | Split string by the occurrences of a character or a pattern.                                  |
| re.sub()     | Replaces all occurrences of a character or patter with a replacement string.                  |
| resubn       | It's similar to re.sub() method but it returns a tuple: (new_string, number_of_substitutions) |
| re.escape()  | Escapes special character                                                                     |
| re.search()  | Searches for first occurrence of character or pattern                                         |

### 1. re.findall()

Returns all non-overlapping matches of a pattern in the string as a list. It scans the string from left to right.

**Example:** This code uses regular expression \d+ to find all sequences of one or more digits in the given string.

```
import re
string = """Hello my Number is 123456789 and
my friend's number is 987654321"""
```

```
regex = '\d+'
match = re.findall(regex, string)
print(match)
```

### Output

```
['123456789', '987654321']
```

### 2. re.compile()

Compiles a regex into a pattern object, which can be reused for matching or substitutions.

**Example 1:** This pattern [a-e] matches all lowercase letters between 'a' and 'e', in the input string "Aye, said Mr. Gibenson Stark". The output should be ['e', 'a', 'd', 'b', 'e'], which are matching characters.

```
import re
```

```
p = re.compile('[a-e]')
print(p.findall("Aye, said Mr. Gibenson Stark"))
```

### Output

```
['e', 'a', 'd', 'b', 'e', 'a']
```

### Explanation:

- First occurrence is 'e' in "Aye" and not 'A', as it is Case Sensitive.
- Next Occurrence is 'a' in "said", then 'd' in "said", followed by 'b' and 'e' in "Gibenson", the Last 'a' matches with "Stark".
- Metacharacter backslash '\' has a very important role as it signals various sequences. If the backslash is to be used without its special meaning as metacharacter, use '\\'.

**Example 2:** The code uses regular expressions to find and list all single digits and sequences of digits in the given input strings. It finds single digits with `\d` and sequences of digits with `\d+`.

```
import re
p = re.compile('\d')
print(p.findall("I went to him at 11 A.M. on 4th July 1886"))
```

```
p = re.compile('\d+')
print(p.findall("I went to him at 11 A.M. on 4th July 1886"))
```

### Output

```
['1', '1', '4', '1', '8', '8', '6']
['11', '4', '1886']
```

**Example 3:** Word and non-word characters

- `\w` matches a single word character.
- `\w+` matches a group of word characters.
- `\W` matches non-word characters.

```
import re
```

```
p = re.compile('\w')
print(p.findall("He said * in some_lang."))
```

```
p = re.compile('\w+')
print(p.findall("I went to him at 11 A.M., he \
said *** in some_language."))
```

```
p = re.compile('\W')
print(p.findall("he said *** in some_language."))
```

### Output

```
['H', 'e', 's', 'a', 'i', 'd', 'i', 'n', 's', 'o', 'm', 'e', '_', 'l', 'a', 'n', 'g']
```

```
['I', 'went', 'to', 'him', 'at', '11', 'A', 'M', 'he', 'said', 'in', 'some_language']
[' ', ' ', '*', '*', '*', ' ', ' ', '!']
```

**Example 4:** The regular expression pattern **'ab\*'** to find and list all occurrences of **'ab'** followed by zero or more **'b'** characters. In the input string "ababbaabbb". It returns the following list of matches: ['ab', 'abb', 'abbb'].

```
import re
p = re.compile('ab*')
print(p.findall("ababbaabbb"))
```

### Output

```
['ab', 'abb', 'a', 'abbb']
```

### Explanation:

- Output 'ab', is valid because of single 'a' accompanied by single 'b'.
- Output 'abb', is valid because of single 'a' accompanied by 2 'b'.
- Output 'a', is valid because of single 'a' accompanied by 0 'b'.
- Output 'abbb', is valid because of single 'a' accompanied by 3 'b'.

### 3. re.split()

Splits a string wherever the pattern matches. The remaining characters are returned as list elements.

### Syntax:

```
re.split(pattern, string, maxsplit=0, flags=0)
```

- **pattern:** Regular expression to match split points.
- **string:** The input string to split.
- **maxsplit (optional):** Limits the number of splits. Default is 0 (no limit).
- **flags (optional):** Apply regex flags like re.IGNORECASE.

### Example 1: Splitting by non-word characters or digits

This example demonstrates how to split a string using different patterns like non-word characters (**\W+**), apostrophes, and digits (**\d+**).

```
from re import split
```

```
print(split('\W+', 'Words, words , Words'))
print(split('\W+', "Word's words Words"))
print(split('\W+', 'On 12th Jan 2016, at 11:02 AM'))
print(split('\d+', 'On 12th Jan 2016, at 11:02 AM'))
```

### Output

```
['Words', 'words', 'Words']
['Word', 's', 'words', 'Words']
['On', '12th', 'Jan', '2016', 'at', '11', '02', 'AM']
['On ', 'th Jan ', ' ', 'at ', ':', ' ', 'AM']
```

**Example 2: Using maxsplit and flags**

This example shows how to limit the number of splits using maxsplit, and how flags can control case sensitivity.

```
import re
print(re.split('\d+', 'On 12th Jan 2016, at 11:02 AM', 1))
print(re.split('[a-f]+', 'Aey, Boy oh boy, come here', flags=re.IGNORECASE))
print(re.split('[a-f]+', 'Aey, Boy oh boy, come here'))
```

**Output**

```
['On ', 'th Jan 2016, at 11:02 AM']
['', 'y, ', 'oy oh ', 'oy, ', 'om', ' h', 'r', '']
['A', 'y, Boy oh ', 'oy, ', 'om', ' h', 'r', '']
```

**Note:** In the second and third cases of the above, [a-f]+ splits the string using any combination of lowercase letters from 'a' to 'f'. The re.IGNORECASE flag includes uppercase letters in the match.

**4. re.sub()**

The re.sub() function replaces all occurrences of a pattern in a string with a replacement string.

**Syntax:**

```
re.sub(pattern, repl, string, count=0, flags=0)
```

- **pattern:** The regex pattern to search for.
- **repl:** The string to replace matches with.
- **string:** The input string to process.
- **count (optional):** Maximum number of substitutions (default is 0, which means replace all).
- **flags (optional):** Regex flags like re.IGNORECASE.

**Example 1:** The following examples show different ways to replace the pattern 'ub' with '~\*', using various flags and count values.

```
import re

Case-insensitive replacement of all 'ub'
print(re.sub('ub', '~*', 'Subject has Uber booked already', flags=re.IGNORECASE))

Case-sensitive replacement of all 'ub'
print(re.sub('ub', '~*', 'Subject has Uber booked already'))

Replace only the first 'ub', case-insensitive
print(re.sub('ub', '~*', 'Subject has Uber booked already', count=1, flags=re.IGNORECASE))

Replace "AND" with "&", ignoring case
print(re.sub(r'\sAND\s', ' & ', 'Baked Beans And Spam', flags=re.IGNORECASE))
```

**Output**

```
S~*ject has ~*er booked already
```

```
S~*ject has Uber booked already
S~*ject has Uber booked already
Baked Beans & Spam
```

### 5. re.subn()

re.subn() function works just like **re.sub()**, but instead of returning only the modified string, it returns a tuple: **(new\_string, number\_of\_substitutions)**

#### Syntax:

```
re.subn(pattern, repl, string, count=0, flags=0)
```

#### Example: Substitution with count

This example shows how re.subn() gives both the replaced string and the number of times replacements were made.

```
import re
```

```
Case-sensitive replacement
```

```
print(re.subn('ub', '~*', 'Subject has Uber booked already'))
```

```
Case-insensitive replacement
```

```
t = re.subn('ub', '~*', 'Subject has Uber booked already', flags=re.IGNORECASE)
```

```
print(t)
```

```
print(len(t)) # tuple length
```

```
print(t[0]) # modified string
```

#### Output

```
('S~*ject has Uber booked already', 1)
```

```
('S~*ject has ~*er booked already', 2)
```

```
2
```

```
S~*ject has ~*er booked already
```

### 6. re.escape()

re.escape() function adds a backslash (\) before all special characters in a string. This is useful when you want to match a string literally, including any characters that have special meaning in regex (like ., \*, [, ], etc.).

#### Syntax:

```
re.escape(string)
```

#### Example: Escaping special characters

This example shows how re.escape() treats spaces, brackets, dashes, and tabs as literal characters.

```
import re
```

```
print(re.escape("This is Awesome even 1 AM"))
```

```
print(re.escape("I Asked what is this [a-9], he said \t ^WoW"))
```

#### Output

```
This\ is\ Awesome\ even\ 1\ AM
I\ Asked\ what\ is\ this\ \[a-9]\,\ he\ said\ \ \ \ ^WoW
```

## 7. re.search()

The `re.search()` function searches for the first occurrence of a pattern in a string. It returns a **match object** if found, otherwise **None**.

**Note:** Use it when you want to check if a pattern exists or extract the first match.

### Example: Search and extract values

This example searches for a date pattern with a month name (letters) followed by a day (digits) in a sentence.

Import re

```
regex = r"([a-zA-Z]+) (\d+)"
match = re.search(regex, "I was born on June 24")

if match:
 print("Match at index %s, %s" % (match.start(), match.end()))
 print("Full match:", match.group(0))
 print("Month:", match.group(1))
 print("Day:", match.group(2))
else:
 print("The regex pattern does not match.")
```

### Output

```
Match at index 14, 21
Full match: June 24
Month: June
Day: 24
```

### Meta-characters

Metacharacters are special characters in regular expressions used to define search patterns. The `re` module in Python supports several metacharacters that help you perform powerful pattern matching.

Below is a quick reference table:

| MetaCharacters | Description                                                   |
|----------------|---------------------------------------------------------------|
| \              | Used to drop the special meaning of character following it    |
| []             | Represent a character class                                   |
| ^              | Matches the beginning                                         |
| \$             | Matches the end                                               |
| .              | Matches any character except newline                          |
|                | Means OR (Matches with any of the characters separated by it. |
| ?              | Matches zero or one occurrence                                |
| *              | Any number of occurrences (including 0 occurrences)           |

|    |                                                                   |
|----|-------------------------------------------------------------------|
| +  | One or more occurrences                                           |
| {} | Indicate the number of occurrences of a preceding regex to match. |
| () | Enclose a group of Regex                                          |

### 1. \ - Backslash

The backslash (\) makes sure that the character is not treated in a special way. This can be considered a way of escaping metacharacters.

For example, if you want to search for the dot(.) in the string then you will find that dot(.) will be treated as a special character as is one of the metacharacters (as shown in the above table). So for this case, we will use the backslash(\) just before the dot(.) so that it will lose its specialty. See the below example for a better understanding.

**Example:** The first search (**re.search(r'.', s)**) matches any character, not just the period, while the second search (**re.search(r'\.', s)**) specifically looks for and matches the period character.

```
import re

s = 'geeks.forgeeks'
```

```
without using \
match = re.search(r'.', s)
print(match)
```

```
using \
match = re.search(r'\.', s)
print(match)
```

### Output

```
<re.Match object; span=(0, 1), match='g'>
<re.Match object; span=(5, 6), match='.'>
```

### 2. [] - Square Brackets

Square Brackets ([]) represent a character class consisting of a set of characters that we wish to match. For example, the character class [abc] will match any single a, b, or c.

We can also specify a range of characters using - inside the square brackets. For example,

- [0, 3] is same as [0123]
- [a-c] is same as [abc]

We can also invert the character class using the caret(^) symbol. For example,

- [^0-3] means any character except 0, 1, 2, or 3
- [^a-c] means any character except a, b, or c

**Example:** In this code, you're using regular expressions to find all the characters in the string that fall within the range of 'a' to 'm'. The **re.findall()** function returns a list of all such characters. In the given string, the characters that match this pattern are: 'c', 'k', 'b', 'f', 'j', 'e', 'h', 'l', 'd', 'g'.

```
import re
```

```
string = "The quick brown fox jumps over the lazy dog"
pattern = "[a-m]"
result = re.findall(pattern, string)

print(result)
```

### Output

```
['h', 'e', 'i', 'c', 'k', 'b', 'f', 'j', 'm', 'e', 'h', 'e', 'l', 'a', 'd', 'g']
```

### 3. ^ - Caret

Caret (^) symbol matches the beginning of the string i.e. checks whether the string starts with the given character(s) or not. For example -

- ^g will check if the string starts with g such as geeks, globe, girl, g, etc.
- ^ge will check if the string starts with ge such as geeks, geeksforgeeks, etc.

**Example:** This code uses regular expressions to check if a list of strings starts with **"The"**. If a string begins with **"The,"** it's marked as **"Matched"** otherwise, it's labeled as **"Not matched"**.

```
import re
regex = r'^The'
strings = ['The quick brown fox', 'The lazy dog', 'A quick brown fox']
for string in strings:
 if re.match(regex, string):
 print(f'Matched: {string}')
 else:
 print(f'Not matched: {string}')
```

### Output

```
Matched: The quick brown fox
Matched: The lazy dog
Not matched: A quick brown fox
```

### 4. \$ - Dollar

Dollar(\$) symbol matches the end of the string i.e checks whether the string ends with the given character(s) or not. For example-

- s\$ will check for the string that ends with a such as geeks, ends, s, etc.
- ks\$ will check for the string that ends with ks such as geeks, geeksforgeeks, ks, etc.

**Example:** This code uses a regular expression to check if the string ends with **"World!"**. If a match is found, it prints **"Match found!"** otherwise, it prints **"Match not found"**.

```
import re

string = "Hello World!"
pattern = r"World!$"

match = re.search(pattern, string)
```



```

if match:
 print("Match found!")
else:
 print("Match not found.")

```

### Output

Match found!

### 5. . - Dot

Dot(.) symbol matches only a single character except for the newline character (\n). For example -

- a.b will check for the string that contains any character at the place of the dot such as acb, acbd, abbb, etc
- .. will check if the string contains at least 2 characters

**Example:** This code uses a regular expression to search for the pattern "**brown.fox**" within the string. The dot (.) in the pattern represents any character. If a match is found, it prints "**Match found!**" otherwise, it prints "**Match not found**".

```
import re
```

```

string = "The quick brown fox jumps over the lazy dog."
pattern = r"brown.fox"

```

```

match = re.search(pattern, string)
if match:
 print("Match found!")
else:
 print("Match not found.")

```

### Output

Match found!

### 6. | - Or

The | operator means either pattern on its left or right can match. a|b will match any string that contains a or b such as acd, bcd, abcd, etc.

### 7. ? - Question Mark

The question mark (?) indicates that the preceding element should be matched zero or one time. It allows you to specify that the element is optional, meaning it may occur once or not at all.

**For example,** ab?c will be matched for the string ac, acb, dabc but will not be matched for abbc because there are two b. Similarly, it will not be matched for abdc because b is not followed by c.

### 8. \* - Star

Star (\*) symbol matches zero or more occurrences of the regex preceding the \* symbol.

**For example,** ab\*c will be matched for the string ac, abc, abbbc, dabc, etc. but will not be matched for abdc because b is not followed by c.

## 9. + - Plus

Plus (+) symbol matches one or more occurrences of the regex preceding the + symbol.

**For example**, `ab+c` will be matched for the string `abc`, `abbc`, `dabc`, but will not be matched for `ac`, `abdc`, because there is no `b` in `ac` and `b`, is not followed by `c` in `abdc`.

## 10. {m, n} - Braces

Braces match any repetitions preceding regex from `m` to `n` both inclusive.

**For example**, `a{2, 4}` will be matched for the string `aaab`, `baaaac`, `gaad`, but will not be matched for strings like `abc`, `bc` because there is only one `a` or no `a` in both the cases.

## 11. (<regex>) - Group

Group symbol is used to group sub-patterns.

**For example**, `(a|b)cd` will match for strings like `acd`, `abcd`, `gacd`, etc.

## Special Sequences

Special sequences do not match for the actual character in the string instead it tells the specific location in the search string where the match must occur. It makes it easier to write commonly used patterns.

### List of special sequences

| Special Sequence | Description                                                                                                                                                                                      | Examples           |                            |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|----------------------------|
| <code>\A</code>  | Matches if the string begins with the given character                                                                                                                                            | <code>\Afor</code> | for geeks<br>for the world |
| <code>\b</code>  | Matches if the word begins or ends with the given character. <code>\b(string)</code> will check for the beginning of the word and <code>(string)\b</code> will check for the ending of the word. | <code>\bge</code>  | geeks<br>get               |
| <code>\B</code>  | It is the opposite of the <code>\b</code> i.e. the string should not start or end with the given regex.                                                                                          | <code>\Bge</code>  | together<br>forge          |
| <code>\d</code>  | Matches any decimal digit, this is equivalent to the set class <code>[0-9]</code>                                                                                                                | <code>\d</code>    | 123<br>gee1                |
| <code>\D</code>  | Matches any non-digit character, this is equivalent to the set class <code>^[^0-9]</code>                                                                                                        | <code>\D</code>    | geeks<br>geek1             |
| <code>\s</code>  | Matches any whitespace character.                                                                                                                                                                | <code>\s</code>    | gee ks<br>a bc a           |
| <code>\S</code>  | Matches any non-whitespace character                                                                                                                                                             | <code>\S</code>    | a bd<br>abcd               |
| <code>\w</code>  | Matches any alphanumeric character, this is equivalent to the class <code>[a-zA-Z0-9_]</code> .                                                                                                  | <code>\w</code>    | 123<br>geeKs4              |
| <code>\W</code>  | Matches any non-alphanumeric character.                                                                                                                                                          | <code>\W</code>    | >\$<br>gee<>               |
| <code>\Z</code>  | Matches if the string ends with the given regex                                                                                                                                                  | <code>ab\Z</code>  | abcdab<br>abababab         |

### Sets for character matching

A **Set** is a set of characters enclosed in '[''] brackets. Sets are used to match a single character in the set of characters specified between brackets. Below is the list of Sets:

| Set        | Description                                                                       |
|------------|-----------------------------------------------------------------------------------|
| \{n,\}     | Quantifies the preceding character or group and matches at least n occurrences.   |
| *          | Quantifies the preceding character or group and matches zero or more occurrences. |
| [0123]     | Matches the specified digits (0, 1, 2, or 3)                                      |
| [^arn]     | matches for any character EXCEPT a, r, and n                                      |
| \d         | Matches any digit (0-9).                                                          |
| [0-5][0-9] | matches for any two-digit numbers from 00 and 59                                  |
| \w         | Matches any alphanumeric character (a-z, A-Z, 0-9, or _).                         |
| [a-n]      | Matches any lower case alphabet between a and n.                                  |
| \D         | Matches any non-digit character.                                                  |
| [arn]      | matches where one of the specified characters (a, r, or n) are present            |
| [a-zA-Z]   | matches any character between a and z, lower case OR upper case                   |
| [0-9]      | matches any digit between 0 and 9                                                 |

### Match Object

A Match object contains all the information about the search and the result and if there is no match found then None will be returned. Let's see some of the commonly used methods and attributes of the match object.

#### 1. Getting the string and the regex

match.re attribute returns the regular expression passed and match.string attribute returns the string passed.

#### Example:

The code searches for the letter "G" at a word boundary in the string "Welcome to GeeksForGeeks" and prints the regular expression pattern (**res.re**) and the original string (**res.string**).

```
import re
s = "Welcome to GeeksForGeeks"
res = re.search(r"\bG", s)
```

```
print(res.re)
print(res.string)
```

**Output**

```
re.compile('\\bG')
Welcome to GeeksForGeeks
```

**2. Getting index of matched object**

- **start()** method returns the starting index of the matched substring
- **end()** method returns the ending index of the matched substring
- **span()** method returns a tuple containing the starting and the ending index of the matched substring

**Example: Getting index of matched object**

The code searches for substring "Gee" at a word boundary in string "Welcome to GeeksForGeeks" and prints start index of the match (**res.start()**), end index of the match (**res.end()**) and span of the match (**res.span()**).

```
import re

s = "Welcome to GeeksForGeeks"
res = re.search(r"\bGee", s)
```

```
print(res.start())
print(res.end())
print(res.span())
```

**Output**

```
11
14
(11, 14)
```

**3. Getting matched substring**

**group()** method returns the part of the string for which the patterns match. See the below example for a better understanding.

**Example: Getting matched substring**

The code searches for a sequence of two non-digit characters followed by a space and the letter 't' in the string "Welcome to GeeksForGeeks" and prints the matched text using **res.group()**.

```
import re

s = "Welcome to GeeksForGeeks"
res = re.search(r"\D{2} t", s)
print(res.group())
```

**Output**

```
me t
```

In the above example, our pattern specifies for the string that contains at least 2 characters which are followed by a space, and that space is followed by a t.

## Basic RegEx Patterns

Let's understand some of the basic regular expressions. They are as follows:

### 1. Character Classes

Character classes allow matching any one character from a specified set. They are enclosed in square brackets [].

```
import re
print(re.findall(r'[Gg]eeks', 'GeeksforGeeks: \
 A computer science portal for geeks'))
```

### Output

```
['Geeks', 'Geeks', 'geeks']
```

### 2. Ranges

In RegEx, a range allows matching characters or digits within a span using - inside []. For example, [0-9] matches digits, [A-Z] matches uppercase letters.

```
import re
print('Range',re.search(r'[a-zA-Z]', 'x'))
```

### Output

```
Range <re.Match object; span=(0, 1), match='x'>
```

### 3. Negation

Negation in a character class is specified by placing a ^ at the beginning of the brackets, meaning match anything except those characters.

#### Syntax:

```
[^a-z]
```

#### Example:

```
import re

print(re.search(r'^a-z', 'c'))
print(re.search(r'G[^e]', 'Geeks'))
```

### Output

```
None
None
```

### 3. Shortcuts

Shortcuts are shorthand representations for common character classes. Let's discuss some of the shortcuts provided by the regular expression engine.

- \w - matches a word character
- \d - matches digit character
- \s - matches whitespace character (space, tab, newline, etc.)
- \b - matches a zero-length character

```
import re
```

```
print('Geeks:', re.search(r'\bGeeks\b', 'Geeks'))
print('GeeksforGeeks:', re.search(r'\bGeeks\b', 'GeeksforGeeks'))
```

### Output

```
Geeks: <_sre.SRE_Match object; span=(0, 5), match='Geeks'>
GeeksforGeeks: None
```

## 4. Beginning and End of String

The ^ character chooses the beginning of a string and the \$ character chooses the end of a string.

```
import re
```

```
Beginning of String
match = re.search(r'^Geek', 'Campus Geek of the month')
print('Beg. of String:', match)
```

```
match = re.search(r'^Geek', 'Geek of the month')
print('Beg. of String:', match)
```

```
End of String
match = re.search(r'Geeks$', 'Compute science portal-GeeksforGeeks')
print('End of String:', match)
```

### Output

```
Beg. of String: None
Beg. of String: <_sre.SRE_Match object; span=(0, 4), match='Geek'>
End of String: <_sre.SRE_Match object; span=(31, 36), match='Geeks'>
```

## 5. Any Character

The . character represents any single character outside a bracketed character class.

```
import re
print('Any Character', re.search(r'p.th.n', 'python 3'))
```

### Output

```
Any Character <_sre.SRE_Match object; span=(0, 6), match='python'>
```

## 6. Optional Characters

Regular expression engine allows you to specify optional characters using the ? character. It allows a character or character class either to present once or else not to occur. Let's consider the example of a word with an alternative spelling - color or colour.

```
import re
```

```
print('Color',re.search(r'colou?r', 'color'))
print('Colour',re.search(r'colou?r', 'colour'))
```

**Output**

```
Color <_sre.SRE_Match object; span=(0, 5), match='color'>
Colour <_sre.SRE_Match object; span=(0, 6), match='colour'>
```

**7. Repetition**

Repetition enables you to repeat the same character or character class. Consider an example of a date that consists of day, month, and year. Let's use a regular expression to identify the date (mm-dd-yyyy).

```
import re
print('Date {mm-dd-yyyy}:', re.search(r'[\d]{2}-[\d]{2}-[\d]{4}', '18-08-2020'))
```

**Output**

```
Date {mm-dd-yyyy}: <_sre.SRE_Match object; span=(0, 10), match='18-08-2020'>
```

Here, the regular expression engine checks for two consecutive digits. Upon finding the match, it moves to the hyphen character. After then, it checks the next two consecutive digits and the process is repeated.

Let's discuss three other regular expressions under repetition.

**7.1 Repetition ranges**

The repetition range is useful when you have to accept one or more formats. Consider a scenario where both three digits, as well as four digits, are accepted. Let's have a look at the regular expression.

```
import re

print('Three Digit:', re.search(r'[\d]{3,4}', '189'))
print('Four Digit:', re.search(r'[\d]{3,4}', '2145'))
```

**Output**

```
Three Digit: <_sre.SRE_Match object; span=(0, 3), match='189'>
Four Digit: <_sre.SRE_Match object; span=(0, 4), match='2145'>
```

**7.2 Open-Ended Ranges**

There are scenarios where there is no limit for a character repetition. In such scenarios, you can set the upper limit as infinitive. A common example is matching street addresses. Let's have a look

```
import re

print(re.search(r'[\d]{1,}','5th Floor, A-118,\
Sector-136, Noida, Uttar Pradesh - 201305'))
```

**Output**

```
<_sre.SRE_Match object; span=(0, 1), match='5'>
```

### 7.3 Shorthand

Shorthand characters allow you to use + character to specify one or more ({1,}) and \* character to specify zero or more ({0,}).

```
import re
```

```
print(re.search(r'[\d]+', '5th Floor, A-118,\nSector-136, Noida, Uttar Pradesh - 201305'))
```

#### Output

```
<_sre.SRE_Match object; span=(0, 1), match='5'>
```

### 8. Grouping

Grouping is the process of separating an expression into groups by using parentheses, and it allows you to fetch each individual matching group.

```
import re
```

```
grp = re.search(r'([\d]{2})-([\d]{2})-([\d]{4})', '26-08-2020')
print(grp)
```

#### Output

```
<_sre.SRE_Match object; span=(0, 10), match='26-08-2020'>
Let's see some of its functionality.
```

#### 8.1 Return the entire match

The re module allows you to return the entire match using the group() method

```
import re
```

```
grp = re.search(r'([\d]{2})-([\d]{2})-([\d]{4})', '26-08-2020')
print(grp.group())
```

#### Output

```
26-08-2020
```

#### 8.2 Return a tuple of matched groups

You can use groups() method to return a tuple that holds individual matched groups

```
import re
```

```
grp = re.search(r'([\d]{2})-([\d]{2})-([\d]{4})', '26-08-2020')
print(grp.groups())
```

#### Output

```
('26', '08', '2020')
```

#### 8.3 Retrieve a single group

Upon passing the index to a group method, you can retrieve just a single group.

```
import re
```

```
grp = re.search(r'([\d]{2})-([\d]{2})-([\d]{4})', '26-08-2020')
print(grp.group(3))
```



**Output**

2020

**8.4 Name your groups**

The re module allows you to name your groups. Let's look into the syntax.

```
import re
match = re.search(r'(?P<dd>[\d]{2})-(?P<mm>[\d]{2})-(?P<yyyy>[\d]{4})',
 '26-08-2020')
print(match.group('mm'))
```

**Output**

08

**8.5 Individual match as a dictionary**

We have seen how regular expression provides a tuple of individual groups. Not only tuple, but it can also provide individual match as a dictionary in which the name of each group acts as the dictionary key.

```
import re
match = re.search(r'(?P<dd>[\d]{2})-(?P<mm>[\d]{2})-(?P<yyyy>[\d]{4})',
 '26-08-2020')
print(match.groupdict())
```

**Output**

```
{'dd': '26', 'mm': '08', 'yyyy': '2020'}
```

**9. Lookahead**

In the case of a negated character class, it won't match if a character is not present to check against the negated character. We can overcome this case by using lookahead; it accepts or rejects a match based on the presence or absence of content.

```
import re
print('negation:', re.search(r'n[^e]', 'Python'))
print('lookahead:', re.search(r'n(!e)', 'Python'))
```

**Output**

negation: None

lookahead: <\_sre.SRE\_Match object; span=(5, 6), match='n'>

Lookahead can also disqualify the match if it is not followed by a particular character. This process is called a positive lookahead, and can be achieved by simply replacing ! character **with** = character.

```
import re
print('positive lookahead', re.search(r'n(=e)', 'jasmine'))
```

**Output**

positive lookahead <\_sre.SRE\_Match object; span=(5, 6), match='n'>

## 10. Substitution

The regular expression can replace the string and returns the replaced one using the `re.sub` method. It is useful when you want to avoid characters such as `/`, `-`, `.`, etc. before storing it to a database. It takes three arguments:

- the regular expression
- the replacement string
- the source string being searched

Let's have a look at the below code that replaces `-` character from a credit card number.

```
import re
print(re.sub(r'([\d]{4})-([\d]{4})-([\d]{4})-([\d]{4})',r'\1\2\3\4',
 '1111-2222-3333-4444'))
```

### Concept of regular expression,

A regular expression is a special sequence of characters that helps you match or find other strings or sets of strings, using a specialized syntax held in a pattern. Regular expression are popularly known as `regex` or `regexp`. Usually, such patterns are used by string-searching algorithms for "find" or "find and replace" operations on strings, or for input validation.

Large scale text processing in data science projects requires manipulation of textual data. The regular expressions processing is supported by many programming languages including Python. Python's standard library has `re` module for this purpose.

Since most of the functions defined in `re` module work with raw strings, let us first understand what the raw strings are.

### Raw Strings

Regular expressions use the backslash character (`\`) to indicate special forms or to allow special characters to be used without invoking their special meaning. Python on the other hand uses the same character as escape character. Hence Python uses the raw string notation.

A string become a raw string if it is prefixed with `r` or `R` before the quotation symbols. Hence `'Hello'` is a normal string were are `r'Hello'` is a raw string.

```
>>> normal="Hello"
>>> print (normal)
Hello
>>> raw=r"Hello"
>>> print (raw)
Hello
```

In normal circumstances, there is no difference between the two. However, when the escape character is embedded in the string, the normal string actually interprets the escape sequence, where as the raw string doesn't process the escape character.

```
>>> normal="Hello\nWorld"
>>> print (normal)
Hello
World
>>> raw=r"Hello\nWorld"
>>> print (raw)
Hello\nWorld
```

In the above example, when a normal string is printed the escape character '\n' is processed to introduce a newline. However because of the raw string operator 'r' the effect of escape character is not translated as per its meaning.

### Metacharacters

Most letters and characters will simply match themselves. However, some characters are special metacharacters, and don't match themselves. Meta characters are characters having a special meaning, similar to \* in wild card.

Here's a complete list of the metacharacters –

```
. ^ $ * + ? { } [] \ | ()
```

The square bracket symbols [ and ] indicate a set of characters that you wish to match. Characters can be listed individually, or as a range of characters separating them by a '-'.  
 Characters can be listed individually, or as a range of characters separating them by a '-'.

| Sr.No. | Metacharacters & Description                                                        |
|--------|-------------------------------------------------------------------------------------|
| 1      | [abc]<br>match any of the characters a, b, or c                                     |
| 2      | [a-c]<br>which uses a range to express the same set of characters.                  |
| 3      | [a-z]<br>match only lowercase letters.                                              |
| 4      | [0-9]<br>match only digits.                                                         |
| 5      | '^'<br>complements the character set in [].[^5] will match any character except'5'. |

'\' is an escaping metacharacter. When followed by various characters it forms various special sequences. If you need to match a [ or \, you can precede them with a backslash to remove their special meaning: \[ or \\\.

Predefined sets of characters represented by such special sequences beginning with '\' are listed below –

| Sr.No. | Metacharacters & Description                                                               |
|--------|--------------------------------------------------------------------------------------------|
| 1      | \d<br>Matches any decimal digit; this is equivalent to the class [0-9].                    |
| 2      | \D<br>Matches any non-digit character; this is equivalent to the class [^0-9].             |
| 3      | \s<br>Matches any whitespace character; this is equivalent to the class [\t\n\r\f\v].      |
| 4      | \S<br>Matches any non-whitespace character; this is equivalent to the class [^\t\n\r\f\v]. |

|    |                                                                                                                                                |
|----|------------------------------------------------------------------------------------------------------------------------------------------------|
| 5  | <code>\w</code><br>Matches any alphanumeric character; this is equivalent to the class <code>[a-zA-Z0-9_]</code> .                             |
| 6  | <code>\W</code><br>Matches any non-alphanumeric character. equivalent to the class <code>[^a-zA-Z0-9_]</code> .                                |
| 7  | <code>.</code><br>Matches with any single character except newline <code>"\n"</code> .                                                         |
| 8  | <code>?</code><br>match 0 or 1 occurrence of the pattern to its left                                                                           |
| 9  | <code>+</code><br>1 or more occurrences of the pattern to its left                                                                             |
| 10 | <code>*</code><br>0 or more occurrences of the pattern to its left                                                                             |
| 11 | <code>\b</code><br>boundary between word and non-word and <code>/B</code> is opposite of <code>/b</code>                                       |
| 12 | <code>[.]</code><br>Matches any single character in a square bracket and <code>[^.]</code> matches any single character not in square bracket. |
| 13 | <code>\</code><br>It is used for special meaning characters like <code>\.</code> to match a period or <code>\+</code> for plus sign.           |
| 14 | <code>{n,m}</code><br>Matches at least n and at most m occurrences of preceding                                                                |
| 15 | <code>a  b</code><br>Matches either a or b                                                                                                     |

Python's re module provides useful functions for finding a match, searching for a pattern, and substitute a matched string with other string etc.

### Various types of regular expressions

#### RegEx Functions

The re module in Python provides various functions that help search, match, and manipulate strings using regular expressions.

Below are main functions available in the re module:

| Function                  | Description                                                                                                |
|---------------------------|------------------------------------------------------------------------------------------------------------|
| <code>re.findall()</code> | finds and returns all matching occurrences in a list                                                       |
| <code>re.compile()</code> | Regular expressions are compiled into pattern objects                                                      |
| <code>re.split()</code>   | Split string by the occurrences of a character or a pattern.                                               |
| <code>re.sub()</code>     | Replaces all occurrences of a character or patter with a replacement string.                               |
| <code>resubn</code>       | It's similar to <code>re.sub()</code> method but it returns a tuple: (new_string, number_of_substitutions) |
| <code>re.escape()</code>  | Escapes special character                                                                                  |
| <code>re.search()</code>  | Searches for first occurrence of character or pattern                                                      |

Let's see the working of these RegEx functions with definition and examples:

**1. re.findall()**

Returns all non-overlapping matches of a pattern in the string as a list. It scans the string from left to right.

**Example:** This code uses regular expression `\d+` to find all sequences of one or more digits in the given string.

```
import re
string = """Hello my Number is 123456789 and
my friend's number is 987654321"""
```

```
regex = '\d+'
match = re.findall(regex, string)
print(match)
```

**Output**

```
['123456789', '987654321']
```

**2. re.compile()**

Compiles a regex into a pattern object, which can be reused for matching or substitutions.

**Example 1:** This pattern `[a-e]` matches all lowercase letters between 'a' and 'e', in the input string `"Aye, said Mr. Gibenson Stark"`. The output should be `['e', 'a', 'd', 'b', 'e']`, which are matching characters.

```
import re
p = re.compile('[a-e]')
print(p.findall("Aye, said Mr. Gibenson Stark"))
```

**Output**

```
['e', 'a', 'd', 'b', 'e', 'a']
```

**Explanation:**

- First occurrence is 'e' in "Aye" and not 'A', as it is Case Sensitive.
- Next Occurrence is 'a' in "said", then 'd' in "said", followed by 'b' and 'e' in "Gibenson", the Last 'a' matches with "Stark".
- Metacharacter backslash '\' has a very important role as it signals various sequences. If the backslash is to be used without its special meaning as metacharacter, use '\\'.

**Example 2:** The code uses regular expressions to find and list all single digits and sequences of digits in the given input strings. It finds single digits with `\d` and sequences of digits with `\d+`.

```
import re
p = re.compile('\d')
print(p.findall("I went to him at 11 A.M. on 4th July 1886"))
```

```
p = re.compile('\d+')
print(p.findall("I went to him at 11 A.M. on 4th July 1886"))
```

**Output**

```
['1', '1', '4', '1', '8', '8', '6']
['11', '4', '1886']
```

**Example 3:** Word and non-word characters

- `\w` matches a single word character.
- `\w+` matches a group of word characters.
- `\W` matches non-word characters.

```
import re
```

```
p = re.compile('\w')
print(p.findall("He said * in some_lang."))
```

```
p = re.compile('\w+')
print(p.findall("I went to him at 11 A.M., he \
said *** in some_language."))
```

```
p = re.compile('\W')
print(p.findall("he said *** in some_language."))
```

### Output

```
['H', 'e', 's', 'a', 'i', 'd', 'i', 'n', 's', 'o', 'm', 'e', '_', 'l', 'a', 'n', 'g']
['I', 'went', 'to', 'him', 'at', '11', 'A', 'M', 'he', 'said', 'in', 'some_language']
[' ', ' ', '*', '*', '*', ' ', ' ', '!']
```

**Example 4:** The regular expression pattern `'ab*'` to find and list all occurrences of `'ab'` followed by zero or more `'b'` characters. In the input string `"ababbaabbb"`. It returns the following list of matches: `['ab', 'abb', 'abbb']`.

```
import re
p = re.compile('ab*')
print(p.findall("ababbaabbb"))
```

### Output

```
['ab', 'abb', 'a', 'abbb']
```

### Explanation:

- Output `'ab'`, is valid because of single `'a'` accompanied by single `'b'`.
- Output `'abb'`, is valid because of single `'a'` accompanied by 2 `'b'`.
- Output `'a'`, is valid because of single `'a'` accompanied by 0 `'b'`.
- Output `'abbb'`, is valid because of single `'a'` accompanied by 3 `'b'`.

## 3. re.split()

Splits a string wherever the pattern matches. The remaining characters are returned as list elements.

### Syntax:

```
re.split(pattern, string, maxsplit=0, flags=0)
```

- **pattern:** Regular expression to match split points.
- **string:** The input string to split.

- **maxsplit (optional):** Limits the number of splits. Default is 0 (no limit).
- **flags (optional):** Apply regex flags like re.IGNORECASE.

### Example 1: Splitting by non-word characters or digits

This example demonstrates how to split a string using different patterns like non-word characters (\W+), apostrophes, and digits (\d+).

```
from re import split
```

```
print(split('\W+', 'Words, words , Words'))
print(split('\W+', "Word's words Words"))
print(split('\W+', 'On 12th Jan 2016, at 11:02 AM'))
print(split('\d+', 'On 12th Jan 2016, at 11:02 AM'))
```

### Output

```
['Words', 'words', 'Words']
['Word', 's', 'words', 'Words']
['On', '12th', 'Jan', '2016', 'at', '11', '02', 'AM']
['On ', 'th Jan ', ' ', 'at ', ':', 'AM']
```

### Example 2: Using maxsplit and flags

This example shows how to limit the number of splits using maxsplit, and how flags can control case sensitivity.

```
import re
print(re.split('\d+', 'On 12th Jan 2016, at 11:02 AM', 1))
print(re.split('[a-f]+', 'Aey, Boy oh boy, come here', flags=re.IGNORECASE))
print(re.split('[a-f]+', 'Aey, Boy oh boy, come here'))
```

### Output

```
['On ', 'th Jan 2016, at 11:02 AM']
[' ', 'y, ', 'oy oh ', 'oy, ', 'om', ' h', 'r', '']
['A', 'y, Boy oh ', 'oy, ', 'om', ' h', 'r', '']
```

**Note:** In the second and third cases of the above, [a-f]+ splits the string using any combination of lowercase letters from 'a' to 'f'. The re.IGNORECASE flag includes uppercase letters in the match.

## 4. re.sub()

The re.sub() function replaces all occurrences of a pattern in a string with a replacement string.

### Syntax:

```
re.sub(pattern, repl, string, count=0, flags=0)
```

- **pattern:** The regex pattern to search for.
- **repl:** The string to replace matches with.
- **string:** The input string to process.
- **count (optional):** Maximum number of substitutions (default is 0, which means replace all).
- **flags (optional):** Regex flags like re.IGNORECASE.

**Example 1:** The following examples show different ways to replace the pattern 'ub' with '~\*', using various flags and count values.

```
import re

Case-insensitive replacement of all 'ub'
print(re.sub('ub', '~*', 'Subject has Uber booked already', flags=re.IGNORECASE))

Case-sensitive replacement of all 'ub'
print(re.sub('ub', '~*', 'Subject has Uber booked already'))

Replace only the first 'ub', case-insensitive
print(re.sub('ub', '~*', 'Subject has Uber booked already', count=1, flags=re.IGNORECASE))

Replace "AND" with "&", ignoring case
print(re.sub(r'\sAND\s', ' & ', 'Baked Beans And Spam', flags=re.IGNORECASE))
```

### Output

```
S~*ject has ~*er booked already
S~*ject has Uber booked already
S~*ject has Uber booked already
Baked Beans & Spam
```

### 5. re.subn()

re.subn() function works just like **re.sub()**, but instead of returning only the modified string, it returns a tuple: (**new\_string**, **number\_of\_substitutions**)

#### Syntax:

```
re.subn(pattern, repl, string, count=0, flags=0)
```

#### Example: Substitution with count

This example shows how re.subn() gives both the replaced string and the number of times replacements were made.

```
import re

Case-sensitive replacement
print(re.subn('ub', '~*', 'Subject has Uber booked already'))

Case-insensitive replacement
t = re.subn('ub', '~*', 'Subject has Uber booked already', flags=re.IGNORECASE)
print(t)
print(len(t)) # tuple length
print(t[0]) # modified string
```

### Output

```
('S~*ject has Uber booked already', 1)
('S~*ject has ~*er booked already', 2)
```



2

S~\*ject has ~\*er booked already

## 6. re.escape()

re.escape() function adds a backslash (\) before all special characters in a string. This is useful when you want to match a string literally, including any characters that have special meaning in regex (like ., \*, [, ], etc.).

### Syntax:

```
re.escape(string)
```

### Example: Escaping special characters

This example shows how re.escape() treats spaces, brackets, dashes, and tabs as literal characters.

```
import re
print(re.escape("This is Awesome even 1 AM"))
print(re.escape("I Asked what is this [a-9], he said \t ^WoW"))
```

### Output

```
This\ is\ Awesome\ even\ 1\ AM
I\ Asked\ what\ is\ this\ \[a\~9\],\ he\ said\ \ \ \^WoW
```

## 7. re.search()

The re.search() function searches for the first occurrence of a pattern in a string. It returns a **match object** if found, otherwise **None**.

**Note:** Use it when you want to check if a pattern exists or extract the first match.

### Example: Search and extract values

This example searches for a date pattern with a month name (letters) followed by a day (digits) in a sentence.

```
import re
regex = r"([a-zA-Z]+) (\d+)"
match = re.search(regex, "I was born on June 24")
```

if match:

```
 print("Match at index %s, %s" % (match.start(), match.end()))
 print("Full match:", match.group(0))
 print("Month:", match.group(1))
 print("Day:", match.group(2))
```

else:

```
 print("The regex pattern does not match.")
```

### Output

```
Match at index 14, 21
Full match: June 24
Month: June
Day: 24
```

## Metacharacters

**Metacharacters** are considered as the building blocks of regular expressions. Regular expressions are patterns used to match character combinations in the strings. Metacharacter has special meaning in finding patterns and are mostly used to define the search criteria and any text manipulations.

Some of the mostly used metacharacters along with their uses are as follows:

| Meta Character | Description                        | Example                               |
|----------------|------------------------------------|---------------------------------------|
| \d             | whole numbers( 0-9 )(single digit) | \d = 7, \d\d=77                       |
| \w             | alphanumeric character             | \w\w\w\w = geek<br>\w\w\w != geek     |
| *              | 0 or more characters               | s* = _,s,ss,sss,ssss.....             |
| +              | 1 or more characters               | s+ = s,ss,sss,ssss.....               |
| ?              | 0 or 1 character                   | s? = _ or s                           |
| {m}            | occurs "m" times                   | sd{3} = sddd                          |
| {m,n}          | min "m" and max "n" times          | sd{2,3}=sdd or sddd                   |
| \W             | symbols                            | \W = %                                |
| [a-z] or [0-9] | character set                      | geek[sy] = geeky<br>geek[sy] != geeki |

Regular expressions can be built by metacharacters, and patterns can be processed using a library in Python for **Regular Expressions** known as "**re**".

import re # used to import regular expressions

The inbuilt library can be used to **compile patterns, find patterns, etc.**

**Example:** In the below code, we will generate all the patterns based on the given regular expression

import re

'''

Meta characters -

\* - 0 or more

+ - 1 or more

? - 0 or 1

{m} - m times

{m,n} - min m and max n

'''

test\_phrase = 'sdds..sssd...sddsddd...dsds...dsssss...sddd'

test\_patterns = [r'sd\*', # s followed by zero or more d's

          r'sd+', # s followed by one or more d's

          r'sd?', # s followed by zero or one d's

          r'sd{3}', # s followed by three d's

          r'sd{2,3}', # s followed by two to three d's

]

```
def multi_re_find(test_patterns, test_phrase):
 for pattern in test_patterns:
 compiledPattern = re.compile(pattern)
 print('finding {} in test_phrase'.format(pattern))
 print(re.findall(compiledPattern, test_phrase))
```

```
multi_re_find(test_patterns, test_phrase)
```

### Output:

```
finding sd* in test_phrase ['sdd', 'sd', 's', 's', 'sddd', 'sddd', 'sddd', 'sd', 's', 's', 's', 's', 's', 's', 's', 's']
finding sd+ in test_phrase ['sdd', 'sd', 'sddd', 'sddd', 'sddd', 'sd', 'sddd'] finding sd? in
test_phrase ['sd', 'sd', 's', 's', 'sd', 'sd', 'sd', 'sd', 's', 's', 's', 's', 's', 's', 'sd'] finding sd{3} in test_phrase
['sddd', 'sddd', 'sddd', 'sddd'] finding sd{2,3} in test_phrase ['sdd', 'sddd', 'sddd', 'sddd', 'sddd']
```

### Using match function.

Introduced in Python 3.10, the **match case** statement offers a powerful mechanism for pattern matching in Python. It allows us to perform more expressive and readable conditional checks. Unlike traditional if-elif-else chains, which can become unwieldy with complex conditions, the match-case statement provides a more elegant and flexible solution.

```
def check_number(x):
 match x:
 case 10:
 print("It's 10")
 case 20:
 print("It's 20")
 case _:
 print("It's neither 10 nor 20")
```

```
check_number(10)
check_number(30)
```

### Explanation:

- In this example, the function `check_number(x)` uses a match-case statement to compare the value of `x` to the constants 10 and 20.
- If `x` equals 10, it prints "It's 10". If `x` equals 20, it prints "It's 20".
- If neither condition is met, the wildcard `_` matches any value, leading to the message "It's neither 10 nor 20".

Let's take a look at python match case statement in detail:

### Python Match Case Statement Syntax

The match-case syntax is based on structural pattern matching, which enables matching against data structures like sequences, mappings and even classes, providing more granularity and flexibility in handling various conditions. This feature is particularly useful when dealing with different types of data in a clear and organized manner.

```

match subject:
case pattern1:
Code block if pattern1 matches
case pattern2:
Code block if pattern2 matches
case _:
Default case (wildcard) if no other pattern matches
 • match subject: The value (or variable) to match against.
 • case pattern: A pattern to match the subject.
 • _ (Wildcard): A default catch-all pattern, similar to a "default" in other languages'
 switch statements.

```

Now, let us see a few examples to know how the match case statement works in Python.

The power of the match-case statement lies in its ability to match a variety of data types, including constants, sequences, mappings and custom classes. Let's explore how to use match-case with different data types.

### Match Case Statements with Constants

Matching constants is one of the simplest uses of the match-case statement. It checks if a variable matches specific constant values and allows for different actions based on the matched constant.

```

def greet(person):
 match person:
 case "A":
 print("Hello, A!")
 case "B":
 print("Hello, B!")
 case _:
 print("Hello, stranger!")

```

```
greet("A")
```

```
greet("B")
```

#### Explanation:

- The function greet(person) checks the value of person using the match-case statement.
- It specifically matches "A" and "B", printing a personalized greeting.
- If the value of person doesn't match any of the constants, the wildcard \_ matches any value and prints "Hello, stranger!".

### Match Case Statement with OR Operator

The match-case statement can also be used with logical operators like or to combine multiple patterns. This allows for a more flexible matching system where you can group patterns and check for a match against any of them.

```

def num_check(x):
 match x:
 case 10 | 20 | 30: # Matches 10, 20, or 30

```

```

 print(f'Matched: {x}')
case _:
 print("No match found")

```

```

num_check(10)
num_check(20)
num_check(25)

```

### Output

```

Matched: 10
Matched: 20
No match found

```

### Explanation:

- The pattern `10 | 20 | 30` uses the or operator to match any of these three values. If `x` equals 10, 20 or 30, the case will execute.
- If `x` doesn't match any of the values, the wildcard `_` catches it and prints "No match found".

### Match Case Statement with Python If Condition

We can also add an if condition after a case to create more granular control over the matching process. This allows us to add additional checks within a case.

```

def num_check(x):
 match x:
 case 10 if x % 2 == 0: # Match 10 only if it's even
 print("Matched 10 and it's even!")
 case 10:
 print("Matched 10, but it's not even.")
 case _:
 print("No match found")

```

```

num_check(10)
num_check(15)

```

### Explanation:

- The first case matches 10 but only if `x % 2 == 0`. If this condition is met, it prints a specific message.
- The second case matches 10 without the condition, ensuring that a fallback message is shown if the first case isn't executed.

### Match Case Statement on Sequences

The match-case statement is particularly powerful when working with sequences such as lists or tuples. We can match individual elements of a sequence or even match the structure of the sequence itself.

```

def process(data):
 match data:

```

```

case [x, y]:
 # A list with two elements
 print(f"Two-element list: {x}, {y}")
case [x, y, z]:
 # A list with three elements
 print(f"Three-element list: {x}, {y}, {z}")
case _:
 print("Unknown data format")

```

```

process([1, 2])
process([1, 2, 3])
process([1, 2, 3, 4])

```

**Explanation:**

- The function `process_data(data)` matches the structure of the list data.
- The first case matches if the list has exactly two elements, binding the first and second elements to `x` and `y` respectively.
- The second case matches if the list has exactly three elements, binding them to `x`, `y` and `z`.
- If the list does not match either pattern, the wildcard `_` is used to print "Unknown data format".

**Match Case Statement on Mappings (Dictionaries)**

A mapping is another common data type in Python and match-case can be used to match against dictionaries, checking for specific keys and values.

```
def person(person):
```

```
 match person:
```

```

 # Dictionary with name and age keys
 case {"name": name, "age": age}:
 print(f"Name: {name}, Age: {age}")

```

```

 # Dictionary with only name key
 case {"name": name}:
 print(f"Name: {name}")
 case _:
 print("Unknown format")

```

```

person({"name": "Alice", "age": 25})
person({"name": "Bob"})
person({"city": "New York"})

```

Explanation:

- The function `person(person)` matches the structure of a dictionary person.
- The first case matches a dictionary that contains both "name" and "age" keys, binding their corresponding values to the variables `name` and `age`.

- The second case matches a dictionary with just the "name" key.
- The wildcard `_` is used to catch any other dictionary structure that doesn't match these patterns, printing "Unknown format".

### Match Case Statement with Python Class

One of the most powerful features of match-case is the ability to match against classes. We can match instances of a class and even extract specific attributes of an object for further handling.

**class Shape:**

**pass**

**class Circle(Shape):**

```
def __init__(self, radius):
 self.radius = radius
```

**class Rectangle(Shape):**

```
def __init__(self, width, height):
 self.width = width
 self.height = height
```

**def check\_shape(shape):**

**match** shape:

**# Match Circle and extract the radius**

```
case Circle(radius):
 print(f"circle radius {radius}.")
```

**# Match Rectangle and extract width and height**

```
case Rectangle(width, height):
 print(f"Rectangle width {width} and height {height}.")
```

**# Default case for any other object**

```
case _:
 print("This is an unknown shape.")
```

**# Create objects of Circle and Rectangle**

```
circle = Circle(10)
```

```
rectangle = Rectangle(4, 6)
```

**# Test with different shapes**

```
check_shape(circle)
```

```
check_shape(rectangle)
```

**Explanation:**

- We have a Shape class and two types of shapes: Circle and Rectangle.

- The `check_shape` function checks if the shape is a Circle or Rectangle and prints their details.
- If the shape isn't one of these, it says "unknown shape."

## Special Sequences

The **special sequence represents the basic predefined character classes**, which have a unique meaning. Each special sequence makes specific common patterns more comfortable to use.

### Special sequence

The **special sequence represents the basic predefined character classes**, which have a unique meaning. Each special sequence makes specific common patterns more comfortable to use.

For example, you can use `\d` sequence as a simplified definition for character class `[0-9]`, which means match any digit from 0 to 9.

Let's see the list of regex special sequences and their meaning. The special sequences consist of `'\'` (backslash) and a character from the table below.

| Special Sequence | Meaning                                                                                                                                                                                                                                                                                                                                                                      |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>\A</code>  | Matches pattern only at the start of the string                                                                                                                                                                                                                                                                                                                              |
| <code>\Z</code>  | Matches pattern only at the end of the string                                                                                                                                                                                                                                                                                                                                |
| <code>\d</code>  | Matches any digit.<br>Short for character classes <code>[0-9]</code>                                                                                                                                                                                                                                                                                                         |
| <code>\D</code>  | Matches any non-digit.<br>short for <code>[^0-9]</code>                                                                                                                                                                                                                                                                                                                      |
| <code>\s</code>  | Matches any whitespace character.<br>short for character class <code>[\t\n\r\f]</code>                                                                                                                                                                                                                                                                                       |
| <code>\S</code>  | Matches any non-whitespace character.<br>short for <code>[^\t\n\r\f]</code>                                                                                                                                                                                                                                                                                                  |
| <code>\w</code>  | Matches any alphanumeric character.<br>short for character class <code>[a-zA-Z_0-9]</code>                                                                                                                                                                                                                                                                                   |
| <code>\W</code>  | Matches any non-alphanumeric character.<br>short for <code>[^a-zA-Z_0-9]</code>                                                                                                                                                                                                                                                                                              |
| <code>\b</code>  | Matches the empty string, but only at the beginning or end of a word.<br>Matches a word boundary where a word character is <code>[a-zA-Z0-9_]</code> .<br>For example, <code>'\bJessa\b'</code> matches <code>'Jessa'</code> , <code>'Jessa.'</code> , <code>'(Jessa)'</code> , <code>'Jessa Emma Kelly'</code> but not <code>'JessaKelly'</code> or <code>'Jessa5'</code> . |
| <code>\B</code>  | Opposite of a <code>\b</code> . Matches the empty string, but only when it is not at the beginning or end of a word                                                                                                                                                                                                                                                          |



## Sets

A Set is used to store a collection of items with the following properties.

- No duplicate elements. If try to insert the same item again, it overwrites previous one.
- An unordered collection. When we access all items, they are accessed without any specific order and we cannot access items using indexes as we do in lists.
- Internally use hashing that makes set efficient for search, insert and delete operations. It gives a major advantage over a list for problems with these operations.
- Mutable, meaning we can add or remove elements after their creation, the individual elements within the set cannot be changed directly.

**Example:** This example shows how to create a set with some numbers and check its type.

```
s = {10, 50, 20}
print(s)
print(type(s))
```

### Output

```
{10, 50, 20}
<class 'set'>
```

**Note:** There is no specific order for set elements to be printed

### Type Casting

set() method is used to convert other data types, such as lists or tuples, into sets.

```
typecasting list to set
s = set(["a", "b", "c"])
print(s)
```

```
Adding element to the set
s.add("d")
print(s)
```

### Output

```
{'c', 'b', 'a'}
{'d', 'c', 'b', 'a'}
```

### Check unique and Immutable

Sets cannot have duplicate values. While you cannot modify the individual elements directly, you can still add or remove elements from the set.

```
a set cannot have duplicate values
s = {"Geeks", "for", "Geeks"}
print(s)
```

```
values of a set cannot be changed
s[1] = "Hello"
```

```
print(s)
```

### Output

```
{'Geeks', 'for'}
```

TypeError: 'set' object does not support item assignment

### Explanation:

- `s = {"Geeks", "for", "Geeks"}` duplicates are removed, only unique values remain.
- `s[1] = "Hello"` Error, set elements cannot be changed directly. Only `add()` or `remove()` can modify a set.

### Heterogeneous Element

Sets can store heterogeneous elements in it, i.e., a set can store a mixture of string, integer, boolean, etc datatypes.

```
s = {"Geeks", "for", 10, 52.7, True}
```

```
print(s)
```

### Output

```
{True, 52.7, 'Geeks', 10, 'for'}
```

### Frozen Sets

Frozenset is an immutable version of a set. Its elements cannot be changed after creation, but you can perform operations like union, intersection and difference. Use `frozenset()` to create one.

```
Normal set (mutable)
```

```
s = set(["a", "b", "c"])
```

```
print("Normal Set:", s)
```

```
Frozen set (immutable)
```

```
fs = frozenset(["e", "f", "g"])
```

```
print("Frozen Set:", fs)
```

### Output

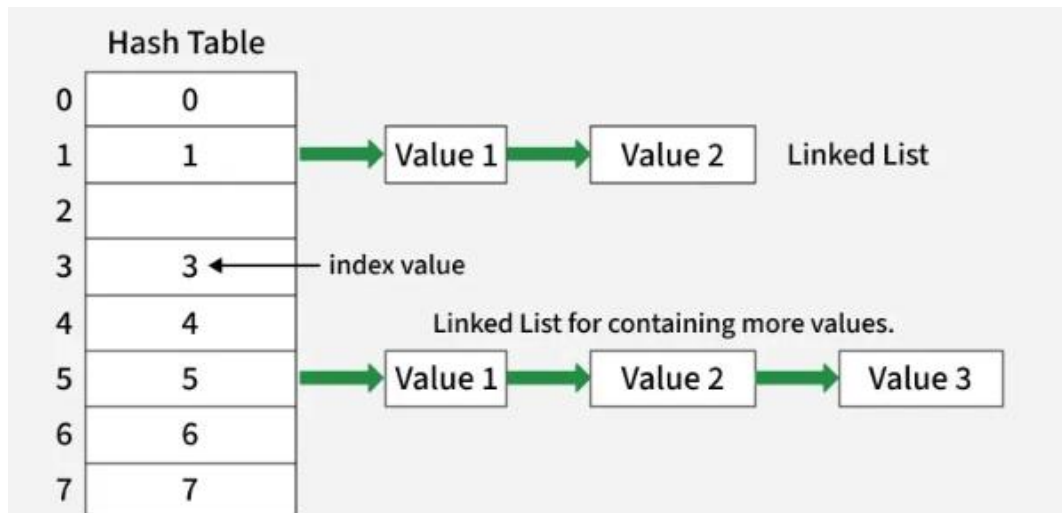
```
('Normal Set:', set(['a', 'c', 'b']))
```

```
('Frozen Set:', frozenset(['e', 'g', 'f']))
```

**Note:** Frozensets are immutable, so methods like `add()` or `remove()` cannot be used. They are also hashable, which allows them to be used as dictionary keys.

### Internal working of Set

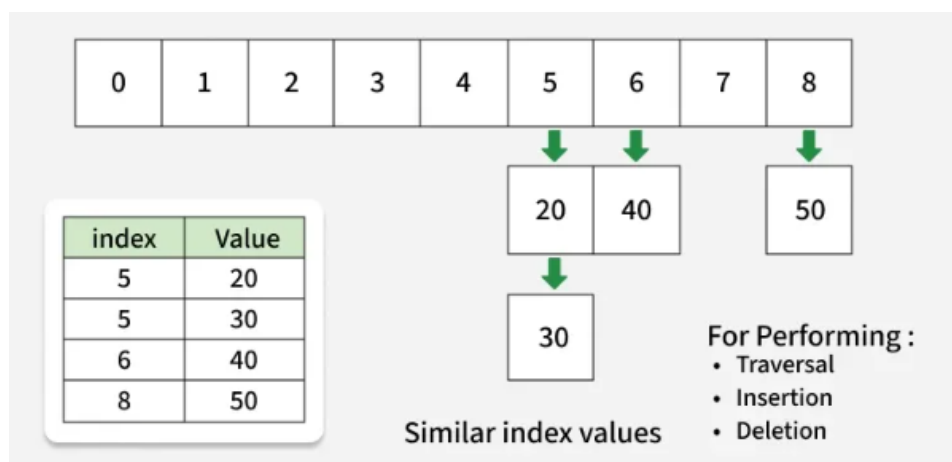
This is based on a data structure known as a hash table. If Multiple values are present at the same index position, then the value is appended to that index position, to form a Linked List. Sets are implemented using a dictionary with dummy variables, where key beings the members set with greater optimizations to the time complexity. Below diagram shows how Python internally manages collisions in sets using linked lists for efficient element storage and retrieval.



### Internal working of Set

- Each index in the hash table represents a possible hash value where elements are stored based on their computed hash key.
- When two elements map to the same index, they are linked together using a linked list, forming a chain at that position.
- This chaining mechanism helps efficiently handle collisions, allowing multiple elements to exist at the same hash index without overwriting each other.

Sets with Numerous operations on a single HashTable:



### Operations on a Single HashTable

The hash table stores elements based on their computed index values for example, values 20 and 30 are placed at index 5, while 40 goes to index 6 and 50 to index 8. When multiple values share the same index, they are linked together, forming a chain (as seen at index 5).

During traversal, insertion or deletion, the algorithm moves through these linked elements at each index to access or modify data efficiently.

### Methods for Sets

**1. Adding elements to Sets:** add() function is used to insert new elements into a set. It automatically ignores duplicates.

```
s = {"a", "b", "c"}
```

```
s.add("d")
print(s)
```

**Output**

```
{'c', 'd', 'a', 'b'}
```

**2. Union of Sets:** union() function combines two sets and returns a new set with all unique elements.

```
a = {"x", "y"}
b = {"y", "z"}
u = a.union(b)
print(u)
```

**Output**

```
{'z', 'y', 'x'}
```

**3. Intersection of Sets:** intersection() function returns a new set containing elements that are common to both sets.

```
a = {1, 2, 3}
b = {2, 3, 4}
i = a.intersection(b)
print(i)
```

**Output**

```
{2, 3}
```

**4. Difference of Sets:** difference() function returns a set containing elements that are in the first set but not in the second.

```
a = {1, 2, 3}
b = {2, 3, 4}
d = a.difference(b)
print(d)
```

**Output**

```
{1}
```

**5. Clearing a Set:** clear() function removes all elements from a set, leaving it empty.

```
s = {1, 2, 3}
s.clear()
print(s)
```

# CHAPTER 5

## FILES, MODULES AND PACKAGES

---

### Files and Exception

File handling refers to the process of performing operations on a file, such as creating, opening, reading, writing and closing it through a programming interface. It involves managing the data flow between the program and the file system on the storage device, ensuring that data is handled safely and efficiently.

#### Need for File Handling

- Store data permanently, even after the program ends.
- Access external files like .txt, .csv, .json, etc.
- Process large files efficiently without using much memory.
- Automate tasks like reading configs or saving outputs.

#### Opening a File

To open a file, we can use `open()` function, which requires file-path and mode as arguments.

**Note:** We will use a sample file named `geek.txt` for all examples in this article. To download click [here](#)

#### Syntax:

```
file = open('filename.txt', 'mode')
```

- **filename.txt:** name (or path) of the file to be opened.
- **mode:** mode in which you want to open the file (read, write, append, etc.).

**Note:** If you don't specify the mode, Python uses **'r'** (read mode) by default.

#### Basic Example: Opening a File

```
f = open("geek.txt", "r")
print(f)
```

**Explanation:** code opens file **geek.txt** in read mode. If the file exists, it returns a file object connected to that file; if the file does not exist, Python raises a `FileNotFoundError`.

#### Closing a File

`file.close()` method closes the file and releases the system resources. If the file was opened in write or append mode, closing ensures that all changes are properly saved.

```
file = open("geek.txt", "r")
Perform file operations
file.close()
```

We will also see later how closing can be handled automatically using the `with` statement and how to ensure files close properly using exception handling.

### Checking File Properties

Once the file is open, we can check some of its properties:

```
f = open("geek.txt", "r")
print("Filename:", f.name)
print("Mode:", f.mode)
print("Is Closed?", f.closed)
```

```
f.close()
print("Is Closed?", f.closed)
```

#### Output

```
Filename: geek.txt
Mode: r
Is Closed? False
Is Closed? True
```

#### Explanation:

- **f.name:** Returns the name of the file that was opened (in this case, "geek.txt").
- **f.mode:** Tells us the mode in which the file was opened. Here, it's 'r' which means read mode.
- **f.closed:** Returns a boolean value- False when file is currently open otherwise True.

### Reading a File

Reading a file can be achieved by **file.read()** which reads the entire content of the file. After reading, it's good practice to close the file to free up system resources.

**Example:** Reading a File in Read Mode (r)

```
file = open("geek.txt", "r")
content = file.read()
print(content)
file.close()
```

#### Output

```
Hello world
GeeksforGeeks
123 456
```

### Writing a File

Writing to a file is done using the mode "w". This creates a new file if it doesn't exist, or overwrites the existing file if it does. The write() method is used to add content. After writing, make sure to close the file.

**Example:** Writing to a file (overwrites if file exists)

```
with open("geek.txt", "w") as file:
 file.write("Hello, Python!\n")
 file.write("File handling is easy with Python.")
```

```
print("File written successfully")
```

### Output

Hello, Python!

File handling is easy with Python.

### Explanation:

- "w" mode opens the file for writing (overwrites existing content if the file already exists).
- write() method adds new text to the file.
- When using with, the file closes automatically at the end of the block.

### Using with Statement

Instead of manually opening and closing the file, you can use the with statement, which automatically handles closing. This reduces the risk of file corruption and resource leakage.

**Example:** Let's assume we have a file named **geek.txt** that contains text **"Hello, World!"**.

with open("geek.txt", "r") as file:

```
 content = file.read()
 print(content)
```

### Output

Hello, World!

### Handling Exceptions When Closing a File

It's important to handle exceptions to ensure that files are closed properly, even if an error occurs during file operations. Here, the finally block ensures the file is closed even if an error occurs.

try:

```
 file = open("geek.txt", "r")
 content = file.read()
 print(content)
```

except FileNotFoundError as e:

```
 print("Error:", e)
```

finally:

```
 file.close()
```

### Output

Hello, World!

### Python Exception Handling

Python Exception Handling allows a program to gracefully handle unexpected events (like invalid input or missing files) without crashing. Instead of terminating abruptly, Python lets you detect the problem, respond to it, and continue execution when possible.

Below is a basic example demonstrating how to catch an exception and handle it gracefully:

```
n = 10
```

try:

```
 res = n / 0
```

except ZeroDivisionError:

```
print("Can't be divided by zero!")
```

### Output

Can't be divided by zero!

**Explanation:** Dividing a number by 0 raises a `ZeroDivisionError`. The try block contains code that may fail and except block catches the error, printing a safe message instead of stopping the program.

### Difference Between Errors and Exceptions

Errors and exceptions are both issues in a program, but they differ in severity and handling.

- **Error:** Issues in the program logic such as `SyntaxError`, etc. It occurs at compile time.
- **Exception:** Problems that occur at runtime and can be managed using exception handling (e.g., invalid input, missing files).

**Example:** This example shows the difference between a syntax error and a runtime exception.

# Syntax Error (Error)

```
print("Hello world" # Missing closing parenthesis
```

# ZeroDivisionError (Exception)

```
n = 10
```

```
res = n / 0
```

**Explanation:** A syntax error stops the code from running at all, while an exception like `ZeroDivisionError` occurs during execution and can be caught with exception handling.

### Syntax of Exception Handling

Python provides four main keywords for handling exceptions: `try`, `except`, `else` and `finally` each plays a unique role. Let's see syntax:

```
try:
```

```
Code
```

```
except SomeException:
```

```
Code
```

```
else:
```

```
Code
```

```
finally:
```

```
Code
```

- **try:** Runs the risky code that might cause an error.
- **except:** Catches and handles the error if one occurs.
- **else:** Executes only if no exception occurs in try.
- **finally:** Runs regardless of what happens useful for cleanup tasks like closing files.

**Example:** This code attempts division and handles errors gracefully using try-except-else-finally.

```
try:
```

```
 n = 0
```

```
 res = 100 / n
```



```

except ZeroDivisionError:
 print("You can't divide by zero!")

except ValueError:
 print("Enter a valid number!")

else:
 print("Result is", res)

finally:
 print("Execution complete.")

```

### Output

You can't divide by zero!

Execution complete.

**Explanation:** try block attempts division, except blocks catch specific errors, else block executes only if no errors occur, while finally block always runs, signaling end of execution. Please refer Python Built-in Exceptions for some common exceptions.

### Catching Exceptions

We can handle errors more efficiently by specifying the types of exceptions we expect. This can make code both safer and easier to debug.

#### 1. Catching Specific Exceptions

Catching specific exceptions makes code to respond to different exception types differently. It precisely makes your code safer and easier to debug. It avoids masking bugs by only reacting to the exact problems you expect.

**Example:** This code handles ValueError and ZeroDivisionError with different messages.

```

try:
 # This will cause ValueError
 x = int("str")
 inv = 1 / x # Inverse calculation

except ValueError:
 print("Not Valid!")

except ZeroDivisionError:
 print("Zero has no inverse!")

```

### Output

Not Valid!

**Explanation:** A ValueError occurs because "str" cannot be converted to an integer. If conversion had succeeded but x were 0, a ZeroDivisionError would have been caught instead.

## 2. Catching Multiple Exceptions

We can catch multiple exceptions in a single block if we need to handle them in the same way or we can separate them if different types of exceptions require different handling.

**Example:** This code attempts to convert list elements and handles `ValueError`, `TypeError` and `IndexError`.

```
a = ["10", "twenty", 30]
try:
 # 'twenty' cannot be converted to int
 total = int(a[0]) + int(a[1])

except (ValueError, TypeError) as e:
 print("Error", e)

except IndexError:
 print("Index out of range.")
```

### Output

Error invalid literal for int() with base 10: 'twenty'

**Explanation:** The `ValueError` is raised when trying to convert "twenty" to an integer. A `TypeError` could occur if incompatible types were used, while `IndexError` would trigger if the list index was out of range.

## 3. Catch-All Handlers and Their Risks

Catch-all handler is used to call to catch any exception (similar to `else` statement). Use only `except` keyword to define it:

**Example:** This code tries dividing a string by a number, which causes a `TypeError`.

```
try:
 # Risky operation: dividing string by number
 res = "100" / 20

except ArithmeticError:
 print("Arithmetic problem.")

except:
 print("Something went wrong!")
```

### Output

Something went wrong!

**Explanation:** A `TypeError` occurs because you can't divide a string by a number. The bare `except` catches it, but this can make debugging harder since the actual error type is hidden. Use bare `except` only as a net.

## Raise an Exception

We raise an exception in Python using the raise keyword followed by an instance of the exception class that we want to trigger. We can choose from built-in exceptions or define our own custom exceptions by inheriting from Python's built-in Exception class.

### Basic Syntax:

```
raise ExceptionType("Error message")
```

**Example:** This code raises a ValueError if an invalid age is given.

```
def set(age):
 if age < 0:
 raise ValueError("Age cannot be negative.")
 print(f"Age set to {age}")
```

```
try:
 set(-5)
except ValueError as e:
 print(e)
```

### Output

Age cannot be negative.

## File Types

When working with files in Python, the file mode tells Python what kind of operations (read, write, etc.) you want to perform on the file. You specify the mode as the second argument to the open() function.

### Different File Mode in Python

Below are the different types of file modes in Python along with their description:

| Mode  | Description                                                        |
|-------|--------------------------------------------------------------------|
| 'r'   | Read-only. Raises I/O error if file doesn't exist.                 |
| 'r+'  | Read and write. Raises I/O error if the file does not exist.       |
| 'w'   | Write-only. Overwrites file if it exists, else creates a new one.  |
| 'w+'  | Read and write. Overwrites file or creates new one.                |
| 'a'   | Append-only. Adds data to end. Creates file if it doesn't exist.   |
| 'a+'  | Read and append. Pointer at end. Creates file if it doesn't exist. |
| 'rb'  | Read in binary mode. File must exist.                              |
| 'rb+' | Read and write in binary mode. File must exist.                    |
| 'wb'  | Write in binary. Overwrites or creates new.                        |
| 'wb+' | Read and write in binary. Overwrites or creates new.               |
| 'ab'  | Append in binary. Creates file if not exist.                       |
| 'ab+' | Read and append in binary. Creates file if it does not exist.      |

### Examples of Common Modes

Let's say we have a file named **example.txt** with the content:

Hello Geeks

We'll use this file for the examples below.

### 1. Read Mode ('r')

This mode allows you to open a file for reading only. If the file does not exist, it will raise a `FileNotFoundError`.

**Example:** In this example, a file named 'example.txt' is opened in read mode ('r'), and its content is read and stored in the variable 'content' using a 'with' statement, ensuring proper resource management by automatically closing the file after use.

**with** open('example.txt', 'r') as file:

    content = file.read()

**Output:**

Hello Geeks

### 2. Write Mode ('w')

Opens the file for writing only. If the file exists, its content is deleted. If not, a new file is created.

**Example:** In this example, a file named 'example.txt' is opened in write mode ('w'), and the string 'Hello, world!' is written into the file.

**with** open('example.txt', 'w') as file:

    file.write('Hello, world!')

**Output** (file content after writing):

Hello, world!

**Note:** If you were to open the file "**example.txt**" after running this code, you would find that it contains the text "**Hello, world!**" as the previous content "Hello Geeks" will be deleted.

### 3. Append Mode ('a')

Opens the file to add content at the end without deleting existing data. If the file doesn't exist, it creates a new one.

**Example:** In this example, a file named 'example.txt' is opened in append mode ('a'), and the string '\n This is a new line.' is written to the end of the file.

**with** open('example.txt', 'a') as file:

    file.write('\nThis is a new line.')

**Output:**

Hello, World!

This is a new line

The code will then write the string "**\nThis is a new line.**" to the file, appending it to the existing content or creating a new line if the file is empty.

### 4. Binary Mode ('b')

Used for non-text files like images or audio. Always combined with 'r', 'w', or 'a'.

**Example:** In this example, a file named 'image.png' is opened in binary read mode ('rb'). The binary data is read from the file using the 'read()' method and stored in the variable 'data'.

**with** open('image.png', 'rb') as file:

    data = file.read()

```
Process the binary data
```

### 5. Read and Write Mode ('r+')

Opens the file for both reading and writing. Starts at the beginning of the file. Raises **FileNotFoundError** if the file doesn't exist.

**with** open('example.txt', 'r+') as file:

```
 content = file.read()
```

```
 file.write("\nThis is a new line.")
```

**Output:** If the initial contents of "example.txt" were:

Hello, World!

This is a new line

After running the code, the new content of the file would be:

This is a new line

Hello, World!

This is a new line

### 6. Write and Read Mode ('w+')

This mode allows you to open a file for both reading and writing. If the file already exists, it will truncate the file to zero length. If the file does not exist, it will create a new file.

**Example:** In this example, a file named 'example.txt' is opened in write and read mode ('w+').

**With** open('example.txt', 'w+') as file:

```
 file.write('Hello, world!')
```

```
 file.seek(0)
```

```
 content = file.read()
```

**Output:**

Hello, world!

## Text files

Before writing to a file, let's first understand the different ways to create one. Creating a file is the first step before writing data. In Python you control creation behaviour with the mode passed to open() (or with pathlib helpers).

| Mode / Option | Description                                                                               |
|---------------|-------------------------------------------------------------------------------------------|
| "w"           | Write mode: creates file if missing, truncates (erases) if it exists                      |
| "a"           | Append mode: creates file if missing, writes data always at the end                       |
| "x"           | Exclusive create: creates new file, but fails with FileExistsError if it already exists   |
| "b"           | Binary flag: used with other modes (e.g., "wb", "ab") for binary files                    |
| "+"           | Read/write flag: combine with other modes (e.g., "r+", "w+") for both reading and writing |
| encoding=     | Specify text encoding (e.g., "utf-8") when working with text files                        |
| newline=      | Control newline translation in text mode (e.g., "\n")                                     |

**Note:** You can combine flags like "wb" (write + binary) or "a+" (append + read). Always pick a mode based on whether you want to overwrite, append, or strictly create a new file.

### Overwrite an existing file

Opening a file in write mode ("w") clears any existing content before writing new data. This is useful when you want to start fresh and replace old information with new output.

**Example:** This code writes two lines to file.txt, overwriting any previous content.

**with** open("file.txt", "w", encoding="utf-8") as f:

    f.write("Created using write mode.\n")

    f.write("Second line.\n")

**with** open("file.txt", "r", encoding="utf-8") as f:

    print(f.read())

### Output

Created using write mode.

Second line.

**Explanation:** Here "w" truncates file.txt and writes fresh content. The with block ensures the file is properly saved and closed.

### Append to an existing file

Using "a" mode (append), we can add new data at the end of a file without removing what's already there.

**Example:** This writes an extra line to file.txt without removing existing content.

**with** open("file.txt", "a", encoding="utf-8") as f:

    f.write("Appended line.\n")

**with** open("file.txt", "r", encoding="utf-8") as f:

    print(f.read())

### Output

Created using write mode.

Second line.

Appended line.

**Explanation:** "a" always writes at the end. Useful for logs, reports, or when you want to keep history intact.

### Create only if it does not exist

With "x" mode, Python creates a new file but raises FileExistsError if the file already exists. This prevents accidental overwrites.

**Example:** This tries to create file.txt only if it doesn't exist, otherwise prints an informative message.

try:

    with open("file.txt", "x", encoding="utf-8") as f:

        f.write("Created using exclusive mode.\n")

except FileExistsError:

```
print("file.txt already exists, exclusive creation aborted.")
```

**Output**

file.txt already exists, exclusive creation aborted.

**Explanation:** "x" mode is useful for safe file creation. It guarantees no overwriting if the file exists, Python raises a `FileExistsError` instead.

**Writing multiple lines**

Sometimes you need to write several lines at once instead of one by one. Python provides two common approaches: `writelines()` for lists of strings and `join()` for building a single text block.

**Example 1:** This writes a list of lines; each element includes a `\n`.

```
lines = ["First line\n", "Second line\n", "Third line\n"]
```

```
with open("file1.txt", "w", encoding="utf-8") as f:
```

```
 f.writelines(lines)
```

```
with open("file1.txt", "r", encoding="utf-8") as f:
```

```
 print(f.read())
```

**Output**

First line

Second line

Third line

**Explanation:** `writelines()` writes each string in the list directly. It does not add newlines automatically you must include `\n` in each string yourself.

**Example 2:** join + single write (often cleaner)

This creates the final text with `'\n'.join(...)` and writes once.

```
lines = ["Line A", "Line B", "Line C"]
```

```
text = "\n".join(lines) + "\n"
```

```
with open("file2.txt", "w", encoding="utf-8") as f:
```

```
 f.write(text)
```

```
with open("file2.txt", "r", encoding="utf-8") as f:
```

```
 print(f.read())
```

**Output**

Line A

Line B

Line C

**Explanation:** Building a single string and calling `write()` once can be more efficient and less error-prone, especially when you must ensure consistent newlines.

**Writing to a Binary File**

For non-text data (like images, audio, or other binary content), use binary write mode ("wb"). This treats the file as raw bytes instead of text.

**Example:** The following code writes a small sequence of bytes into a file called `file.bin` and then reads it back in binary mode.

```
data = b'\x00\x01\x02\x03\x04'
```

```
with open("file.bin", "wb") as f:
 f.write(data)
```

**Output**

```
b'\x00\x01\x02\x03\x04'
```

**Explanation:**

- **data = b'\x00\x01\x02\x03\x04':** Creates a bytes object (raw binary data).
- **open("file.bin", "wb"):** Opens file in binary write mode (creates/overwrites).
- **f.write(data):** Saves the bytes to the file.
- **open("file.bin", "rb"):** Opens file in binary read mode.
- **print(f.read()):** Reads and prints the stored bytes.

**Using pathlib**

pathlib provides an object-oriented way to work with files and paths. It is more modern but slightly advanced compared to open().

```
from pathlib import Path
Path("file.txt").write_text("Hello World\n")
content = Path("file.txt").read_text()
print(content)
```

**Output**

```
Hello World
```

**Explanation:**

- **Path()** represents a file path
- **write\_text()** writes data to file
- **read\_text()** reads file content

## Reading and Writing files

Python provides built-in functions for creating, reading, and writing files. Python can handle two types of files:

- **Text files:** Each line of text is terminated with a special character called EOL (End of Line), which is new line character ('\n') in Python by default.
- **Binary files:** There is no terminator for a line and data is stored after converting it into a machine-understandable binary format.

This article focuses on opening, reading, writing, and appending text files in Python.

**Opening a Text File**

In Python, files can be opened using the built-in open() function. No additional modules are required.

```
File_object = open(r"File_Name","Access_Mode")
```

**Example:** Opening text files in different modes using the open() function.

```
f1 = open("MyFile1.txt", "a")
f2 = open(r"D:\Text\MyFile2.txt", "w+")
```

**Reading a Text File**



Python provides three main methods to read a file:

### 1. Using read()

Returns the read bytes in form of a string. Reads 'n' bytes, if no 'n' specified, reads the entire file.

```
File_object.read([n])
```

### 2. Using readline()

Reads a single line from the file as a string. If n is specified, it reads at most n characters, but never more than one line.

```
File_object.readline([n])
```

### 3. Using readlines()

Reads all the lines and return them as each line a string element in a list.

```
File_object.readlines()
```

**Note:** '\n' is treated as a special character of two bytes.

**Example:** This example writes data to myfile.txt, then reopens it in r+ mode to demonstrate various read methods before closing the file.

```
f1 = open("myfile.txt", "w")
```

```
L = ["This is Delhi \n", "This is Paris \n", "This is London \n"]
```

```
f1.write("Hello \n")
```

```
f1.writelines(L)
```

```
f1.close()
```

```
f1 = open("myfile.txt", "r+")
```

```
print("Output of read():")
```

```
print(f1.read())
```

```
print()
```

```
f1.seek(0)
```

```
print("Output of readline():")
```

```
print(f1.readline())
```

```
print()
```

```
f1.seek(0)
```

```
print("Output of read(9):")
```

```
print(f1.read(9))
```

```
print()
```

```
f1.seek(0)
```

```
print("Output of readline(9):")
```

```
print(f1.readline(9))
```

```
print()
```

```
f1.seek(0)
print("Output of readlines():")
print(f1.readlines())
print()
```

```
f1.close()
```

### **Output**

Output of read():

Hello

This is Delhi

This is Paris

This is London

Output of readline():

Hello

Output of read(9):

Hello

Th

Output of readline(9):

Hello

Output of readlines():

['Hello \n', 'This is Delhi \n', 'This is Paris \n', 'This is London \n']

## **Writing in a Text File**

There are two ways to write in a file:

### **1. Using write()**

Inserts the string str1 in a single line in the text file.

```
File_object.write(str1)
```

```
f1 = open("Employees.txt", "w")
```

```
for i in range(3):
```

```
 name = input("Enter the name of the employee: ")
```

```
 f1.write(name)
```

```
 f1.write("\n")
```

```
f1.close()
```

```
print("Data is written into the file.")
```

### **Output**

Data is written into the file.

### **2. Using writelines()**

For a list of string elements, each string is inserted in text file. Used to insert multiple strings at a single time.

```
File_object.writelines(L) for L = [str1, str2, str3]
```

```
f1 = open("Employees.txt", "w")
lst = []
for i in range(3):
 name = input("Enter the name of the employee: ")
 lst.append(name + '\n')
```

```
f1.writelines(lst)
f1.close()
print("Data is written into the file.")
```

**Output**

Data is written into the file.

**Appending in a Text File**

Appending allows you to add new data at the end of an existing file without overwriting its current content. Use mode 'a' to open a file for appending.

In this example, "myfile.txt" is written with initial lines, then "Today" is appended, and finally overwritten with "Tomorrow".

```
f1 = open("myfile.txt", "w")
L = ["This is Delhi \n", "This is Paris \n", "This is London \n"]
f1.writelines(L)
f1.close()
```

```
Append-adds at last
f1 = open("myfile.txt", "a")
f1.write("Today \n")
f1.close()
```

```
f1 = open("myfile.txt", "r")
print("Output of Readlines after appending")
print(f1.readlines())
print()
f1.close()
```

```
Write-Overwrites
f1 = open("myfile.txt", "w")
f1.write("Tomorrow \n")
f1.close()
```

```
f1 = open("myfile.txt", "r")
print("Output of Readlines after writing")
print(f1.readlines())
print()
f1.close()
```

**Output**

Output of Readlines after appending

```
['This is Delhi \n', 'This is Paris \n', 'This is London \n', 'Today \n']
```

Output of Readlines after writing

```
['Tomorrow \n']
```

**Closing a Text File**

Python close() function closes file and frees memory space acquired by that file. It is used at the time when file is no longer needed or if it is to be opened in a different file mode.

```
File_object.close()
```

```
f1 = open("MyFile.txt","a")
```

```
f1.close()
```

**Format operator**

String formatting in Python is used to insert variables and expressions into strings in a readable and structured way. It helps create dynamic output and improves the clarity and presentation of text in programs.

**Methods of String Formatting in Python**

There are five different ways to perform string formatting in Python:

- Formatting with % Operator.
- Formatting with format() string method.
- Formatting with string literals, called f-strings.
- Formatting with String Template Class
- Formatting with center() string method.

**1. Format String using % Operator**

It is the oldest method of string formatting. Here we use the modulo % operator. The modulo % is also known as the “string-formatting operator”.

**Python Format String Using the % Operator**

In the expression "The mangy, scrawny stray dog %s gobbled down" % 'hurriedly', the %s placeholder within the string is replaced by the value 'hurriedly'.

```
print("The mangy, scrawny stray dog %s gobbled down" % 'hurriedly' +
 "the grain-free, organic dog food.")
```

**Output**

The mangy, scrawny stray dog hurriedly gobbled downthe grain-free, organic dog food.

**Injecting Multiple Strings using the modulo Operator**

Here we are inserting multiple strings with the % operator.

```
x = 'looked'
```

```
print("Misha %s and %s around"%('walked',x))
```

**Output**

Misha walked and looked around

**Precision Handling in Python using % operator**

Floating-point numbers use the format %a.bf. Here, 'a' would be the minimum number of digits to be present in the string; these might be padded with white space if the whole number doesn't have these many digits.

Close to this, bf represents how many digits are to be displayed after the decimal point.

In this code, the string 'The value of pi is: %5.4f' contains a format specifier %5.4f. The %5.4f format specifier is used to format a floating-point number with a minimum width of 5 and a precision of 4 decimal places.

```
print('The value of pi is: %5.4f' %(3.141592))
```

**Output**

The value of pi is: 3.1416

**Multiple format conversion types**

In the given code, the formatting string Python is converted to Integer and floating point with %d,%f.

```
variable = 12
```

```
string = "Variable as integer = %d \n\
```

```
Variable as float = %f" %(variable, variable)
```

```
print (string)
```

**Output**

Variable as integer = 12

Variable as float = 12.000000

**2. Format String using format() Method**

Format() method was introduced with Python3 for handling complex string formatting more efficiently.

Formatters work by putting in one or more replacement fields and placeholders defined by a pair of curly braces { } into a string and calling the str.format(). The value we wish to put into the placeholders and concatenate with the string passed as parameters into the format function.

**Syntax**

```
'String here { } then also { }'.format('something1','something2')
```

**Formatting String Python using format() Method**

This code is using { } as a placeholder and then we have called.format() method on the 'equal' to the placeholder.

```
print('We all are { }.'.format('equal'))
```

**Output**

We all are equal.

**Index-based Insertion**

In this code, curly braces {} with indices are used within the string '{2} {1} {0}' to indicate the positions where the corresponding values will be placed.

```
print('{2} {1} {0}'.format('directions',
 'the', 'Read'))
```

### Output

Read the directions

### Insert object by Assigning Keywords

In this code, curly braces {} with named placeholders ({a}, {b}, {c}) are used within the string 'a: {a}, b: {b}, c: {c}' to indicate the positions where the corresponding named arguments will be placed.

```
print('a: {a}, b: {b}, c: {c}'.format(a = 1,
 b = 'Two',
 c = 12.3))
```

### Output

a: 1, b: Two, c: 12.3

### Reuse the inserted objects

In this code, curly braces {} with named placeholders ({p}) are used within the string 'The first {p} was alright, but the {p} {p} was tough.' to indicate the positions where the corresponding named argument p will be placed.

```
print(
 'The first {p} was alright, but the {p} {p} was tough.'.format(p='second'))
```

### Output

The first second was alright, but the second second was tough.

### Float Precision with the.format() Method

#### Syntax

```
{[index]:[width][.precision][type]}
```

The type can be used with format codes:

- 'd' for integers
- 'f' for floating-point numbers
- 'b' for binary numbers
- 'o' for octal numbers
- 'x' for octal hexadecimal numbers
- 's' for string
- 'e' for floating-point in an exponent format

**Example:** Both the codes are doing string formatting. The first String is formatted with '%' and the second String is formatted with .format().

```
print('The value of pi is: %1.5f %3.141592')
print('The value of pi is: {0:1.5f}'.format(3.141592))
```

**Output**

The value of pi is: 3.14159

The value of pi is: 3.14159

**3. Understanding Python f-string**

PEP 498 introduced a new string formatting mechanism known as Literal String Interpolation or more commonly as F-strings (because of the leading f character preceding the string literal). The idea behind f-String in Python is to make string interpolation simpler.

To create an f-string in Python, prefix the string with the letter "f". The string itself can be formatted in much the same way that you would with str. format(). F-strings provide a concise and convenient way to embed Python expressions inside string literals for formatting.

**String Formatting with F-Strings**

In this code, the f-string f"My name is {name}." is used to interpolate the value of the name variable into the string.

```
name = 'Ele'
```

```
print(f"My name is {name}.")
```

**Output**

My name is Ele.

This new formatting syntax is very powerful and easy. You can also insert arbitrary Python expressions and you can even do arithmetic operations in it.

**Arithmetic operations using F-strings**

In this code, the f-string f" He said his age is {2 \* (a + b)}." is used to interpolate the result of the expression 2 \* (a + b) into the string.

```
a = 5
```

```
b = 10
```

```
print(f" He said his age is {2 * (a + b)}.")
```

**Output**

He said his age is 30.

We can also use lambda expressions in f-string formatting.

**Lambda Expressions using F-strings**

In this code, an anonymous lambda function is defined using lambda x: x\*2. This lambda function takes an argument x and returns its double.

```
print(f" He said his age is {(lambda x: x*2)(3)}")
```

**Output**

He said his age is 6

**Float precision in the f-String Method**

In this code, f-string formatting is used to interpolate the value of the num variable into the string.

**Syntax**

```
{value:{width}.{precision}}
num = 3.14159

print(f"The value of pi is: {num:{1}.{5}}")
```

**Output**

The value of pi is: 3.1416

**4. Python String Template Class**

In the String module, Template Class allows us to create simplified syntax for output specification. The format uses placeholder names formed by \$ with valid Python identifiers (alphanumeric characters and underscores). Surrounding the placeholder with braces allows it to be followed by more alphanumeric letters with no intervening spaces. Writing \$\$ creates a single escaped \$:

**Formatting String Python Using Template Class**

This code imports the Template class from the string module. The Template class allows us to create a template string with placeholders that can be substituted with actual values. Here we are substituting the values n1 and n2 in place of n3 and n4 in the string n.

```
from string import Template
```

```
n1 = 'Hello'
n2 = 'GeeksforGeeks'

n = Template('$n3 ! This is $n4.')
print(n.substitute(n3=n1, n4=n2))
```

**Output**

Hello ! This is GeeksforGeeks.

**5. Format String using center() Method**

The center() method is a built-in method in Python's str class that returns a new string that is centered within a string of a specified width.

**Formatting string using center() method**

This code returns a new string padded with spaces on the left and right sides.

```
string = "GeeksForGeeks!"
width = 30
```

```
centered_string = string.center(width)
print(centered_string)
```



## Output

GeeksForGeeks!

### Python Format String: % vs .format vs. f-string literal

f-strings are faster and better than both %-formatting and str.format(). F-strings expressions are evaluated at runtime, and we can also embed expressions inside f-string, using a very simple and easy syntax.

The expressions inside the braces are evaluated in runtime and then put together with the string part of the f-string and then the final string is returned.

## Command line arguments

In Python, command line arguments are values passed to a script when running it from the terminal or command prompt. They act like inputs, allowing you to change a program's behavior without modifying the code.

### Example:

```
python add.py 10 20
```

### Here:

- **python:** runs the interpreter
- **add.py:** script name
- **10 20:** arguments passed to the program

## Ways to Handle Command Line Arguments

Python provides the following built-in modules to handle command-line arguments:

- **sys module:** gives access to arguments as a list (sys.argv).
- **getopt module:** helps parse arguments similar to how options work in Unix/Linux commands.
- **argparse module:** the most powerful and user-friendly way to define, parse, and handle arguments.

### 1. Using sys.argv

The simplest way to access command line arguments in Python is through the sys module. It provides a list called sys.argv that stores everything you type after python in the command line.

- **sys.argv:** A list in Python that contains all the command-line arguments.
- **sys.argv[0]:** Always holds the name of the script being executed (e.g., add.py).
- **sys.argv[1:]:** Holds the actual arguments you pass (e.g., numbers like 10 20, file names, or other inputs).

```
import sys
```

```
print("Total arguments:", len(sys.argv))
print("Script name:", sys.argv[0])
print("Arguments:", sys.argv[1:])
```

```
total = 0
```

```
for arg in sys.argv[1:]:
```

```
total += int(arg)
```

```
print("Sum =", total)
```

### Run

```
python add.py 5 10 15
```

### Output

Total arguments: 4

Script name: add.py

Arguments: ['5', '10', '15']

Sum = 30

### Explanation:

- **sys.argv[0]:** script name (e.g., add.py).
- **sys.argv[1:]:** user-provided arguments (e.g., numbers).
- **len(sys.argv):** gives total count of arguments.
- A loop converts arguments to integers and sums them.

## 2. Using getopt

getopt module is used when you want to handle flags and options (both short -h and long --help styles). It is helpful when your script needs command-line options similar to Linux commands.

### Example:

```
import getopt, sys
```

```
args = sys.argv[1:]
```

```
options = "hmo:"
```

```
long_options = ["Help", "My_file", "Output="]
```

```
try:
```

```
 arguments, values = getopt.getopt(args, options, long_options)
```

```
 for currentArg, currentVal in arguments:
```

```
 if currentArg in ("-h", "--Help"):
```

```
 print("Showing Help")
```

```
 elif currentArg in ("-m", "--My_file"):
```

```
 print("File name:", sys.argv[0])
```

```
 elif currentArg in ("-o", "--Output"):
```

```
 print("Output mode:", currentVal)
```

```
except getopt.error as err:
```

```
 print(str(err))
```

### Run

```
python script.py -o result.txt
```

### Output

Output mode: result.txt

**Explanation:**

- **sys.argv[1:]**: gets arguments passed to the script.
- **options = "hmo:"**: short options: -h, -m, -o (-o expects a value).
- **long\_options**: long versions: --Help, --My\_file, --Output=<val>.
- **getopt.getopt()**: parses arguments into (option, value) pairs.

**Loop:**

- **-h / --Help**: prints help.
- **-m / --My\_file**: prints script name.
- **-o / --Output**: prints the provided value.
- **Error handling**: catches invalid options.

**3. Using argparse**

argparse is the easiest and most powerful way to handle command-line arguments in Python. It has the following features:

- Automatically creates help messages.
- Supports default values, type checking, and options.
- Comes with a built-in -h / --help flag.

**Example 1: Basic Parser**

```
import argparse
```

```
parser = argparse.ArgumentParser()
parser.parse_args()
```

**Run**

```
python script.py -h
```

**Output**

```
usage: script.py [-h]
```

```
optional arguments:
```

```
-h, --help show this help message and exit
```

**Explanation:**

- **argparse.ArgumentParser()**: creates a parser object to handle arguments.
- **parser.parse\_args()**: processes the arguments from the command line.
- Since no arguments are defined, the script only supports the default -h/--help option.

**Example 2: Adding Description**

```
import argparse
```

```
parser = argparse.ArgumentParser(description="Demo of argparse")
parser.parse_args()
```

**Run**

```
python script.py -h
```

**Output**

```
usage: script.py [-h]
```

```
Demo of argparse
```

options:

-h, --help show this help message and exit

**Explanation:**

- **argparse module:** used to handle command-line arguments in Python.
- **ArgumentParser(description=...):** creates a parser object. The description is shown when you use the help option.
- **parse\_args():** processes the arguments given when running the script.

**Example 3: Optional Argument**

```
import argparse
```

```
parser = argparse.ArgumentParser()
parser.add_argument("-o", "--Output", help="Show output message")
args = parser.parse_args()
```

```
if args.Output:
 print("Output:", args.Output)
```

**Run**

```
python script.py -o Hello
```

**Output**

```
Output: Hello
```

**Explanation:**

- **add\_argument("-o", "--Output", ...):** defines an optional argument with short form -o and long form --Output.
- **help="...":** shows a helpful description in the usage message.
- **args.Output:** stores the value passed with the -o/--Output flag.
- If provided, the script prints it.

## Errors and exceptions

Errors are problems in a program that causes the program to stop its execution. On the other hand, exceptions are raised when some internal events change the program's normal flow.

### Syntax Errors in Python

Syntax error occurs when the code doesn't follow Python's rules, like using incorrect grammar in English. Python stops and points out the issue before running the program.

Example 1: In this example, this code returns a syntax error because there is a missing colon (:) after the if statement. The correct syntax requires a colon to indicate the start of the block of code to be executed if the condition is true.

```
a = 10000
if a > 2999
 print("Eligible")
```

Output

```
File "<ipython-input-1-94302ab5dc97>", line 3
 if a > 2999
 ^
SyntaxError: expected ':'
```

Syntax error

Indentation Error

Python uses indentation to define blocks of code. An IndentationError occurs when the required indentation is missing or incorrect.

This code returns an IndentationError because Python expects an indented block after the if statement, but the statement inside the block is not properly indented.

```
if a<3:
```

```
print("gfg")
```

Output

```
File "<ipython-input-2-9d7cace61905>", line 2
 print("gfg")
 ^
IndentationError: expected an indented block after 'if' statement on line 1
```

Indentation Error

Python Logical Errors (Exceptions)

Logical errors are subtle bugs in the program that allow the code to run but produce incorrect or unintended results. These are often harder to detect since the program doesn't crash, but the output is not as expected.

Characteristics of Logical Errors

- No Syntax Error: The code runs without issues.
- Unexpected Output: The results produced by the program are not what the programmer intended.
- Difficult to Detect: These errors can be tricky to spot since there's no obvious problem with the code itself.
- Causes: Faulty logic, incorrect assumptions, or improper use of operators.

Example:

```
a = [10, 20, 30, 40, 50]
```

```
b = 0
```

```
for I in a:
```

```
 b += i
```

```
res = b / len(a) - 1
```

```
print(res)
```

Output

29.0

Explanation: Expected output is that the average of a should be 30, but the program outputs 29.0. The logical error occurs because the formula  $b / \text{len}(a) - 1$  incorrectly subtracts 1, leading to an incorrect result. The correct formula should be  $b / \text{len}(a)$ .

Common Built-in Exceptions

Some common built-in exceptions other than above mention exceptions are:

| Exception      | Description                                                               |
|----------------|---------------------------------------------------------------------------|
| IndexError     | When the wrong index of a list is retrieved.                              |
| AssertionError | It occurs when the assert statement fails                                 |
| AttributeError | It occurs when an attribute assignment is failed.                         |
| ImportError    | It occurs when an imported module is not found.                           |
| KeyError       | It occurs when the key of the dictionary is not found.                    |
| NameError      | It occurs when the variable is not defined.                               |
| MemoryError    | It occurs when a program runs out of memory.                              |
| TypeError      | It occurs when a function and operation are applied in an incorrect type. |

### Handling exceptions

Python Exception Handling allows a program to gracefully handle unexpected events (like invalid input or missing files) without crashing. Instead of terminating abruptly, Python lets you detect the problem, respond to it, and continue execution when possible.

Below is a basic example demonstrating how to catch an exception and handle it gracefully:

```
n = 10
try:
 res = n / 0
except ZeroDivisionError:
 print("Can't be divided by zero!")
```

### Output

Can't be divided by zero!

**Explanation:** Dividing a number by 0 raises a ZeroDivisionError. The try block contains code that may fail and except block catches the error, printing a safe message instead of stopping the program.

### Difference Between Errors and Exceptions

Errors and exceptions are both issues in a program, but they differ in severity and handling.

- **Error:** Issues in the program logic such as SyntaxError, etc. It occurs at compile time.
- **Exception:** Problems that occur at runtime and can be managed using exception handling (e.g., invalid input, missing files).

**Example:** This example shows the difference between a syntax error and a runtime exception.

```
Syntax Error (Error)
print("Hello world" # Missing closing parenthesis
```

```
ZeroDivisionError (Exception)
n = 10
res = n / 0
```

**Explanation:** A syntax error stops the code from running at all, while an exception like ZeroDivisionError occurs during execution and can be caught with exception handling.

## Syntax of Exception Handling

Python provides four main keywords for handling exceptions: try, except, else and finally each plays a unique role. Let's see syntax:

```
try:
Code
except SomeException:
Code
else:
Code
finally:
Code
```

- **try:** Runs the risky code that might cause an error.
- **except:** Catches and handles the error if one occurs.
- **else:** Executes only if no exception occurs in try.
- **finally:** Runs regardless of what happens useful for cleanup tasks like closing files.

**Example:** This code attempts division and handles errors gracefully using try-except-else-finally.

```
try:
 n = 0
 res = 100 / n

except ZeroDivisionError:
 print("You can't divide by zero!")

except ValueError:
 print("Enter a valid number!")

else:
 print("Result is", res)

finally:
 print("Execution complete.")
```

## Output

```
You can't divide by zero!
Execution complete.
```

**Explanation:** try block attempts division, except blocks catch specific errors, else block executes only if no errors occur, while finally block always runs, signaling end of execution. Please refer Python Built-in Exceptions for some common exceptions.

## Catching Exceptions

We can handle errors more efficiently by specifying the types of exceptions we expect. This can make code both safer and easier to debug.

### 1. Catching Specific Exceptions

Catching specific exceptions makes code to respond to different exception types differently. It precisely makes your code safer and easier to debug. It avoids masking bugs by only reacting to the exact problems you expect.

**Example:** This code handles `ValueError` and `ZeroDivisionError` with different messages.

try:

```
This will cause ValueError
x = int("str")
inv = 1 / x # Inverse calculation
```

except `ValueError`:

```
print("Not Valid!")
```

except `ZeroDivisionError`:

```
print("Zero has no inverse!")
```

### Output

Not Valid!

**Explanation:** A `ValueError` occurs because "str" cannot be converted to an integer. If conversion had succeeded but x were 0, a `ZeroDivisionError` would have been caught instead.

## 2. Catching Multiple Exceptions

We can catch multiple exceptions in a single block if we need to handle them in the same way or we can separate them if different types of exceptions require different handling.

**Example:** This code attempts to convert list elements and handles `ValueError`, `TypeError` and `IndexError`.

```
a = ["10", "twenty", 30]
```

try:

```
'twenty' cannot be converted to int
total = int(a[0]) + int(a[1])
```

except (`ValueError`, `TypeError`) as e:

```
print("Error", e)
```

except `IndexError`:

```
print("Index out of range.")
```

### Output

Error invalid literal for int() with base 10: 'twenty'

**Explanation:** The `ValueError` is raised when trying to convert "twenty" to an integer. A `TypeError` could occur if incompatible types were used, while `IndexError` would trigger if the list index was out of range.

## 3. Catch-All Handlers and Their Risks



Catch-all handler is used to call to catch any exception (similar to else statement). Use only `except` keyword to define it:

**Example:** This code tries dividing a string by a number, which causes a `TypeError`.

`try:`

```
Risky operation: dividing string by number
res = "100" / 20
```

`except ArithmeticError:`

```
print("Arithmetic problem.")
```

`except:`

```
print("Something went wrong!")
```

### Output

Something went wrong!

**Explanation:** A `TypeError` occurs because you can't divide a string by a number. The bare `except` catches it, but this can make debugging harder since the actual error type is hidden. Use bare `except` only as a net.

### Raise an Exception

We raise an exception in Python using the `raise` keyword followed by an instance of the exception class that we want to trigger. We can choose from built-in exceptions or define our own custom exceptions by inheriting from Python's built-in `Exception` class.

#### Basic Syntax:

```
raise ExceptionType("Error message")
```

**Example:** This code raises a `ValueError` if an invalid age is given.

`Def set(age):`

```
 if age < 0:
 raise ValueError("Age cannot be negative.")
 print(f'Age set to {age}')
```

`try:`

```
 set(-5)
```

`except ValueError as e:`

```
 print(e)
```

### Output

Age cannot be negative.

## Modules

Module is a file containing definitions and statements. A module can define functions, classes and variables. Modules help organize code into separate files so that programs become easier to maintain and reuse. Instead of writing everything in one place, related functionality can be grouped into its own module and imported whenever needed.

### Create a Module

To create a module, write the desired code and save that in a file with .py extension.

**Example:** Let's create a `calc.py` in which we define two functions, one `add` and another `subtract`.

```
calc.py
def add(x, y):
 return (x+y)
```

```
def subtract(x, y):
 return (x-y)
```

This is all that is required to create a module.

### Import module

Modules can be used in another file using the `import` statement. When Python sees an `import`, it loads the module if it exists in the interpreter's search path. Below is the syntax to import a module:

```
import module
```

**Example:** Here, we are importing the `calc` that we created earlier to perform `add` operation.

```
import calc
print(calc.add(10, 2))
```

### Output

```
12
```

**Explanation:** `import calc` loads the module and `calc.add()` accesses a function through dot notation.

### Types of Import Statements

**1. Import From Module:** This allows importing specific functions, classes, or variables rather than the whole module.

```
from math import sqrt, factorial
print(sqrt(16))
print(factorial(6))
```

### Output

```
4.0
720
```

**Explanation:** Only `sqrt` and `factorial` are brought into the local namespace, so the prefix `math.` is not required.

**2. Import All Names:** \* imports everything from a module into the current namespace.

```
From math import *
print(sqrt(16))
print(factorial(6))
```

### Output

```
4.0
720
```

**Explanation:** Every public name of math becomes directly accessible. (Not recommended in large projects due to namespace conflicts.)

**3. Import With Alias:** You can shorten a module's name using as.

```
Import math as m
print(m.pi)
```

### Output

```
3.141592653589793
```

**Explanation:** math is accessed through the shorter alias m.

### Types of Modules

Python provides several kinds of modules. Each type plays a different role in application development.

- 4. Built-in Modules:** These come bundled with Python and require no installation – e.g., math, random, os.

```
Import random
print(random.randint(1, 5))
```

### Output

```
4
```

**Explanation:** random.randint() returns a random number within the given range.

**2. User-Defined Modules:** These are modules you create yourself, such as **calc.py**.

```
import calc
print(calc.subtract(20, 5))
```

### Output

```
15
```

**Explanation:** The module is created manually and then imported into another script.

**3. External (Third-Party) Modules:** These modules are installed using pip – e.g., NumPy, Pandas, Requests.

### Import requests

```
r = requests.get("https://example.com")
print(r.status_code)
```

### Output

```
200
```

**Explanation:** requests is installed separately (pip install requests) and provides HTTP utilities.

**4. Package Modules:** A package is a directory containing multiple modules, usually with an `__init__.py` file.

Example Directory

```
mypkg/
__init__.py
```

```
calc.py
```

```
utils.py
```

Using a module from a package

```
from mypkg import utils
```

```
print(utils.some_func())
```

calls a function named `some_func()`, the output will be whatever that function returns.

If `utils.py` contains something like:

```
def some_func():
 return "Hello"
```

### Output

```
Hello
```

### Locating a Module

Python searches for modules in a predefined list of directories known as the module search path. You can view this list using `sys.path`.

```
import sys
```

```
for p in sys.path:
```

```
 print(p)
```

### Output

```
/home/guest/sandbox
```

```
/usr/local/lib/python3.13.zip
```

```
/usr/local/lib/python3.13
```

```
/usr/local/lib/python3.13/lib-dynload
```

```
/usr/local/lib/python3.13/site-packages
```

## Python Packages

Python packages are a way to organize and structure code by grouping related modules into directories.

- A package is essentially a folder that contains an `__init__.py` file and one or more Python files (modules).
- Allows modules to be easily shared and distributed across different applications.

### Key Components of a Python Package

- **Module:** A single Python file containing reusable code (e.g., `math.py`).
- **Package:** A directory containing modules and a special `__init__.py` file.
- **Sub-Packages:** Packages nested within other packages for deeper organization.

### Creating and Accessing Packages

1. **Create a Directory:** create a folder that will act as the package root.

2. **Add Modules:** Inside the directory, add Python files (modules). Each module can contain related functions or classes.
3. **Add `__init__.py`:** add an `__init__.py` file to the directory. This file tells Python that the directory should be treated as a package.
4. **Create Sub-packages (Optional):** One can organize code further by creating subdirectories with their own `__init__.py` files.
5. **Import Modules:** Modules or functions inside the package can be imported using dot notation, for example:

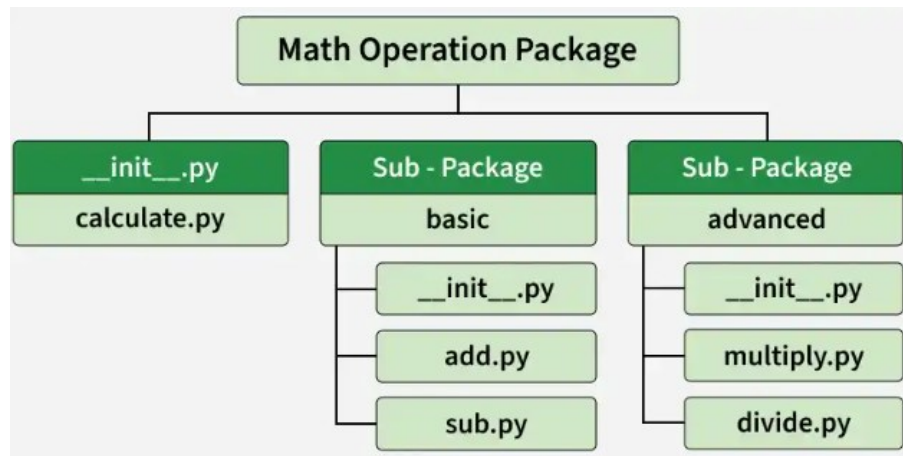
```
from mypackage.module1 import greet
```

### Example

In this example, we create a package called `math_operations` to organize mathematical functions. The package contains two sub-packages:

- **basic:** addition and subtraction
- **advanced:** multiplication and division

Each operation is stored in its own module, which makes the code modular, reusable and easier to maintain.



maths operation package

**math\_operations/ `__init__.py`** - This file initializes the main package and exposes commonly used functions so they can be imported directly from the package.

```
from .calculate import calculate
```

```
from .basic import add, subtract
```

```
from .advanced import multiply, divide
```

**math\_operations/calculate.py** - This module contains a simple function that prints a message indicating that a calculation is being performed.

```
def calculate():
```

```
 print("Performing calculation...")
```

**math\_operations/basic/ `__init__.py`** - This file initializes the basic sub-package and makes the add and subtract functions available when the sub-package is imported.

```
from .add import add
```

```
from .sub import subtract
```

**math\_operations/basic/add.py** - This module contains the function for performing addition.

```
def add(a, b):
```

```
 return a + b
```

**math\_operations/basic/sub.py** - This module contains the function for performing subtraction.

```
def subtract(a, b):
 return a - b
```

**math\_operations/advanced/\_\_init\_\_.py** - This file initializes the advanced sub-package and exposes the multiply and divide functions from their respective modules.

```
from .mul import multiply
from .div import divide
```

**math\_operations/advanced/mul.py** - This module contains the function for performing multiplication.

```
def multiply(a, b):
 return a * b
```

**math\_operations/advanced/div.py** - This module contains the function for performing division.

```
def divide(a, b):
 return a / b
```

### Using the Package

Now we can import functions from the package and use them in our program.

```
from math_operations import calculate, add, subtract, multiply, divide
```

```
Using the placeholder calculate function
calculate()
```

```
Perform basic operations
print("Addition:", add(5, 3))
print("Subtraction:", subtract(10, 4))
```

```
Perform advanced operations
print("Multiplication:", multiply(4, 2))
print("Division:", divide(10, 2))
```

### Output

```
Performing calculation...
Addition: 8
Subtraction: 6
Multiplication: 8
Division: 5.0
```

## Creating and Accessing Packages

In Python, a package is a way to organize related modules into a directory structure. Instead of keeping all files in one place, packages help group similar functionality together, making code easier to manage, reuse and maintain especially in large projects.

### Creating a Package

Creating a package involves just three steps:

- Create a directory (this will be package name)
- Add Python files (modules) inside it
- Add an `__init__.py` file to mark it as a package

Let's create a package named Cars and build three modules in it namely, Bmw, Audi and Nissan. Below is the folder structure:

```
cars/
| — __init__.py
| — bmw.py
| — audi.py
| — nissan.py
```

**bmw.py:** This module defines the BMW class, which contains a method to return a list of BMW car models.

```
class BMW:
```

```
 def models(self):
 return ["i8", "X1", "X5"]
```

**audi.py:** This module defines the Audi class, which provides Audi car models through a method.

```
class Audi:
```

```
 def models(self):
 return ["A3", "A6", "Q7"]
```

**nissan.py:** This module defines the Nissan class, used to return popular Nissan car models.

```
class Nissan:
```

```
 def models(self):
 return ["Altima", "Rogue", "370Z"]
```

**init.py:** The `__init__.py` file makes the cars directory a package. By importing classes here, it allows direct access to BMW, Audi, and Nissan classes when the package is imported.

```
from .bmw import BMW
from .audi import Audi
from .nissan import Nissan
```

This file allows direct access to classes when importing the package.

### Accessing the Package

Now, create a file named `main.py` in the same directory where the cars package exists. This file will be used to import and access the classes defined inside the package modules.

```
From cars import BMW, Audi, Nissan
```

```
bmw = BMW()
print(bmw.models())
```

```
audi = Audi()
print(audi.models())
```

```
261 nissan = Nissan()
print(261 nissan.models())
```

**Output**

```
['i8', 'X1', 'X5']
['A3', 'A6', 'Q7']
['Altima', 'Rogue', '370Z']
```

**Explanation:**

- **from cars import BMW, Audi, Nissan:** imports the classes directly from the package using `__init__.py`.
- **BMW(), Audi(), and Nissan():** create objects of their respective classes.
- **models():** is called on each object to retrieve the list of car models.

**Different Ways to Import from a Package**

Python provides multiple ways to import content from a package depending on how much access you need.

3. **Import a Specific Module:** used when only one class or module is required from the package

```
from cars.bmw import BMW
```

This statement imports only the BMW class from the `bmw` module inside the `cars` package.

2. **Import the Whole Package:** used when modules need to be accessed using the package name

```
import cars
```

This statement imports the entire `cars` package, and its modules or classes must be accessed using dot notation.

3. **Import Everything Using \*:** used to import all objects defined in `__init__.py`

```
from cars import *
```

This statement imports all classes and objects listed in the `__init__.py` file.

**Python Packages for Web Frameworks**

Web frameworks help developers build websites and APIs by providing tools for routing, request handling and database integration. These frameworks range from minimal setups for small apps to structured solutions for large-scale web projects.

- Flask
- Django
- FastAPI
- Pyramid
- Tornado
- Falcon
- CherryPy
- Bottle



## Flask

Flask is a lightweight web framework used to build web applications and APIs. It follows a minimal design and provides core features like routing, request handling and template rendering while allowing developers to add extensions as needed. It is widely used for building small to medium web applications due to its simplicity and flexibility.

- **Microframework:** Lightweight framework with minimal dependencies, giving developers full control over application structure.
- **Werkzeug & Jinja2:** Uses Werkzeug for request/response handling and Jinja2 for dynamic HTML templating.
- **Routing:** Maps URLs to Python functions using simple decorators.
- **Flexible Database Choice:** No built-in ORM, allowing use of tools like SQLAlchemy or raw SQL.
- **API Development:** Well-suited for building RESTful APIs and backend services.
- **Development Server:** Includes a built-in server for local testing during development.

## Django

Django is a Python web framework that simplifies web development through reusable components and built-in features such as authentication, database connectivity, and CRUD operations, following the DRY (Don't Repeat Yourself) principle. It follows the MVT (Model-View-Template) design pattern:

- **Model:** Represents the data you want to display, typically sourced from a database.
- **View:** Handles incoming requests and returns the appropriate template and content based on the user's request.
- **Template:** A text file (usually HTML) that defines the structure of the web page and includes logic for displaying the data.

## Why Use Django Framework?

- **Built-in Admin:** It comes with a ready-to-use admin interface to manage your website data easily.
- **Security:** Django has built-in protections to guard against common security threats like SQL injection, XSS, and CSRF.
- **Reusability:** Django follows the DRY (Don't Repeat Yourself) principle, encouraging reusable code components.
- **Scalable:** Django is designed to handle websites of all sizes, from small apps to large-scale projects.
- **ORM (Object-Relational Mapping):** Easily interact with your database without writing SQL queries.
- **URL Routing:** Clean and simple URL structure that maps URLs to views in your app.
- **Templating Engine:** Separate your HTML from Python code using Django's templating system, making dynamic content easy to manage.

## FastAPI - Introduction

FastAPI is a modern and high-performance Python web framework used to build APIs quickly and efficiently. Designed with simplicity it allows developers to create RESTful APIs using Python's type hints which also enable automatic validation and error handling.

One of FastAPI's key features is its ability to generate interactive API documentation automatically making it easier to test and understand API endpoints. It is an ideal choice for beginners and professionals who want to build fast, secure and scalable web applications with minimal effort.

## Features of FastAPI

- **Automatic Documentation:** FastAPI auto-generates interactive API docs using OpenAPI standard making it easy to test and understand your APIs.
- **Python Type Hints:** uses type hints to validate inputs and generate docs automatically making code more readable and less error-prone.
- **Data Validation:** FastAPI uses Pydantic models to validate and serialize/deserialize request data automatically.
- **Asynchronous Support:** Supports async and await allowing non-blocking code for better performance in I/O-heavy apps.
- **Dependency Injection:** Built-in support for dependency injection helps keep code modular, testable and clean.
- **Built-in Security:** Includes OAuth2, JWT and request validation to protect against common security issues.

## Installation and Setup of FastAPI

To get started with FastAPI:

**1. Install Python 3:** Make sure Python 3.7 or above is installed.

**2. Install FastAPI:** Use pip to install the FastAPI library

```
pip install fastapi
```

**3. Install Uvicorn (ASGI server):** Uvicorn is a lightweight server used to run FastAPI apps

```
pip install uvicorn
```

## Create a Simple API

### Example 1:

Let's create a simple web service that responds with **"Hello, FastAPI!"** when a specific URL is accessed. Using FastAPI, this can be achieved in just a few lines of code. The code is saved in a Python file named **main.py**.

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def read_root():
```

```
 return {"message": "Hello, FastAPI!"}
```

Here `@app.get("/")` tells FastAPI to handle HTTP GET requests sent to the root URL (/) by executing the function below it (`read_root`).

Now, run the following command to start the FastAPI app with live reload enabled:

```
uvicorn main:app --reload
```

Once the application is running, open your web browser and navigate to

`http://localhost:8000/`

You should see a message displayed in your browser or the response if you are using an API testing tool like curl or Postman.

```
{"message": "Hello, FastAPI!"}
```

## Python Pyramid

This project is a dynamic web application built on the powerful **Python Pyramid framework**. In this, we'll go through the comprehensive overview of the prerequisites you should have before diving into Pyramid, as well as the essential steps to get started with this powerful web framework. Pyramid enables programmers to create web applications of various sizes, from modestly sized side projects to expansive business systems. and we'll get a look at how to start with the pyramid framework.

### Prerequisites

- Download and Install Python 3 Latest Version

### Why should we use Pyramid?

Python Pyramid is a flexible web framework with many benefits, making it a fantastic option for many different web development projects. In the python, there are many different frameworks available for web development. Pyramid is one such framework that stands out for its ease of use, adaptability and scalability. Here are several justifications for considering Pyramid:

- **Flexibility:** Pyramid is frequently referred to as a "micro" or "minimal" web framework, meaning that it only offers the necessities for creating web applications.
- **Scalability:** The architecture of the Pyramid is made to grow gradually from simple applications to substantial, sophisticated ones.
- **Modularity:** Pyramid encourages the use of plug-and-play modules in the development process. It makes use of the idea of "add-ons" or "extensions," which let you add features and capabilities to our application as independent components.
- **Performance:** The high performance of Pyramid is well known. Pyramid is a fantastic option for projects that need to be highly responsive and low latency.
- **Security:** Pyramid takes security very seriously and offers tools to make sure our web apps are safe to use. The entire security of your application is enhanced by features like integrated CSRF protection.
- **Cross-Platform Capability:** Because Pyramid is a cross-platform framework, we can easily create on several operating systems (Windows, Linux, and macOS).
- **Vast of Libraries and Extensions:** Pyramid offers a thriving ecosystem of libraries and add-ons, referred to as Pyramid "scaffolds" and "cookiecutters," that enable RESTful APIs, database integration, user authentication, and other popular functionality.

- **Pyramid Used By:** Mozilla, AdRoll, Bravewords, Niteoweb, Survey Monkey etc.

### Pyramid Installation

**Step 1 :** Make sure Python is set up on the machine or From the Python website, we can get the most recent version. To check python installed or not run this command

```
>> python --version
```

[this command is use to check the current version of the python which is installed in the machine]

```
C:\Users\Haxkd>python --version
Python 3.11.1

C:\Users\Haxkd>_
```

check python installation

**Step 2 :** It's a good practice to create a virtual environment for your project to manage dependencies and isolate your project's environment from the system-wide Python packages. To create a virtual environment, and activate it, please follow the command...

```
>> python -m venv myenv
```

[this command is use to create a new virtual invironment with the name of **myenv**(name can be change)]

```
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Haxkd\Documents\PyramidProjects>python -m venv myenv

C:\Users\Haxkd\Documents\PyramidProjects>
```

create a virtual environment

```
>> myenv\Scripts\activate
```

[this command is use to activate the virtual invironment which is previously created with the name of **myenv**]

```
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Haxkd\Documents\PyramidProjects>myenv\Scripts\activate

(myenv) C:\Users\Haxkd\Documents\PyramidProjects>
```

activating virtual invironment

**Step 3 :** To Install the Pyramid framework Run the following command....

```
>> pip install pyramid
```

[the above command is use to install the pyramid framework into the machine]

```
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Haxkd\Documents\PyramidProjects>pip install pyramid
Collecting pyramid
 Obtaining dependency information for pyramid from https://files.pythonhosted.org/.../pyramid-2.0.2-py3-none-any.whl.metadata
 Downloading pyramid-2.0.2-py3-none-any.whl.metadata (20 kB)
Collecting hupper>=1.5 (from pyramid)
```

to install pyramid

**Step 4 :** another tool name cookiecutter and waitress is required so to install this follow the command...

>> pip install cookiecutter waitress

[this command is used to install some other requirement for building the pyramid project.]

```
Select C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Haxkd\Documents\PyramidProjects>pip install cookiecutter
Collecting cookiecutter
 Obtaining dependency information for cookiecutter from https://files.pythonhosted.org/.../cookiecutter-2.4.0-py3-none-any.whl.metadata
 Downloading cookiecutter-2.4.0-py3-none-any.whl.metadata (7.0 kB)
Collecting binaryornot>=0.4.4 (from cookiecutter)
```

installing cookiecutter

**Step 5 :** now we have to create a new pyramid project. So to do this write that command.....

>> cookiecutter gh:Pylons/pyramid-cookiecutter-starter

[this command is use to create a fresh new pyramid project] And then choose appropriate options such as project name, template language and backend.

```
Select C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Haxkd\Documents\PyramidProjects>cookiecutter gh:Pylons/pyramid-cookiecutter-starter
[1/4] project_name (Pyramid Scaffold): firstproject
[2/4] repo_name (firstproject): firstproject
[3/4] Select template_language
1 - jinja2
2 - chameleon
3 - mako
Choose from [1/2/3] (1): 1
[4/4] Select backend
1 - none
2 - sqlalchemy
3 - zodb
Choose from [1/2/3] (1): 1
```

creating the project

Now the project has been created. then now **navigate to project folder** and install the dependencies by running this command....

>> pip install -e .

[this command is used to install all the required dependencies to start with the pyramid project]

```
Select C:\Windows\System32\cmd.exe - pip install -e .
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Haxkd\Documents\PyramidProjects>cd firstproject

C:\Users\Haxkd\Documents\PyramidProjects\firstproject>pip install -e .
Obtaining file:///C:/Users/Haxkd/Documents/PyramidProjects/firstproject
Installing build dependencies ... |
```

installing project dependencies

Now all the required dependencies installed for the pyramid project. now the project is ready to serve.

**Step 6 :** To run the development server follow this command.

```
>> pserve development.ini
```

[the mention command is use to start the pyramid development server to run the project]

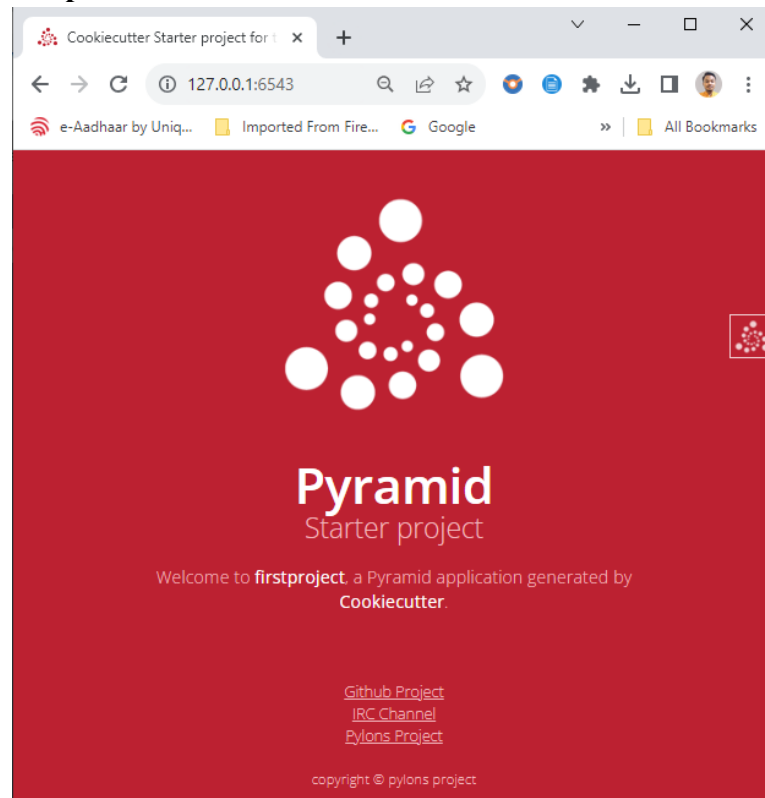
```
C:\Windows\System32\cmd.exe - pserve development.ini
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Haxkd\Documents\PyramidProjects\firstproject>pserve development.ini
Starting server in PID 15408.
2023-10-04 11:40:22,600 INFO [waitress:486][MainThread] Serving on http://[::1]:6543
2023-10-04 11:40:22,601 INFO [waitress:486][MainThread] Serving on http://127.0.0.1:6543
```

running the project

The project runs successfully to see that open the serving URL(<http://127.0.0.1:6543>) and open it on the browser.

**Output:**



project output

## Introduction to Tornado Framework

Tornado is a robust Python asynchronous networking library and web framework, is available as an open-source project. Given that it is made to manage non-blocking, asynchronous processes, it is appropriate for developing high-performance, scalable web apps. Since its creation, Tornado—which was first created by FriendFeed—has been well-known for its ease of use, quickness, and ability to handle stable network connections.

## Prerequisites

- Download and Install Python 3 Latest Version

**Key features of Tornado:**

- **Asynchronous I/O:** Tornado is well-suited for applications requiring high concurrency since it is designed to manage thousands of simultaneous connections in an efficient manner.
- **WebSockets:** Tornado supports WebSockets, which allows clients and servers to communicate in real time.
- **Request Handlers:** To handle incoming HTTP requests, Tornado employs the request-handler pattern. Classes called handlers specify how to reply to various kinds of requests.
- **Built-in Authentication and Authorization:** Tornado comes with built-in ways to manage user authentication and authorization, which simplify the process of securing your apps.
- **Scalability:** Tornado is well-suited for developing scalable web applications due to its capacity to manage several connections at once.

**Lets create a basic tonado project:**

**Step 1 :** Make sure Python is set up on the machine or From the Python website, we can get the most recent version. To check python installed or not run this command

```
>> python --version
```

**Step 2 :** Now, we have to install tornado framework, by using this command

```
>> pip install tornado
```

**Step 3 :** After the installation now we have to create a app.py and write some basic code in it.

```
import tornado.ioloop
```

```
import tornado.web
```

```
class HomeHandler(tornado.web.RequestHandler):
```

```
 def get(self):
 self.write("Hello, Tornado Home!")
```

```
class AnotherHandler(tornado.web.RequestHandler):
```

```
 def get(self):
 self.write("Hello, Tornado Another!")
```

```
def make_app():
```

```
 return tornado.web.Application([
 (r"/", HomeHandler),
 (r"/another", AnotherHandler),
])
```

```
if __name__ == "__main__":
```

```
 app = make_app()
 app.listen(8888)
 print("running in http://localhost:8888/")
 tornado.ioloop.IOLoop.current().start()
```

**Step 4 :** Now the project has been created. now the project is ready to serve. To run the development server follow this command.

```
>> python app.py
```

The project runs successfully to see that open the serving URL(<http://localhost:8888/>) and open it on the browser.

In this project, the HomeHandler responds with "Hello, Tornado Home!" when the root URL ("/") is accessed.

## Python Falcon Introduction

Python Falcon is a lightweight, high-performance web framework that's well-suited for building RESTful APIs. It's easy to learn, efficient, and perfect for projects where speed and simplicity are priorities. In this article, we introduced Falcon and created a basic "Hello World" application to help you get started with this powerful framework.

## What is Python Falcon?

Python Falcon is a simple web framework designed to build efficient APIs. If you're looking for a system that prioritizes speed and performance while having a simple and easy-to-understand design, Falcon could be the right choice for your project.

## Features of Python Falcon

Below are some features of Python Falcon:

- **Lightweight:** Falcon is known for its minimalistic design. It doesn't include unnecessary features or dependencies, making it a lightweight option. This focus on simplicity results in faster execution and lower memory overhead.
- **High Performance:** Falcon is optimized for high performance. It's designed to handle a large number of requests efficiently, making it an excellent choice for building APIs where speed is crucial.
- **RESTful by Design:** Falcon is built with RESTful API development in mind. It encourages best practices for designing REST APIs and provides tools to help you create clean and maintainable code.
- **Easy to Learn:** Falcon's simplicity makes it easy for developers to pick up quickly. If you're familiar with Python and REST principles, you'll find it straightforward to start building APIs with Falcon.
- **Flexibility:** Falcon applications are flexible and is very useful for applications with high complexity and need high-performance tuning.

## Installation and Setup of Python Falcon

To get started with Falcon, you need to install Python, if not then install Python3 and Install pip. Then, you need to install Falcon using the following command  

```
pip install falcon
```



## Create First Project

We import the Falcon framework. We define a class called `HelloWorldResource` that will handle the GET requests to the `/hello` endpoint. Inside the `on_get` method, we set the HTTP response status to 200 OK and provide the response text.

```
import falcon
from waitress import serve

class HelloWorldResource:
 def on_get(self, req, resp):
 resp.status = falcon.HTTP_200 # Set the HTTP status code to 200 OK
 resp.text = "Hello, Falcon World!" # The response body

Create a Falcon app
app = falcon.App()

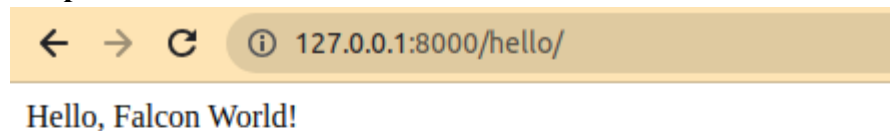
Add a route that maps the URL path to the HelloWorldResource
app.add_route('/hello', HelloWorldResource())

if __name__ == '__main__':
 serve(app, host='127.0.0.1', port=8000)
```

Now, you can run your Falcon application. In your terminal, navigate to the directory containing `app.py` and run the following command:

```
gunicorn app:app
```

### Output



## Advantages of Falcon

- **Need for Speed:** Falcon's forte is speed and efficiency. If your project involves a high volume of requests or you're working with limited resources, Falcon's got your back.
- **Simple and User-Friendly:** The Falcon's simplicity is one of its greatest assets. It doesn't take long to get the hang of it, especially if you already know Python. This means you can easily jump right into building an API.
- **Built to Scale:** Falcon's minimalistic architecture makes it highly scalable. Whether you're deploying on a small virtual server or a massive cloud infrastructure, Falcon can handle it.

- **Community and Documentation:** While Falcon may not be as popular as some other programs, it boasts an active community and well-maintained documentation. You will not be left in the dark when you need help or resources.

### Disadvantages of Falcon

- **Features Lite:** Falcon's minimalistic approach has its drawbacks. It lacks some of the features you might find in more feature-rich frameworks like Django or Flask. If your project relies heavily on such features, Falcon may not be your best bet.
- **Async Learning Curve:** Although the Falcon supports asynchronous programming, it can be a bit difficult for newbies to understand. If your project involves a lot of asynchronous processing, be prepared to spend some time understanding this aspect of Falcon.
- **Smaller Ecosystem:** Falcon's smaller user base compared to other Python frameworks means you'll have access to fewer third-party extensions and libraries. You might need to build certain functionality from scratch or rely on custom solutions.

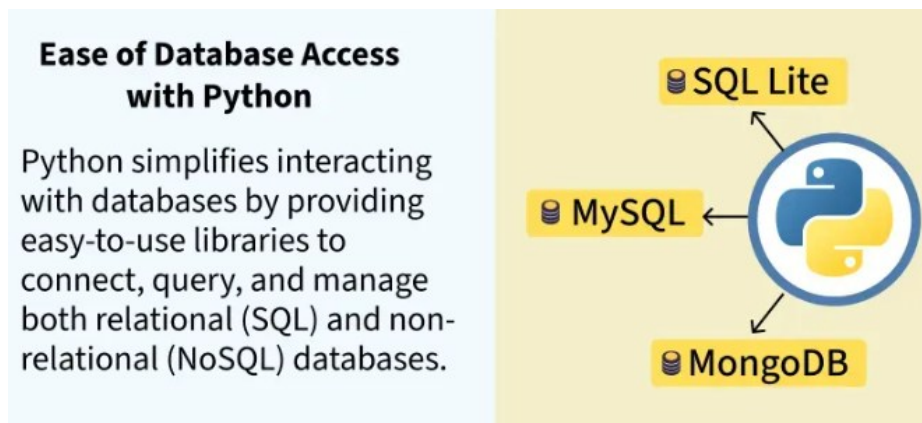
# CHAPTER 6

## DATABASE CONNECTIVITY

---

### Python Database

Python being a high-level language provides support for various databases. We can connect and run queries for a particular database using Python and without writing raw queries in the terminal or shell of that particular database, we just need to have that database installed in our system.



A database is an organized collection of data stored electronically. It is managed by a Database Management System (DBMS) that allows users to access, update, and manage data efficiently. Databases are primarily categorized into two types:

#### 1. Relational Databases (RDBMS)

Store data in structured tables with rows and columns. They support relationships between data entities. Examples:

- MySQL
- PostgreSQL
- SQLite

#### 2. Non-relational Databases (NoSQL)

Store data in various formats like documents, key-value pairs, or graphs, offering more flexibility. For Examples:

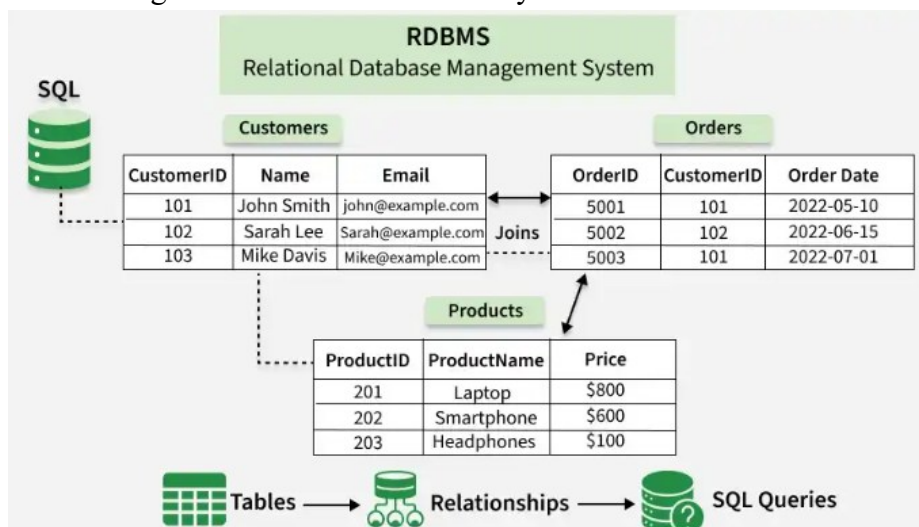
- MongoDB
- Cassandra

In this tutorial, we will focus on how to use Python with the most commonly used databases: **MySQL**, **SQLite**, and **MongoDB**. We will cover how to connect to these databases, run queries, and manage data efficiently using Python. Additionally, we will explore how to work with **JSON** data in Python.

## Relational Databases (RDBMS)

RDBMS stands for Relational Database Management Systems. A database is an organized collection of data stored in a computer system and usually controlled by a database management system (DBMS). It is a program that allows us to create, delete, and update a relational database.

- Data is organized in a structured, tabular format
- Tables are related to each other using keys
- Supports easy and powerful query execution
- Can handle large volumes of data efficiently

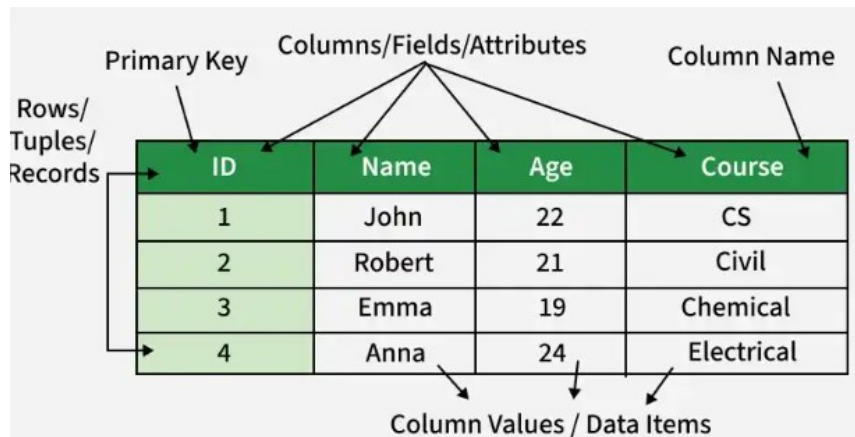


Relational Database Management Systems maintain data integrity by simulating the following features:

- **Entity Integrity:** No two records of the database table can be completely duplicates.
- **Referential Integrity:** Only the rows of those tables can be deleted which are not used by other tables. Otherwise, it may lead to data inconsistency.
- **User-defined Integrity:** Rules defined by the users based on confidentiality and access.
- **Domain integrity:** The columns of the database tables are enclosed within some structured limits, based on default values, type of data or ranges.

### Database Table

A table is a collection of related data in an organized manner in the form of rows and columns. Here is the pictorial representation of the table and its different components containing the data about different students that is ID, name, Age, and course.



### Features of RDBMS

- Data must be stored in tabular form in DB file and it should be organized in the form of rows and columns.
- Each row of table is called record/tuple . Collection of such records is known as the cardinality of the table.
- Each column of the table is called an attribute/field. Collection of such columns is called the arity of the table.
- No two records of the DB table can be same. Data duplicity is therefore avoided by using a candidate key. Candidate Key is a minimum set of attributes required to identify each record uniquely.
- Tables are related to each other with the help for foreign keys.
- Database tables also allow NULL values, that is if the values of any of the element of the table are not filled or are missing, it becomes a NULL value. (NOTE: Primary key cannot have a NULL value).

### Working of Relational Database Model

The relational database enables any table to be associated to another table by means of a common attribute. A change to a data model where data is stored, accessed, and related in tables without restructuring the tables that hold them in place of hierarchical structures.

- Each spreadsheet in the relational database model is a table that contains data, which is shown as rows (tuples) and columns (attributes).
- A data type is specified by an attribute (column), and the value of that particular data type is contained in each record (or row).
- A primary key is an attribute found in all tables in a relational database that serves as a unique row identifier.

| Roll number | Name   | Section |
|-------------|--------|---------|
| 1           | Sophia | A       |
| 2           | Jack   | B       |
| 3           | David  | A       |
| 4           | Olivia | C       |

### Uses of RDBMS

- Customer Relationship Management.
- Online Retail Platforms.

- Hospital Management Systems.
- Business Intelligence.
- Data Warehousing

## SQL Query in RDBMS

### 1. Creating a Table

#### Syntax:

```
CREATE TABLE table_name (
column1_name datatype constraint,
column2_name datatype constraint,
);
```

#### Example:

```
CREATE TABLE Employees (
EmployeeID INT PRIMARY KEY,
FirstName VARCHAR(50),
LastName VARCHAR(50),
BirthDate DATE,
Salary DECIMAL(10, 2)
);
```

### 2. Inserting Data into a Table

#### Syntax:

```
INSERT INTO table_name (column1_name, column2_name, ...)
VALUES (value1, value2, ...);
```

#### Example:

```
INSERT INTO Employees (EmployeeID, FirstName, LastName, BirthDate, Salary)
VALUES (1, 'John', 'Doe', '1985-06-15', 55000.00);
```

### 3. Querying Data (SELECT)

#### Syntax:

```
SELECT column1_name, column2_name, ...
FROM table_name
WHERE condition;
```

#### Example:

```
SELECT FirstName, LastName, Salary
FROM Employees
WHERE Salary > 50000;
```

### 4. Deleting Data from a Table

#### Syntax:

```
DELETE FROM table_name
WHERE condition;
```

#### Example:

```
DELETE FROM Employees
```

WHERE EmployeeID = 1;

## 5. Dropping a Table

### Syntax:

DROP TABLE table\_name;

### Example:

DROP TABLE Employees;

## Advantages of RDBMS

- **Easy to Manage:** Each table can be independently manipulated without affecting others.
- **Security:** It is more secure consisting of multiple levels of security. Access of data shared can be limited.
- **Flexible:** Updating of data can be done at a single point without making amendments at multiple files.
- **Users:** RDBMS supports client-side architecture storing multiple users together. Facilitates storage and retrieval of large amount of data.
- **Fault Tolerance:** Replication of databases provides simultaneous access and helps the system recover in case of disasters, such as power failures .
- Data consistency is ensured as RDBMS is based on ACID properties for data transactions.

## Disadvantages of RDBMS

- **High Cost and Extensive Hardware :** Huge costs and setups are required to make these systems functional.
- **Scalability:** In case of addition of more data, servers along with additional power, and memory are required.
- **Complexity:** Voluminous data creates complexity in understanding of relations and may lower down the performance.
- **Structured Limits:** The fields or columns of a relational database system is enclosed within various limits, which may lead to loss of data.

## DBMS vs RDBMS

| DBMS                                        | RDBMS                                                                 |
|---------------------------------------------|-----------------------------------------------------------------------|
| DBMS stores data as file.                   | RDBMS stores data in tabular form.                                    |
| Data elements need to access individually.  | Multiple data elements can be accessed at the same time               |
| No relationship between data.               | Data is stored in the form of tables which are related to each other. |
| Normalization is not present.               | Normalization is present.                                             |
| DBMS does not support distributed database. | RDBMS supports distributed database.                                  |
| It deals with small quantity of data.       | It deals with large amount of data.                                   |
| Not all Codd rules are satisfied.           | All 12 Codd rules are satisfied.                                      |

|                                                       |                                                       |
|-------------------------------------------------------|-------------------------------------------------------|
| Security is less                                      | More security measures provided.                      |
| Data fetching is slower for the large amount of data. | Data fetching is fast because of relational approach. |

## Non-relational Databases (NoSQL)

In the area of database management, the data is arranged in two ways which are Relational Databases (SQL) and Non-Relational Databases (NoSQL). While relational databases organize data into structured tables, non-relational databases use various flexible data models like key-value pairs, documents, graphs, and wide-column stores.

### Non-Relational Database

A non-relational database is a type of database that does not rely on the traditional tabular structure of rows and columns found in relational databases. Instead, it uses flexible data models such as key-value pairs, documents, graphs, and wide-column stores.

This flexibility allows non-relational databases to manage unstructured, semi-structured, and structured data efficiently. They were designed when data was expected to be partitioned across multiple machines to scale, in contrast to relational databases, which assumed the data would stay on a single machine.

### Key Features of Non-Relational Databases

1. **Scalability:** Non-relational databases like MongoDB and Cassandra are designed to horizontally scale across clusters of cheap commodity hardware, offering seamless scalability as data volumes and user loads increase.
2. **Flexibility in Data Models:** Unlike rigid table-based structures in relational databases, non-relational databases support flexible data models like document stores (e.g., JSON in MongoDB), key-value pairs (e.g., Redis), and wide-column stores (e.g., Cassandra), making it easier to store and manage unstructured or semi-structured data.
3. **Performance:** Non-relational databases are optimized for specific use cases such as real-time data ingestion, high-speed transactions, and rapid access to large volumes of data. They often outperform relational databases in these scenarios due to their distributed architecture and optimized data storage formats.
4. **Schemaless Design:** Non-relational databases typically do not enforce a rigid schema, allowing developers to evolve the data structure over time without downtime or complex migrations. This advantage is particularly beneficial in agile development environments and for handling diverse and unpredictable data types.
5. **High Availability and Fault Tolerance:** Many non-relational databases are designed with built-in replication and automatic failover capabilities, ensuring high availability and data redundancy. This makes them suitable for mission-critical applications where continuous uptime is essential.
6. **Cost-Effectiveness:** By using commodity hardware and open-source software, non-relational databases often provide a more cost-effective solution compared to traditional relational databases, especially at scale.



## Why Use a Non-Relational Database?

Non-relational databases are particularly suitable for:

- Applications requiring real-time data processing.
- Scenarios with massive and rapidly growing datasets.
- Use cases involving unstructured or semi-structured data.
- Agile development environments where data models evolve frequently.

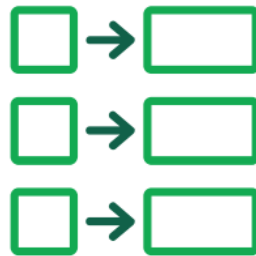
## Types of Non-Relational Databases

Non-relational databases come in four primary types, each catering to specific use cases:



### 1. Key/Value Database

Key-value databases use a straightforward schema: a unique key is paired with a collection of values, where the values can be anything from a string to a large binary object. One of the benefits of using this structure in a database is that we don't have to worry about complex queries. Because the system knows where the data is stored, it only sends a request to that particular server.



Key/Value Database

#### Key/ Value Database

Use Cases: Session management, caching, and real-time analytics.

Examples:

- Redis: Known for its in-memory data storage for high-speed applications.
- Amazon DynamoDB: A fully managed key-value database optimized for serverless applications.

| Key  | Value     |
|------|-----------|
| Name | John Snow |
| Age  | 23        |

## 2. Graph Database

Graph database is another type of non-relational database. A popular example of a graph database is Neo4J. This database stores information as a collection of nodes and edges, where the edges represent the relationships between the nodes.



Graph Database

Graph Database

Use Cases: Social networks, recommendation engines, and fraud detection.

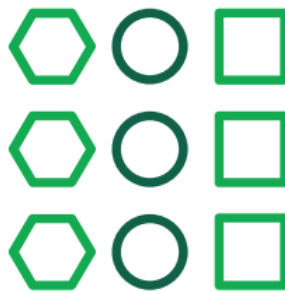
Examples:

- Neo4j: A widely used graph database for complex relationship queries.
- ArangoDB: A multi-model database supporting graph data structures

## 3. Column Oriented Database

A column-oriented or wide-column non-relational database is primarily designed for analytics. Cassandra is a commonly used column-oriented database. Column-Oriented databases organize data in columns rather than rows. This structure improves query performance by allowing efficient retrieval of relevant columns.

The primary key in a column-oriented database is the data or value, which is then mapped to row keys. This is the inverse, or opposite, of how the primary key works in a relational database.



Column Oriented  
or Wide Column

Wide Column

Use Cases: Analytics, time-series data, and IoT applications.

Examples:

- Apache Cassandra: Known for its high availability and scalability.

- HBase: Built on top of Hadoop for handling large datasets

| Name     | ID  |
|----------|-----|
| John     | 001 |
| Sherlock | 002 |

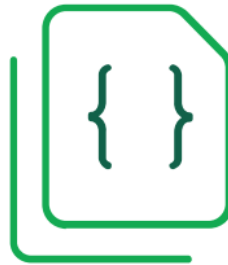
| Height | ID  |
|--------|-----|
| 5'9    | 001 |
| 6'3    | 002 |

| Age | ID  |
|-----|-----|
| 23  | 001 |
| 28  | 002 |

#### 4. Document Database

Document databases, such as MongoDB, store data in a single document, which can have different shapes within the single collection or table that stores the documents. It provides a clear means of capturing relationships using sub-documents and embedded arrays within a single document.

Document databases store data as documents, typically in JSON or BSON format. Each document can contain nested data and arrays, making it easy to represent complex relationships.



Document Database

Use Cases: Content management systems, e-commerce platforms, and mobile apps.

Examples:

- MongoDB: Popular for its scalability and support for flexible data models.
- Couchbase: Combines document storage with caching capabilities.

```
{
 "_id":
 ObjectId("5ef2d4b45b7f11b6d7a"),
 "user_id": "John Snow",
 "age": 23,
 "address":
 {
 "Country": "England"
 "City": "London",
 "Street": "Baker St."
 },
 "Hobbies": [violin]
}
```

#### Advantages of Non-Relational Databases

1. Scalability: Seamlessly scale horizontally across distributed systems.
2. Flexibility: Support for multiple data formats allows easy integration of diverse datasets.
3. Performance: Ideal for high-speed transactions and real-time applications.
4. Agility: Quick adaptation to evolving requirements due to schemaless designs.

5. Cost Efficiency: Reduced costs through open-source software and commodity hardware.

### Popular Non-Relational Database Systems

These are some Non-relational database names, that you might hear in the market. Decide on which Non-relational database software is best for your work, and master that. Here are some widely adopted non-relational database systems:

1. MongoDB: Versatile document database for unstructured and semi-structured data.
2. Redis: High-performance key-value store for caching and real-time analytics.
3. Apache Cassandra: Scalable wide-column store for handling massive data loads.
4. Neo4j: Graph database for modeling complex relationships.
5. Couchbase: Combines document storage and in-memory caching.

### Relational vs Non-Relational Database

Here's a comparison of Relational and Non-Relational Databases in tabular format:

| Feature         | Relational Database                         | Non-Relational Database                                                                               |
|-----------------|---------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Data Structure  | Tables with rows and columns                | Various formats (document, key-value, columnar, graph)                                                |
| Schema          | Structured schema enforced by schemas       | Flexible schema, often schema-less or dynamic                                                         |
| Query Language  | SQL (Structured Query Language)             | Query languages specific to the database type (e.g., JSON query languages, graph traversal languages) |
| ACID Compliance | ACID transactions                           | May vary; some offer ACID compliance, others eventual consistency                                     |
| Scalability     | Vertical and horizontal scaling options     | Horizontal scaling typically easier and more flexible                                                 |
| Flexibility     | Less flexible with rigid schema definitions | Highly flexible due to schema-less or dynamic schema                                                  |
| Performance     | Excellent for complex queries and joins     | Optimal for hierarchical data storage and retrieval                                                   |
| Examples        | MySQL, PostgreSQL, SQL Server               | MongoDB, Cassandra, Redis, DynamoDB                                                                   |

### Choosing the Right Non-Relational Database

When selecting a non-relational database, consider the following:

1. Data Requirements: Evaluate whether the data is structured, semi-structured, or unstructured.
2. Scalability Needs: Determine the expected data volume and transaction rates.
3. Query Patterns: Identify whether the application needs simple lookups or complex relationships.
4. Performance Expectations: Understand the application's latency and throughput requirements.
5. Integration: Check compatibility with existing tools and systems.

## Database Programming:

Data refers to raw, unorganized facts and figures that can be processed to generate meaningful information. A database is a structured system used to store, manage, and retrieve data efficiently for multiple users and applications. It's important due to following features:

- **Scalability:** Handles large volumes of data efficiently.
- **Data Integrity:** Maintains accuracy using rules and constraints.
- **Security:** Protects data through controlled access and permissions.
- **Analytics:** Helps analyze data to support better decision-making.

### Working of Databases

Databases work by organizing and storing information in a structured or unstructured format, allowing easy access, retrieval, and modification. At the core of every database system is the Database Management System (DBMS)- a software layer that acts as an intermediary between users and the raw data.

- The DBMS handles tasks like querying, updating, deleting and managing access permissions, without requiring users to know the physical details of where data is stored.
- When a user submits a request (such as a search or update), the DBMS processes the query, locates the relevant data, and returns results in a structured format.
- DBMSs provide features like backup, recovery, performance optimization and data security to ensure the system runs efficiently and reliably.

### According to industry rankings, the most popular databases of 2025 are:

Databases consist of several critical components that work together to **store, organize and retrieve data** effectively. Here is a detailed explanation of each component:

- **Data:** The actual information stored in the database, such as text, numbers, images, or files.
- **Schema:** The structural blueprint that defines how data is organized—tables, fields, data types, and relationships.
- **DBMS:** The software that manages database operations like storage, retrieval, and security (e.g., MySQL, Oracle).
- **Queries:** Instructions (usually SQL) used to retrieve or manipulate data within the database.
- **Users:** People or systems that interact with the database, each with specific roles and access permissions.

### Types of Databases

Databases can be classified into two primary types Relational (SQL) and NoSQL Databases. NoSQL is then further divided into four types: Document-oriented, Key-Value, Wide-Column, and Graph databases.

## 1. Relational Databases (RDBMS)

Relational databases organize data into tables made up of rows (records) and columns (fields). They use schemas (blueprints) to define how data is structured and how different tables relate to each other.

- Strict schema-based structure.
- Primary Keys (unique IDs) and Foreign Keys (relationships between tables).
- ACID compliant (ensures Atomicity, Consistency, Isolation, and Durability in transactions).
- Ideal for structured data.



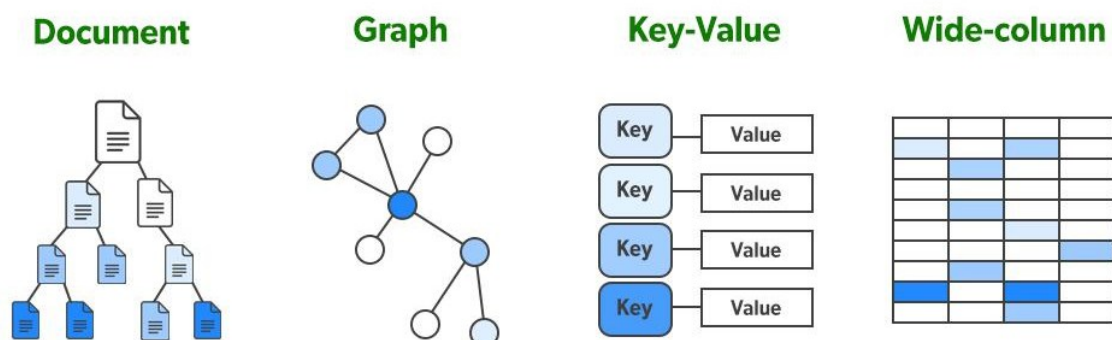
**Examples:** MySQL, PostgreSQL, Oracle, Microsoft SQL Server.

## 2. NoSQL Databases

"NoSQL" stands for "Not Only SQL". These databases are designed to handle unstructured or semi-structured data, such as text, images, videos or sensor data. They don't rely on the traditional table format.

- Flexible data models (no fixed schema).
- Scales horizontally for high-volume data.
- Often optimized for specific use cases like graphs or time-series data.

### Main Sub-Types of NoSQL Databases:



1. **Document Databases** – Store data as JSON-like documents. (Example: MongoDB)
2. **Key-Value Stores** – Store simple key-value pairs for fast lookups. (Example: Redis)
3. **Columnar Databases** – Store data by columns for analytics. (Example: Apache Cassandra)

#### 4. **Graph Databases** – Store nodes & relationships for connected data. (Example: Neo4j)

### **ACID Properties**

ACID stands for Atomicity, Consistency, Isolation, and Durability—four essential principles that ensure your database transactions are reliable, accurate, and secure.

- **Atomicity:** Ensures transactions complete fully or not at all.
- **Consistency:** Ensures the database moves from one valid state to another.
- **Isolation:** Ensures that multiple transactions can happen at the same time without affecting each other.
- **Durability:** Saves changes permanently after completion.

### **Real-World Applications of Databases**

Databases are essential part of our life. There are several everyday activities that involve our interaction with databases.

- **Banking:** Stores transactions and account details.
- **Transportation:** Manages bookings and schedules.
- **Education:** Tracks student records and grades.
- **Retail:** Handles inventory and customer orders.
- **Social Media:** Stores user data, messages, and media.
- **Multimedia:** Manages images, audio, and video.
- **Business & Data Science:** Analyzes trends and supports predictions.

### **Importance of Databases for Different Technology**

Databases are the engine behind every digital experience—whether you are building an app, training AI models, or running infrastructure at scale. Here is a breakdown of the most suitable database types for various technology domains:

#### **1. Databases for Web Development**

Web applications rely heavily on databases to store and manage user data, content, and transactions. Whether it is a blog or a large e-commerce platform, developers typically use:

- **Popular Databases:** MySQL, PostgreSQL, MongoDB, Firebase
- **Use Case:** Dynamic content, authentication, product catalogs

#### **2. Databases for Mobile Development**

Mobile apps require fast, lightweight databases optimized for limited device resources and offline access.

- **Popular Databases:** SQLite, Realm, Firebase Realtime DB
- **Use Case:** Local storage, syncing user preferences, offline-first apps

#### **3. Databases for DevOps**

DevOps teams manage CI/CD pipelines and infrastructure, where databases must support automation, monitoring, and scale.

- **Popular Databases:** PostgreSQL, Redis, InfluxDB, Cassandra
- **Use Case:** Logging, monitoring metrics, configuration storage

#### **4. Databases for Data Engineering**

Data engineers handle massive volumes and real-time pipelines. Their databases must be highly scalable and performant.

- **Popular Databases:** Apache Hadoop (HDFS), Apache Cassandra, Amazon Redshift, Google BigQuery
- **Use Case:** ETL processes, big data pipelines, real-time data streaming

## 5. Databases for Data Science

Data scientists need flexible querying, data aggregation, and easy integration with tools like Python and R.

- **Popular Databases:** PostgreSQL, MongoDB, Apache Hive, Snowflake
- **Use Case:** Feature extraction, exploratory analysis, modeling datasets

## 6. Databases for Artificial Intelligence

AI systems depend on structured, unstructured, and streaming data for model training and predictions.

- **Popular Databases:** MongoDB, Apache Cassandra, Google BigQuery, AWS S3 (data lake)
- **Use Case:** Storing training datasets, real-time inference, model feedback loops

## 7. Database for Cloud Computing

In cloud-native environments, databases must support autoscaling, high availability, and service integration.

- **Popular Databases:** Amazon Aurora, Google Cloud Spanner, Microsoft Azure Cosmos DB
- **Use Case:** SaaS platforms, distributed apps, serverless environments

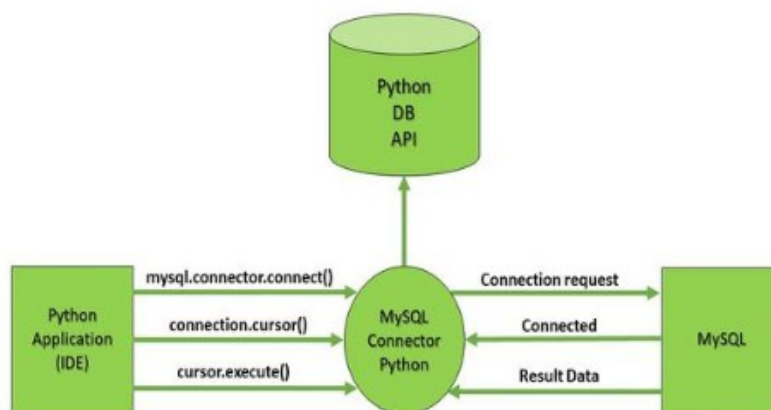
## 8. Database for Blockchain/Web3.0

Blockchain-powered systems need tamper-proof, decentralized databases for trustless transactions and transparency.

- **Popular Databases:** BigchainDB, IPFS, Ethereum (as data ledger), Chainlink
- **Use Case:** Immutable ledgers, decentralized identity, smart contract data.

## Connecting to database

In this article, we will learn how to connect SQL with Python using the MySQL Connector Python module. Below diagram illustrates how a connection request is sent to MySQL connector Python, how it gets accepted from the database and how the cursor is executed with result data.





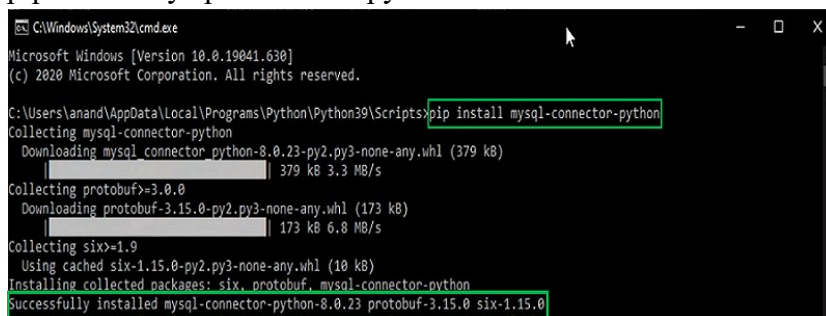
To create a connection between the MySQL database and Python the **connect()** method of **mysql.connector** module is used. We pass the database details like HostName, username and the password in the method call and then the method returns the connection object. Steps to Connect SQL with Python involve:

### 1. Install MySQL Database

Download and Install MySQL database in your system.

### 2. Open Command Prompt and Navigate to the location of PIP

After installing the MySQL database, open your Command prompt and run the commands given below to download and install "MySQL Connector". Here mysql.connector statement will help you to communicate with the MySQL database. Click here to see How to install PIP? pip install mysql-connector-python



```

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19041.630]
(c) 2020 Microsoft Corporation. All rights reserved.

C:\Users\lanand\AppData\Local\Programs\Python\Python39\Scripts>pip install mysql-connector-python
Collecting mysql-connector-python
 Downloading mysql_connector_python-8.0.23-py2.py3-none-any.whl (379 kB)
 |#####| 379 kB 3.3 MB/s
Collecting protobuf>=3.0.0
 Downloading protobuf-3.15.0-py2.py3-none-any.whl (173 kB)
 |#####| 173 kB 6.8 MB/s
Collecting six>=1.9
 Using cached six-1.15.0-py2.py3-none-any.whl (10 kB)
Installing collected packages: six, protobuf, mysql-connector-python
Successfully installed mysql-connector-python-8.0.23 protobuf-3.15.0 six-1.15.0

```

### 3. Test MySQL Connector

To check if the installation was successful or if you already installed "MySQL Connector" go to your IDE and run the given below code :

```
import mysql.connector
```

If the above code gets executed with no errors "MySQL Connector" is ready to be used.

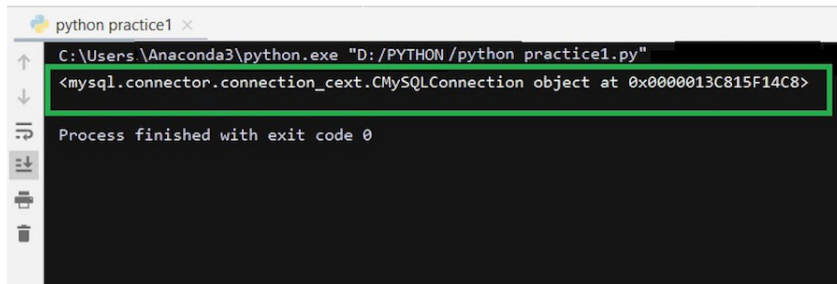
### 4. Create Connection

Now to connect SQL with Python run the code given below in your IDE.

- **mysql.connector** allows Python programs to access MySQL databases.
- **connect()** method of the MySQL Connector class with the arguments will connect to MySQL and would return a MySQLConnection object if the connection is established successfully.
- **host = "localhost"**: The host refers to the server where your MySQL database is hosted. In most cases for local development you can set it as "localhost". If the MySQL server is hosted on a different machine, you would replace "localhost" with the appropriate IP address or hostname of that machine.
- **user = "yourusername"**: here "yourusername" should be the same username as you set during MySQL installation.
- **password = "your\_password"**: here "your\_password" should be the same password as you set during MySQL installation.

**Import mysql.connector**

```
mydb = mysql.connector.connect(
 host = "localhost",
 user = "yourusername",
 password = "your_password"
)
print(mydb)
```

**Output:**

Now that we know how to connect SQL database with python we can use it to create databases and tables, store data and manipulate them all directly through python. We can create many projects using this way.

**Creating Tables**

MySQL is a Relational Database Management System (RDBMS) whereas the structured Query Language (SQL) is the language used for handling the RDBMS using commands i.e Creating, Inserting, Updating and Deleting (**CRUD**) the data from the databases. **SQL commands are case insensitive** i.e "**CREATE**" and "**create**" signify the same command.

In this article, we will discuss how to create a table in MySQL using Python.

**Installation**

To get started, you'll need to install the MySQL Connector for Python. This allows Python to interact with a MySQL database.

1. Open the command prompt or terminal.
2. Navigate to your Python script directory.
3. Run the following command to install the MySQL connector:

```
pip install mysql-connector
```

**Python Mysql Connector Module Methods****1. connect()**

This method is used to establish a connection with the MySQL server. It takes the following arguments:

- **user:** Username associated with the MySQL server.
- **password:** Password for authentication.
- **database:** The database to connect to.

**2. cursor()**

This method creates a cursor in the system memory. The cursor is the workspace for executing SQL commands. It remains open for the entire session.

**3. execute()**

The execute() method takes an SQL query as an argument and executes it. You can use this method to perform operations like creating, inserting, updating, or deleting data.

**Creating Database**

A database is an organized collection of data stored in **tables**. These tables are designed to make data manipulation (like creation, insertion, updates, and deletions) easy and efficient.

**SQL command for Creating Database:**

```
CREATE DATABASE database_name;
```

**Example:** Creating a database called "College".

```
import mysql.connector as SQLC
```

```
Connect to MySQL
DataBase = SQLC.connect(
 host="localhost",
 user="root",
 password="your_password"
)

Create a cursor object
Cursor = DataBase.cursor()

Execute command to create the database
Cursor.execute("CREATE DATABASE College")

print("College database is created")
```

**Output:**

College Data base is created

**Note:** Make sure to enter your own MySQL user and password to connect with MySQL server.

**What is a Table**

The table is a collection of data organized in the form of rows and columns. Table is present within a database as you can see from the above image.

- Rows are also called tuples
- Columns are called the attributes of the table

**SQL command for Creating a Table:**

As we have discussed that tables are created inside a database so we need to first make sure to select or move into to the database before creating a table. To select the "College" database use this command:

```
USE college;
```

This will select the "College" database, now we can proceed to create tables inside it. Here's how to create a table:

```
CREATE TABLE table_name
(
column_name_1 column_Data_type,
column_name_2 column_Data_type,
:
:
column_name_n column_Data_type
);
```

### SQL Data Types:

- **Numeric:** INT, FLOAT, DOUBLE
- **Character/String:** VARCHAR, CHAR, TEXT
- **Date/Time:** DATE, TIME, DATETIME
- **Unicode:** NCHAR, NVARCHAR
- **Binary:** BLOB

For instance, let's suppose we have to create a table named "**STUDENT**" having two columns, here's how we can do it:

```
CREATE TABLE Student (
Name VARCHAR(255),
Roll_no INT
);
```

### Creating a Table in MySQL Using Python

Let's consider an example where we create a table named Student inside the "**College**" database. The Student table will have two columns: **Name** (for the student's name) and **Roll\_no** (for the student's roll number).

### Python Code to Create a Table:

```
import mysql.connector as SQLC
```

```
def CreateTable():
```

```
 # Connect to the College database
 DataBase = SQLC.connect(
 host="localhost", # Server name
 user="root", # MySQL username
 password="your_password", # MySQL password
 database="College" # Database name
)

 # Create a cursor object
 Cursor = DataBase.cursor()
```

```
SQL query to create the table
TableName = """CREATE TABLE Student (
 Name VARCHAR(255),
 Roll_no INT
);"""

Execute the query to create the table
Cursor.execute(TableName)
print("Student Table is Created in the Database")
```

```
Calling the CreateTable function
```

```
CreateTable()
```

**Output:**

Student Table is Created in the Database

## Insert

In Python, `append()`, `extend()` and `insert()` are list methods used to add elements to a list. Each method has different behaviors and is used in different scenarios.

### Append

The `append()` method adds a single element which can be string, integer, tuple or another list to the end of the list. If we pass a list or another iterable, it will add the entire list as a single element.

**Syntax:**

```
list.append(element)
```

The element can be a string, integer, tuple or another list.

**Example:**

```
li = ['geeks']
```

```
use method
li.append('for')
li.append('geeks')
```

```
display list
print(l)
```

### Insert

The `insert()` method allows us to insert an element(string, object or integer) at any specified position (index) in the list. If the index is greater than the list's length, the element is added to the end. If the index is negative, it starts counting from the end of the list.

**Syntax:**

```
list.insert(index, element)
```

**Example:**

```
li = ['geeks', 'geeks']
```

```
use method
li.insert(1, 'for')
```

```
display list
print(li)
```

**Output:**

```
['geeks', 'for', 'geeks']
```

**Extend**

The extend() method adds all the elements of an iterable (list, tuple, string) to the end of the list. Unlike append(), which adds a single element, extend() adds each element from the iterable to the list.

**Syntax:**

```
list.extend(iterable)
```

**Example:**

```
python program to demonstrate
working of extend function
```

```
assign list
li = ['hello', 'welcome', 'to']
```

```
use method
li.extend(['geeks', 'for', 'geeks'])
```

```
display list
print(li)
```

**Output:**

```
['hello', 'welcome', 'to', 'geeks', 'for', 'geeks']
```

**Difference between Append, Extend and Insert**

| Parameter                    | append()                                            | insert()                                             | extend()                                                   |
|------------------------------|-----------------------------------------------------|------------------------------------------------------|------------------------------------------------------------|
| <b>Purpose</b>               | Adds a single element to the end of the list.       | Inserts an element at a specified index in the list. | Adds each element from an iterable to the end of the list. |
| <b>Type of Argument</b>      | A single element (any data type, including a list). | An index (integer) and an element (any data type).   | An iterable (list, tuple, string).                         |
| <b>Location of Insertion</b> | Always at the end of the list.                      | At the specified index position in the list.         | At the end of the list (each element from the iterable).   |
| <b>Modifies the List</b>     | Yes, modifies the list in place.                    | Yes, modifies the list in place.                     | Yes, modifies the list in place.                           |

|                       |                                          |                                                                 |                                                   |
|-----------------------|------------------------------------------|-----------------------------------------------------------------|---------------------------------------------------|
| <b>Multiple Items</b> | No, only one element is added at a time. | No, only one element is added at a specified position.          | Yes, it can add multiple items from the iterable. |
| <b>Iterables</b>      | It adds the list as a single element.    | It can add a list, tuple, or any object at the specified index. | Adds elements from an iterable.                   |

## Update

**update()** method add multiple elements to a set. It allows you to modify the set in place by adding elements from an iterable (like a list, tuple, dictionary, or another set). The method also ensures that duplicate elements are not added, as sets inherently only contain unique elements. **Example:**

```
s = {1, 2, 3}
```

```
s.update([3, 4, 5])
print(s)
```

### Output

```
{1, 2, 3, 4, 5}
```

### Syntax of update()

```
set.update(*others)
```

- **\*others:** One or more iterables (sets, lists, tuples, strings, etc.) from which elements will be added to the set.

### Examples of update()

**Example 1:** We can use **update()** to add all elements from one set to another. Any duplicate elements will be ignored.

```
d1 = {'a': 1, 'b': 2}
d2 = {'b': 3, 'c': 4}
```

```
d1.update(d2)
print(d1)
```

### Output

```
{'a': 1, 'b': 3, 'c': 4}
```

**Explanation:** Since 3 is present in both sets, but it only appears once in the updated set1 because sets don't allow duplicates.

**Example 2:** We can also pass a list (or any iterable) to **update()** to add its elements to the set.

```
d1 = {'a': 1, 'b': 2}
d1.update(['c', 3], ('d', 4))
print(d1)
```

### Output

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

**Explanation:** `update()` method is called on **d1** with the iterable `[('c', 3), ('d', 4)]`, which adds the key-value pairs 'c': 3 and 'd': 4 to **d1**.

**Example 3:** You can update a set with multiple iterables in a single operation.

```
d1 = {'a': 1, 'b': 2}
d1.update(c=3, d=4)
print(d1)
```

### Output

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

**Explanation:** `update()` adds the key-value pairs 'c': 3 and 'd': 4 to **d1**, creating new keys with the specified values.

**Example 4:** `update()` method can also be used to add individual characters from a string to the set.

```
s1 = {1, 2, 3}
s1.update("hello")
print(s1)
```

### Output

```
{1, 2, 3, 'e', 'l', 'h', 'o'}
```

**Explanation:** Each character in the string "hello" is treated as a unique element and added to the set. Duplicates (like 'l') are ignored.

## Delete and Read Operations

When data is no longer needed, it's important to free up space for more relevant information. Python's file handling capabilities allow us to manage files easily, whether it's deleting entire files, clearing contents or removing specific data.

For more on file handling, check out:

- [File Handling in Python](#)
- [Reading and Writing to Text Files in Python](#)

Let's explore the various ways to delete data from files. Consider our sample file **sample.txt**, which contains the following data:

```
≡ sample.txt
1 Python is versatile.
2 Used in tech fields.
```

sample.txt file

### Using `os.remove()`

This method is used when you want to permanently delete the entire file from the file system. The `os.remove()` function checks if the file exists and then removes it. It's useful for file cleanup tasks or when the data is no longer needed. Note that this method cannot delete directories, only files.

**import os**



```
if os.path.exists("sample.txt"):
 os.remove("sample.txt")
 print("Deleted")
```

**else:**

```
 print("Not found")
```

### Output

Deleted

**Explanation:** This code checks if sample.txt exists. If it does, it deletes the file using `os.remove()` and prints "Deleted". If not, it prints "Not found". This prevents errors by ensuring the file is only deleted if it exists.

### Using truncate()

`truncate()` method is used to clear all contents of a file without deleting the file itself. This is useful when you want to reuse the same file but start fresh with no data inside. The file is opened in read/write mode and `truncate()` cuts off all existing content.

**with** open("sample.txt", "r+") as f:

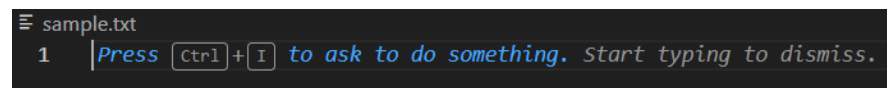
```
 f.seek(0)
```

```
 f.truncate()
```

```
print("Cleared")
```

### Output

Cleared



```
sample.txt
1 | Press Ctrl + I to ask to do something. Start typing to dismiss.
```

Using `truncate()`

**Explanation:** This code opens sample.txt in read and write mode. It moves the file pointer to the beginning using `seek(0)` and then clears all its contents with `truncate()`. The file remains, but its data is erased.

### Using filtering

This method helps when you want to remove specific lines or content from a file, while keeping the rest intact. You read all lines, filter out the ones you don't want and write the rest back to the same file. This approach is ideal for making precise edits to a file's contents.

```
file_path = r"C:\Users\GFG0578\Desktop\nw\sample.txt"
```

**with** open(file\_path, "r+") as f:

```
 lines = [line for line in f if line.strip() != "Used in tech fields."]
```

```
 f.seek(0)
```

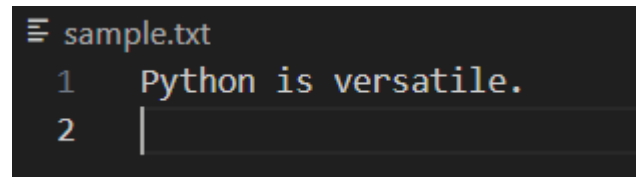
```
 f.writelines(lines)
```

```
 f.truncate()
```

```
print("Removed")
```

### Output

Removed



```

≡ sample.txt
1 Python is versatile.
2 |

```

Using filtering

**Explanation:** This code filters out the line "Used in tech fields." from the file, rewrites the remaining lines and truncates extra data, removing only the targeted line while keeping the rest.

### Using temporary file

This is a safe method to remove specific data from a file. You read the original file line by line and write only the necessary content to a temporary file. After filtering, the original file is replaced with the temp file. This avoids direct editing of the original file, reducing the risk of data corruption.

### Import os

```
src = r"C:\Users\GFG0578\Desktop\nw\sample.txt"
```

```
temp = r"C:\Users\GFG0578\Desktop\nw\temp.txt"
```

```
with open(src, "r") as infile, open(temp, "w") as outfile:
```

```
 for line in infile:
```

```
 if "Used in tech fields." Not in line:
```

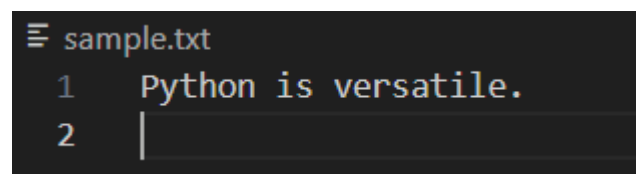
```
 outfile.write(line)
```

```
os.replace(temp, src)
```

```
print("file filtered")
```

### Output

file filtered



```

≡ sample.txt
1 Python is versatile.
2 |

```

Using temporary file

**Explanation:** This code filters out lines containing "Used in tech fields." By copying the remaining lines to a temporary file, then replaces the original file with the updated one.

## Transaction Control,

In Python, the **commit()** and **rollback()** methods play an essential role in database transactions, ensuring atomicity, consistency, isolation, and durability (**ACID properties**).

These methods help manage changes to the database in a reliable way, ensuring data integrity even in case of errors or failures.

In this article, we will focus on the use of `commit()` and `rollback()` methods for handling database transactions.

### MySQL Table in used in this article:

```
mysql> select * from STUDENT;
```

| NAME         | BRANCH                           | ROLL    | SECTION | AGE | Roll_no |
|--------------|----------------------------------|---------|---------|-----|---------|
| Ritam Barik  | Information Technology           | 1706254 | IT-3    | 21  | 1       |
| Rishi Kumar  | Information Technology           | 1706253 | IT-3    | 23  | 2       |
| Rituraj Saha | Information Technology           | 1706256 | IT-3    | 20  | 3       |
| S.K. Anirban | Computer Science and Engineering | 1706271 | CSE-5   | 21  | 5       |
| Soumya Das   | Computer Science and Engineering | 1706273 | CSE-2   | 21  | 55      |

### 1. `commit()` method

The `commit()` method is used to save changes made to the database permanently. It confirms that all the changes executed during the current session, such as `INSERT`, `UPDATE`, or `DELETE` queries, should be saved in the database. Without `commit()`, any changes would be lost once the connection is closed.

### Syntax

`comm.commit()`

- **comm:** refers to the database connection object
- **commit()** method ensures that changes are persistent in the database.

For a better understanding of the concept look into the code below followed by the code explanation. The below demonstration of the `commit()` method is performed on a MySQL database.

### Example: Updating a Record and Committing the Changes

The following example demonstrates how to update a student's age and commit the change to the database:

```
import mysql.connector
```

```
Connecting to the Database
```

```
mydb = mysql.connector.connect(
 host='localhost',
 database='College',
 user='root',
 password='your_password' # Use your actual password
)
```

```
cs = mydb.cursor()
```

```
SQL query to update the age of a student named 'Rishi Kumar'
```

```
statement = "UPDATE STUDENT SET AGE = 25 WHERE Name = 'Rishi Kumar'"
```

```
cs.execute(statement)
```

```
Commit the changes to make them permanent in the database
mydb.commit()
```

```
Disconnecting from the database
mydb.close()
```

### Output:

```
mysql> mysql> select * from STUDENT;
```

| NAME         | BRANCH                           | ROLL    | SECTION | AGE | Roll_no |
|--------------|----------------------------------|---------|---------|-----|---------|
| Ritam Barik  | Information Technology           | 1706254 | IT-3    | 21  | 1       |
| Rishi Kumar  | Information Technology           | 1706253 | IT-3    | 25  | 2       |
| Rituraj Saha | Information Technology           | 1706256 | IT-3    | 20  | 3       |
| S.K. Anirban | Computer Science and Engineering | 1706271 | CSE-5   | 21  | 5       |
| Soumya Das   | Computer Science and Engineering | 1706273 | CSE-2   | 21  | 55      |

### Explanation:

- program updates the **AGE** field for the student **Rishi Kumar**.
- **commit()** method ensures that this update is saved permanently to the College database.

## 2. rollback() method

The `rollback()` method is used to undo changes made during the current transaction. If a condition arises where one is not satisfied with the changes made to the database or a database transaction fails, the `rollback()` method can be used to retrieve the original data that was changed through the `commit()` method.

### Syntax

```
comm.rollback()
```

- **comm:** refers to the database connection object
- **rollback()** method discards all changes since the last commit.

### Example: Handling Errors with rollback()

The following example demonstrates how to use the `rollback()` method to revert changes if a database operation fails:

```
import mysql.connector
```

```
from mysql.connector import Error
```

```
try:
```

```
 # Connecting to the Database
```

```
 mydb = mysql.connector.connect(
```

```
 host='localhost',
```

```
 database='College',
```

```
 user='root',
```

```
 password='your_password' # Use your actual password
```

```
)
```

```
 cs = mydb.cursor()
```

```
 # SQL query to update the age of a student named 'Rishi Kumar'
```

```

statement = "UPDATE STUDENT SET AGE = 25 WHERE Name = 'Rishi Kumar'"

cs.execute(statement)

Commit the changes to make them permanent
mydb.commit()

Success message
print("Database Updated!")

except mysql.connector.Error as error:
 # Error message if the transaction fails
 print(f"Database Update Failed! Error: {error}")

 # Rollback changes if there's an error
 mydb.rollback()

Disconnecting from the database
mydb.close()

```

**Output:**

If the database transaction is successful the output will be,  
Database Updated!

If the database transaction fails the output is an error raised as,  
Database Update Failed!

### Disconnecting from a database, Exception Handling in Databases

JDBC is a Java API that allows interaction with databases. JDBC exceptions like SQLException commonly occur due to connection issues, SQL syntax errors or data type mismatches and are handled using standard Java exception handling.

- Uses try-catch blocks to handle SQLException and related exceptions
- Provides detailed error information like error code and SQL state using methods like getMessage() and getErrorCode()
- Ensures proper resource cleanup using finally block or try-with-resources

**Importance of JDBC Exception Handling**

When we communicate with databases, some problems occur like:

- Invalid SQL syntax bad query
- Connection time out problem
- Wrong data types mentioned
- miss database drivers

The below code describes how we handle exceptions.

```
try {
```

```
//Risky code is define in that block
}
catch(Exception ex) {
//Exception handling code define hare

}
finally {
 // This block always used for closing resource
}
```

### Types of JDBC Exceptions in Java

Below are some common exceptions in JDBC

**1. SQLException:** it is a Checked Exception. it is occur when we have defined wrong sql syntax like Incorrect SQL syntax, Invalid table or column name.

#### Incorrect SQL syntax:

##### Example:

```
SELECT * FORM person; // 'FROM' is misspelled
```

##### Error:

```
java.sql.SQLException: Syntax error in SQL statement
```

#### Invalid table or column names

##### Example:

```
SELECT * FROM perso; // 'person' is misspelled
```

##### Error:

```
java.sql.SQLException: Table 'person' doesn't exist
```

**2. ClassNotFoundException:** It is a checked exception. This occur when the JDBC driver class is not found.

##### Example:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

If the MySQL driver is missing from the classpath.

**3. SQLTimeoutException:** It is a subclass of SQLException. It occurs when a query takes too long and exceeds the configured timeout limit.

##### Example:

```
statement.setQueryTimeout(seconds)
```

**Example:** The below-described code describes how to handle Jdbc Exception.

```
package org.vishnu;
```

```
import java.sql.*;
```

```
import javax.sound.midi.Soundbank;
```

```
public class App {
```

```
 public static void main(String[] args) throws Exception
```

```
 {
```

```
 // fetching data from database
```

```
 String email = "gaurav@gmail.com";
```

```
 // we have use try with resource for automatically close JDBC resources
```

```

try {
 // Load and register the driver
 Class.forName("com.mysql.cj.jdbc.Driver");
 try (
 // create connection
 Connection con
 = DriverManager.getConnection(
 "jdbc:mysql://localhost:3306/jdbc_db",
 "root", "password");
 // execute dynamic query
 PreparedStatement ps = con.prepareStatement(
 "select * from ragister");
 // store the result
 ResultSet rs = ps.executeQuery();
 // process the result
) {
 while (rs.next()) {
 String name1 = rs.getString("name");
 System.out.println("name->" + name1);
 String email1 = rs.getString("email");
 System.out.println("email->" + email1);
 String poss = rs.getString("password");
 System.out.println("poss->" + poss);
 String gen = rs.getString("gender");
 System.out.println("gender->" + gen);
 String city1 = rs.getString("city");
 System.out.println("city->" + city1);
 System.out.println(
 "-----");
 }
 }
}
catch (SQLException se) {
 System.err.println("Error in sql syntax: "
 + se.getMessage());
}
catch (SQLException te) {
 System.err.println("Query Time Expire: "
 + te.getMessage());
}
catch (ClassNotFoundException e) {
 System.err.println("jdbc driver not found.");
}
catch (Exception e) {

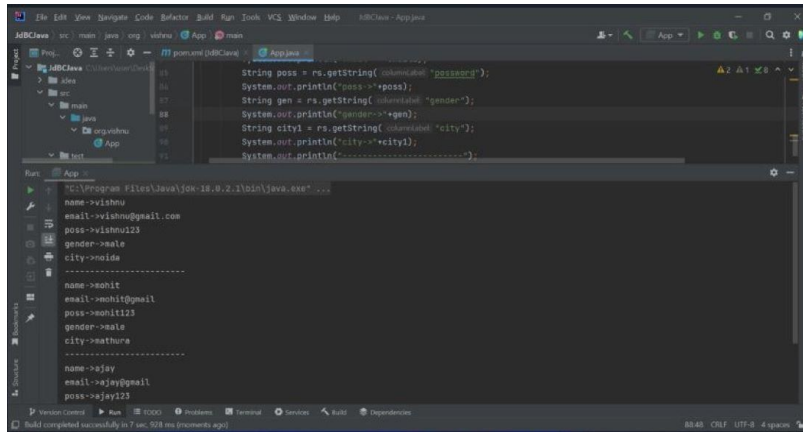
```

```

 System.err.println("Unexpected error: "
 + e.getMessage());
 }
}
}

```

**Output:**



output

### Explanation

- Connects to a MySQL database named jdbc\_db using JDBC.
- Uses try-with-resources to automatically close JDBC resources.
- Loads the MySQL JDBC driver using Class.forName.
- Establishes a connection using DriverManager.getConnection.
- Prepares and executes a SQL query to select all records from the register table.
- Iterates through the ResultSet to print values of name, email, password, gender and city.
- Catches and handles SQLException for SQL syntax issues.
- Catches and handles SQLException if the query exceeds time limits.
- Catches and handles ClassNotFoundException if the JDBC driver is missing.
- Catches any other unexpected exceptions.

### Best Way for JDBC Exception Handling

- Always close resources using try-with-resources.
- Avoid exposing DB details in production error messages.
- Use meaningful messages in catch blocks.
- Don't catch generic Exception unless necessary



# CHAPTER 7

## DATA ANALYSIS USING PYTHON

---

### Introduction to Data Analysis

Data Analysis involves collecting, transforming and organizing data to generate insights, support decision making and solve business problems.

- Helps in making informed, data driven decisions
- Identifies patterns and trends for better predictions
- Supports solving real world business problems
- Converts raw data into meaningful insights

### Analyzing Numerical Data with NumPy

NumPy is a Python library used for fast and efficient numerical computations. It provides multidimensional arrays and built in functions that simplify data analysis, mathematical operations and large scale data processing.

### Arrays in NumPy

NumPy arrays store elements of the same data type and support multiple dimensions. The number of dimensions is called rank and the size of each dimension is called shape.

**import numpy as np**

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Array:\n", arr)
print("Shape:", arr.shape)
print("Dimensions:", arr.ndim)
```

**Output:**

```
... Array:
 [[1 2 3]
 [4 5 6]]
 Shape: (2, 3)
 Dimensions: 2
```

### Creating NumPy Arrays

Arrays can be created using lists, tuples or built in functions like zeros, ones, arange and empty.

```
a = np.array([1, 2, 3])
b = np.zeros((2, 2))
c = np.ones((2, 2))
d = np.arange(0, 10, 2)
e = np.empty((2, 2), dtype=int)
```

```
print("Matrix A\n", a, "\n Matrix B\n", b, "\n Matrix C\n", c, "\n Matrix D\n", d, "\n Matrix E\n", e)
```

**Output:**

```
... Matrix A
 [1 2 3]
 Matrix B
 [[0. 0.]
 [0. 0.]]
 Matrix C
 [[1. 1.]
 [1. 1.]]
 Matrix D
 [0 2 4 6 8]
 Matrix E
 [[4607182418800017408 4607182418800017408]
 [4607182418800017408 4607182418800017408]]
```

### Operations on Numpy Arrays

NumPy allows efficient element wise operations on arrays, making numerical computations faster and more optimized compared to traditional Python methods.

- **Addition:** Adds corresponding elements of two arrays
- **Subtraction:** Subtracts elements of one array from another
- **Multiplication:** Performs element wise multiplication
- **Division:** Divides elements of one array by another

**import numpy as np**

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
```

```
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
```

**Output:**

```
... Addition: [11 22 33]
 Subtraction: [9 18 27]
 Multiplication: [10 40 90]
 Division: [10. 10. 10.]
```

### NumPy Array Indexing

Indexing is used to access individual elements in an array using their position. It works similarly to Python lists but is more useful for multi dimensional data.

```
arr = np.array([10, 20, 30, 40])
```

```
print("First element:", arr[0])
```

```
print("Last element:", arr[-1])
```

**Output:**

```
... First element: 10
 Last element: 40
```

Array Indexing

### NumPy Array Slicing

Slicing allows accessing a range of elements from an array. It is useful for working with subsets of data.

```
arr = np.array([10, 20, 30, 40, 50])
```

```
print("Slice (1 to 3):", arr[1:4])
print("First three elements:", arr[:3])
```

**Output:**

```
... Slice (1 to 3): [20 30 40]
 First three elements: [10 20 30]
```

---

### NumPy Array Broadcasting

Broadcasting allows operations between arrays of different shapes without explicitly resizing them, improving efficiency and reducing code complexity.

```
arr = np.array([1, 2, 3])
```

```
print("Add scalar:", arr + 5)
```

```
b = np.array([[1], [2], [3]])
c = np.array([10, 20, 30])
```

```
print("Broadcasting result:\n", b + c)
```

**Output:**

```
... Add scalar: [6 7 8]
 Broadcasting result:
 [[11 21 31]
 [12 22 32]
 [13 23 33]]
```

---

### Analyzing Data Using Pandas

Pandas is a Python library used for handling structured (relational or labeled) data. Built on top of NumPy, it provides flexible data structures and tools for data manipulation, analysis and time series operations.

- Used for working with structured and tabular data
- Built on top of NumPy for high performance
- Supports data cleaning, transformation and analysis

## Series in Pandas

A Series is a one dimensional labeled array capable of holding any data type (integers, strings, floats, etc.). Each element has an associated index.

- Represents a single column of data
- Supports indexing and labeling
- Can store different data types

**import pandas as pd**

```
data = [10, 20, 30, 40]
series = pd.Series(data)
```

```
print(series)
print("First element:", series[0])
```

**Output:**

```
... 0 10
 1 20
 2 30
 3 40
 dtype: int64
 First element: 10
```

## DataFrame in Pandas

A DataFrame is a two dimensional labeled data structure with rows and columns, similar to a table or spreadsheet.

- Represents tabular data (rows and columns)
- Each column can have a different data type
- Most commonly used Pandas structure

**import pandas as pd**

```
data = {
 "Name": ["A", "B", "C"],
 "Age": [20, 21, 22]
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
print("Column Access:\n", df["Name"])
```

**Output:**

---

```

... Name Age
 0 A 20
 1 B 21
 2 C 22
 Column Access:
 0 A
 1 B
 2 C
 Name: Name, dtype: object

```

---

### Pandas CRUD Operations

Pandas allows easy Create, Read, Update and Delete operations on data stored in CSV files, making it practical for real-world datasets. It is known as CRUD Operations.

- **Create:** Create and save a DataFrame as a CSV file
- **Read:** Load data from a CSV file
- **Update:** Modify values or add new columns
- **Delete:** Remove rows or columns

#### import pandas as pd

```

data = {"Name": ["A", "B", "C"], "Age": [20, 21, 22]}
df = pd.DataFrame(data)
df.to_csv("data.csv", index=False)
df = p.read_csv("data.csv")
print("Read:\n", df)

```

```

df["Age"] = df["Age"] + 1
df["City"] = ["Delhi", "Mumbai", "Pune"]
print("\nUpdate:\n", df)

```

```

df = df.drop("City", axis=1)
df = df.drop(0, axis=0)
print("\nDelete:\n", df)

```

#### Output:

```

... Read:
 Name Age
 0 A 20
 1 B 21
 2 C 22

 Update:
 Name Age City
 0 A 21 Delhi
 1 B 22 Mumbai
 2 C 23 Pune

 Delete:
 Name Age
 1 B 22
 2 C 23

```

## Exploratory Data Analysis (EDA)

### 1. Data Inspection

Pandas provides quick methods to understand the structure, summary and content of a dataset.

These functions help in exploring data before analysis.

- **info():** Displays dataset structure, column names, data types and non null values
- **describe():** Shows statistical summary like mean, min, max and standard deviation
- **value\_counts():** Counts frequency of unique values in a column
- **head():** Displays first few rows of the dataset
- **tail():** Displays last few rows of the dataset

**import pandas as pd**

```
data = {
 "Name": ["A", "B", "C", "A", "B"],
 "Age": [20, 21, 22, 20, 21]
}
df = pd.DataFrame(data)

print("Info:")
print(df.info())

print("\nDescribe:\n", df.describe())

print("\nValue Counts:\n", df["Name"].value_counts())

print("\nHead:\n", df.head())

print("\nTail:\n", df.tail())
```

**Output:**

```

... Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
Column Non-Null Count Dtype
--- ------ -
0 Name 5 non-null object
1 Age 5 non-null int64
dtypes: int64(1), object(1)
memory usage: 212.0+ bytes
None

```

Describe:

|       | Age       |
|-------|-----------|
| count | 5.000000  |
| mean  | 20.800000 |
| std   | 0.83666   |
| min   | 20.000000 |
| 25%   | 20.000000 |
| 50%   | 21.000000 |
| 75%   | 21.000000 |
| max   | 22.000000 |

Value Counts:

|   | Name |
|---|------|
| A | 2    |
| B | 2    |
| C | 1    |

Name: count, dtype: int64

Head:

|   | Name | Age |
|---|------|-----|
| 0 | A    | 20  |
| 1 | B    | 21  |
| 2 | C    | 22  |
| 3 | A    | 20  |
| 4 | B    | 21  |

Tail:

|   | Name | Age |
|---|------|-----|
| 0 | A    | 20  |
| 1 | B    | 21  |
| 2 | C    | 22  |
| 3 | A    | 20  |
| 4 | B    | 21  |

## 2. Data Manipulation in Pandas

Pandas provides multiple operations to efficiently select, organize and transform data for analysis.

### Indexing and Selection

Indexing and Selection are used to access specific rows, columns or subsets of data.

**import pandas as pd**

```

df = pd.DataFrame({
 "Name": ["A", "B", "C"],

```

```
"Age": [20, 21, 22]
})
```

```
print(df["Name"])
print(df.iloc[0:2])
```

**Output:**

```
... 0 A
 1 B
 2 C
 Name: Name, dtype: object
 Name Age
0 A 20
1 B 21
```

**Grouping and Aggregation**

Grouping and Aggregation Groups data based on a column and applies aggregate functions like mean, sum, etc.

```
import pandas as pd
```

```
df = pd.DataFrame({
 "Name": ["A", "B", "A"],
 "Marks": [80, 90, 85]
})
```

```
print(df.groupby("Name")["Marks"].mean())
```

**Output:**

```
... Name
 A 82.5
 B 90.0
 Name: Marks, dtype: float64
```

**Merging and Joining**

Merging and Joining combines multiple DataFrames based on common columns.

```
import pandas as pd
```

```
df1 = pd.DataFrame({"Name": ["A", "B"], "Age": [20, 21]})
df2 = pd.DataFrame({"Name": ["A", "B"], "City": ["Delhi", "Mumbai"]})
```

```
print(pd.merge(df1, df2, on="Name"))
```

**Output:**

```
... Name Age City
 0 A 20 Delhi
 1 B 21 Mumbai
```

**Sort**

Sorts data based on column values.

```
import pandas as pd
```



```
df = pd.DataFrame({
 "Name": ["A", "B", "C"],
 "Marks": [85, 95, 90]
})

print(df.sort_values(by="Marks", ascending=False))
```

**Output:**

---

| ... | Name | Marks |
|-----|------|-------|
| 1   | B    | 95    |
| 2   | C    | 90    |
| 0   | A    | 85    |

---

### Filter

Filter selects data based on conditions.

**import pandas as pd**

```
df = pd.DataFrame({
 "Name": ["A", "B", "C"],
 "Marks": [85, 95, 90]
})
```

```
print(df[df["Marks"] > 88])
```

**Output:**

---

| ... | Name | Marks |
|-----|------|-------|
| 1   | B    | 95    |
| 2   | C    | 90    |

---

### set\_index

Sets a column as the index of the DataFrame.

**import pandas as pd**

```
df = pd.DataFrame({
 "Name": ["A", "B", "C"],
 "Marks": [85, 95, 90]
})
```

```
print(df.set_index("Name"))
```

**Output:**

---

| ...  | Marks |
|------|-------|
| Name |       |
| A    | 85    |
| B    | 95    |
| C    | 90    |

---

### reset\_index

Resets the index back to default numeric indexing.

```
import pandas as pd
```

```
df = pd.DataFrame({
 "Name": ["A", "B", "C"],
 "Marks": [85, 95, 90]
}).set_index("Name")
```

```
print(df.reset_index())
```

**Output:**

---

```
... Name Marks
0 A 85
1 B 95
2 C 90
```

---

### 3. Working With Missing Data

Working with missing data is a key step in EDA to ensure data quality and accurate analysis. It involves identifying missing values and applying appropriate techniques to handle them without affecting results.

#### Checking Missing Data

Used to detect null values present in the dataset.

```
import pandas as pd
```

```
df = pd.DataFrame({
 "Name": ["A", "B", "C"],
 "Age": [20, None, 22]
})
```

```
print(df.isnull())
```

```
print(df.isnull().sum())
```

**Output:**

---

```
... Name Age
0 False False
1 False True
2 False False
Name 0
Age 1
dtype: int64
```

---

#### Dropping Missing Values

There are different methods to handle missing data based on requirements, here we just drop the missing values.

```
import pandas as pd
```

```
df = pd.DataFrame({
 "Name": ["A", "B", "C"],
```

```

 "Age": [20, None, 22]
})

print(df.dropna())

df["Age"].fillna(df["Age"].mean(), inplace=True)
print(df)

```

**Output:**

```

...
 Name Age
0 A 20.0
2 C 22.0

```

---

#### 4. Checking and Handling Duplicate Values

Duplicate values can lead to incorrect analysis and biased results. Identifying and removing duplicates is an important step in data cleaning during EDA.

##### Checking Duplicate Values

Used to detect duplicate rows in the dataset.

**import pandas as pd**

```

df = pd.DataFrame({
 "Name": ["A", "B", "A", "C"],
 "Age": [20, 21, 20, 22]
})

```

```

print(df.duplicated())
print(df.duplicated().sum())

```

**Output:**

```

... 0 False
 1 False
 2 True
 3 False
 dtype: bool
 1

```

---

##### Handling Duplicate Values

Remove duplicate rows to clean the dataset.

**import pandas as pd**

```

df = pd.DataFrame({
 "Name": ["A", "B", "A", "C"],
 "Age": [20, 21, 20, 22]
})

```

```
})
```

```
df_clean = df.drop_duplicates()
print(df_clean)
```

**Output:**

```
... Name Age
0 A 20
1 B 21
3 C 22
```

## 5. Outlier Detection and Handling

Outliers are extreme values that differ significantly from other data points. Detecting and handling them is important to improve data quality and model performance during EDA.

### IQR (Interquartile Range) Method

Outliers are values below  $Q1 - 1.5 \text{ IQR}$  or above  $Q3 + 1.5 \text{ IQR}$ .

**import pandas as pd**

```
df = pd.DataFrame({
 "Values": [10, 12, 14, 15, 100]
})
```

```
Q1 = df["Values"].quantile(0.25)
Q3 = df["Values"].quantile(0.75)
IQR = Q3 - Q1
```

```
outliers = df[(df["Values"] < Q1 - 1.5*IQR) | (df["Values"] > Q3 + 1.5*IQR)]
print("Outliers:\n", outliers)
```

**Output:**

```
... Outliers:
 Values
4 100
```

### Z-Score Method

Outliers are values with Z-score greater than 3 or less than -3.

**import pandas as pd**

**import numpy as np**

```
df = pd.DataFrame({
 "Values": [10, 12, 14, 15, 100]
})
```

```
mean = np.mean(df["Values"])
std = np.std(df["Values"])
```

```
df["Z_score"] = (df["Values"] - mean) / std
outliers = df[df["Z_score"].abs() > 3]
```

```
print("Outliers:\n", outliers)
```

**Output:**

```
... Outliers:
 Empty DataFrame
 Columns: [Values, Z_score]
 Index: []
```

## Handling Outliers

Outliers can be handled by removing or capping depending on the use case.

**import pandas as pd**

```
df = pd.DataFrame({
 "Values": [10, 12, 14, 15, 100]
})
```

```
Q1 = df["Values"].quantile(0.25)
```

```
Q3 = df["Values"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
df_clean = df[(df["Values"] >= Q1 - 1.5*IQR) & (df["Values"] <= Q3 + 1.5*IQR)]
```

```
print("After Removing Outliers:\n", df_clean)
```

**Output:**

```
... After Removing Outliers:
 Values
0 10
1 12
2 14
3 15
```

## 6. Data Visualization Using Matplotlib

Matplotlib is a widely used Python library for creating visualizations and graphs. It helps in understanding patterns, trends, and relationships in data through visual representation during EDA.

- Used to create plots like line charts, bar graphs, histograms and scatter plots
- Helps in identifying trends, distributions and outliers
- Works well with NumPy and Pandas data

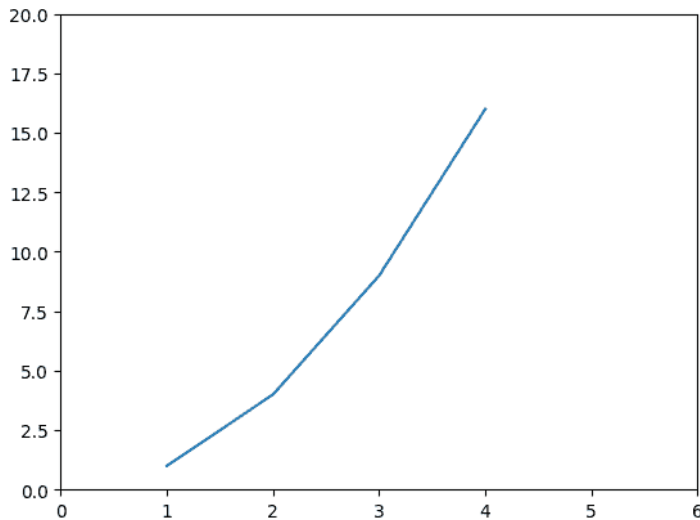
### Pyplot

Pyplot is a Matplotlib module that provides a simple interface to create and customize plots. It helps in generating figures, adding labels, and displaying visualizations.

**import matplotlib.pyplot as plt**

```
plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
plt.axis([0, 6, 0, 20])
plt.show()
```

**Output:**



### Bar chart

A bar chart is used to compare values across different categories using rectangular bars. The height or length of each bar represents the value of that category.

- Used for comparing discrete categories
- Can be plotted vertically or horizontally
- Created using bar() method

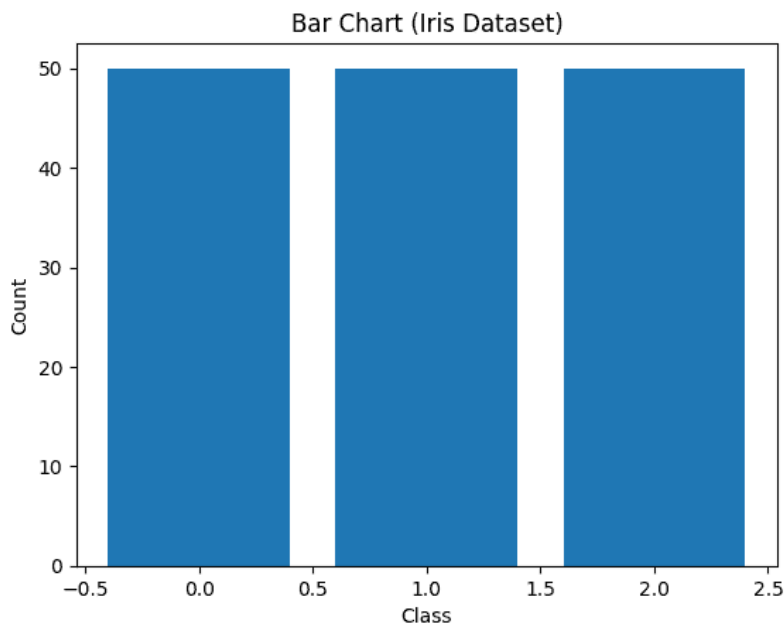
```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd
```

```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
```

```
counts = df["target"].value_counts()
```

```
plt.bar(counts.index, counts.values)
plt.title("Bar Chart (Iris Dataset)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

**Output:**



## Histograms

A histogram is used to show the distribution of data by grouping values into bins (ranges). The X-axis represents the bins, and the Y-axis shows the frequency of values in each bin.

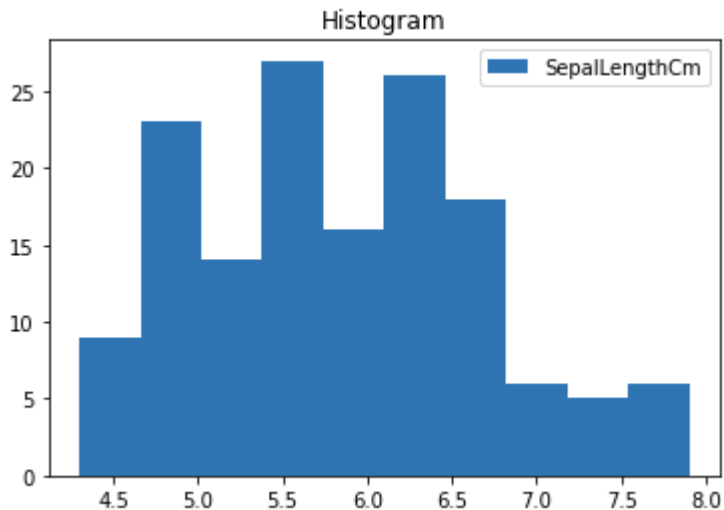
- Used to understand data distribution
- Groups data into non-overlapping intervals (bins)
- Created using hist() method

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd
```

```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

```
plt.hist(df["sepal length (cm)"], bins=10)
plt.title("Histogram (Iris Dataset)")
plt.xlabel("Sepal Length")
plt.ylabel("Frequency")
plt.show()
```

**Output:**



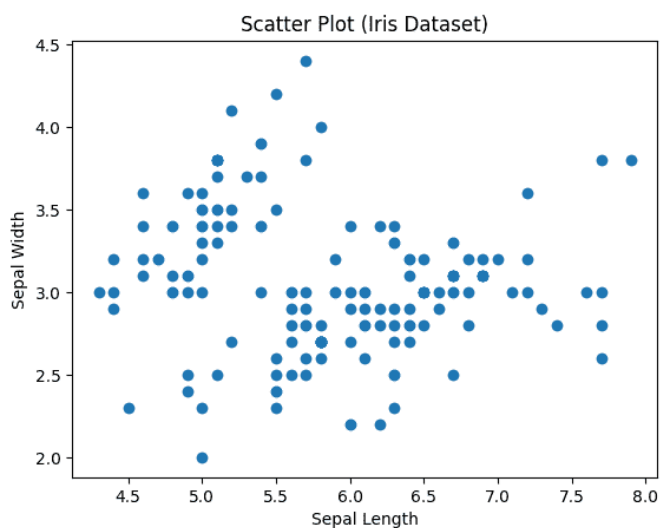
### Scatter Plot

Scatter plots are used to observe relationship between variables and uses dots to represent the relationship between them. The scatter() method in the matplotlib library is used to draw a scatter plot.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["species"] = iris.target
plt.scatter(df["sepal length (cm)"], df["sepal width (cm)"])
plt.title("Scatter Plot (Iris Dataset)")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
```

```
plt.show()
```

**Output:**





## Box Plot

A boxplot (box-and-whisker plot) is used to visualize data distribution and identify outliers using quartiles. The minimum is shown at the far left of the chart, at the end of the left 'whisker'

- First quartile, Q1, is the far left of the box (left whisker)
- The median is shown as a line in the center of the box
- Third quartile, Q3, shown at the far right of the box (right whisker)
- The maximum is at the far right of the box

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd
```

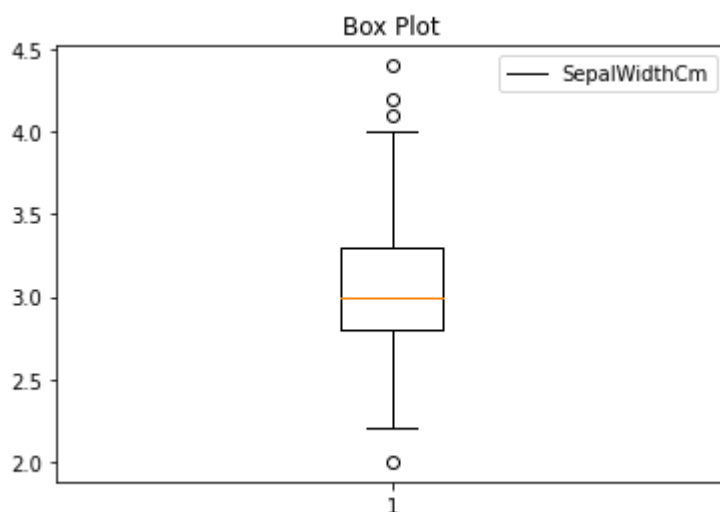
```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

```
plt.boxplot(df["sepal width (cm)"], labels=["Sepal Width"])
```

```
plt.title("Box Plot (Iris Dataset)")
plt.ylabel("Value")
```

```
plt.show()
```

**Output:**



## Correlation Heatmaps

A correlation heatmap is a visual tool that shows relationships between variables using colors. It is based on a correlation matrix, where each cell represents how strongly two variables are related.

- Uses colors to represent correlation values (-1 to +1)
- Helps identify patterns, relationships, and dependencies
- Created using `corr()` to compute correlation matrix
- Color intensity shows strength of relationship

- Works only with numerical data

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

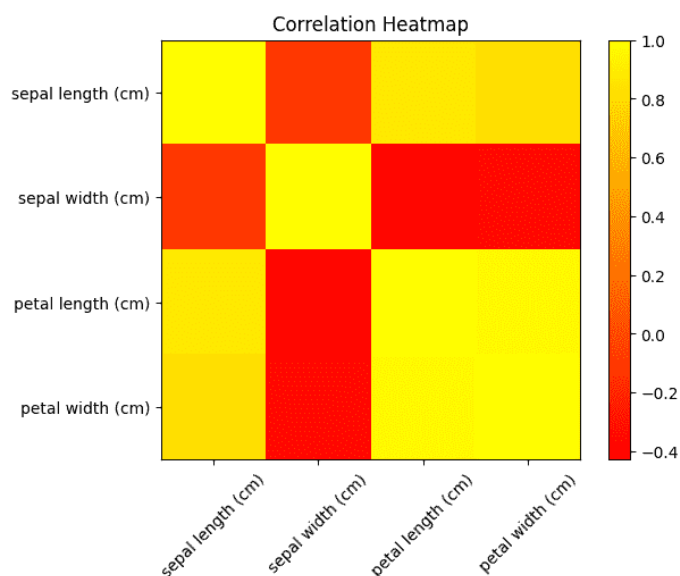
corr = df.corr()

plt.imshow(corr, cmap='autumn', interpolation='nearest')

plt.title("Correlation Heatmap")
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45)
plt.yticks(range(len(corr.columns)), corr.columns)

plt.show()
```

**Output:**



## 7. Data Visualization using Seaborn

Seaborn is a high level visualization library built on Matplotlib that provides more attractive and informative statistical plots.

- Better styling and built in themes
- Simplifies complex visualizations
- Works directly with Pandas DataFrames

### Scatter Plot

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
```

```
import pandas as pd
```

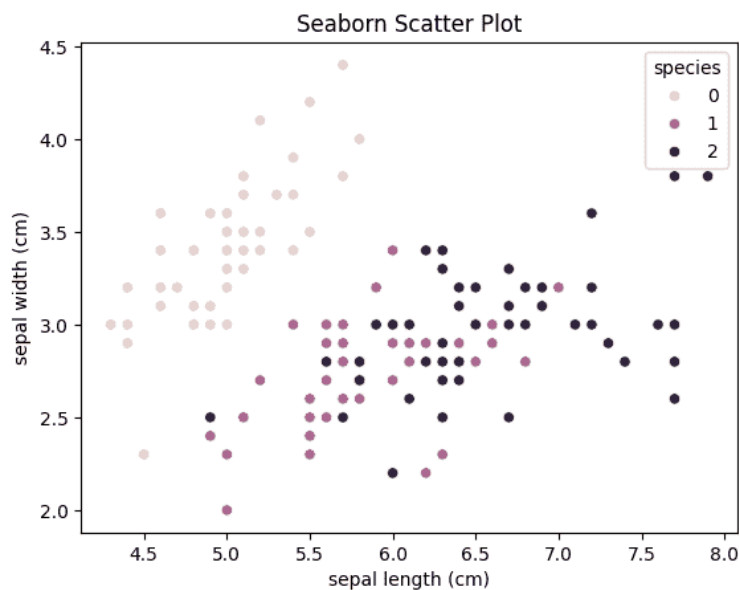
```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["species"] = iris.target
```

```
sns.scatterplot(x="sepal length (cm)", y="sepal width (cm)", hue="species", data=df)
```

```
plt.title("Seaborn Scatter Plot")
```

```
plt.show()
```

**Output:**



## Introduction to NumPy,

NumPy (Numerical Python) is a fundamental library for Python numerical computing. It provides efficient multi-dimensional array objects and various mathematical functions for handling large datasets, making it a critical tool for professionals in fields that require heavy computation.

NumPy has various features that make it popular over lists:

- **N-Dimensional Arrays:** NumPy's core feature is `ndarray`, a N-dimensional array object that supports homogeneous data types.
- **Arrays with High Performance:** Arrays are stored in contiguous memory locations, enabling faster computations than Python lists.
- **Broadcasting:** allows element-wise computations between arrays of different shapes. It simplifies operations on arrays of various shapes by automatically aligning their dimensions without creating new data.
- **Vectorization:** Eliminates the need for explicit loops by applying operations directly on entire arrays.

- **Linear algebra:** NumPy contains routines for linear algebra operations, such as matrix multiplication, decompositions and determinants.

### Installation

To begin using NumPy, you need to install it first. This can be done using the following pip command:

```
pip install numpy
```

Once installed, import the library with the alias np

```
import numpy as np
```

### Creating NumPy Arrays

**1. Using np.array:** Use np.array() when you want to convert Python lists into NumPy arrays.

```
import numpy as np
```

```
a1 = np.array([1, 2, 3]) # 1D array
```

```
a2 = np.array([[1, 2], [3, 4]]) # 2D array
```

```
a3 = np.array([[[1, 2], [3, 4]],
 [[5, 6], [7, 8]]]) # 3D array
```

```
print(a1)
```

```
print(a2)
```

```
print(a3)
```

### Output

```
[1 2 3]
```

```
[[1 2]
```

```
 [3 4]]
```

```
[[[1 2]
```

```
 [3 4]]
```

```
[[5 6]
```

```
 [7 8]]]
```

**2. Using Numpy Functions:** NumPy provides quick utility functions for creating arrays filled with zeros, ones, or ranges:

```
import numpy as np
```

```
a0 = np.zeros((3, 3))
```

```
a1 = np.ones((2, 2))
```

```
ar = np.arange(0, 10, 2)
```

```
print(a0)
```

```
print(a1)
```

```
print(ar)
```

### Output

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
[[1. 1.]
 [1. 1.]]
[0 2 4 6 8]
```

### NumPy Array Indexing

Advanced indexing in NumPy uses arrays of integers or boolean masks to extract complex patterns of elements, enabling non-contiguous and condition-based selection.

```
import numpy as np
```

```
a1 = np.array([10, 20, 30, 40, 50])
print(a1[2]) # single element
print(a1[-1]) # last element
```

```
a2 = np.array([[1, 2, 3],
 [4, 5, 6],
 [7, 8, 9]])
print(a2[1, 0]) # row 1, column 0
```

### Output

```
30
50
4
```

### Slicing

Slicing follows the same indexing rules as lists, but extends them to multiple dimensions, allowing to select rows, columns, or sub-arrays efficiently.

```
import numpy as np
```

```
a = np.array([[1, 2, 3],
 [4, 5, 6]])

print(a[1:4]) # row slice
print(a[:, 1]) # all rows, column 1
```

### Output

```
[[4 5 6]]
[2 5]
```

### Advanced Indexing

Advanced Indexing in NumPy provides more flexible ways to access and manipulate array elements.

```
import numpy as np

a = np.array([10, 20, 30, 40, 50, 60])
idx = np.array([1, 3, 5])

print(a[idx]) # integer indexing

cond = a > 30
print(a[cond]) # boolean indexing
```

**Output**

```
[20 40 60]
[40 50 60]
```

**Basic Arithmetic Operations**

Element-wise operations in NumPy allow to perform mathematical operations on each element of an array individually, without the need for explicit loops. We can perform arithmetic operations like addition, subtraction, multiplication and division directly on NumPy arrays.

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

print(x + y) # add
print(x - y) # subtract
print(x * y) # multiply
print(x / y) # divide
```

**Output**

```
[5 7 9]
[-3 -3 -3]
[4 10 18]
[0.25 0.4 0.5]
```

**Unary Operation**

Unary operations in NumPy apply a single-operand transformation-such as negation, absolute value, or trigonometric evaluation-across entire arrays efficiently without the need for multiple arrays (as in binary operations).

```
import numpy as np

a = np.array([-3, -1, 0, 1, 3]) # array with both positive and negative values

Applying a unary operation: absolute value
```

```
print(np.absolute(a))
```

### Output

```
[3 1 0 1 3]
```

### Binary Operators

Binary Operations apply to the array elementwise and a new array is created. We can use all basic arithmetic operators like +, -, /, etc. Operators like +=, -=, \*= modify the existing array in-place and = operator simply assigns a new reference to a variable, it does not modify the original array.

```
import numpy as np
```

```
Two example arrays
```

```
a1 = np.array([1, 2, 3])
```

```
a2 = np.array([4, 5, 6])
```

```
Applying a binary operation: addition
```

```
res = np.add(a1, a2)
```

```
print(res)
```

### Output

```
[5 7 9]
```

### Mathematical Functions

NumPy provides familiar mathematical functions such as sin, cos, exp, etc. These functions also operate elementwise on an array, producing an array as output.

```
import numpy as np
```

```
create an array of sine values
```

```
a = np.array([0, np.pi/2, np.pi])
```

```
print(np.sin(a))
```

```
exponential values
```

```
b = np.array([0, 1, 2, 3])
```

```
print(np.exp(b))
```

```
print(np.sqrt(b))
```

```
square root of array values
```

```
print (np.sqrt(a))
```

### Output

```
[0.0000000e+00 1.0000000e+00 1.2246468e-16]
```

```
[1. 2.71828183 7.3890561 20.08553692]
```

```
[0. 1. 1.41421356 1.73205081]
```

```
[0. 1.25331414 1.77245385]
```

### Sorting Arrays

We can use a simple `np.sort()` method for sorting Python NumPy arrays.

**Note:** Strings with prefix `b"` indicate byte strings (fixed-length string in NumPy). `S10` means each string can store up to 10 bytes.

```
import numpy as np
```

```
dtype = [('name', 'S10'), ('year', int), ('cgpa', float)]
vals = [('Hrithik', 2009, 8.5),
 ('Ajay', 2008, 8.7),
 ('Pankaj', 2008, 7.9),
 ('Aakash', 2009, 9.0)]
```

```
a = np.array(vals, dtype=dtype)
```

```
print(np.sort(a, order='name'))
print(np.sort(a, order=['year', 'cgpa']))
```

### Output

```
[(b'Aakash', 2009, 9.) (b'Ajay', 2008, 8.7) (b'Hrithik', 2009, 8.5)
 (b'Pankaj', 2008, 7.9)]
[(b'Pankaj', 2008, 7.9) (b'Ajay', 2008, 8.7) (b'Hrithik', 2009, 8.5)
 (b'Aakash', 2009, 9.)]
```

## Array Creation using NumPy

**NumPy Arrays** are grid-like structures similar to lists in Python but optimized for numerical operations. The most straightforward way to create a NumPy array is by converting a regular Python list into an array using the `np.array()` function.

Let's understand this with the help of an example:

```
import numpy as np
```

```
One-dimensional array
arr1 = np.array([1, 2, 3, 4, 5])
print(arr1)
```

```
Two-dimensional array
arr2 = np.array([[1, 2], [3, 4]])
print(arr2)
```

### Output

```
[1 2 3 4 5]
[[1 2]
 [3 4]]
```



### Creating Arrays with Specific Values

For assigning a specific values. NumPy provides several function to create arrays filled with zeros, ones, or a specific constant value.

- **Zeros Array:** `np.zeros()` function creates an array filled with zeros. It requires the shape of the array as a parameter.

#### Example:

```
import numpy as np
```

```
3x4 array filled with zeros
```

```
arr_zero = np.zeros((3, 4))
```

```
print(arr_zero)
```

#### Output

```
[[0. 0. 0. 0.]
```

```
 [0. 0. 0. 0.]
```

```
 [0. 0. 0. 0.]]
```

- **Ones Array:** `np.ones()` creates an array filled with ones.

#### Example:

```
import numpy as np
```

```
2x3 array filled with ones
```

```
arr_one = np.ones((2, 3))
```

```
print(arr_one)
```

#### Output

```
[[1. 1. 1.]
```

```
 [1. 1. 1.]]
```

- **Full Array :** `np.full()` function allows you to create an array filled with a specific value.

#### Example:

```
import numpy as np
```

```
2x2 array filled with 7
```

```
arr_full = np.full((2, 2), 7)
```

```
print(arr_full)
```

#### Output

```
[[7 7]
```

```
 [7 7]]
```

### Creating Arrays with Random Values

NumPy also has functions for generating arrays with random values, useful for simulations and testing.

- **Random Float Array :** `np.random.rand()` function generates an array of random values between 0 and 1.

**Example:**

```
import numpy as np

2x3 array of random floats
arr_rand = np.random.rand(2, 3)
print(arr_rand)
```

**Output**

```
[[0.67820861 0.64484802 0.48673431]
 [0.00263043 0.55383721 0.43240166]]
```

- **Random Integers :** If we need random integers, we can use **np.random.randint()** to create arrays with integer values in a specified range.

**Example:**

```
import numpy as np

3x3 array of random integers from 1 to 9
arr_int = np.random.randint(1, 10, size=(3, 3))
print(arr_int)
```

**Output**

```
[[4 6 5]
 [7 4 8]
 [8 5 2]]
```

**Creating Arrays with a Range of Values**

Another common method of creating arrays is using a range of values. NumPy provides functions like **np.arange()** and **np.linspace()** for this purpose.

- **Using np.arange() :** **np.arange()** creates arrays with values spaced according to a given interval. It's similar to Python's built-in **range()** but returns a NumPy array.

**Example:**

```
import numpy as np

Array from 0 to 10 with step 2
arr_range = np.arange(0, 10, 2)
print(arr_range)
```

**Output**

```
[0 2 4 6 8]
```

- **Using np.linspace():** **np.linspace()** generates an array with a specified number of evenly spaced values over a specified range.

**Example:**

```
import numpy as np

5 values from 0 to 1
```

```
arr_linspace = np.linspace(0, 1, 5)
print(arr_linspace)
```

**Output**

```
[0. 0.25 0.5 0.75 1.]
```

**Identity and Diagonal Matrices**

NumPy also provides functions for creating identity matrices and diagonal matrices, which are often used in linear algebra.

- **Identity Matrix** : `np.eye()` function creates an identity matrix, a square matrix with ones on the diagonal and zeros elsewhere.

**Example:**

```
import numpy as np
```

```
3x3 identity matrix
identity_matrix = np.eye(3)
print(identity_matrix)
```

**Output**

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

- **Diagonal Matrix** : Use `np.diag()` to create a diagonal matrix, where the provided array elements form the diagonal.

**Example:**

```
import numpy as np
```

```
Diagonal matrix with [1, 2, 3] on the diagonal
diag_matrix = np.diag([1, 2, 3])
print(diag_matrix)
```

**Output**

```
[[1 0 0]
 [0 2 0]
 [0 0 3]]
```

**Methods for array creation in Numpy**

| Function                  | Description                                                              |
|---------------------------|--------------------------------------------------------------------------|
| <code>empty()</code>      | Return a new array of given shape and type, without initializing entries |
| <code>empty_like()</code> | Return a new array with the same shape and type as a given array         |
| <code>eye()</code>        | Return a 2-D array with ones on the diagonal and zeros elsewhere.        |

|                            |                                                                        |
|----------------------------|------------------------------------------------------------------------|
| <b>identity()</b>          | Return the identity array                                              |
| <b>ones()</b>              | Return a new array of given shape and type, filled with ones           |
| <b>ones_like()</b>         | Return an array of ones with the same shape and type as a given array  |
| <b>zeros()</b>             | Return a new array of given shape and type, filled with zeros          |
| <b>zeros_like()</b>        | Return an array of zeros with the same shape and type as a given array |
| <b>full_like()</b>         | Return a full array with the same shape and type as a given array.     |
| <b>array()</b>             | Create an array                                                        |
| <b>asarray()</b>           | Convert the input to an array                                          |
| <b>asanyarray()</b>        | Convert the input to an ndarray, but pass ndarray subclasses through   |
| <b>ascontiguousarray()</b> | Return a contiguous array in memory (C order)                          |
| <b>asmatrix()</b>          | Interpret the input as a matrix                                        |
| <b>copy()</b>              | Return an array copy of the given object                               |
| <b>frombuffer()</b>        | Interpret a buffer as a 1-dimensional array                            |
| <b>fromfile()</b>          | Construct an array from data in a text or binary file                  |
| <b>fromfunction()</b>      | Construct an array by executing a function over each coordinate        |
| <b>fromiter()</b>          | Create a new 1-dimensional array from an iterable object               |
| <b>fromstring()</b>        | A new 1-D array initialized from text data in a string                 |
| <b>loadtxt()</b>           | Load data from a text file                                             |
| <b>arange()</b>            | Return evenly spaced values within a given interval                    |
| <b>linspace()</b>          | Return evenly spaced numbers over a specified interval                 |
| <b>logspace()</b>          | Return numbers spaced evenly on a log scale                            |
| <b>geomspace()</b>         | Return numbers spaced evenly on a log scale (a geometric progression)  |
| <b>meshgrid()</b>          | Return coordinate matrices from coordinate vectors                     |
| <b>mgrid()</b>             | nd_grid instance which returns a dense multi-dimensional “meshgrid     |
| <b>ogrid()</b>             | nd_grid instance which returns an open multi-dimensional “meshgrid     |
| <b>diag()</b>              | Extract a diagonal or construct a diagonal array                       |
| <b>diagflat()</b>          | Create a two-dimensional array with the flattened input as a diagonal  |
| <b>tri()</b>               | An array with ones at and below the given diagonal and zeros elsewhere |
| <b>tril()</b>              | Lower triangle of an array                                             |
| <b>triu()</b>              | Upper triangle of an array                                             |
| <b>vander()</b>            | Generate a Vandermonde matrix                                          |
| <b>mat()</b>               | Interpret the input as a matrix                                        |
| <b>bmat()</b>              | Build a matrix object from a string, nested sequence, or array         |

## Operations on Arrays

An array is a collection of items of the same variable type that are stored at contiguous memory locations. It is one of the most popular and simple data structures used in programming.

### Basic terminologies of Array

- **Array Element:** Elements are items stored in an array.
- **Array Index:** Elements are accessed by their indexes. Indexes in most of the programming languages start from 0.

### Memory representation of Array

In an array, all the elements or their references are stored in contiguous memory locations. This allows for efficient access and manipulation of elements.

### Declaration of Array

Arrays can be declared in various ways in different languages. For better illustration, below are some language-specific array declarations:

# In Python, all types of lists are created same way  
arr = []

### Initialization of Array

Arrays can be initialized in different ways in different languages. Below are some language-specific array initialization:

# This list will store integer type elements  
arr = [1, 2, 3, 4, 5]

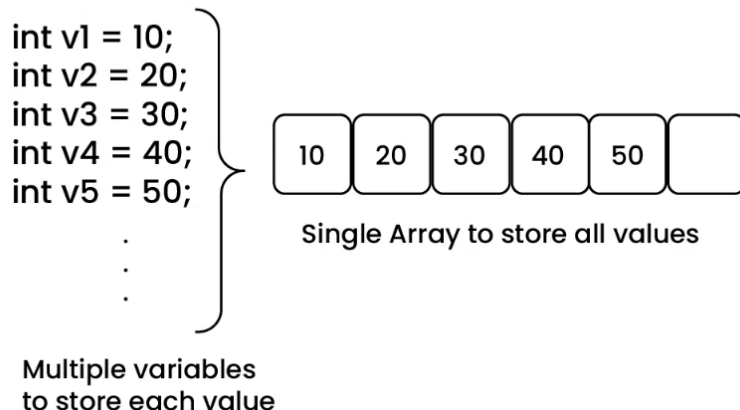
# This list will store character type elements (strings in Python)  
arr = ['a', 'b', 'c', 'd', 'e']

# This list will store float type elements  
arr = [1.4, 2.0, 24.0, 5.0, 0.0] # All float values

### Why do we Need Arrays?

Assume there is a class of five students and if we have to keep records of their marks in examination then, we can do this by declaring five variables individual and keeping track of records but what if the number of students becomes very large, it would be challenging to manipulate and maintain the data. So we use an array of students.

### Importance of Array



## Types of Arrays

Arrays can be classified in two ways:

- On the basis of Size
- On the basis of Dimensions

### Types of Arrays on the basis of Size

#### 1. Fixed Sized Arrays

- We cannot alter or update the size of this array. Here only a fixed size (i.e. the size that is mentioned in square brackets []) of memory will be allocated for storage.
- In case, we don't know the size of the array then if we declare a larger size and store a lesser number of elements, it will result in a wastage of memory. And if we declare a lesser size than the number of elements then we won't get enough memory to store all the elements.

```
Create a fixed-size list of length 5,
initialized with zeros
arr = [0] * 5
```

```
Output the fixed-size list
print(arr)
```

#### 2. Dynamic Sized Arrays

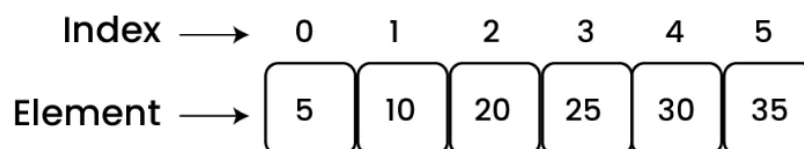
The size of the array changes as per user requirements during execution of code so the coders do not have to worry about sizes. They can add and removed the elements as per the need. The memory is mostly dynamically allocated and de-allocated in these arrays.

```
Dynamic Array
arr = []
```

### Types of Arrays on the basis of Dimensions

**1. One-dimensional Array(1-D Array):** You can imagine a 1d array as a row, where elements are stored one after another.

#### One-Dimensional Array (1-D Array)



**2. Multi-dimensional Array:** A multi-dimensional array is an array with more than one dimension. We can use multidimensional array to store complex data in the form of tables, etc. We can have 2-D arrays, 3-D arrays, 4-D arrays and so on.

- **Two-Dimensional Array(2-D Array or Matrix):** 2-D Multidimensional arrays can be considered as an array of arrays or as a matrix consisting of rows and columns.

To read more about Matrix Refer, Matrix Data Structure

## Two-Dimensional Array (2-D Array or Matrix)

|           |   |          |          |          |
|-----------|---|----------|----------|----------|
| Columns → |   |          |          |          |
|           |   | 0        | 1        | 2        |
| Rows ↓    | 0 | $a_{00}$ | $a_{01}$ | $a_{02}$ |
|           | 1 | $a_{10}$ | $a_{11}$ | $a_{12}$ |
|           | 2 | $a_{20}$ | $a_{21}$ | $a_{22}$ |

- **Three-Dimensional Array(3-D Array):** A 3-D Multidimensional array contains three dimensions, so it can be considered an array of two-dimensional arrays.

## Three-Dimensional Array (3-D Array)

|      |       |           |           |           |            |
|------|-------|-----------|-----------|-----------|------------|
|      |       | Columns   |           |           |            |
|      |       | Column 1  | Column 2  | Column 3  |            |
| Rows | Row 1 | $a_{000}$ | $a_{001}$ | $a_{002}$ | → Matrix 1 |
|      | Row 2 | $a_{010}$ | $a_{100}$ | $a_{101}$ | → Matrix 2 |
|      | Row 3 | $a_{020}$ | $a_{110}$ | $a_{200}$ | → Matrix 3 |
|      |       |           | $a_{120}$ | $a_{210}$ |            |
|      |       |           | $a_{220}$ | $a_{221}$ |            |
|      |       |           |           | $a_{222}$ |            |

### Operations on Array

- Traversal in Array
- **Insertions** : At Beginning, At given position and At the end.
- **Deletion** : From Beginning, Given Position, First Occurrence, All occurrences and From End
- **Searching** : Linear Search and Binary Search

## Introduction to Pandas

Pandas is an open-source Python library used for data manipulation, analysis and cleaning. It provides fast and flexible tools to work with tabular data, similar to spreadsheets or SQL tables. Pandas is widely used in data science and analytics due to its integration with libraries such as:

- **NumPy:** numerical operations
- **Matplotlib and Seaborn:** data visualization
- **SciPy:** statistical analysis
- **Scikit-learn:** machine learning workflows

Pandas allows efficient handling and analysis of data in a few lines of code.

### Installation

Before using Pandas, make sure it is installed:

```
pip install pandas
```

After the Pandas have been installed in the system we need to import the library. This module is imported using:

```
import pandas as pd
```

**Note:** pd is just an alias for Pandas. It's not required but using it makes the code shorter when calling methods or properties.

### Data Structures in Pandas

Pandas provides two data structures for manipulating data which are as follows:

#### 1. Pandas Series

A Pandas Series is one-dimensional labeled array capable of holding data of any type (integer, string, float, Python objects etc.). The axis labels are collectively called **indexes**. Series is created by loading the datasets from existing storage which can be a SQL database, a CSV file or an Excel file.

```
import pandas as pd
```

```
import numpy as np
```

```
s = pd.Series()
print("Pandas Series: ", s)
data = np.array(['g', 'e', 'e', 'k', 's'])
```

```
s = pd.Series(data)
print("Pandas Series:\n", s)
```

#### Output

```
Pandas Series: Series([], dtype: object)
Pandas Series:
0 g
1 e
2 e
3 k
4 s
dtype: object
```



Pandas Series

## 2. Pandas DataFrame

Pandas DataFrame is a two-dimensional data structure with labeled axes (rows and columns). It is created by loading the datasets from existing storage which can be a SQL database, a CSV file or an Excel file. It can be created from lists, dictionaries, a list of dictionaries etc.

**import pandas as pd**

```
df = pd.DataFrame()
print(df)
lst = ['Geeks', 'For', 'Geeks', 'is', 'portal', 'for', 'Geeks']
```

```
df = pd.DataFrame(lst)
print(df)
```

**Output:**

```
Empty DataFrame
Columns: []
Index: []
0
0 Geeks
1 For
2 Geeks
3 is
4 portal
5 for
6 Geeks
```

Pandas DataFrame

## Operations in Pandas

Pandas provides essential operations for working with structured data efficiently. The sections below introduce the most commonly used functionalities with short explanations and simple examples.

**1. Loading Data:** This operation reads data from files such as CSV, Excel or JSON into a DataFrame.

**import pandas as pd**

```
df = pd.read_csv("data.csv")
print(df.head())
```

**Output**

|   | name  | age | city    | category | sales | a | b  |
|---|-------|-----|---------|----------|-------|---|----|
| 0 | John  | 25  | Delhi   | A        | 200   | 5 | 10 |
| 1 | Riya  | 22  | Mumbai  | B        | 120   | 2 | 3  |
| 2 | Aman  | 30  | Pune    | A        | 250   | 7 | 8  |
| 3 | Neha  | 27  | Kolkata | C        | 300   | 4 | 6  |
| 4 | Arjun | 35  | Jaipur  | B        | 200   | 1 | 9  |

Output of Loading Dataset

**Explanation:** `pd.read_csv("data.csv")` reads the CSV file and loads it into a DataFrame and `df.head()` shows the first 5 rows of the data.

You can download the data.csv file from [here](#)

**2. Viewing and Exploring Data:** After loading data, it is important to understand its structure and content. This methods allow you to inspect rows, summary statistics and metadata.

```
print(df.info())
```

**Output**

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
Column Non-Null Count Dtype
--- -
0 name 5 non-null object
1 age 5 non-null int64
2 city 5 non-null object
3 category 5 non-null object
4 sales 5 non-null int64
5 a 5 non-null int64
6 b 5 non-null int64
dtypes: int64(4), object(3)
memory usage: 412.0+ bytes
None
```

Output of `df.info()`

**3. Handling Missing Data:** Datasets often contain empty or missing values. Pandas provides functions to detect, remove or replace these values.

```
print(df.isnull().sum())
```

```
df = df.fillna(0)
```

**Output**

```
name 0
age 0
city 0
category 0
sales 0
a 0
b 0
dtype: int64
```

No Columns have NAN value

**Explanation:** `df.fillna(0)` replaces missing values with 0.

**4. Selecting and Filtering Data:** This operation retrieves specific columns, rows or records that match a condition. It allows precise extraction of required information.

```
ages = df[df['age'] > 25]
```

```
print(ages)
```

**Output**

|   | name  | age | city    | category | sales | a | b |
|---|-------|-----|---------|----------|-------|---|---|
| 2 | Aman  | 30  | Pune    | A        | 250   | 7 | 8 |
| 3 | Neha  | 27  | Kolkata | C        | 300   | 4 | 6 |
| 4 | Arjun | 35  | Jaipur  | B        | 200   | 1 | 9 |

Output of Filtering Data

**Explanation** `df[df['age'] > 25]` returns rows where the "age" value is greater than 25.

**5. Adding and Removing Columns:** You can create new columns based on existing ones or delete unwanted columns from the DataFrame.

```
df['total'] = df['a'] + df['b']
```

```
print(df.head())
```

**Output**

|   | name  | age | city    | category | sales | a | b  | total |
|---|-------|-----|---------|----------|-------|---|----|-------|
| 0 | John  | 25  | Delhi   | A        | 200   | 5 | 10 | 15    |
| 1 | Riya  | 22  | Mumbai  | B        | 120   | 2 | 3  | 5     |
| 2 | Aman  | 30  | Pune    | A        | 250   | 7 | 8  | 15    |
| 3 | Neha  | 27  | Kolkata | C        | 300   | 4 | 6  | 10    |
| 4 | Arjun | 35  | Jaipur  | B        | 200   | 1 | 9  | 10    |

Adding new column "total"

**Explanation:** `df['total'] = df['a'] + df['b']` creates a new column named "total".

**6. Grouping Data (GroupBy):** Grouping allows you to organize data into categories and compute values for each group for example, sums, counts or averages.

```
res = df.groupby('category')['sales'].sum()
```

```
print(res)
```

**Output**

```
category
A 450
B 320
C 300
Name: sales, dtype: int64
```

Grouping Data

**Explanation:** `df.groupby('category')` divides the dataset based on the "category" column.

## Series and Data Frames

**Pandas** is a widely-used **Python** library for data analysis that provides two essential data structures: **Series** and **DataFrame**. These structures are potent tools for handling and examining data, but they have different features and applications.

In this article, we will explore the differences between Series and DataFrames.

**What are pandas?**

Pandas is a popular open-source data manipulation and analysis library for Python. It provides easy-to-use data structures like DataFrame and Series, which are designed to make working with structured data fast, easy, and expressive. Pandas are widely used in data science, machine learning, and data analysis for tasks such as data cleaning, transformation, and exploration.

### What is the Pandas series?

A Pandas Series is a one-dimensional array-like object that can hold data of any type (integer, float, string, etc.). It is labelled, meaning each element has a unique identifier called an index. You can think of a Series as a column in a spreadsheet or a single column of a database table. Series are a fundamental data structure in Pandas and are commonly used for data manipulation and analysis tasks. They can be created from lists, arrays, dictionaries, and existing Series objects. Series are also a building block for the more complex Pandas DataFrame, which is a two-dimensional table-like structure consisting of multiple Series objects.

### Creating a Series data structure from a list, dictionary, and custom index:

**import pandas as pd**

# Initializing a Series from a list

```
data = [1, 2, 3, 4, 5]
```

```
series_from_list = pd.Series(data)
```

```
print(series_from_list)
```

# Initializing a Series from a dictionary

```
data = {'a': 1, 'b': 2, 'c': 3}
```

```
series_from_dict = pd.Series(data)
```

```
print(series_from_dict)
```

# Initializing a Series with custom index

```
data = [1, 2, 3, 4, 5]
```

```
index = ['a', 'b', 'c', 'd', 'e']
```

```
series_custom_index = pd.Series(data, index=index)
```

```
print(series_custom_index)
```

#### Output:

```
0 1
```

```
1 2
```

```
2 3
```

```
3 4
```

```
4 5
```

```
dtype: int64
```

```
a 1
```

```
b 2
```

```
c 3
```

```
dtype: int64
```

```
a 1
```

```
b 2
c 3
d 4
e 5
dtype: int64
```

### Key Features of Series data structure:

#### Indexing:

Each element in a Series has a corresponding index, which can be used to access or manipulate the data.

```
print(series_from_list[0])
print(series_from_dict['b'])
```

#### Output:

```
1
2
```

#### Vectorized Operations:

Series supports vectorized operations, allowing you to perform arithmetic operations on the entire series efficiently.

```
series_a = pd.Series([1, 2, 3])
series_b = pd.Series([4, 5, 6])
sum_series = series_a + series_b
print(sum_series)
```

#### Output:

```
0 5
1 7
2 9
dtype: int64
```

#### Alignment:

When performing operations between two Series objects, Pandas automatically aligns the data based on the index labels.

```
series_a = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
series_b = pd.Series([4, 5, 6], index=['b', 'c', 'd'])
sum_series = series_a + series_b
print(sum_series)
```

#### Output:

```
a NaN
b 6.0
c 8.0
d NaN
dtype: float64
```

#### NaN Handling:

Missing values, represented by NaN (Not a Number), can be handled gracefully in Series operations.

```
series_a = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
series_b = pd.Series([4, 5], index=['b', 'c'])
sum_series = series_a + series_b
print(sum_series)
```

**Output:**

```
a NaN
b 6.0
c 8.0
dtype: float64
```

**What is Pandas Dataframe?**

A Pandas DataFrame is a two-dimensional, tabular data structure with rows and columns. It is similar to a spreadsheet or a table in a relational database. The DataFrame has three main components: the data, which is stored in rows and columns; the rows, which are labeled by an index; and the columns, which are labeled and contain the actual data.

**Creating a dataframe from lists, dictionary**

**import pandas as pd**

```
Initializing a DataFrame from a dictionary
data = {'Name': ['John', 'Alice', 'Bob'],
 'Age': [25, 30, 35],
 'City': ['New York', 'Los Angeles', 'Chicago']}
df = pd.DataFrame(data)
print(df)
```

```
Initializing a DataFrame from a list of lists
data = [['John', 25, 'New York'],
 ['Alice', 30, 'Los Angeles'],
 ['Bob', 35, 'Chicago']]
columns = ['Name', 'Age', 'City']
df = pd.DataFrame(data, columns=columns)
print(df)
```

**Output:**

```
 Name Age City
0 John 25 New York
1 Alice 30 Los Angeles
2 Bob 35 Chicago
 Name Age City
0 John 25 New York
```

```
1 Alice 30 Los Angeles
2 Bob 35 Chicago
```

### Key Features of Data Frame data structures:

#### Indexing:

DataFrame provides flexible indexing options, allowing access to rows, columns, or individual elements based on labels or integer positions.

```
Accessing a column
```

```
print(df['Name'])
```

```
Accessing a row by label
```

```
print(df.loc[0])
```

```
Accessing a row by integer position
```

```
print(df.iloc[0])
```

```
Accessing an individual element
```

```
print(df.at[0, 'Name'])
```

#### Output:

```
0 John
```

```
1 Alice
```

```
2 Bob
```

```
Name: Name, dtype: object
```

```
Name John
```

```
Age 25
```

```
City New York
```

```
Name: 0, dtype: object
```

```
Name John
```

```
Age 25
```

```
City New York
```

```
Name: 0, dtype: object
```

```
John
```

### Column Operations:

Columns in a DataFrame are Series objects, enabling various operations such as arithmetic operations, filtering, and sorting.

```
Adding a new column
```

```
df['Salary'] = [50000, 60000, 70000]
```

```
Filtering rows based on a condition
```

```
high_salary_employees = df[df['Salary'] > 60000]
```

```
print(high_salary_employees)
```

```
Sorting DataFrame by a column
```

```
sorted_df = df.sort_values(by='Age', ascending=False)
print(sorted_df)
```

**Output:**

```
 Name Age City Salary
2 Bob 35 Chicago 70000
 Name Age City Salary
2 Bob 35 Chicago 70000
1 Alice 30 Los Angeles 60000
0 John 25 New York 50000
```

**Missing Data Handling:**

DataFrames provide methods for handling missing or NaN values, including dropping or filling missing values.

```
Dropping rows with missing values
df.dropna()
print(df)
```

```
Filling missing values with a specified value
df.fillna(0)
print(df)
```

**Output:**

```
 Name Age City Salary
0 John 25 New York 50000
1 Alice 30 Los Angeles 60000
2 Bob 35 Chicago 70000
 Name Age City Salary
0 John 25 New York 50000
1 Alice 30 Los Angeles 60000
2 Bob 35 Chicago 70000
```

**Grouping and Aggregation:**

DataFrames support group-by operations for summarizing data and applying aggregation functions.

```
Grouping by a column and calculating mean
avg_age_by_city = df.groupby('City')['Age'].mean()
print(avg_age_by_city)
```

**Output:**

```
City
Chicago 35.0
Los Angeles 30.0
New York 25.0
Name: Age, dtype: float64
```



**DataFrame vs Series**

| Series                              | DataFrame                        |
|-------------------------------------|----------------------------------|
| One- dimensional                    | Two- dimensional                 |
| Series elements must be homogenous. | Can be heterogeneous.            |
| Immutable(size cannot be changed).  | Mutable(size can be changeable). |
| Element wise computations.          | Column wise computations.        |
| Functionality is less.              | Functionality is more.           |
| Alignment not supported.            | Alignment is supported.          |

**Creating Data Frames from Excel Sheet and .csv file**

A DataFrame is essentially a 2-dimensional labeled data structure with rows and columns, making it ideal for handling structured data from Excel. Pandas library provides different way to load Excel files and convert them into DataFrame objects a tabular data structure similar to an Excel sheet.

**Note:** We will use a sample Excel file named SampleWork.xlsx you can download it from here. Make sure to keep the file in the same folder as your Python script.

**Reading Default Sheet of an Excel File**

When you read an Excel file using `pd.read_excel()` without specifying any sheet name, Pandas automatically loads the first sheet of the workbook. This is the most common method, especially when your file contains only one sheet or the required data is already on the first sheet.

**import pandas as pd**

```
df = pd.read_excel("SampleWork.xlsx")
print(df)
```

**Output**

```
Name Age Stream Percentage
0 Ankit 18 Math 95
1 Rahul 19 Science 90
2 Shaurya 20 Commerce 85
3 Aishwarya 18 Math 80
4 Priyanka 19 Science 75
```

**Explanation:**

- `pd.read_excel("SampleWork.xlsx")` reads the default (first) sheet
- `df` stores the loaded DataFrame

**Reading a Specific Sheet Using sheet\_name**

When an Excel file contains multiple sheets, you can tell Pandas exactly which sheet to load using the `sheet_name` parameter. You may provide either the sheet's index (starting from 0) or its name to directly access the data you need.

**import pandas as pd**

```
df = pd.read_excel("SampleWork.xlsx", sheet_name=1)
print(df)
```

**Output**

Name Age Stream Percentage

```
0 Priya 18 Math 95
1 shivangi 19 Science 90
2 Jeet 20 Commerce 85
3 Ananya 18 Math 80
4 Swapnil 19 Science 75
```

**Explanation:** `sheet_name=1` loads the second sheet

**Reading Only Specific Columns Using usecols**

When we need the entire dataset from an Excel sheet, we can use the `usecols` parameter allows you to load only the required columns by specifying their names or index positions.

**import pandas as pd**

```
df = pd.read_excel("SampleWork.xlsx", usecols=[0, 3])
print(df)
```

**Output**

Name Percentage

```
0 Ankit 95
1 Rahul 90
2 Shaurya 85
3 Aishwarya 80
4 Priyanka 75
```

**Explanation:** `usecols=[0, 3]` loads only selected columns and ignores all other data

**Handling Missing Values Using na\_values**

Excel files sometimes contain placeholder text such as "Missing" or "NA" to indicate empty data. With the `na_values` parameter, Pandas can automatically convert these placeholders into proper NaN values.

**import pandas as pd**

```
df = pd.read_excel("SampleWork.xlsx", sheet_name=2, na_values="Missing")
print(df)
```

**Output**

Name Age Stream Percentage

```
0 Priya 18 Math 95
1 shivangi 19 Science 90
2 Jeet 20 NaN 85
3 Ananya 18 Math 80
4 Swapnil 19 Science 75
```

**Explanation:** `na_values="Missing"` converts the word "Missing" into NaN

**Skipping Rows Using skiprows**

When Excel files contain extra information at the top like titles, notes, or empty rows that you don't want in your DataFrame, we can use `skiprows` parameter which helps you ignore these unwanted rows so Pandas starts reading only from the actual data.

```
import pandas as pd
df = pd.read_excel("SampleWork.xlsx", sheet_name=1, skiprows=2)
print(df)
```

**Output**

```
shivangi 19 Science 90
0 Jeet 20 Commerce 85
1 Ananya 18 Math 80
2 Swapnil 19 Science 75
```

**Explanation:** `skiprows=2` ignores first two rows before reading data

**Setting a Custom Header Row Using header**

Not all Excel files have column names in the first row. With the `header` parameter, you can tell Pandas which row contains the actual column names, ensuring your DataFrame loads with clean and correct headers.

```
import pandas as pd
df = pd.read_excel("SampleWork.xlsx", sheet_name=1, header=2)
print(df)
```

**Output**

```
shivangi 19 Science 90
0 Jeet 20 Commerce 85
1 Ananya 18 Math 80
2 Swapnil 19 Science 75
```

**Explanation:** `header=2` treats the third row as header

**Reading Multiple Sheets Together**

If you want to load more than one sheet at a time, Pandas allows you to pass a list of sheet names or indexes. This is useful when your data is spread across multiple sheets but still related.

```
import pandas as pd
df = pd.read_excel("SampleWork.xlsx", sheet_name=[0, 1], na_values="Missing")
print(df)
```

**Output**

```
{0: Name Age Stream Percentage
0 Ankit 18 Math 95
1 Rahul 19 Science 90
2 Shaurya 20 Commerce 85
3 Aishwarya 18 Math 80
4 Priyanka 19 Science 75,
1: Name Age Stream Percentage
0 Priya 18 Math 95
1 shivangi 19 Science 90
2 Jeet 20 Commerce 85
3 Ananya 18 Math 80
4 Swapnil 19 Science 75}
```

**Explanation:** It returns an OrderedDict where, each key = sheet index and each value = DataFrame.

### Reading All Sheets Together Using `sheet_name=None`

When you want to read every sheet in the Excel file at once, simply set `sheet_name=None`. Pandas will load all sheets and return them in a dictionary like structure, where each sheet becomes a separate DataFrame.

```
import pandas as pd
```

```
df = pd.read_excel("SampleWork.xlsx", sheet_name=None, na_values="Missing")
```

```
print(df)
```

### Output

```
{'Sheet1': Name Age Stream Percentage
0 Ankit 18 Math 95
1 Rahul 19 Science 90
2 Shaurya 20 Commerce 85
3 Aishwarya 18 Math 80
4 Priyanka 19 Science 75,
'Sheet2': Name Age Stream Percentage
0 Priya 18 Math 95
1 shivangi 19 Science 90
2 Jeet 20 Commerce 85
3 Ananya 18 Math 80
4 Swapnil 19 Science 75,
'Sheet3': Name Age Stream Percentage
0 Priya 18 Math 95
1 shivangi 19 Science 90
2 Jeet 20 NaN 85
3 Ananya 18 Math 80
4 Swapnil 19 Science 75}
```

**Explanation:** `sheet_name=None` loads all sheets

## Dictionary and Tuples

### Python Dictionary

Dictionary in Python is an ordered (since Py 3.7) collection of data values, used to store data values like a map, which, unlike other Data Types that hold only a single value as an element. Dictionaries store data in **key: value** pairs. They are ideal for fast lookups and representing structured data. Here are five important applications of Python dictionaries:

#### 1. Database Record Representation

Dictionaries are often used to represent database records where each **key-value** pair corresponds to a column and its associated value. This makes it easy to access and manipulate data based on field names.

**Example:**

```

rec = {
 'id': 1,
 'name': 'Prajjwal',
 'email': 'prajjwal@example.com',
 'age': 30
}
print(rec['name'])

```

**Output**

Prajjwal

**2. Counting Frequency of Elements**

Dictionaries can be used to count the frequency of elements in a list, string, or any iterable. The element itself serves as the key, and its frequency is the value.

**Example:**

```

i = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple']
freq = {}
for item in i:
 freq[item] = freq.get(item, 0) + 1
print(freq)

```

**Output**

```
{'apple': 3, 'banana': 2, 'orange': 1}
```

**Explanation:** We use a dictionary to count how often each fruit appears in the list.

**3. Fast Lookup Tables**

Dictionaries provide **O(1)** average time complexity for lookups, making them ideal for creating fast lookup tables. This can be used in applications like caching, memoization, or mapping operations.

**Example:**

```

lookup = {
 'USD': 'United States Dollar',
 'EUR': 'Euro',
 'JPY': 'Japanese Yen'
}
print(lookup.get('USD'))

```

**Output**

United States Dollar

**4. Storing and Accessing JSON Data**

JSON data, which is commonly used for APIs and web services, maps directly to Python dictionaries. This allows for easy storage, access, and manipulation of JSON data.

**Example:**

```
import json

a = '{"name": "prajjwal", "age": 23, "city": "Prayagraj"}'
data = json.loads(a)

print(data['name'])
```

**Output**

```
prajjwal
```

**Explanation:** We parse a JSON string into a Python dictionary and access a value by key.

**5. Grouping Data by Keys**

Dictionaries can be used to group data by certain keys, which is particularly useful in data analysis and aggregation tasks.

**Example:**

```
emp = [
 {'name': 'prajj', 'dept': 'HR'},
 {'name': 'brij', 'dept': 'IT'},
 {'name': 'kareena', 'dept': 'HR'},
 {'name': 'aryan', 'dept': 'IT'}
]

grp = {}
for emp in emp:
 dept = emp['dept']
 grp.setdefault(dept, []).append(emp['name'])

print(grp)
```

**Output**

```
{'HR': ['prajj', 'kareena'], 'IT': ['brij', 'aryan']}
```

**Python Tuple**

A Tuple is a collection of Python objects separated by commas. In some ways, a tuple is similar to a list in terms of indexing, nested objects, and repetition but a tuple is **immutable**, unlike lists that are mutable.

**1. Immutable Data Storage**

Tuples are used when data should not be changed, such as coordinates or constants.

**Example:**

```
p= (10, 20)
print(p)
```

**Output**

```
(10, 20)
```

**2. Dictionary Keys**

Tuples are hashable and can be used as dictionary keys, unlike lists.

**Example:**

```
p = {
 ("Paris", "France"): "Eiffel Tower",
 ("New York", "USA"): "Statue of Liberty"
}
print(p[("Paris", "France")])
```

**Output**

Eiffel Tower

### 3. Storing Heterogeneous Data

Tuples are ideal for grouping data of different types like user records.

**Example:**

```
p = ("Prajwal", 22, "prajwal@gfg.org")
print(p)
```

**Output**

('Prajwal', 22, 'prajwal@gfg.org')

### Python Set

A Python set is an unordered collection of unique elements. It is mutable, meaning you can add or remove elements (only regular sets), and it supports mathematical operations like union, intersection, and difference.

**Important Note:**

- Regular sets (set) are mutable and therefore not hashable.
- They cannot be used as dictionary keys or stored inside other sets.
- Python provides frozenset, an immutable version of a set, which is hashable and can be used as a key or element in another set.

### 1. Removing Duplicates from a Set

When you need to eliminate duplicate elements from a list, converting the list to a set is a quick and efficient way to do so.

**Example:**

```
a = [1, 2, 2, 3, 4, 4, 5]
u = set(a)
print(u)
```

**Output**

{1, 2, 3, 4, 5}

### 2. Set Operations

Sets are ideal for performing mathematical operations like **union**, **intersection**, and **difference**, which are useful in fields like data analysis, database management, and computational biology.

**Example:**

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print("Union:", a | b)
print("Intersection:", a & b)
print("Difference:", a - b)
```

**Output**

```
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference: {1, 2}
```

**Explanation:** We perform **union** (`|`), **intersection** (`&`), and **difference** (`-`) between two sets.

**3. Membership Testing**

Checking for the existence of an element in a collection is very efficient with sets due to their underlying hash table implementation. This makes sets a good choice for membership testing in scenarios like checking if an item is in a list of prohibited items.

**Example:**

```
ban = {"knife", "gun", "drugs"}
i = "knife"
if i in ban:
 print(f'{i} is prohibited.')
```

**Output**

```
knife is prohibited.
```

**Explanation:** We check if a given item is in the banned set. Membership testing in sets is fast.

**4. Finding Common Elements**

Sets are highly efficient for finding common elements between two or more collections. This is useful in scenarios like finding common interests, mutual friends, or common values between datasets.

**Example:**

```
a_f = {"Aryan", "Anurag", "Vishakshi"}
b_f = {"Prajwal", "Brijkant", "Aryan", "Kareena"}
comm = a_f & b_f
print(comm)
```

**Output**

```
{'Aryan'}
```

**Explanation:** We use intersection (`&`) to find common friends between two sets.

**5. Handling Data in Multi-Set Scenarios**

Sets are useful when dealing with multiple data sources where you need to ensure the uniqueness of the combined dataset or identify overlaps between different data sources.



**Example:**

```
d1 = {"apple", "banana", "cherry"}
d2 = {"banana", "cherry", "date"}
comb = d1 | d2
over = d1 & d2
print("Combined:", comb)
print("Overlap:", over)
```

**Output**

Combined: {'apple', 'cherry', 'banana', 'date'}

Overlap: {'cherry', 'banana'}

**Explanation:** We merge two sets using **union (|)** and find shared items using **intersection (&)**.

**Operations on Data Frames**

A Pandas DataFrame is a two-dimensional table-like structure in Python where data is arranged in rows and columns. It's one of the most commonly used tools for handling data and makes it easy to organize, analyze and manipulate data. It can store different types of data such as numbers, text and dates across its columns. The main parts of a DataFrame are:

- **Data:** Actual values in the table.
- **Rows:** Labels that identify each row.
- **Columns:** Labels that define each data category.

In this article, we'll see the key components of a DataFrame and see how to work with it to make data analysis easier and more efficient.

The diagram shows a table representing a DataFrame. Arrows point from the labels 'Rows', 'Columns', and 'Data' to their respective parts in the table. 'Rows' points to the index column (0-6). 'Columns' points to the header row (Name, Team, Number, Position, Age). 'Data' points to the body of the table.

|   | Name            | Team           | Number | Position | Age  |
|---|-----------------|----------------|--------|----------|------|
| 0 | Avery Bradley   | Boston Celtics | 0.0    | PG       | 25.0 |
| 1 | John Holland    | Boston Celtics | 30.0   | SG       | 27.0 |
| 2 | Jonas Jerebko   | Boston Celtics | 8.0    | PF       | 29.0 |
| 3 | Jordan Mickey   | Boston Celtics | NaN    | PF       | 21.0 |
| 4 | Terry Rozier    | Boston Celtics | 12.0   | PG       | 22.0 |
| 5 | Jared Sullinger | Boston Celtics | 7.0    | C        | NaN  |
| 6 | Evan Turner     | Boston Celtics | 11.0   | SG       | 27.0 |

DataFrame

**Creating a Pandas DataFrame**

Pandas allows us to create a DataFrame from many data sources. We can create DataFrames directly from Python objects like lists and dictionaries or by reading data from external files like CSV, Excel or SQL databases.

Here are some ways by which we create a dataframe:

**1. Creating DataFrame using a List**

If we have a simple list of data, we can easily create a DataFrame by passing that list to the `pd.DataFrame()` function.

import pandas as pd

```
lst = ['Geeks', 'For', 'Geeks', 'is',
 'portal', 'for', 'Geeks']
```

```
df = pd.DataFrame(lst)
```

```
print(df)
```

**Output:**

|   | 0      |
|---|--------|
| 0 | Geeks  |
| 1 | For    |
| 2 | Geeks  |
| 3 | is     |
| 4 | portal |
| 5 | for    |
| 6 | Geeks  |

---

## 2. Creating DataFrame from dict of ndarray/lists

We can create a DataFrame from a dictionary where the keys are column names and the values are lists or arrays.

- All arrays/lists must have the same length.
- If an index is provided, it must match the length of the arrays.
- If no index is provided, Pandas will use a default range index (0, 1, 2, ...).

import pandas as pd

```
data = {'Name':['Tom', 'nick', 'krish', 'jack'],
 'Age':[20, 21, 19, 18]}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

**Output:**

|   | Name  | Age |
|---|-------|-----|
| 0 | Tom   | 20  |
| 1 | nick  | 21  |
| 2 | krish | 19  |
| 3 | jack  | 18  |

For more details refer to Creating a Pandas DataFrame.

### Working With Rows and Columns in Pandas DataFrame

We can perform basic operations on rows/columns like selecting, deleting, adding and renaming.

#### 1. Column Selection

In Order to select a column in Pandas DataFrame, we can either access the columns by calling them by their columns name.

```
import pandas as pd
```

```
data = {'Name':['Jai', 'Princi', 'Gaurav', 'Anuj'],
 'Age':[27, 24, 22, 32],
 'Address':['Delhi', 'Kanpur', 'Allahabad', 'Kannauj'],
 'Qualification':['Msc', 'MA', 'MCA', 'Phd']}
```

```
df = pd.DataFrame(data)
```

```
print(df[['Name', 'Qualification']])
```

**Output:**

|   | Name   | Qualification |
|---|--------|---------------|
| 0 | Jai    | Msc           |
| 1 | Princi | MA            |
| 2 | Gaurav | MCA           |
| 3 | Anuj   | Phd           |

#### 2. Row Selection

Pandas provide unique methods for selecting rows from a Data frame. `DataFrame.loc[]` method is used for label-based selection

Here we'll be using nba.csv dataset in below examples for better understanding.

```
import pandas as pd

data = pd.read_csv("/content/nba.csv", index_col="Name")

first = data.loc["Avery Bradley"]
second = data.loc["R.J. Hunter"]

print(first, "\n\n", second)
```

**Output:**

```
Team Boston Celtics
Number 0
Position PG
Age 25
Height 6-2
Weight 180
College Texas
Salary 7.73034e+06
Name: Avery Bradley, dtype: object
```

```
Team Boston Celtics
Number 28
Position SG
Age 22
Height 6-5
Weight 185
College Georgia State
Salary 1.14864e+06
Name: R.J. Hunter, dtype: object
```

# CHAPTER 8

## DATA VISUALISATION USING PYTHON

---

### Data Visualisation

Data visualization uses charts, graphs and maps to present information clearly and simply. It turns complex data into visuals that are easy to understand. With large amounts of data in every industry, visualization helps spot patterns and trends quickly, leading to faster and smarter decisions.

#### Common Types of Data Visualization

1. **Charts and Graphs:** They are used to visualize data, with charts comparing data points across categories or showing trends over time and graphs analyzing relationships between variables to identify correlations, trends and outliers. Examples: Bar Charts, Line Charts, Pie Charts, Scatter Plots, Histograms, Box Plots.
2. **Maps:** They are used to display geographical data which provides spatial context to trends and patterns. Examples: Geographic Maps, Heat Maps
3. **Dashboards:** They combine multiple visualizations into a single interface which provides real-time insights and interactive features for users to explore data.

For more details refer to Types of Data Visualization

#### Importance of Data Visualization

Data visualization is essential for understanding and communicating information effectively. Here are some key reasons why it's important:

1. **Simplifies Complex Data:** It turns large and complicated data into visual formats like charts and graphs, making the information easier to understand.
2. **Reveals Patterns and Trends:** It helps identify trends, relationships and patterns that are not easily seen in raw data or tables.
3. **Saves Time:** Visuals allow quicker interpretation of data, helping users spot key information at a glance instead of manually scanning through numbers.
4. **Improves Communication:** It makes it easier to explain data insights to others, especially those who may not be familiar with the technical details.
5. **Tells a Clear Story:** Data visuals guide the audience through the information step-by-step, making it easier to reach conclusions and make informed decisions.

#### Real-World Use Cases

Data visualization is used across various industries to improve decision-making and drive results. Here are a few examples:

1. **Business Analytics:** Used to monitor company performance, track KPIs and make data-driven decisions by visualizing trends, sales and customer metrics.
2. **Healthcare:** Helps in analyzing patient records, tracking disease outbreaks and managing hospital operations through easy-to-read charts and dashboards.

3. **Sports:** Used to visualize player statistics, team performance and match outcomes, helping coaches and analysts improve strategies and training plans.
4. **Retail and E-commerce:** Enables tracking of sales, customer preferences and inventory levels, helping businesses adjust stock and marketing efforts effectively.

### Challenges

1. **Data Quality:** Accuracy of visualizations depends on the quality of the data. If the data is inaccurate or incomplete, the insights from the visualization will be misleading.
2. **Over-Simplification:** Simplifying data too much can lead to important details being lost like using a pie chart that oversimplifies complex relationships between categories.
3. **Choosing the Right Visualization:** Using the wrong type of visualization can distort the message. For example, a pie chart might not work well with many categories which leads to confusion.
4. **Overload of Information:** Too much information in a visualization can overwhelm viewers. It's important to focus on key data points and avoid clutter.

### Best Practices for Effective Data Visualization

1. **Audience-Centric Design:** Tailor visualizations to our audience's knowledge. A technical audience may need detailed graphs while a general audience benefits from simpler charts.
2. **Design Clarity and Consistency:** Choose the right chart for our data and keep the design clean with consistent colors, fonts and labels and also avoid clutter to ensure clarity.
3. **Provide Context:** Always provide context by including labels, titles and data source acknowledgments. This helps viewers to understand the significance of the data and builds trust in the results.
4. **Interactive and Accessible Design:** Make visualizations interactive with features like tooltips and filters and ensure accessibility for all users regardless of device or visual needs.

## Introduction to Data Visualisation

In today's world, a lot of data is being generated on a daily basis. And sometimes to analyze this data for certain trends, patterns may become difficult if the data is in its raw format. To overcome this data visualization comes into play. Data visualization provides a good, organized pictorial representation of the data which makes it easier to understand, observe, analyze. In this tutorial, we will discuss how to visualize data using Python.

Python provides various libraries that come with different features for visualizing data. All these libraries come with different features and can support various types of graphs. In this tutorial, we will be discussing four such libraries.

- Seaborn
- Bokeh

- Plotly

We will discuss these libraries one by one and will plot some most commonly used graphs. Before diving into these libraries, at first, we will need a database to plot the data. We will be using the tips database for this complete tutorial. Let's discuss see a brief about this database.

## Database Used

### Tips Database

Tips database is the record of the tip given by the customers in a restaurant for two and a half months in the early 1990s. It contains 6 columns such as total\_bill, tip, sex, smoker, day, time, size.

You can download the tips database from [here](#).

### Example:

```
import pandas as pd
```

```
reading the database
data = pd.read_csv("tips.csv")
```

```
printing the top 10 rows
display(data.head(10))
```

### Output:

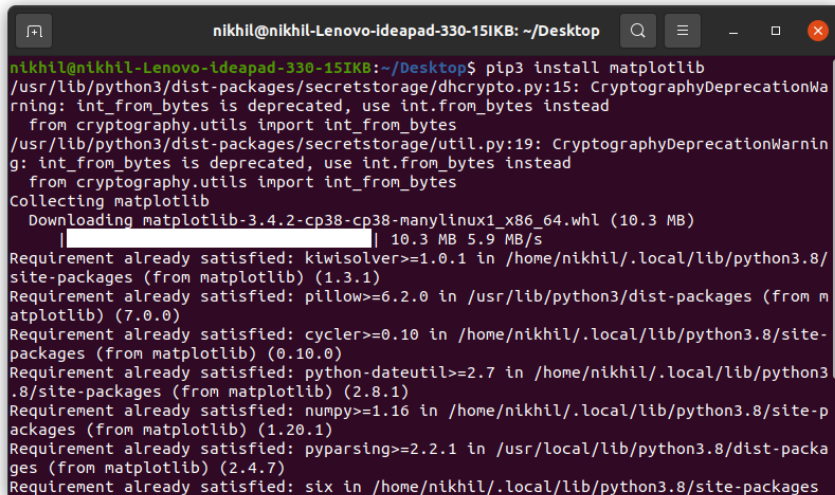
|   | total_bill | tip  | sex    | smoker | day | time   | size |
|---|------------|------|--------|--------|-----|--------|------|
| 0 | 16.99      | 1.01 | Female | No     | Sun | Dinner | 2    |
| 1 | 10.34      | 1.66 | Male   | No     | Sun | Dinner | 3    |
| 2 | 21.01      | 3.50 | Male   | No     | Sun | Dinner | 3    |
| 3 | 23.68      | 3.31 | Male   | No     | Sun | Dinner | 2    |
| 4 | 24.59      | 3.61 | Female | No     | Sun | Dinner | 4    |
| 5 | 25.29      | 4.71 | Male   | No     | Sun | Dinner | 4    |
| 6 | 8.77       | 2.00 | Male   | No     | Sun | Dinner | 2    |
| 7 | 26.88      | 3.12 | Male   | No     | Sun | Dinner | 4    |
| 8 | 15.04      | 1.96 | Male   | No     | Sun | Dinner | 2    |
| 9 | 14.78      | 3.23 | Male   | No     | Sun | Dinner | 2    |

## Matplotlib

Matplotlib is an easy-to-use, low-level data visualization library that is built on NumPy arrays. It consists of various plots like scatter plot, line plot, histogram, etc. Matplotlib provides a lot of flexibility.

To install this type the below command in the terminal.

```
pip install matplotlib
```



```

nikhil@nikhil-Lenovo-Ideapad-330-15IKB: ~/Desktop
nikhil@nikhil-Lenovo-Ideapad-330-15IKB:~/Desktop$ pip3 install matplotlib
/usr/lib/python3/dist-packages/secretstorage/dhcrypto.py:15: CryptographyDeprecationWarning: int_from_bytes is deprecated, use int.from_bytes instead
 from cryptography.utils import int_from_bytes
/usr/lib/python3/dist-packages/secretstorage/util.py:19: CryptographyDeprecationWarning: int_from_bytes is deprecated, use int.from_bytes instead
 from cryptography.utils import int_from_bytes
Collecting matplotlib
 Downloading matplotlib-3.4.2-cp38-cp38-manylinux1_x86_64.whl (10.3 MB)
 | 10.3 MB 5.9 MB/s
Requirement already satisfied: kiwisolver>=1.0.1 in /home/nikhil/.local/lib/python3.8/site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: pillow>=6.2.0 in /usr/lib/python3/dist-packages (from matplotlib) (7.0.0)
Requirement already satisfied: cython>=0.10 in /home/nikhil/.local/lib/python3.8/site-packages (from matplotlib) (0.10.0)
Requirement already satisfied: python-dateutil>=2.7 in /home/nikhil/.local/lib/python3.8/site-packages (from matplotlib) (2.8.1)
Requirement already satisfied: numpy>=1.16 in /home/nikhil/.local/lib/python3.8/site-packages (from matplotlib) (1.20.1)
Requirement already satisfied: pyparsing>=2.2.1 in /usr/local/lib/python3.8/dist-packages (from matplotlib) (2.4.7)
Requirement already satisfied: six in /home/nikhil/.local/lib/python3.8/site-packages

```

Refer to the below articles to get more information setting up an environment with Matplotlib.

- Environment Setup for Matplotlib
- Using Matplotlib with Jupyter Notebook

After installing Matplotlib, let's see the most commonly used plots using this library.

### Scatter Plot

Scatter plots are used to observe relationships between variables and uses dots to represent the relationship between them. The **scatter()** method in the matplotlib library is used to draw a scatter plot.

**Example:**

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
reading the database
data = pd.read_csv("tips.csv")
```

```
Scatter plot with day against tip
plt.scatter(data['day'], data['tip'])
```

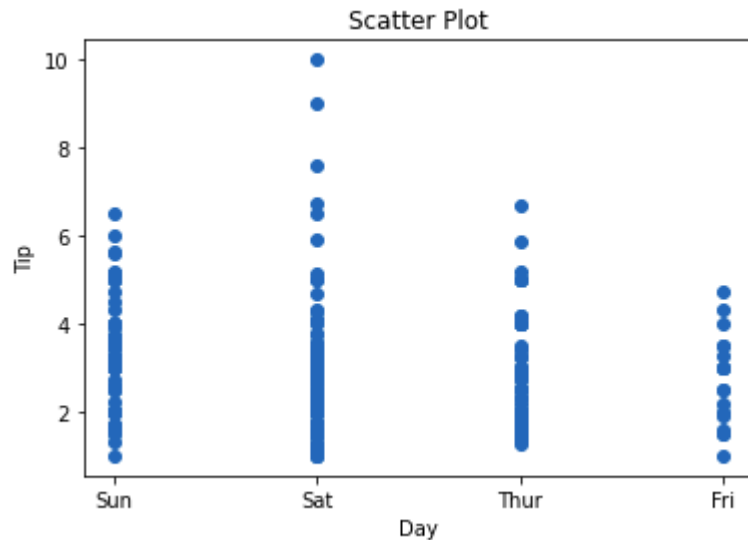
```
Adding Title to the Plot
plt.title("Scatter Plot")
```

```
Setting the X and Y labels
plt.xlabel('Day')
plt.ylabel('Tip')
```

```
plt.show()
```

**Output:**





This graph can be more meaningful if we can add colors and also change the size of the points. We can do this by using the **c** and **s** parameter respectively of the scatter function. We can also show the color bar using the `colorbar()` method.

**Example:**

**import pandas as pd**

**import matplotlib.pyplot as plt**

`# reading the database`

`data = pd.read_csv("tips.csv")`

`# Scatter plot with day against tip`

`plt.scatter(data['day'], data['tip'], c=data['size'],  
s=data['total_bill'])`

`# Adding Title to the Plot`

`plt.title("Scatter Plot")`

`# Setting the X and Y labels`

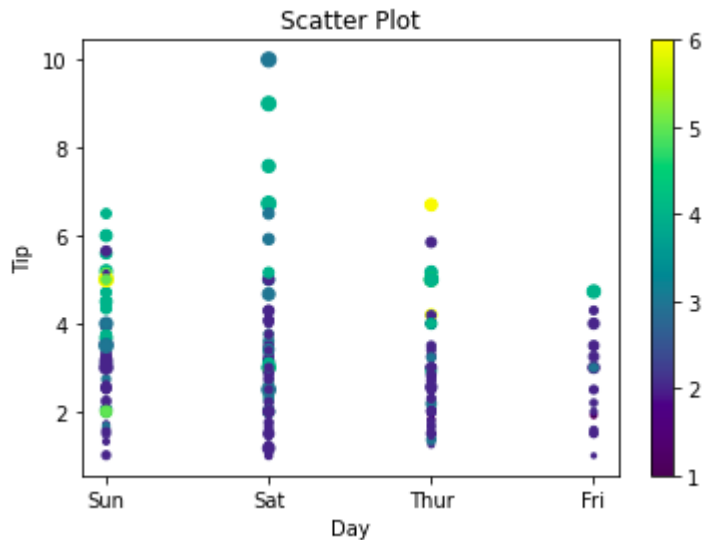
`plt.xlabel('Day')`

`plt.ylabel('Tip')`

`plt.colorbar()`

`plt.show()`

**Output:**



### Line Chart

Line Chart is used to represent a relationship between two data X and Y on a different axis. It is plotted using the **plot()** function. Let's see the below example.

#### Example:

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
reading the database
data = pd.read_csv("tips.csv")
```

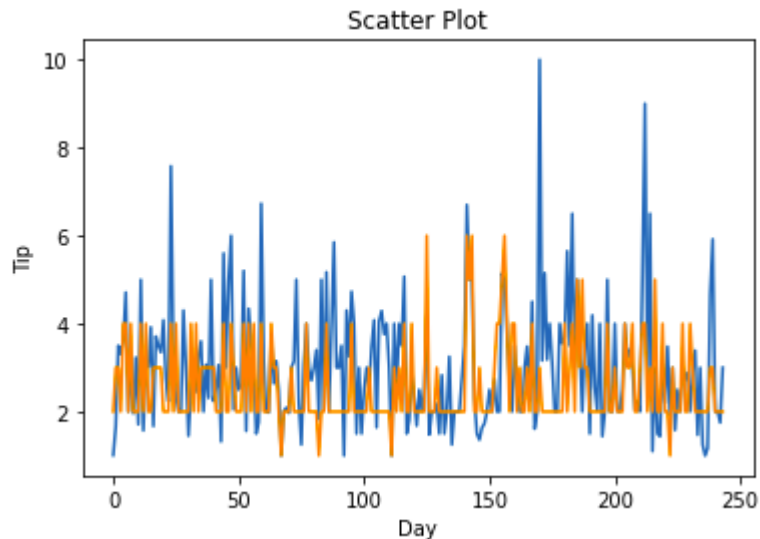
```
Scatter plot with day against tip
plt.plot(data['tip'])
plt.plot(data['size'])
```

```
Adding Title to the Plot
plt.title("Scatter Plot")
```

```
Setting the X and Y labels
plt.xlabel('Day')
plt.ylabel('Tip')
```

```
plt.show()
```

#### Output:



### Bar Chart

A bar plot or bar chart is a graph that represents the category of data with rectangular bars with lengths and heights that is proportional to the values which they represent. It can be created using the **bar()** method.

#### Example:

```
import pandas as pd
import matplotlib.pyplot as plt
```

```
reading the database
data = pd.read_csv("tips.csv")
```

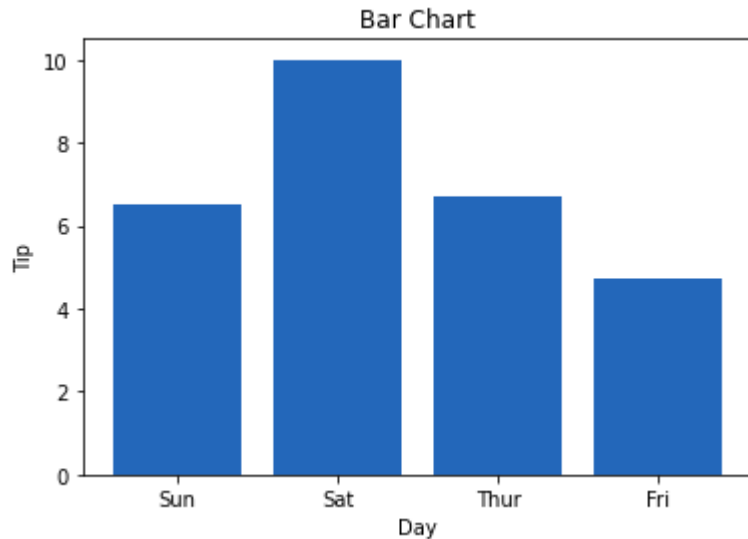
```
Bar chart with day against tip
plt.bar(data['day'], data['tip'])
```

```
plt.title("Bar Chart")
```

```
Setting the X and Y labels
plt.xlabel('Day')
plt.ylabel('Tip')
```

```
Adding the legends
plt.show()
```

#### Output:



### Histogram

A histogram is basically used to represent data in the form of some groups. It is a type of bar plot where the X-axis represents the bin ranges while the Y-axis gives information about frequency. The **hist()** function is used to compute and create a histogram. In histogram, if we pass categorical data then it will automatically compute the frequency of that data i.e. how often each value occurred.

#### Example:

```
import pandas as pd
import matplotlib.pyplot as plt
```

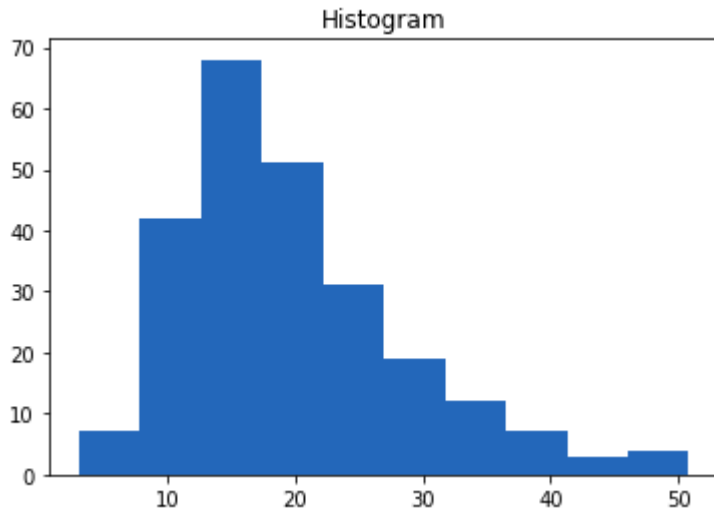
```
reading the database
data = pd.read_csv("tips.csv")
```

```
histogram of total_bills
plt.hist(data['total_bill'])
```

```
plt.title("Histogram")
```

```
Adding the legends
plt.show()
```

#### Output:



**Note:** For complete Matplotlib Tutorial, refer Matplotlib Tutorial

### Seaborn

**Seaborn** is a high-level interface built on top of the Matplotlib. It provides beautiful design styles and color palettes to make more attractive graphs.

To install seaborn type the below command in the terminal.

pip install seaborn

```

nikhil@nikhil-Lenovo-Ideapad-330-151KB: ~
Collecting scipy>=1.0
 Downloading scipy-1.6.0-cp38-cp38-manylinux1_x86_64.whl (27.2 MB)
 | 27.2 MB 33 kB/s
Requirement already satisfied: matplotlib>=2.2 in ./local/lib/python3.8/site-packages
(from seaborn) (3.3.3)
Requirement already satisfied: numpy>=1.15 in ./local/lib/python3.8/site-packages (fro
m seaborn) (1.19.5)
Requirement already satisfied: pandas>=0.23 in ./local/lib/python3.8/site-packages (fr
om seaborn) (1.2.0)
Requirement already satisfied: python-dateutil>=2.1 in /usr/lib/python3/dist-packages (
from matplotlib>=2.2->seaborn) (2.7.3)
Requirement already satisfied: kiwisolver>=1.0.1 in ./local/lib/python3.8/site-packag
es (from matplotlib>=2.2->seaborn) (1.3.1)
Requirement already satisfied: cycler>=0.10 in ./local/lib/python3.8/site-packages (fr
om matplotlib>=2.2->seaborn) (0.10.0)
Requirement already satisfied: pillow>=6.2.0 in /usr/lib/python3/dist-packages (from ma
tplotlib>=2.2->seaborn) (7.0.0)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.3 in /usr/local/l
ib/python3.8/dist-packages (from matplotlib>=2.2->seaborn) (2.4.7)
Requirement already satisfied: pytz>=2017.3 in /usr/lib/python3/dist-packages (from pan
das>=0.23->seaborn) (2019.3)
Requirement already satisfied: six in /usr/lib/python3/dist-packages (from cycler>=0.10
->matplotlib>=2.2->seaborn) (1.14.0)
Installing collected packages: scipy, seaborn
Successfully installed scipy-1.6.0 seaborn-0.11.1
nikhil@nikhil-Lenovo-Ideapad-330-151KB:~$

```

Seaborn is built on the top of Matplotlib, therefore it can be used with the Matplotlib as well. Using both Matplotlib and Seaborn together is a very simple process. We just have to invoke the Seaborn Plotting function as normal, and then we can use Matplotlib's customization function.

**Note:** Seaborn comes loaded with dataset such as tips, iris, etc. but for the sake of this tutorial we will use Pandas for loading these datasets.

### Example:

```

importing packages
import seaborn as sns
import matplotlib.pyplot as plt

```

```
import pandas as pd
```

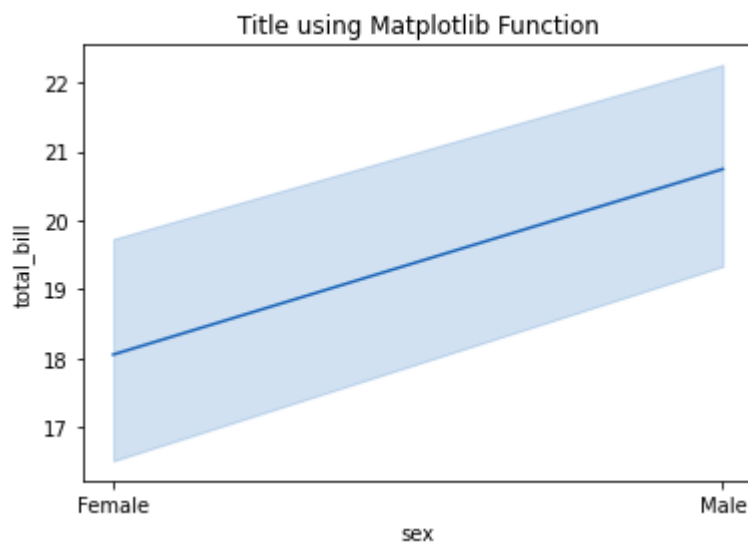
```
reading the database
data = pd.read_csv("tips.csv")
```

```
draw lineplot
sns.lineplot(x="sex", y="total_bill", data=data)
```

```
setting the title using Matplotlib
plt.title('Title using Matplotlib Function')
```

```
plt.show()
```

**Output:**



### Scatter Plot

Scatter plot is plotted using the **scatterplot()** method. This is similar to Matplotlib, but additional argument data is required.

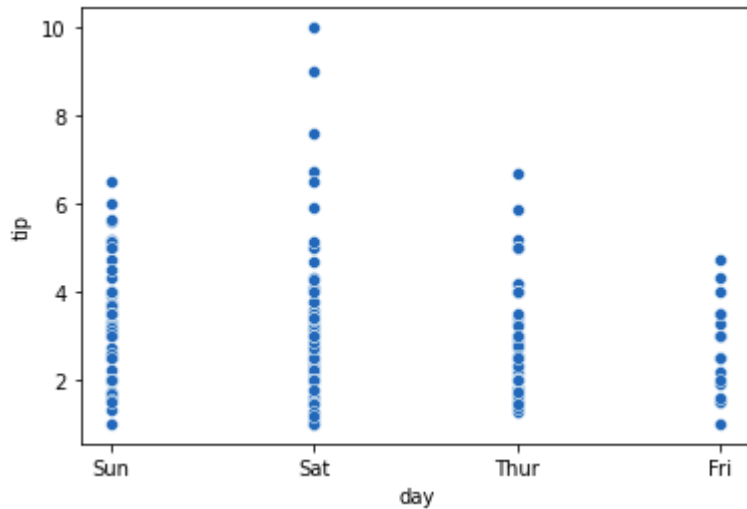
**Example:**

```
importing packages
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

```
reading the database
data = pd.read_csv("tips.csv")
```

```
sns.scatterplot(x='day', y='tip', data=data,)
plt.show()
```

**Output:**



### Matplotlib Library

matplotlib is a Python library for creating static, interactive and animated visualizations from data. It provides flexible and customizable plotting functions that help in understanding data patterns, trends and relationships effectively.

**Example:** Let's create a simple line plot using Matplotlib, showcasing the ease with which you can visualize data.

```
import matplotlib.pyplot as plt
```

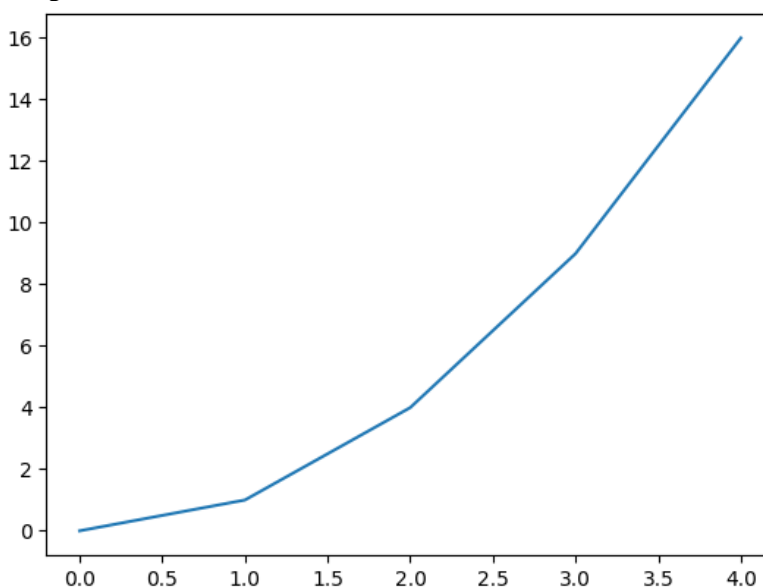
```
x = [0, 1, 2, 3, 4]
```

```
y = [0, 1, 4, 9, 16]
```

```
plt.plot(x, y)
```

```
plt.show()
```

### Output



## Components or Parts of Matplotlib Figure

**Anatomy of a Matplotlib Plot:** This section dives into the key components of a Matplotlib plot, including figures, axes, titles and legends, essential for effective data visualization.

The parts of a Matplotlib figure include (as shown in the figure above):

- **Figure:** The overarching container that holds all plot elements, acting as the canvas for visualizations.
- **Axes:** The areas within the figure where data is plotted; each figure can contain multiple axes.
- **Axis:** Represents the x-axis and y-axis, defining limits, tick locations and labels for data interpretation.
- **Lines and Markers:** Lines connect data points to show trends, while markers denote individual data points in plots like scatter plots.
- **Title and Labels:** The title provides context for the plot, while axis labels describe what data is being represented on each axis.

## Matplotlib Pyplot

Pyplot is a module within Matplotlib that provides a MATLAB-like interface for making plots. It simplifies the process of adding plot elements such as lines, images and text to the axes of the current figure.

### Steps to Use Pyplot

- **Import Matplotlib:** Start by importing matplotlib.pyplot as plt.
- **Create Data:** Prepare your data in the form of lists or arrays.
- **Plot Data:** Use plt.plot() to create the plot.
- **Customize Plot:** Add titles, labels and other elements using methods like plt.title(), plt.xlabel() and plt.ylabel().
- **Display Plot:** Use plt.show() to display the plot.

Let's visualize a basic plot and understand basic components of matplotlib figure:

```
import matplotlib.pyplot as plt
```

```
x = [0, 2, 4, 6, 8]
```

```
y = [0, 4, 16, 36, 64]
```

```
fig, ax = plt.subplots()
```

```
ax.plot(x, y, marker='o', label="Data Points")
```

```
ax.set_title("Basic Components of Matplotlib Figure")
```

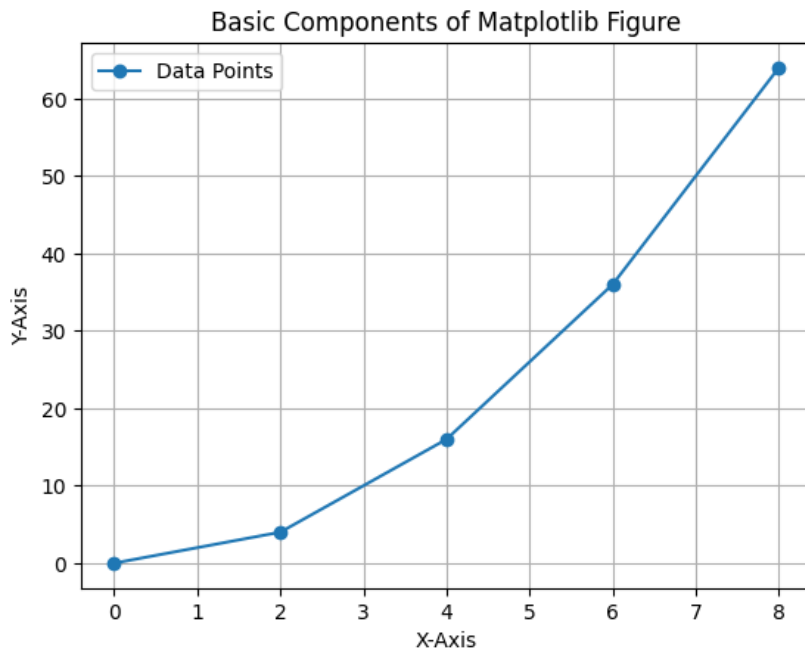
```
ax.set_xlabel("X-Axis")
```

```
ax.set_ylabel("Y-Axis")
```

```
plt.show()
```

## Output





Basic Components of matplotlib figure

### Different Types of Plots in Matplotlib

Matplotlib offers a wide range of plot types to suit various data visualization needs. Here are some of the most commonly used types of plots in Matplotlib:

#### 1. Line Chart

Line chart is one of the basic plots and can be created using `plot()` function. It is used to represent a relationship between two data X and Y on a different axis.

##### Syntax:

```
matplotlib.pyplot.plot(x, y)
```

**Parameter:** x, y Coordinates for data points.

**Example:** This code plots a simple line chart with labeled axes and a title using Matplotlib.  
`import matplotlib.pyplot as plt`

```
x = [10, 20, 30, 40]
```

```
y = [20, 25, 35, 55]
```

```
plt.plot(x, y)
```

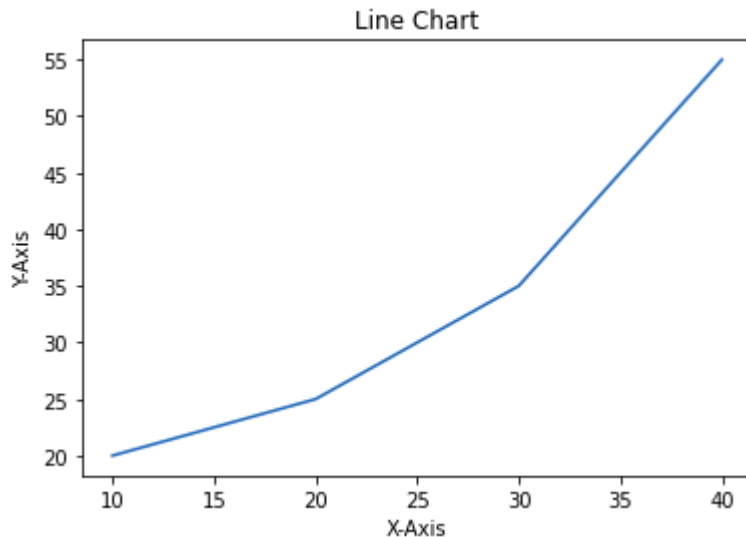
```
plt.title("Line Chart")
```

```
plt.ylabel('Y-Axis')
```

```
plt.xlabel('X-Axis')
```

```
plt.show()
```

##### Output



## 2. Bar Chart

Bar chart displays categorical data using rectangular bars whose lengths are proportional to the values they represent. It can be plotted vertically or horizontally to compare different categories.

### Syntax:

`matplotlib.pyplot.bar(x, height)`

### Parameter:

- **x:** Categories or positions on x-axis.
- **height:** Heights of the bars (y-axis values).

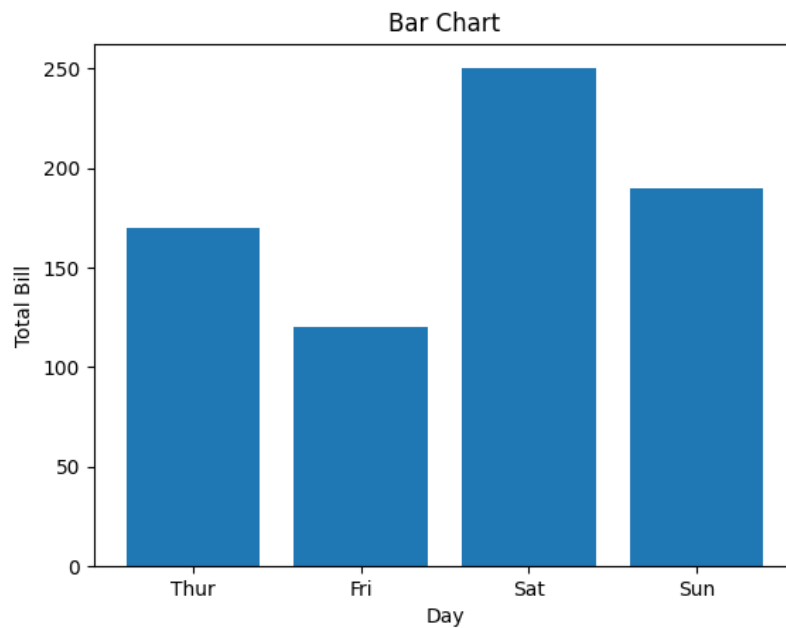
**Example:** This code creates a simple bar chart to show total bills for different days. X-axis represents the days and Y-axis shows total bill amount.

```
import matplotlib.pyplot as plt
```

```
x = ['Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190]
```

```
plt.bar(x, y)
plt.title("Bar Chart")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```

### Output



### 3. Histogram

Histogram shows the distribution of data by grouping values into bins. The `hist()` function is used to create it, with X-axis showing bins and Y-axis showing frequencies.

#### Syntax:

```
matplotlib.pyplot.hist(x, bins=None)
```

#### Parameter:

- **x:** Input data.
- **bins:** Number of bins (intervals) to group data.

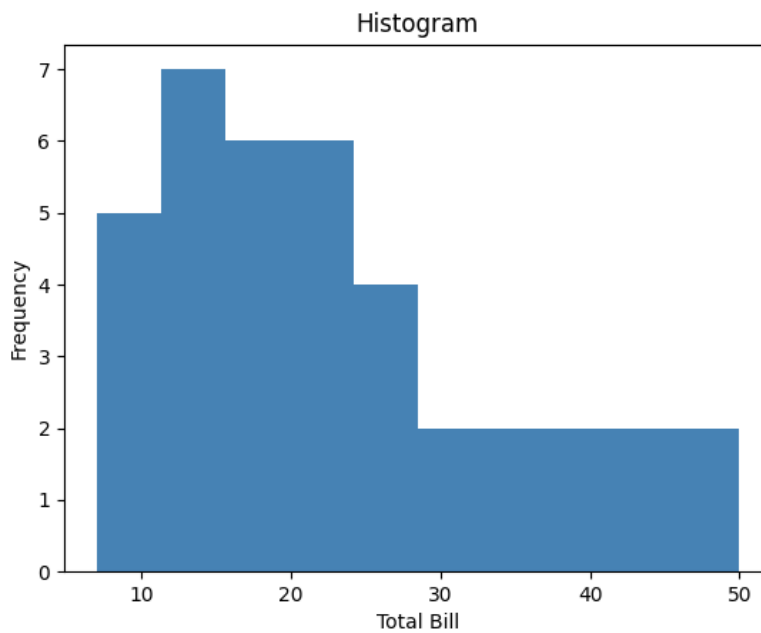
**Example:** This code plots a histogram to show frequency distribution of total bill values from the list `x`. It uses 10 bins and adds axis labels and a title for clarity.

```
import matplotlib.pyplot as plt
```

```
x = [7, 8, 9, 10, 10, 12, 12, 12, 13, 14, 14, 15, 16, 16, 17, 18, 18, 19, 20, 20,
 21, 22, 23, 24, 25, 25, 26, 28, 30, 32, 35, 36, 38, 40, 42, 44, 48, 50]
```

```
plt.hist(x, bins=10, color='steelblue')
plt.title("Histogram")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")
plt.show()
```

#### Output



#### 4. Scatter Plot

Scatter plots are used to observe relationships between variables. The `scatter()` method in the `matplotlib` library is used to draw a scatter plot.

##### Syntax:

```
matplotlib.pyplot.scatter(x, y)
```

**Parameter:** `x, y` Coordinates of the points.

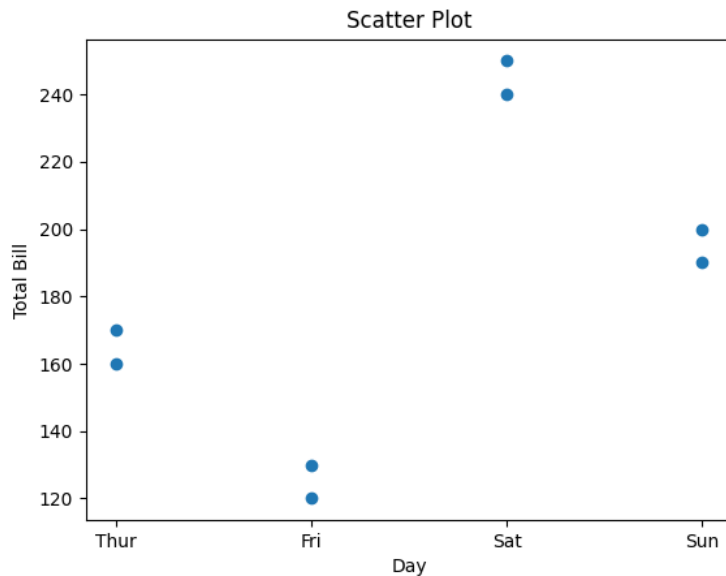
**Example:** This code creates a scatter plot to visualize the relationship between days and total bill amounts using `scatter()`.

```
import matplotlib.pyplot as plt
```

```
x = ['Thur', 'Fri', 'Sat', 'Sun', 'Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190, 160, 130, 240, 200]
```

```
plt.scatter(x, y)
plt.title("Scatter Plot")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```

##### Output



## 5. Pie Chart

Pie chart is a circular chart used to show data as proportions or percentages. It is created using the `pie()`, where each slice (wedge) represents a part of the whole.

### Syntax:

```
matplotlib.pyplot.pie(x, labels=None, autopct=None)
```

### Parameter:

- **x:** Data values for pie slices.
- **labels:** Names for each slice.
- **autopct:** Format to display percentage (e.g., '%1.1f%%').

**Example:** This code creates a simple pie chart to visualize distribution of different car brands. Each slice of pie represents the proportion of cars for each brand in the dataset.

```
import matplotlib.pyplot as plt
import pandas as pd
```

```
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR',]
data = [23, 10, 35, 15, 12]
```

```
plt.pie(data, labels=cars)
plt.title(" Pie Chart")
plt.show()
```

### Output

## 6. Box Plot

Box plot is a simple graph that shows how data is spread out. It displays the minimum, maximum, median and quartiles and also helps to spot outliers easily.

### Syntax:

```
matplotlib.pyplot.boxplot(x, notch=False, vert=True)
```

### Parameter:

- **x:** Data for which box plot is to be drawn (usually a list or array).
- **notch:** If True, draws a notch to show the confidence interval around the median.
- **vert:** If True, boxes are vertical. If False, they are horizontal.

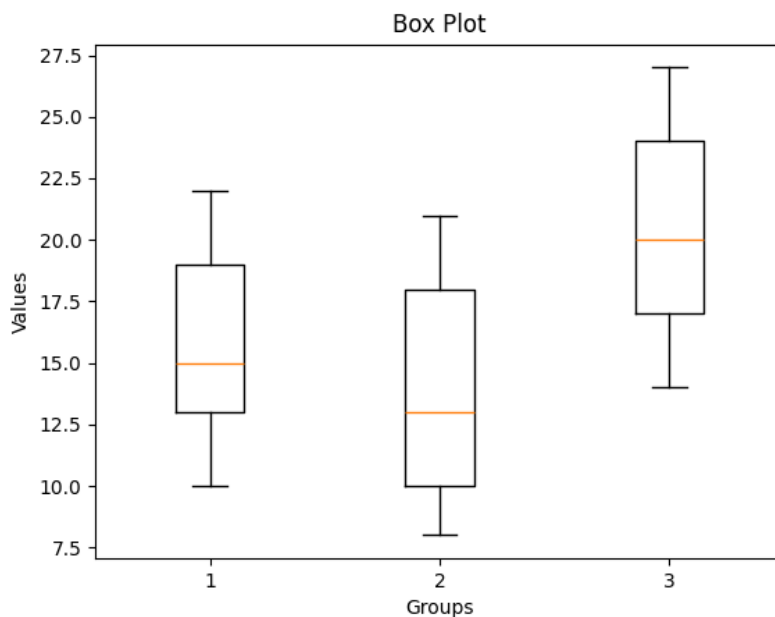
**Example:** This code creates a box plot to show the data distribution and compare three groups using matplotlib

```
import matplotlib.pyplot as plt
```

```
data = [[10, 12, 14, 15, 18, 20, 22],
 [8, 9, 11, 13, 17, 19, 21],
 [14, 16, 18, 20, 23, 25, 27]]
```

```
plt.boxplot(data)
plt.xlabel("Groups")
plt.ylabel("Values")
plt.title("Box Plot")
plt.show()
```

**Output**



## 7. Heatmap

Heatmap is a graphical representation of data where values are shown as colors. It helps visualize patterns, correlations or intensity in a matrix-like format. It is created using **imshow()** method in Matplotlib.

**Syntax:**

```
matplotlib.pyplot.imshow(X, cmap='viridis')
```

**Parameter:**

- **X:** 2D array (data to display as an image or heatmap).
- **cmap:** Sets the color map.

**Example:** This code creates a heatmap of random 10×10 data using imshow(). It uses 'viridis' color map and colorbar() adds a color scale.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(0)
data = np.random.rand(10, 10)

plt.imshow(data, cmap='viridis', interpolation='nearest')

plt.colorbar()
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Heatmap')
plt.show()
```

### Output

#### Explanation:

- **np.random.seed(0):** Ensures same random values every time (reproducibility).
- **np.random.rand(10, 10):** Generates a 10×10 array of random numbers between 0 and 1.

#### Key Features of Matplotlib

- **Versatile Plotting:** Create a wide variety of visualizations, including line plots, scatter plots, bar charts and histograms.
- **Extensive Customization:** Control every aspect of your plots, from colors and markers to labels and annotations.
- **Seamless Integration with NumPy:** Effortlessly plot data arrays directly, enhancing data manipulation capabilities.
- **High-Quality Graphics:** Generate publication-ready plots with precise control over aesthetics.
- **Cross-Platform Compatibility:** Use Matplotlib on Windows, macOS and Linux without issues.
- **Interactive Visualizations:** Engage with your data dynamically through interactive plotting features.

### Different Types of Charts using Pyplot-

Matplotlib is a Python library used for creating static, animated and interactive data visualizations. It is built on the top of NumPy and it can easily handles large datasets for creating various types of plots such as line charts, bar charts, scatter plots, etc.

#### Visualizing Data with Pyplot using Matplotlib

Pyplot is a module in Matplotlib that provides a simple interface for creating plots. It allows users to generate charts like line graphs, bar charts and histograms with minimal code. Let's explore some examples with simple code to understand how to use it effectively.

## 1. Line Chart

Line chart is one of the basic plots and can be created using `plot()` function. It is used to represent a relationship between two data X and Y on a different axis.

### Syntax:

```
matplotlib.pyplot.plot(x, y)
```

**Parameter:** x, y Coordinates for data points.

**Example:** This code plots a simple line chart with labeled axes and a title using Matplotlib.

```
import matplotlib.pyplot as plt
```

```
x = [10, 20, 30, 40]
```

```
y = [20, 25, 35, 55]
```

```
plt.plot(x, y)
```

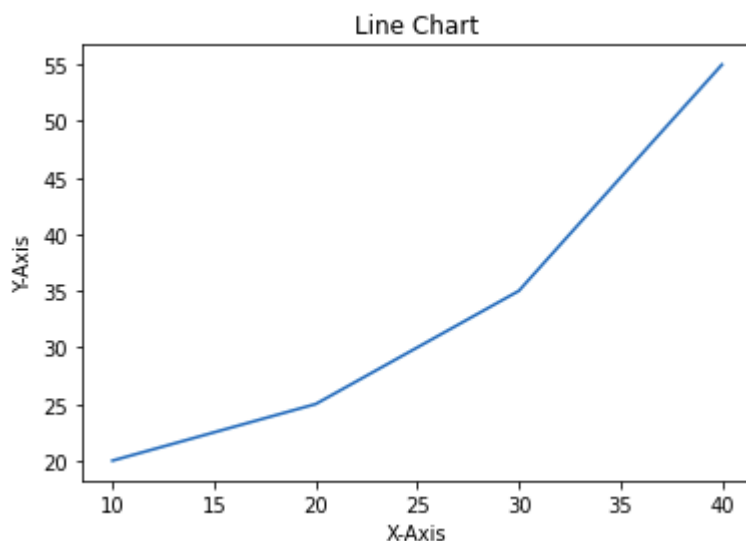
```
plt.title("Line Chart")
```

```
plt.ylabel('Y-Axis')
```

```
plt.xlabel('X-Axis')
```

```
plt.show()
```

### Output



## 2. Bar Chart

Bar chart displays categorical data using rectangular bars whose lengths are proportional to the values they represent. It can be plotted vertically or horizontally to compare different categories.

### Syntax:

```
matplotlib.pyplot.bar(x, height)
```

### Parameter:

- **x:** Categories or positions on x-axis.
- **height:** Heights of the bars (y-axis values).

**Example:** This code creates a simple bar chart to show total bills for different days. X-axis represents the days and Y-axis shows total bill amount.

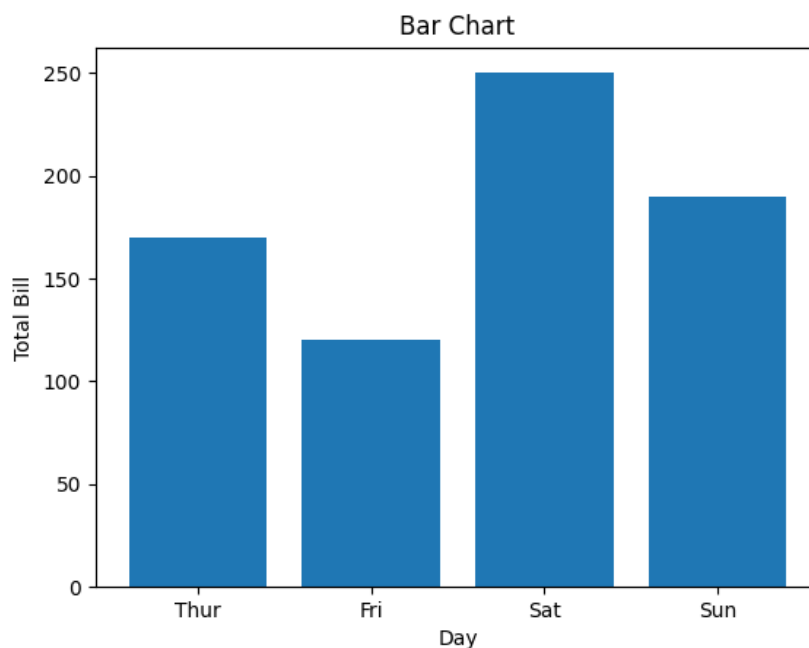
```
import matplotlib.pyplot as plt
```



```
x = ['Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190]
```

```
plt.bar(x, y)
plt.title("Bar Chart")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```

### Output



Bar Chart

### 3. Histogram

Histogram shows the distribution of data by grouping values into bins. The `hist()` function is used to create it, with X-axis showing bins and Y-axis showing frequencies.

#### Syntax:

```
matplotlib.pyplot.hist(x, bins=None)
```

#### Parameter:

- **x:** Input data.
- **bins:** Number of bins (intervals) to group data.

**Example:** This code plots a histogram to show frequency distribution of total bill values from the list `x`. It uses 10 bins and adds axis labels and a title for clarity.

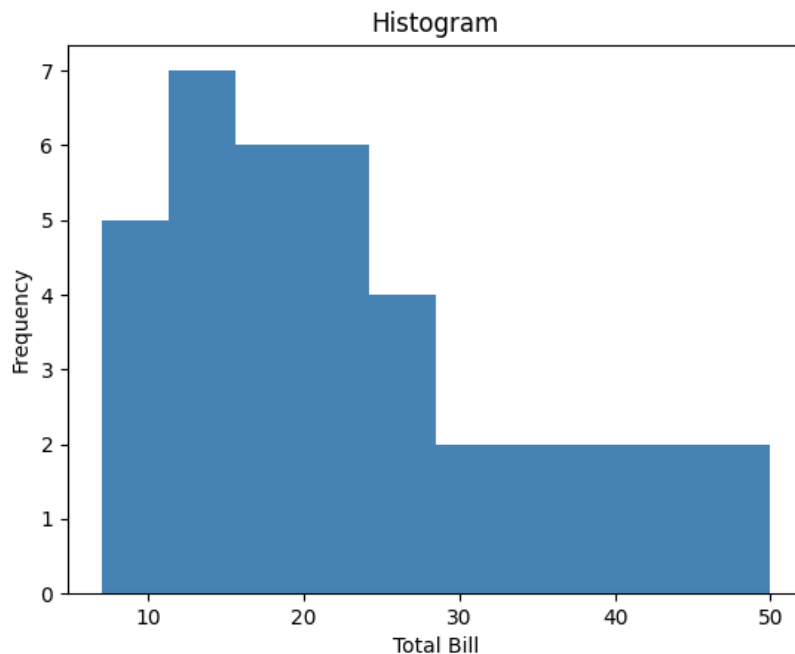
```
import matplotlib.pyplot as plt
```

```
x = [7, 8, 9, 10, 10, 12, 12, 12, 13, 14, 14, 15, 16, 16, 17, 18, 18, 19, 20, 20,
 21, 22, 23, 24, 25, 25, 26, 28, 30, 32, 35, 36, 38, 40, 42, 44, 48, 50]
```

```
plt.hist(x, bins=10, color='steelblue')
```

```
plt.title("Histogram")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")
plt.show()
```

### Output



### Histogram

#### 4. Scatter Plot

Scatter plots are used to observe relationships between variables. The `scatter()` method in the `matplotlib` library is used to draw a scatter plot.

#### Syntax:

```
matplotlib.pyplot.scatter(x, y)
```

**Parameter:** `x, y` Coordinates of the points.

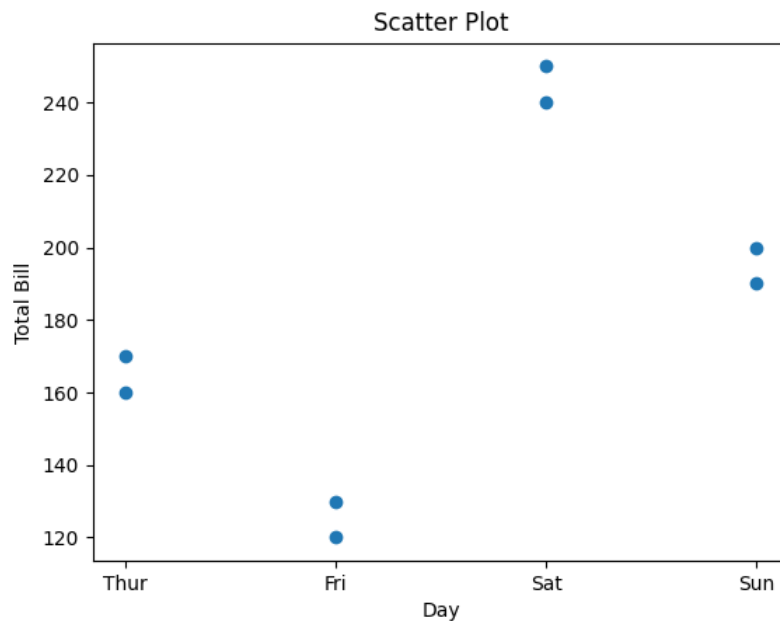
**Example:** This code creates a scatter plot to visualize the relationship between days and total bill amounts using `scatter()`.

```
import matplotlib.pyplot as plt
```

```
x = ['Thur', 'Fri', 'Sat', 'Sun', 'Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190, 160, 130, 240, 200]
```

```
plt.scatter(x, y)
plt.title("Scatter Plot")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```

### Output



Scatter Plot

## 5. Pie Chart

Pie chart is a circular chart used to show data as proportions or percentages. It is created using the `pie()`, where each slice (wedge) represents a part of the whole.

### Syntax:

```
matplotlib.pyplot.pie(x, labels=None, autopct=None)
```

### Parameter:

- **x:** Data values for pie slices.
- **labels:** Names for each slice.
- **autopct:** Format to display percentage (e.g., '%1.1f%%').

**Example:** This code creates a simple pie chart to visualize distribution of different car brands. Each slice of pie represents the proportion of cars for each brand in the dataset.

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR',]
```

```
data = [23, 10, 35, 15, 12]
```

```
plt.pie(data, labels=cars)
```

```
plt.title(" Pie Chart")
```

```
plt.show()
```

### Output

## 6. Box Plot

Box plot is a simple graph that shows how data is spread out. It displays the minimum, maximum, median and quartiles and also helps to spot outliers easily.

### Syntax:

```
matplotlib.pyplot.boxplot(x, notch=False, vert=True)
```

**Parameter:**

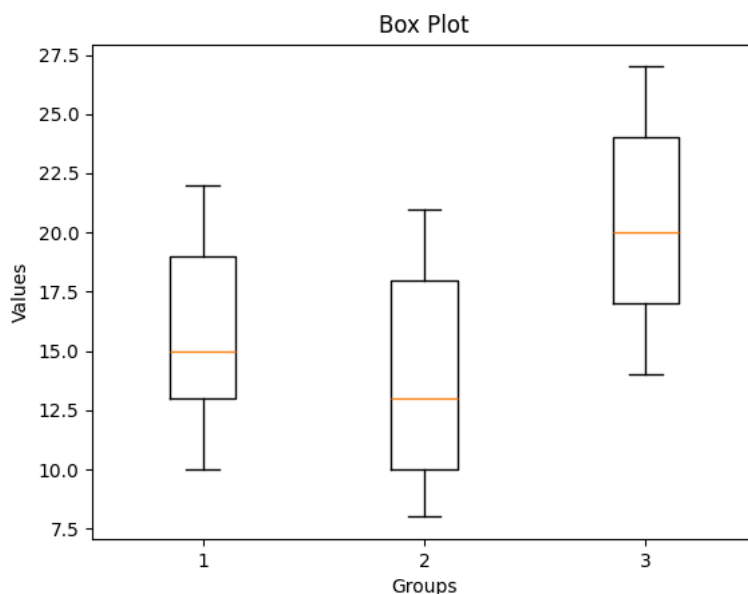
- **x:** Data for which box plot is to be drawn (usually a list or array).
- **notch:** If True, draws a notch to show the confidence interval around the median.
- **vert:** If True, boxes are vertical. If False, they are horizontal.

**Example:** This code creates a box plot to show the data distribution and compare three groups using matplotlib

```
import matplotlib.pyplot as plt
```

```
data = [[10, 12, 14, 15, 18, 20, 22],
 [8, 9, 11, 13, 17, 19, 21],
 [14, 16, 18, 20, 23, 25, 27]]
```

```
plt.boxplot(data)
plt.xlabel("Groups")
plt.ylabel("Values")
plt.title("Box Plot")
plt.show()
```

**Output**

Box Plot

**7. Heatmap**

Heatmap is a graphical representation of data where values are shown as colors. It helps visualize patterns, correlations or intensity in a matrix-like format. It is created using **imshow()** method in Matplotlib.

**Syntax:**

```
matplotlib.pyplot.imshow(X, cmap='viridis')
```

**Parameter:**

- **X:** 2D array (data to display as an image or heatmap).
- **cmap:** Sets the color map.

**Example:** This code creates a heatmap of random 10×10 data using `imshow()`. It uses 'viridis' color map and `colorbar()` adds a color scale.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
np.random.seed(0)
data = np.random.rand(10, 10)

plt.imshow(data, cmap='viridis', interpolation='nearest')
```

```
plt.colorbar()
plt.xlabel('X-axis Label')
plt.ylabel('Y-axis Label')
plt.title('Heatmap')
plt.show()
```

### Output

### Explanation:

- **np.random.seed(0):** Ensures same random values every time (reproducibility).
- **np.random.rand(10, 10):** Generates a 10×10 array of random numbers between 0 and 1.

### How to Customize Matplotlib Visualizations

Customization in Matplotlib allows you to improve look and clarity of plots by adjusting elements like colors, styles, labels, titles and gridlines. It helps make visualizations more informative and visually appealing for better data communication. Let's explore different ways to customize visualizations:

#### 1. Customizing Line Chart

Line charts can be customized using various properties:

1. **Color:** Change the color of the line
2. **Linewidth:** Adjust the width of the line
3. **Marker:** Change the style of plotted points
4. **Markersize:** Change the size of the markers
5. **Linestyle:** Define the style of the line like solid, dashed, etc.

**Example:** This code creates a customized line chart with a green dashed line, thicker width, large circular markers and labeled axes and title.

```
import matplotlib.pyplot as plt
```

```
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

plt.plot(x, y, color='green', linewidth=3, marker='o', markersize=15, linestyle='--')

plt.title("Customizing Line Chart")
plt.ylabel("Y-Axis")
```

```
plt.xlabel('X-Axis')
plt.show()
```

### Output

## 2. Customizing Bar Chart

Bar charts can be made more informative and visually appealing by customizing:

- **Color:** Fill color of the bars
- **Edgecolor:** Color of the bar edges
- **Linewidth:** Thickness of the edges
- **Width:** Width of each bar

**Example:** This code creates a customized bar chart with green bars, blue edges, thicker border lines and labeled axes and title.

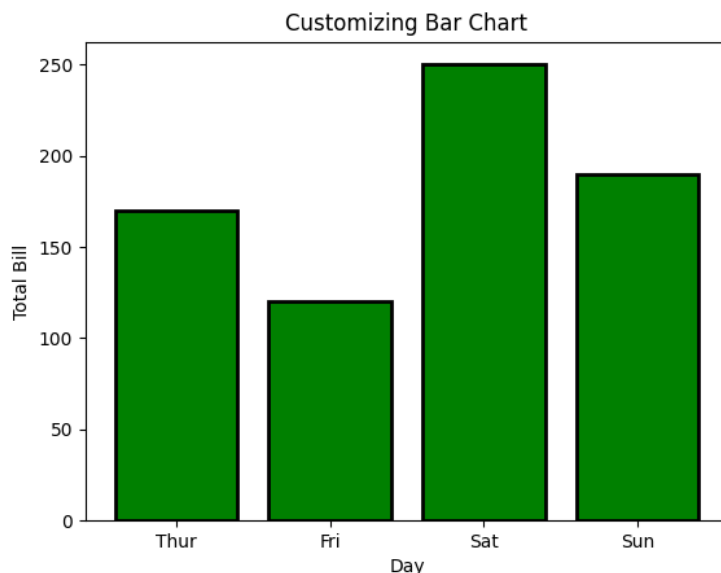
```
import matplotlib.pyplot as plt
```

```
x = ['Thur', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190]
```

```
plt.bar(x, y, color='green', edgecolor='black', linewidth=2)
```

```
plt.title("Customizing Bar Chart")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```

### Output



Customizing Bar plot

## 3. Customizing Histogram Plot

To make histogram plots more effective various customizations can be applied:

- **Bins:** Number of groups (bins) to divide data into
- **Color:** Bar fill color

- **Edgecolor:** Bar edge color
- **Linestyle:** Style of the edges like solid, dashed, etc.
- **Alpha:** Transparency level (0 = transparent, 1 = opaque)

**Example:** This code creates a customized histogram with green bars, blue edges, dashed border lines, semi-transparent fill and labeled axes and title.

```
import matplotlib.pyplot as plt
```

```
x = [7, 8, 9, 10, 10, 12, 12, 12, 13, 14, 14, 15, 16, 16, 17,
 18, 18, 19, 20, 20, 21, 22, 23, 24, 25, 25, 26, 28, 30,
 32, 35, 36, 38, 40, 42, 44, 48, 50]
```

```
plt.hist(x, bins=10, color='green', edgecolor='blue',linestyle='--', alpha=0.5)
```

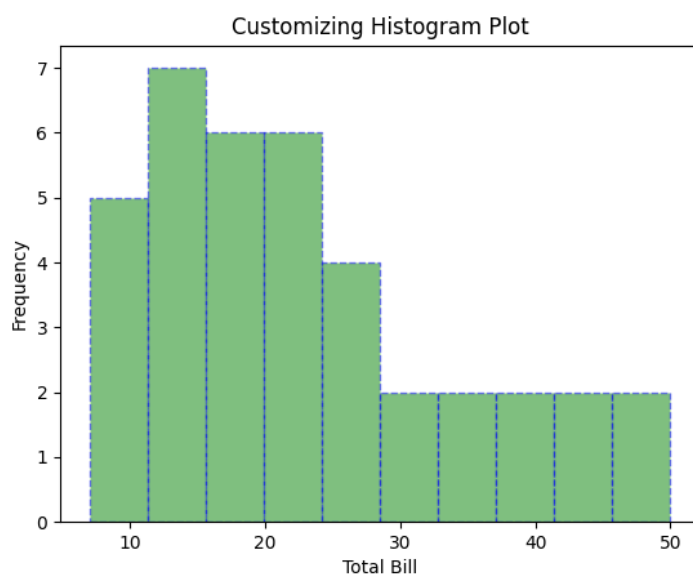
```
plt.title("Customizing Histogram Plot")
```

```
plt.xlabel("Total Bill")
```

```
plt.ylabel("Frequency")
```

```
plt.show()
```

**Output**



Customizing Histogram

#### 4. Customizing Scatter Plot

Scatter plots can be enhanced with:

- **S:** Marker size (single value or array)
- **C:** Color of markers or sequence of colors
- **Marker:** Marker style like circle, diamond, etc.
- **Linewidths:** Width of marker borders
- **Edgecolor:** Color of marker borders
- **Alpha:** Blending value, between 0 (transparent) and 1 (opaque)

**Example:** This code creates a customized scatter plot using diamond-shaped markers, where color represents size, marker size reflects the total bill and transparency is added for better visualization. It includes labeled axes and a title.

```
import matplotlib.pyplot as plt
```

```
x = ['Thur', 'Fri', 'Sat', 'Sun', 'Thur', 'Fri', 'Sat', 'Sun']
```

```
y = [170, 120, 250, 190, 180, 130, 260, 200]
```

```
size = [2, 3, 4, 2, 3, 2, 4, 3]
```

```
bill = [170, 120, 250, 190, 180, 130, 260, 200]
```

```
plt.scatter(x, y, c=size, s=bill, marker='D', alpha=0.5)
```

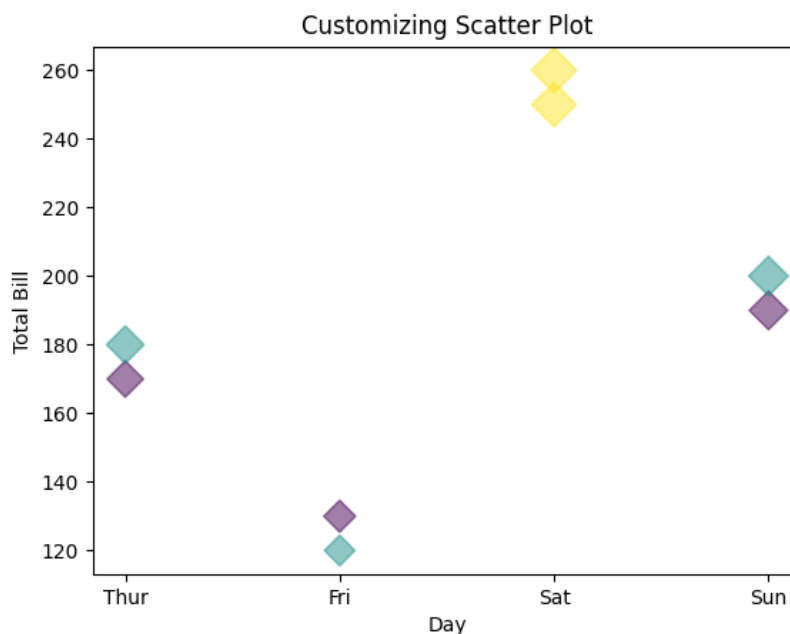
```
plt.title("Customizing Scatter Plot")
```

```
plt.xlabel("Day")
```

```
plt.ylabel("Total Bill")
```

```
plt.show()
```

**Output**



Customizing Scatter Plot

## 5. Customizing Pie Chart

To make pie charts more effective and visually appealing use:

- **Explode:** Moving the wedges of the plot
- **Autopct:** Label the wedge with their numerical value.
- **Color:** Colors of the slices
- **Shadow:** Used to create a shadow effect

**Example:** This code creates a customized pie chart with colored slices, exploded segments for emphasis, percentage labels with two decimal places and a shadow effect for better visual appeal.

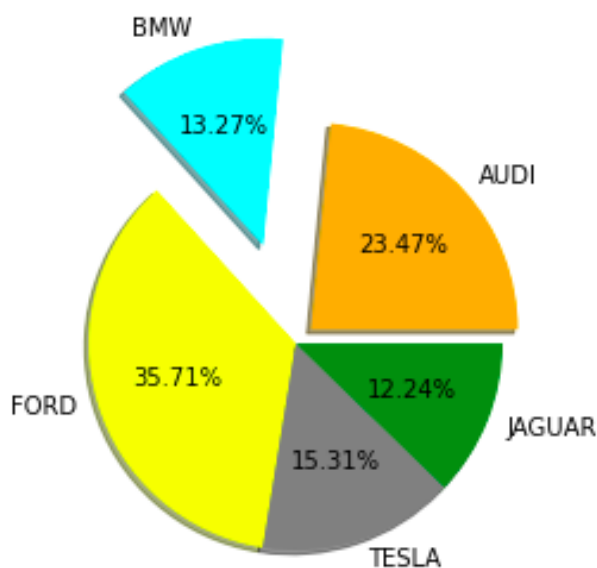


```
import matplotlib.pyplot as plt
import pandas as pd
```

```
cars = ['AUDI', 'BMW', 'FORD', 'TESLA', 'JAGUAR',]
data = [23, 13, 35, 15, 12]
explode = [0.1, 0.5, 0, 0, 0]
colors = ("orange", "cyan", "yellow", "grey", "green",)
```

```
plt.pie(data, labels=cars, explode=explode, autopct='%1.2f%%', colors=colors,
shadow=True)
plt.show()
```

**Output**



## Matplotlib's Core Components: Figures and Axes

Before we proceed let's understand two classes which are important for working with Matplotlib.

### 1. Figure class

The Figure class represents the full drawing area or canvas that can hold one or more plots. It is created using the figure() function and lets user control the overall size, layout and background of the plot window.

#### Syntax:

```
matplotlib.figure.Figure(figsize=None, facecolor=None)
```

#### Parameter:

- **figsize:** Sets size of the figure (width, height) in inches.
- **facecolor:** Sets background color of the figure.

**Example:** This code demonstrates how to use Figure class to create a simple line plot. It sets figure size and background color, adds custom axes, plots data and labels the axes and title.

```
import matplotlib.pyplot as plt
```

```
Create a Figure with basic size and background color
```

```
fig = plt.figure(figsize=(6, 4), facecolor='lightblue')
```

```
Add Axes to the Figure
```

```
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])
```

```
x = [1, 2, 3, 4]
```

```
y = [10, 20, 15, 25]
```

```
ax.plot(x, y)
```

```
plt.title("Simple Line Plot")
```

```
plt.xlabel("X-Axis")
```

```
plt.ylabel("Y-Axis")
```

```
plt.show()
```

### Output

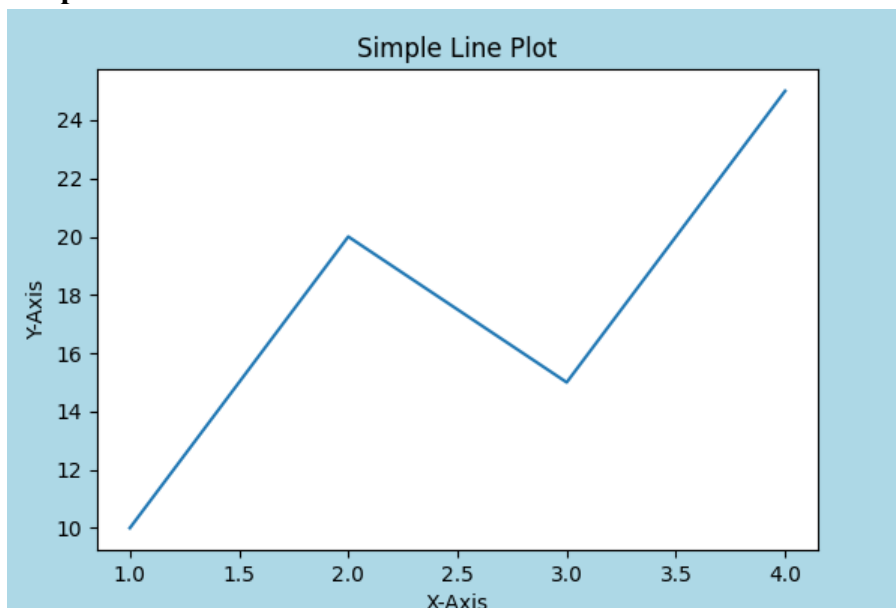


Figure class Representation

### Explanation:

- **fig.add\_axes()** adds an Axes object to the figure.
- **[0.1, 0.1, 0.8, 0.8]** defines the position left, bottom, width and height as a fraction of figure size.

## 2. Axes Class

Axes class represents actual plotting area where data is drawn. It is the most basic and flexible for creating plots or subplots within a figure. A single figure can contain multiple axes but each Axes object belongs to only one figure. It can create an Axes object using `axes()` function.

### Syntax:

```
axes([left, bottom, width, height])
```

**Example:** This code creates a figure using Figure class and adds a custom axes area to it. It then plots two line graphs one for x vs y and another for y vs x. The graph includes axis labels, a title and a legend for better clarity and presentation.

```
import matplotlib.pyplot as plt
```

```
from matplotlib.figure import Figure
```

```

x = [10, 20, 30, 40]
y = [20, 25, 35, 55]

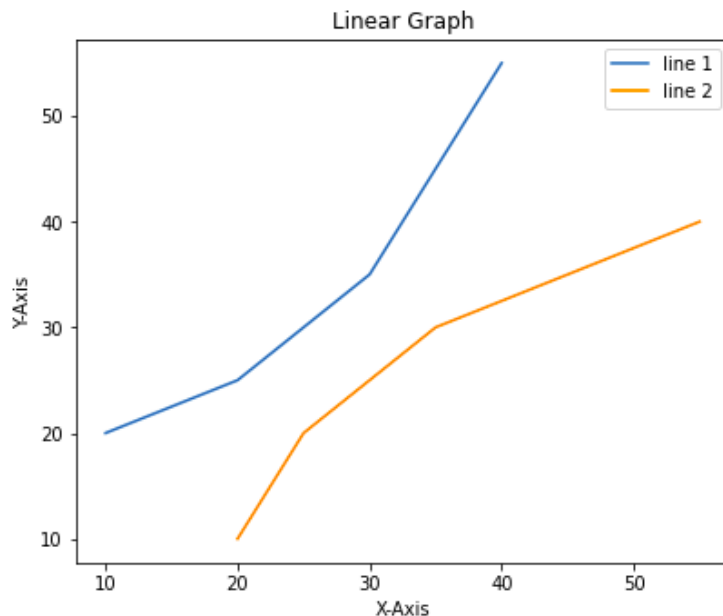
fig = plt.figure(figsize=(5, 4))
ax = fig.add_axes([0.1, 0.1, 0.8, 0.8])

ax.plot(x, y)
ax.plot(y, x)

ax.set_title("Linear Graph")
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")
ax.legend(labels=('line 1', 'line 2'))
plt.show()

```

### Output



### Advanced Techniques for Visualizing Subplots

We have learned how to add basic parts to a graph to show more information. One method can be by calling the plot function again and again with a different set of values as shown in the above example. Now let's see how to draw multiple graphs in one figure using some Matplotlib functions and how to create subplots.

#### 1. Using add\_axes()

add\_axes() method allows us to manually add axes to a figure in Matplotlib. It takes a list of four values [left, bottom, width, height] to specify the position and size of the axes.

**Example:** This code creates two side-by-side plots using add\_axes() one for x vs y and another for y vs x all in a single figure.

```

import matplotlib.pyplot as plt
from matplotlib.figure import Figure

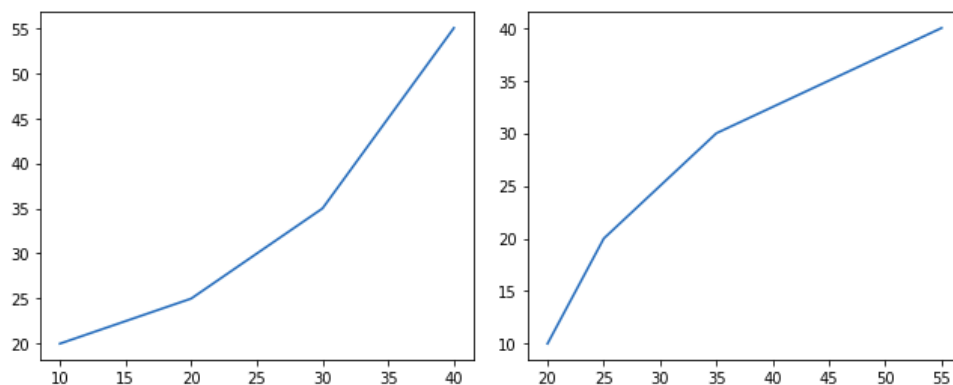
```

```
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]
```

```
fig = plt.figure(figsize=(10, 4))
ax1 = fig.add_axes([0.1, 0.1, 0.35, 0.8])
ax2 = fig.add_axes([0.55, 0.1, 0.35, 0.8])
```

```
ax1.plot(x, y)
ax2.plot(y, x)
plt.show()
```

### Output



## 2. Using subplot()

The subplot() method adds a plot to a specified grid position within the current figure. It takes three arguments: the number of rows, columns and plot index.

**Example:** This code uses subplot() to create two plots side by side one for x vs y and the other for y vs x within same figure.

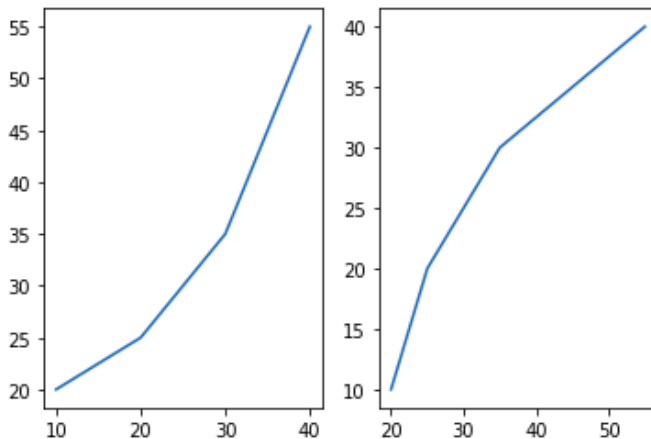
```
import matplotlib.pyplot as plt
```

```
x = [10, 20, 30, 40]
y = [20, 25, 35, 55]
plt.figure()
```

```
plt.subplot(121)
plt.plot(x, y)
```

```
plt.subplot(122)
plt.plot(y, x)
plt.show()
```

### Output



### 3. Using subplot2grid()

The subplot2grid() creates axes object at a specified location inside a grid and also helps in spanning the axes object across multiple rows or columns.

**Example:** This code uses subplot2grid() to place two stacked plots in a single figure one on top of the other by defining their positions in a 7-row grid layout.

```
import matplotlib.pyplot as plt
```

```
x = [10, 20, 30, 40]
```

```
y = [20, 25, 35, 55]
```

```
axes1 = plt.subplot2grid (
(7, 1), (0, 0), rowspan = 2, colspan = 1)
```

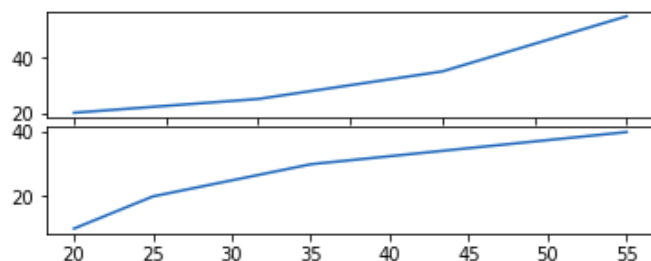
```
axes2 = plt.subplot2grid (
(7, 1), (2, 0), rowspan = 2, colspan = 1)
```

```
axes1.plot(x, y)
```

```
axes2.plot(y, x)
```

```
plt.show()
```

#### Output



### Saving Plots Using savefig()

When plots are created using Matplotlib, users may want to save them as image files for use in reports, presentations or sharing. Matplotlib offers savefig() function to save the current plot to a file. By changing file extension, users can store the plot in different formats such as .png, .jpg, .pdf or .svg.

**Example:**

```
import matplotlib.pyplot as plt
```

```
year = ['2010', '2002', '2004', '2006', '2008']
```

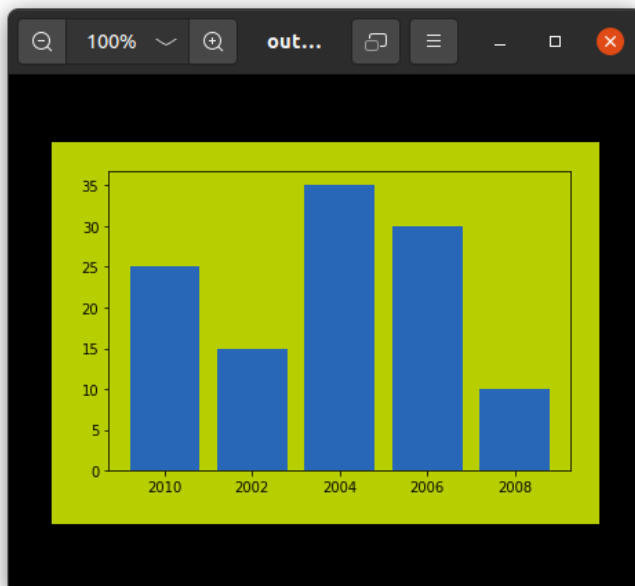
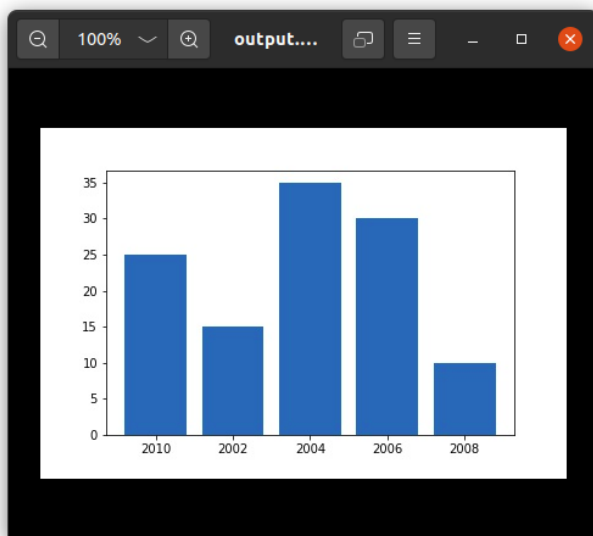
```
production = [25, 15, 35, 30, 10]
```

```
plt.bar(year, production)
```

```
plt.savefig("output.jpg")
```

```
plt.savefig("output1", facecolor='y', bbox_inches="tight", pad_inches=0.3, transparent=True)
```

**Output**



## Line chart

A line chart or line plot is a graphical representation used to show the relationship between two continuous variables by connecting data points with a straight line. It is commonly used to visualize trends, patterns or changes over time.

In Matplotlib line charts are created using the pyplot sublibrary which provides simple and flexible functions for plotting data. In a line chart, the x-axis typically represents the independent variable while the y-axis represents the dependent variable.

**Syntax:** `plt.plot(x, y, color, linestyle, linewidth, marker)`

where:

- **x:** The values for the x-axis
- **y:** The corresponding values for the y-axis
- **color:** Specifies the color of the line
- **linestyle:** Defines the style of the line
- **linewidth:** Controls the thickness of the line
- **marker:** Adds markers at data points

Consider a simple example where we visualise the weekly temperature using a line chart in Matplotlib

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']
```

```
temperature = [22, 24, 23, 26, 25]
```

```
plt.plot(days, temperature, marker='o')
```

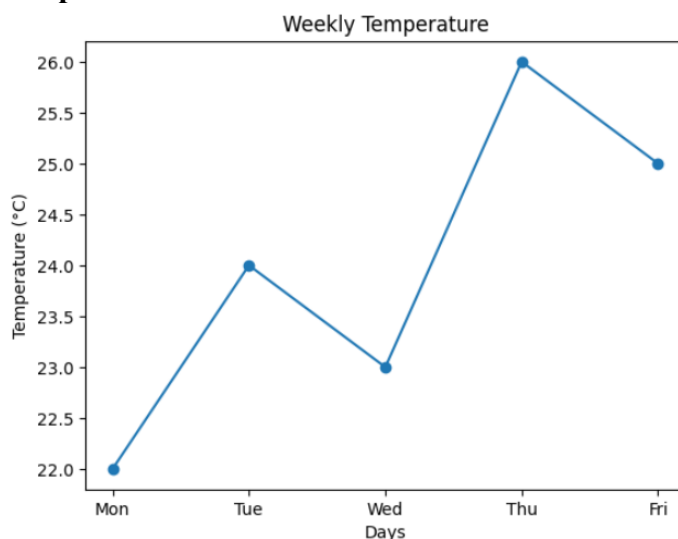
```
plt.title('Weekly Temperature')
```

```
plt.xlabel('Days')
```

```
plt.ylabel('Temperature (°C)')
```

```
plt.show()
```

**Output:**



## Line Chart

### 1. Matplotlib Simple Line Plot

A line plot is used to represent the relationship between two continuous variables. Matplotlib provides the `plot()` function through its `pyplot` module to create simple and advanced line charts efficiently.

#### Simple Line Plot

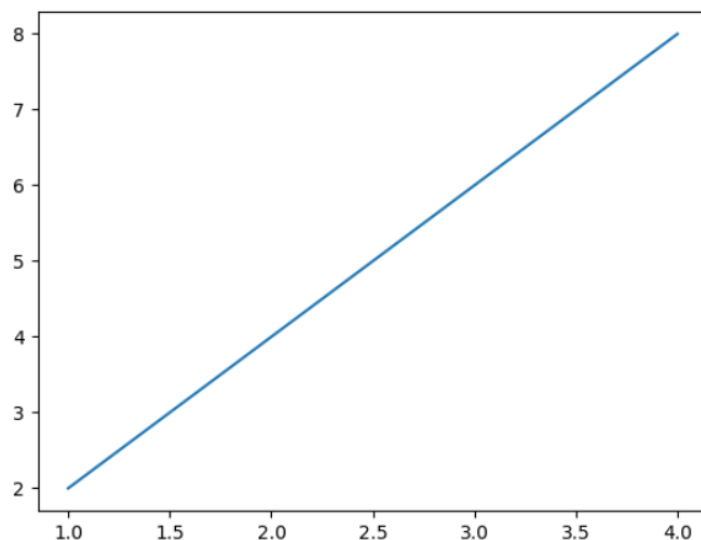
For creating a basic line chart, you can use the `plot()` function. This function draws a line by connecting data points on the x-axis and y-axis, making it easy to visualize relationships between two continuous variables.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array([1, 2, 3, 4])
y = x * 2
```

```
plt.plot(x, y)
plt.show()
```

**Output:**



Simple Line chart

#### Adding Labels and Title to a Line Plot

To improve readability, you can use the `xlabel()`, `ylabel()` and `title()` functions. These functions help in clearly identifying the axes and the purpose of the line chart.

```
import matplotlib.pyplot as plt
import numpy as np
```

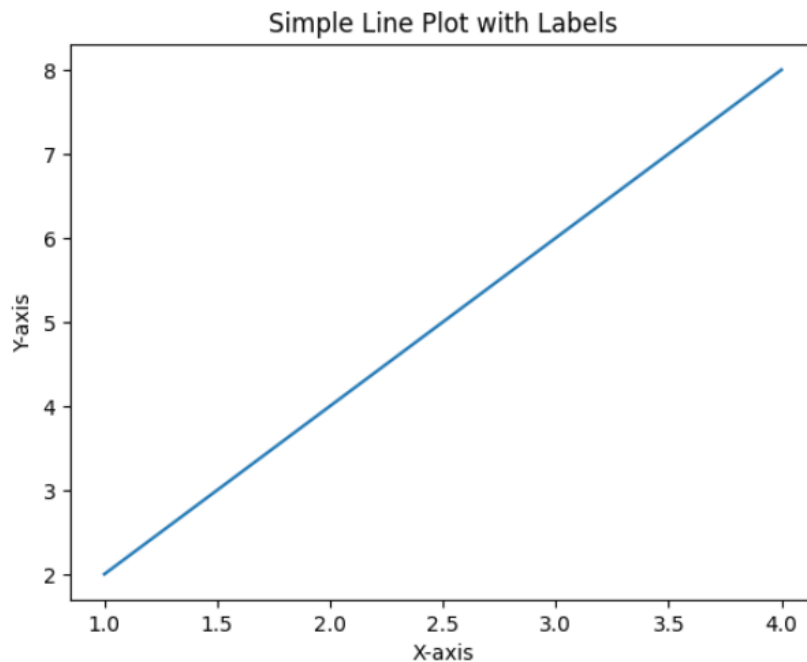
```
x = np.array([1, 2, 3, 4])
y = x * 2
```

```
plt.plot(x, y)
plt.xlabel("X-axis")
```



```
plt.ylabel("Y-axis")
plt.title("Simple Line Plot with Labels")
plt.show()
```

**Output:**



Simple line plot with axis labels and title.

### Using Markers in Line Plots

Markers help highlight individual data points on a line plot making it easier to identify exact values. They are especially useful for small datasets or precise comparisons. The marker parameter in `plt.plot()` specifies the shape of symbols used to mark each data point.

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
x = np.array([1, 2, 3, 4, 5])
```

```
y = [3, 6, 9, 12, 15]
```

```
plt.plot(x, y, marker='o', linestyle='-', label='Data Points')
```

```
plt.xlabel("X-axis")
```

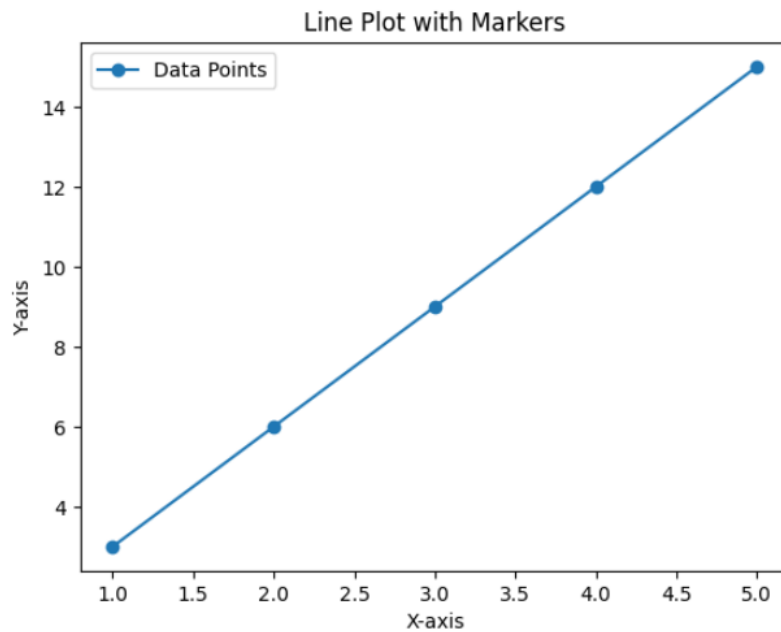
```
plt.ylabel("Y-axis")
```

```
plt.title("Line Plot with Markers")
```

```
plt.legend()
```

```
plt.show()
```

**Output:**



Markers in Line Plots

## 2. Adding Grid to a Line Chart

Grids make it easier to read values and follow trends across the chart. You can enable grids using the `grid()` function

**import matplotlib.pyplot as plt**

```
x = [1, 2, 3, 4]
```

```
y = [1, 4, 9, 16]
```

```
plt.plot(x, y)
```

```
plt.grid(True)
```

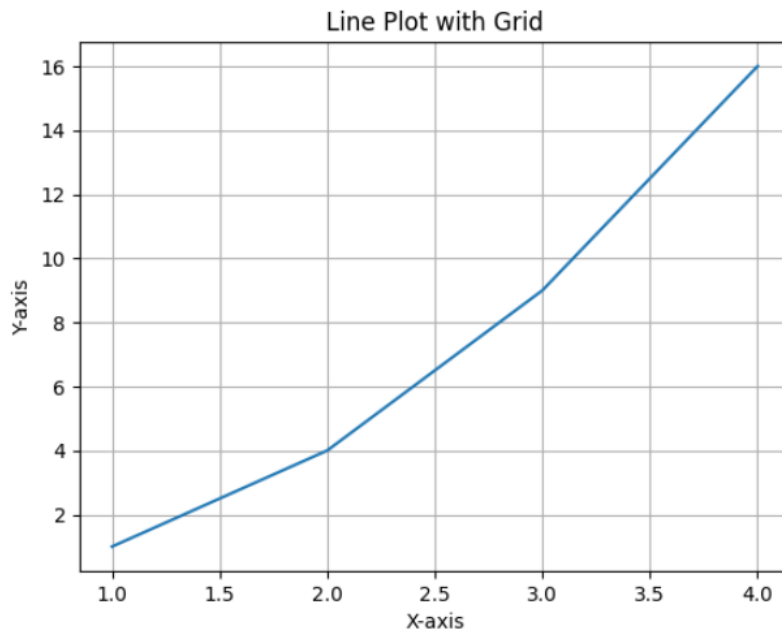
```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.title("Line Plot with Grid")
```

```
plt.show()
```

**Output:**



Grid

### 3. Line Chart with Annotations

For adding annotations to a line chart you can use the `annotate()` function. This function allows you to display additional information such as the exact x and y values directly on the data points, improving clarity and data interpretation.

**import matplotlib.pyplot as plt**

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
plt.figure(figsize=(8, 6))
```

```
plt.plot(x, y, marker='o', linestyle='-')
```

```
for xi, yi in zip(x, y):
```

```
 plt.annotate(f'({xi}, {yi})',
 (xi, yi),
 textcoords="offset points",
 xytext=(0, 10),
 ha='center')
```

```
plt.title('Line Chart with Annotations')
```

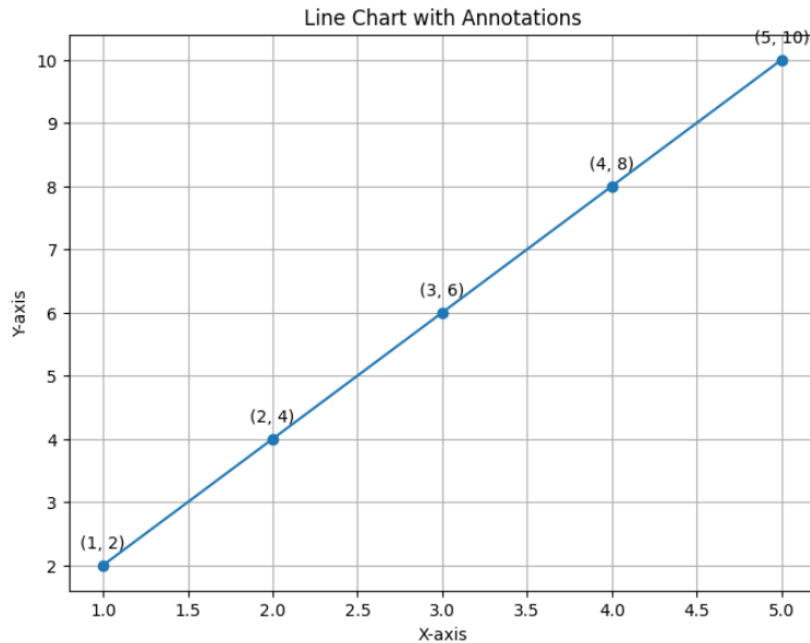
```
plt.xlabel('X-axis')
```

```
plt.ylabel('Y-axis')
```

```
plt.grid(True)
```

```
plt.show()
```

**Output:**



Line chart with annotated data points.

#### 4. Multiple Line Charts Using Matplotlib

To create multiple line charts in separate containers you can use the `figure()` function. Each call to `figure()` generates a new plotting area, making it easier to visualize and compare different datasets independently.

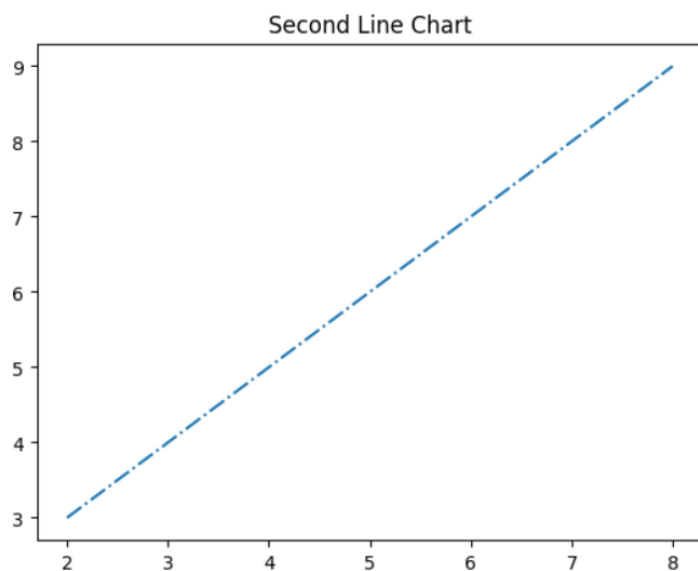
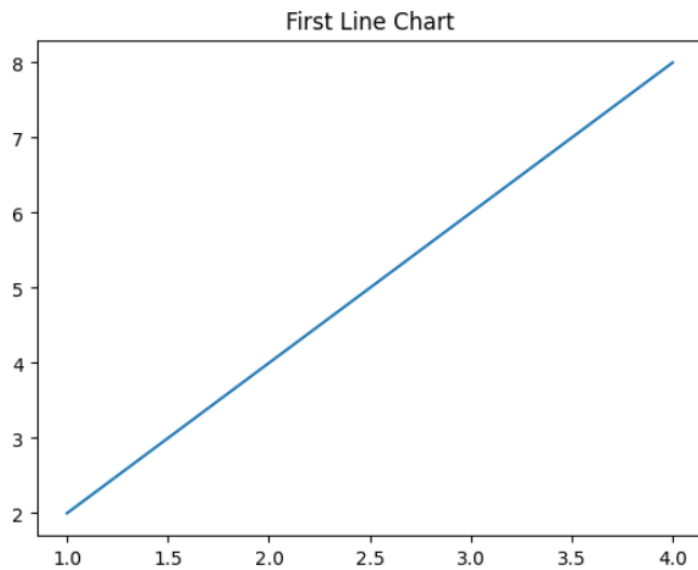
```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array([1, 2, 3, 4])
y = x * 2
```

```
plt.plot(x, y)
plt.title("First Line Chart")
plt.show()
```

```
plt.figure()
x1 = [2, 4, 6, 8]
y1 = [3, 5, 7, 9]
plt.plot(x1, y1, '-.')
plt.title("Second Line Chart")
plt.show()
```

**Output:**



Multiple Line Charts

### 5. Multiple Plots on the Same Axis

To plot multiple lines on the same axis, you can call the `plot()` function multiple times before displaying the graph. This approach is useful for comparing different datasets within the same coordinate system.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array([1, 2, 3, 4])
y = x * 2
```

```
x1 = [2, 4, 6, 8]
y1 = [3, 5, 7, 9]
```

```
plt.plot(x, y, label='y = 2x')
```

```
plt.plot(x1, y1, '-.', label='Second Line')
```

```
plt.xlabel("X-axis data")
```

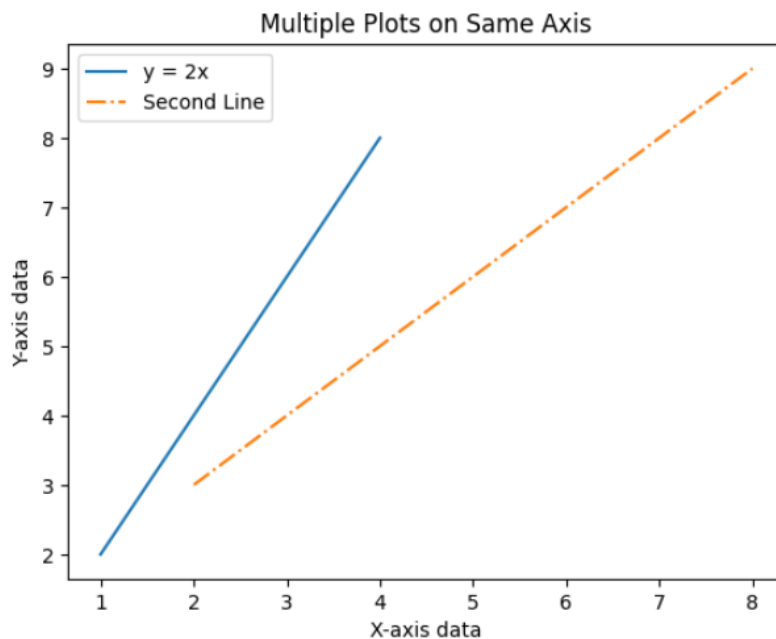
```
plt.ylabel("Y-axis data")
```

```
plt.title("Multiple Plots on Same Axis")
```

```
plt.legend()
```

```
plt.show()
```

**Output:**



Multiple Plots on the Same Axis

## 6. Fill the Area Between Two Lines

To fill the region between two line plots, you can use the `fill_between()` function. This function shades the area between two curves, helping visualize the difference or range between datasets.

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
x = np.array([1, 2, 3, 4])
```

```
y = x * 2
```

```
y1 = [3, 5, 7, 9]
```

```
plt.plot(x, y, label='y = 2x')
```

```
plt.plot(x, y1, '-.', label='y1')
```

```
plt.fill_between(x, y, y1, color='green', alpha=0.4)
```

```
plt.xlabel("X-axis data")
```

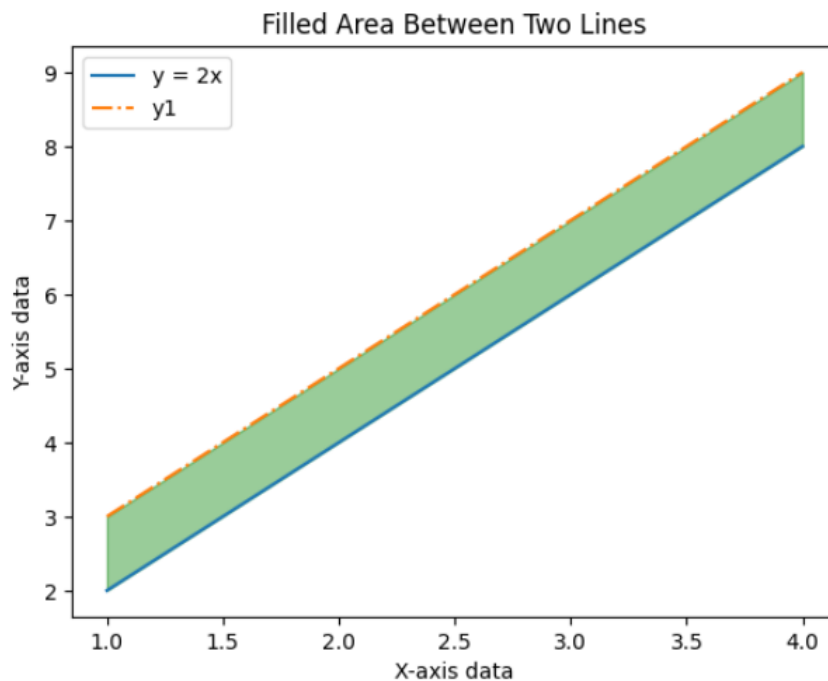
```
plt.ylabel("Y-axis data")
```

```
plt.title("Filled Area Between Two Lines")
```

```
plt.legend()
```

```
plt.show()
```

**Output:**



Filled Area Between Two Lines

## 7. Saving a Line Chart to a File

Instead of displaying a plot you can save it as an image using the `savefig()` function.

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4]
```

```
y = [2, 4, 6, 8]
```

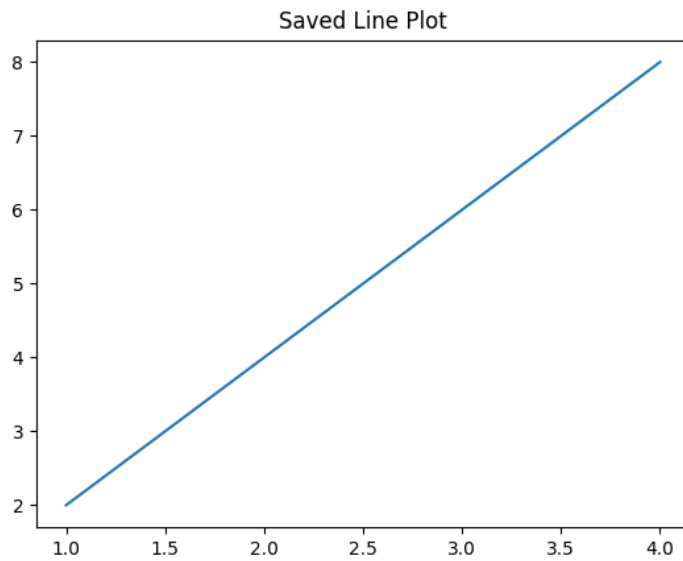
```
plt.plot(x, y)
```

```
plt.title("Saved Line Plot")
```

```
plt.savefig("line_plot.png")
```

```
plt.show()
```

**Output:**



Saved Line Chart

### 8. Plotting Trigonometric Functions

Trigonometric functions such as sine, cosine and tangent can be visualized using `plt.plot()` by generating input values over a continuous range. The `numpy.linspace()` function creates evenly spaced values, allowing smooth and accurate curves.

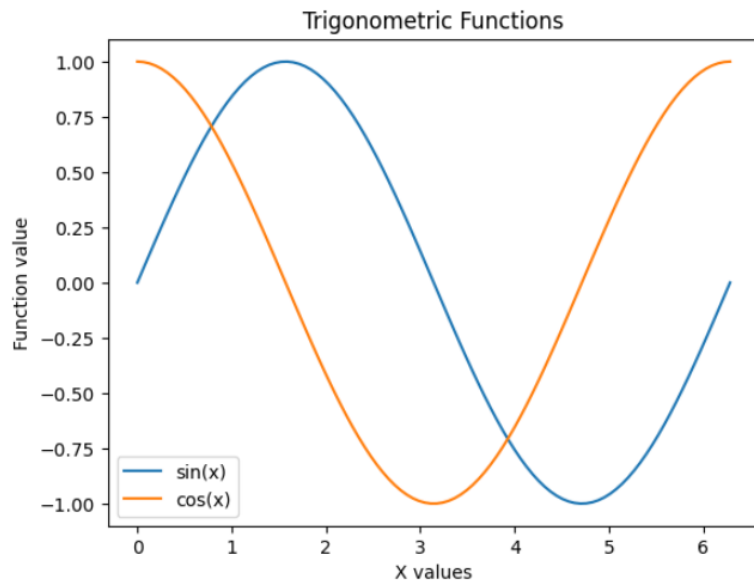
```
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(0, 2 * np.pi, 100)
y_sin = np.sin(x)
y_cos = np.cos(x)
```

```
plt.plot(x, y_sin, label="sin(x)")
plt.plot(x, y_cos, label="cos(x)")
plt.xlabel("X values")
plt.ylabel("Function value")
plt.title("Trigonometric Functions")
plt.legend()
plt.show()
```

**Output:**

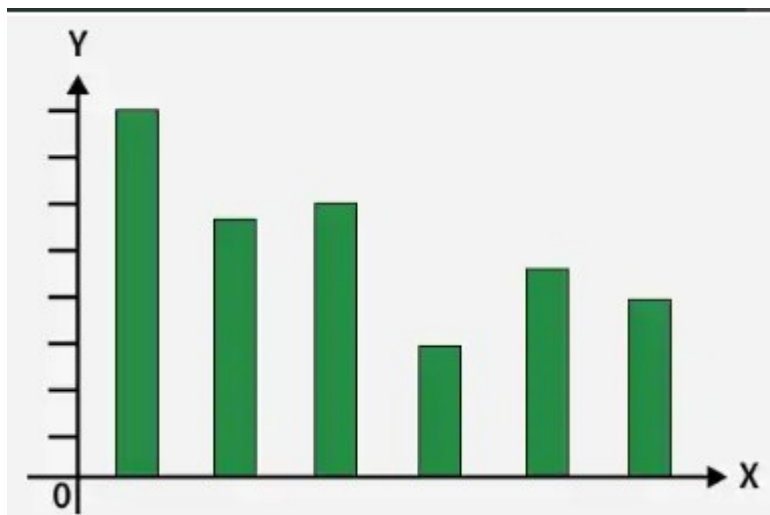




Sine and cosine functions

### Bar chart

A bar graph is a pictorial representation of data using rectangular bars of equal width, where the length or height of each bar shows the value or frequency of the data. It helps in comparing different categories easily and clearly. The bars may be drawn vertically or horizontally, making data simple to understand and interpret.



### Properties of a Bar Graph

- All the bars start from a common base line.
- Each bar has the same width.
- The height or length of each bar shows the value of the data.
- The distance between every two bars is equal.
- Bars can be drawn either vertically or horizontally.

### Types of Bar Graphs or Bar Diagrams

The different types of Bar Graphs are as follows:

- Simple Bar Graph
- Multiple Bar Graph
- Sub-Divided Bar Graph or Component Bar Graph
- Percentage Bar Graph
- Broken-Scale Bar Graph
- Deviation Bar Graph

### 1. Simple Bar Graph

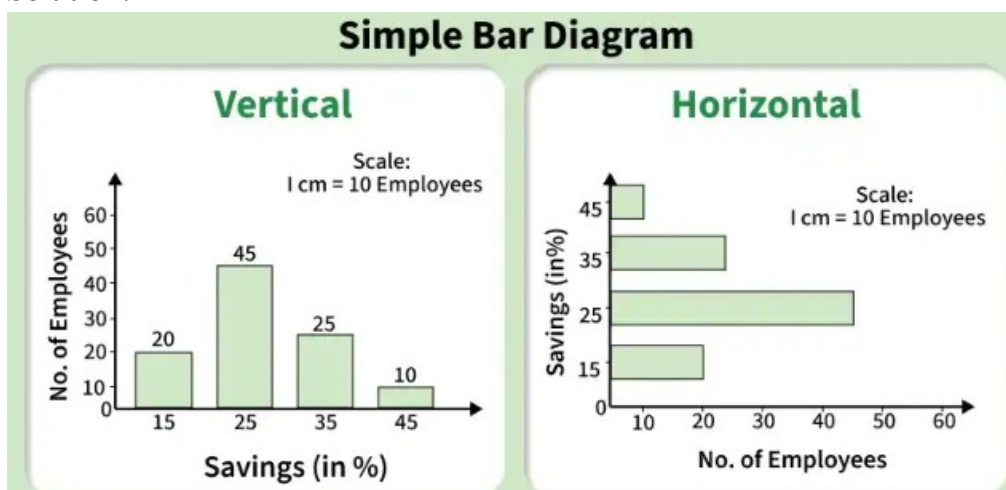
A simple bar graph is a graphical representation of data in which only one variable is represented by rectangular bars of equal width, and the height or length of each bar is proportional to the value of that variable.

- **Vertical Bar Graph:** The diagram in which the magnitude of the data is presented vertically, i.e., along the Y-axis, is a Vertical Bar Diagram.
- **Horizontal Bar Graph:** The diagram in which the magnitude of the data is presented horizontally; i.e., along the X-axis is a Horizontal Bar Diagram

The following table shows the percentage of monthly salary saved by each employee in a 100-person company. Create a vertical and a horizontal bar diagram to represent it.

| Savings (in percentage) | Number of Employees |
|-------------------------|---------------------|
| 15                      | 20                  |
| 25                      | 45                  |
| 35                      | 25                  |
| 45                      | 10                  |
| <b>Total</b>            | <b>100</b>          |

**Solution:**



Simple Bar - vertical and horizontal

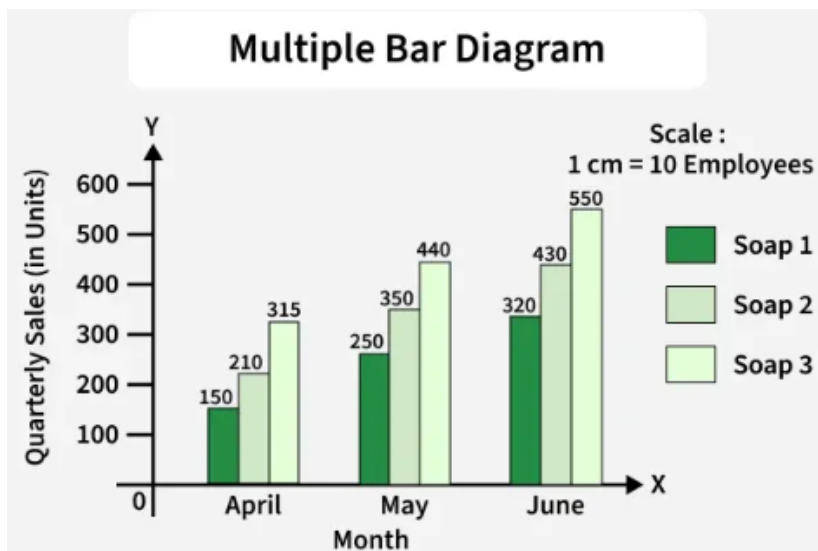
### 2. Multiple Bar Graph

A multiple bar diagram is used to compare two or more related variables at the same time, such as income and expenditure or marks in different subjects over the years. It is also called a compound bar diagram. It is drawn in the same way as a simple bar diagram, but different colours or shades are used to clearly identify and compare each variable.

A company manufactures three varieties of soap. Represent the following information showing the quarterly sales (three months) using a multiple-bar diagram.

| Month | Quarterly Sales (in units) |        |        |
|-------|----------------------------|--------|--------|
|       | Soap 1                     | Soap 2 | Soap 3 |
| April | 150                        | 210    | 315    |
| May   | 250                        | 350    | 440    |
| June  | 320                        | 430    | 550    |

**Solution:**



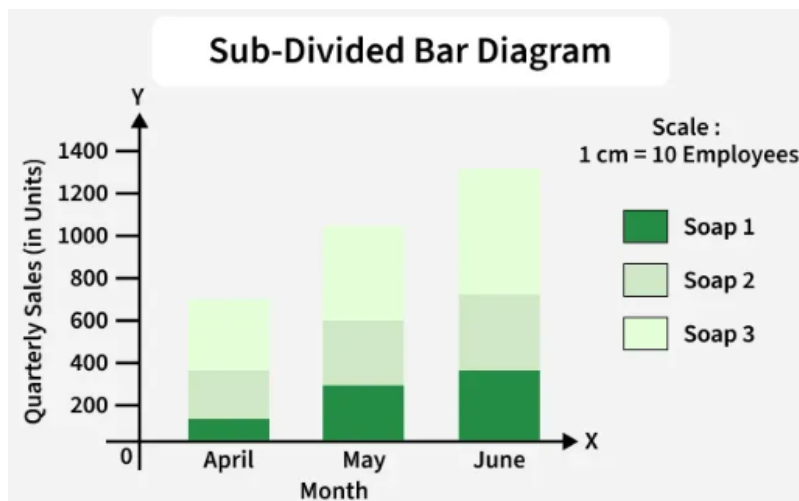
### 3. Sub-Divided Bar Graph

A sub-divided bar graph is used to show how a whole is divided into different parts. In this diagram, each bar represents the total value and is further divided into smaller sections according to the share of each component. Different colours or shades are used to represent each part, with a key to explain them. This type of graph helps students easily understand and compare the parts of a whole, such as boys and girls in a class or different types of expenses in a family.

Represent the following information using a sub-divided bar diagram, showing the quarterly sales of three varieties of soap manufactured by a company.

| Month | Quarterly Sales (in units) |        |        |       |
|-------|----------------------------|--------|--------|-------|
|       | Soap 1                     | Soap 2 | Soap 3 | Total |
| April | 150                        | 210    | 315    | 675   |
| May   | 250                        | 350    | 440    | 1,040 |
| June  | 320                        | 430    | 550    | 1,300 |

**Solution:**



#### 4. Percentage Bar Graph

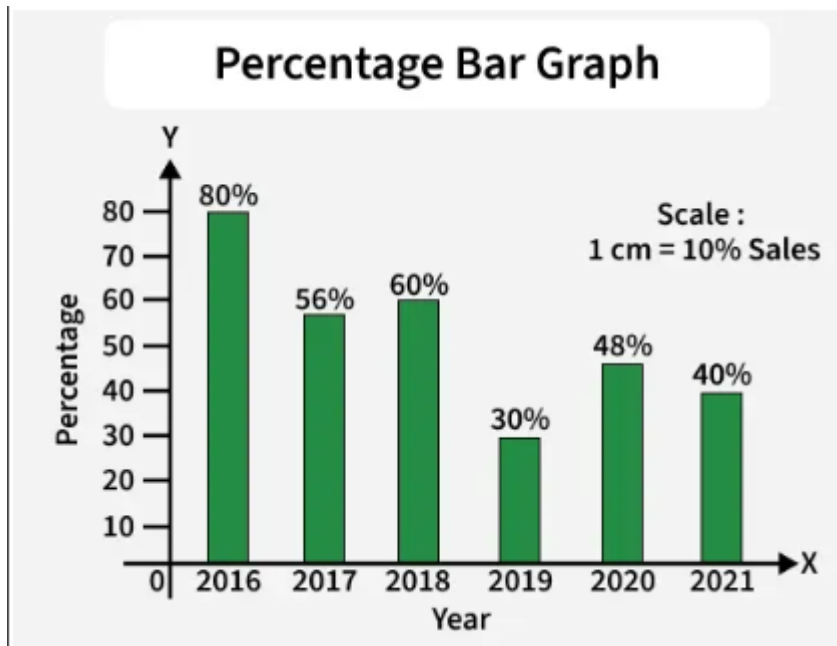
A percentage bar graph is used to represent data in the form of percentages. In this graph, each bar represents a total of 100% and is divided into parts according to the percentage of each component. It helps in comparing the relative contribution of different parts to the whole and can be drawn vertically or horizontally using different colours or shades for clarity.

The table below displays the number of newspapers sold out by the company, out of the total printed newspaper from 2016 to 2021.

| Year | Sale of Newspaper (out of 150) |
|------|--------------------------------|
| 2016 | 120                            |
| 2017 | 80                             |
| 2018 | 90                             |
| 2019 | 45                             |
| 2020 | 72                             |
| 2021 | 60                             |

**Solution:**

| Year | Sale of Newspaper (out of 150) | Percentage of Newspaper sold |
|------|--------------------------------|------------------------------|
| 2016 | 120                            | $\frac{120}{150} * 100 = 80$ |
| 2017 | 84                             | $\frac{84}{150} * 100 = 56$  |
| 2018 | 90                             | $\frac{90}{150} * 100 = 60$  |
| 2019 | 45                             | $\frac{45}{150} * 100 = 30$  |
| 2020 | 72                             | $\frac{72}{150} * 100 = 48$  |
| 2021 | 60                             | $\frac{60}{150} * 100 = 40$  |



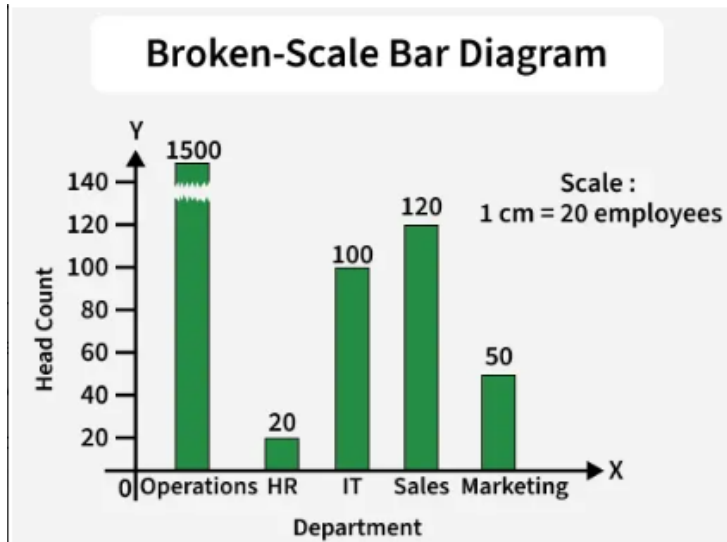
### 5. Broken-Scale Bar Graph

A broken scale graph is a graph in which the scale on an axis is intentionally broken to show both very large and small values clearly. It is used when data values vary widely so that large values are compressed and the graph is easier to read.

Prepare an appropriate diagram using the following data.

| Department | Headcount |
|------------|-----------|
| Operations | 1500      |
| HR         | 20        |
| IT         | 100       |
| Sales      | 120       |
| Marketing  | 50        |

**Solution:**

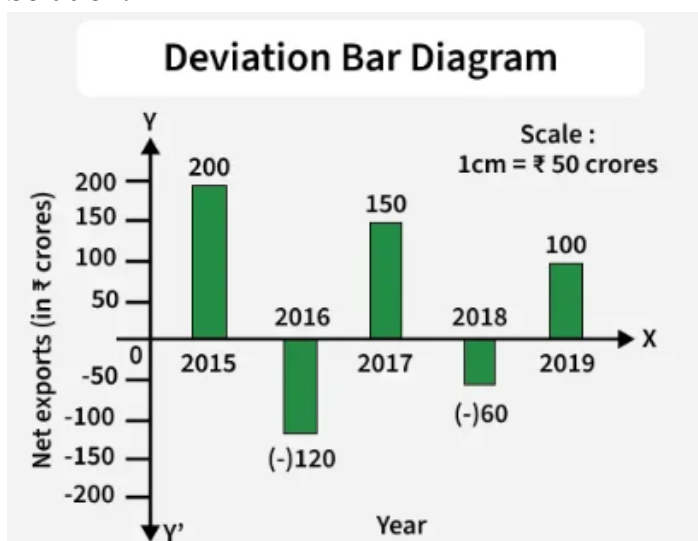


## 6. Deviation Bar Graph

A deviation bar diagram is a bar graph in which bars are drawn on both sides of a baseline (usually zero) to show positive and negative values. Bars above the baseline represent positive deviations, while bars below represent negative deviations from a standard or average value. The following table shows the data relating to the net exports of a company in different years. Use a deviation bar diagram to represent the profit/loss made by the firm.

| Year | Net Exports (in ₹ crores) |
|------|---------------------------|
| 2015 | 200                       |
| 2016 | (-)120                    |
| 2017 | 150                       |
| 2018 | (-) 60                    |
| 2019 | 100                       |

**Solution:**



### Steps to Draw a Bar Graph

- **Collecting Data:** Gather the data to be represented. Make sure the data is categorical and each category has a value.
- **Choosing the Type of Bar Graph:** Select the suitable bar graph according to the data and comparison needed.
- **Drawing the Axes:** Draw the x-axis (horizontal) and y-axis (vertical) and label them properly.
- **Plotting the Bars:** Draw bars of equal width and equal spacing. The height or length of each bar should represent its value.
- **Adding Labels and Title:** Label the categories and values clearly and give a suitable title to the bar graph.

### Key Points of Bar Graph

- Bars should have equal width and equal spacing between them.
- Always read the x-axis and y-axis carefully to understand what the data represents.
- The height or length of each bar shows the value or frequency of the data.
- A proper scale should be chosen so that the data is shown clearly.
- Bar graphs are useful for large data because they are easier and quicker to draw than pictographs.

## Histogram

A Bar graph or a Histogram is a tool used for visual representation of data. Representing the data in a bar graphs or histograms, makes it easy to understand the concepts and relationships among data.

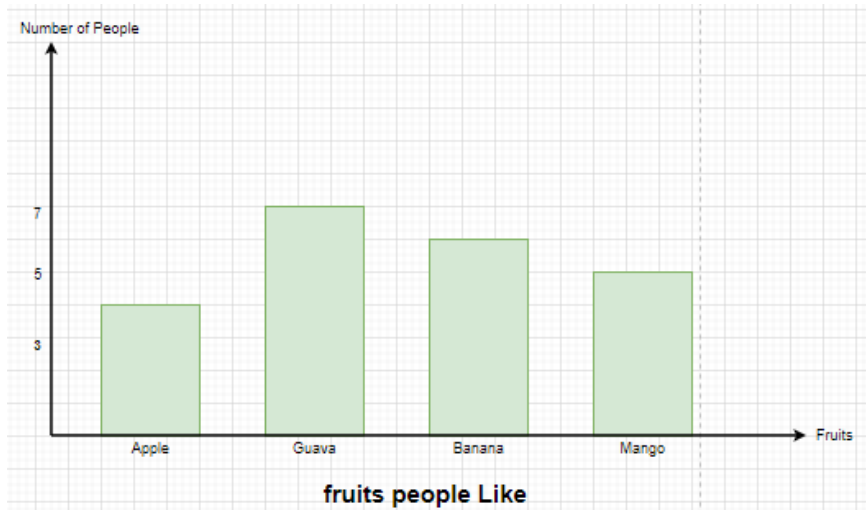
A Histogram is used to display the distribution of continuous data by grouping values into intervals, or bins. Whereas, a Bar graph is used to compare discrete categories, with rectangular bars representing the frequency or value of each category.

**The pictorial representation of data in groups, either in horizontal or vertical bars where the length of the bar represents the value of the data present on the axis.** They (bar graphs) are usually used to display or impart the information belonging to 'categorical data' i.e., data that fit in some category.

### Reading a Bar Graph and comparing two sets of data

To read a Bar graph, we need to ask questions to ourselves looking at the displayed graph. Let's understand reading a Bar graph through a fundamental example,

Different types of fruits are liked by People,



**What does the X-axis and Y-axis on the graph are representing?**

The X-axis represents the different types of fruits like apple, guava. while Y-axis represents the Number of people.

**What is the Common base for the Bars?**

The bars are showing a common base of category of fruits.

**What is the scale used on the Y-axis?**

The scale used is normal, i.e; 1 Unit = 1 person

**Overall, what kind of information the bar graph displaying?**

The bar graph is displaying the number of People liking different types of fruits.

**Looking at the bar Graph, can one answer, how many people like Mango?**

Yes, By observing the length of the bar, one can tell that there are 5 people who like Mango.

**Comparing two sets of data**

To compare two sets of data we simply need to compare the heights of the bars that represent that specific data. Also to get the exact value represented by each bar we can look at the value on the y axis corresponding to that height. The bar with more height represents more value and the bar with less height represents less value.

**It can be concluded the following just by looking at the graph:**

- 4 People like Apples.
- 7 People like guava.
- 6 People like banana.
- 5 People like Mango.

**Properties of Bar Graph**

- All Bars have a common base.
- The length of each bar corresponds to its respective data mentioned on the axis (**Y-axis for Vertical Graph, X-axis for Horizontal Graph**).
- Each bar displayed has the same width.
- The distance between consecutive bars is the same

**Histogram**



Histograms are the tools used for graphical representation of data with the help of bars, the heights of the bars may vary due to different data. The histogram is similar to a bar graph, but histogram groups numbers into different ranges. The length of each bar tells how many fall into each range, and it can be decided which ranges are to be used.

It organizes the numbers into groups, or "bins," and shows how many numbers fall into each group with bars. This makes it easy to see patterns, like which numbers appear most often or how the data is spread out.

### Construction and interpretation of Histogram

- First, mark the class intervals on X-axis and frequencies on Y-axis.
- Make sure that the scale of both axes is the same.
- The class Intervals shall always be exclusive.
- Create bars with class intervals on the x-axis and corresponding frequencies on the y-axis.
- The length of each bar reflects the Frequency when intervals are equal.
- The area of each bar is the same as its respective frequency when intervals are unequal.

### Difference Between Bar Graphs and Histograms

| Feature                            | Bar Graph                            | Histogram                    |
|------------------------------------|--------------------------------------|------------------------------|
| <b>Dimensionality</b>              | One dimension                        | Two dimensions               |
| <b>Representation of frequency</b> | Length of the bars                   | Area of the bar              |
| <b>Significance of Bar Width</b>   | No special significance              | Represents interval or bin   |
| <b>Spacing between bars</b>        | Bars are separated with equal spaces | Bars are touching each other |

### Solved Examples on Bar Graphs and Histograms

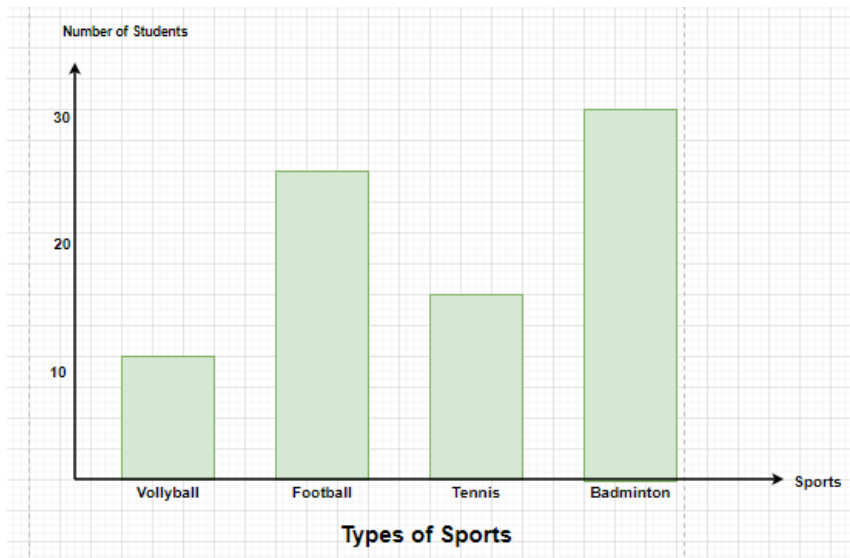
#### Bar Graph Questions:

**Question 1:** Draw the Bar Graph for the following table,

| Sports    | Number of Students |
|-----------|--------------------|
| Vollyball | 10                 |
| Football  | 25                 |
| Tennis    | 15                 |
| Badminton | 30                 |

#### Solution:

The Bar Graph for the following table is,

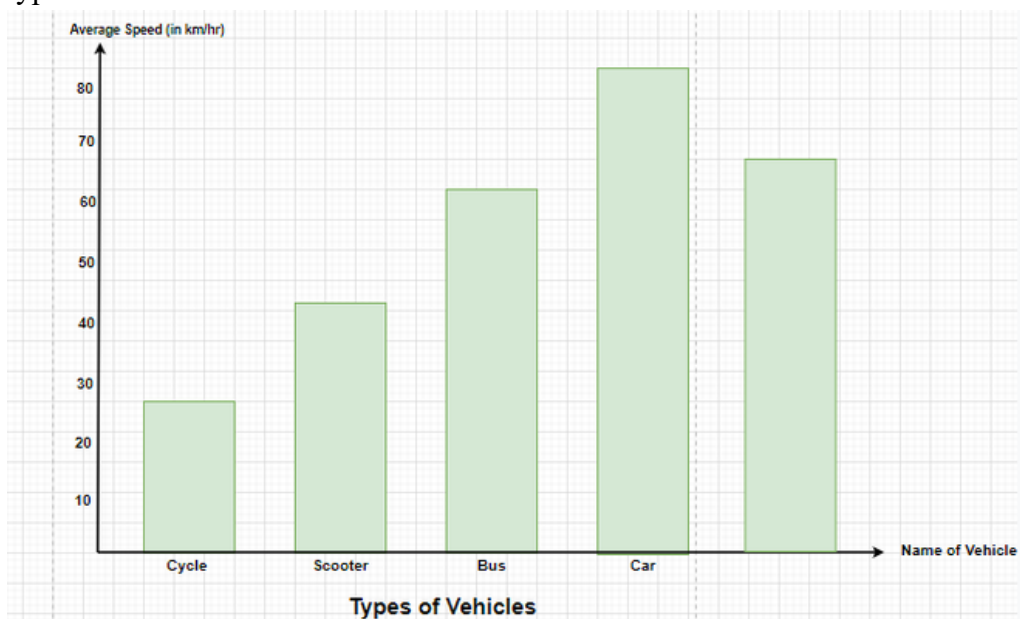


**Question 2:** The average speed of some vehicles is shown below, Represent it through a bar graph.

| Name of Vehicles         | Cycle | Scooter | Bus | Car | Train |
|--------------------------|-------|---------|-----|-----|-------|
| Average Speed (In km/hr) | 25    | 42      | 60  | 80  | 65    |

**Solution:**

The average speed mentioned in the table is the frequency that decides the length of each bar graph, therefore, if the graph is vertical, the Average speed will be shown on the y-axis and the types of vehicles will be shown on the x-axis.



**Histogram Questions:**

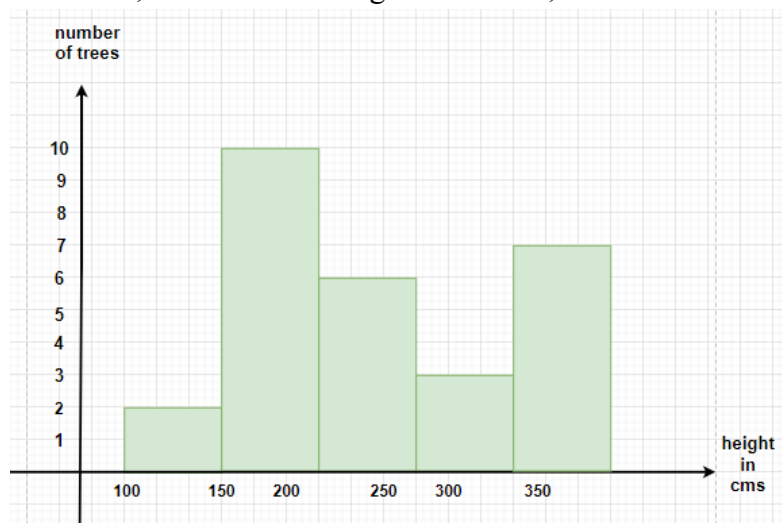
**Question 1:** In a Park, there are 28 trees of different heights, the heights can be measured in centimeters and the range of the trees lie between 100-350 cm. Draw the Histogram for the following data,

| Range of Height of Tree (in cms) | 100-150 | 150-200 | 200-250 | 250-300 | 300-350 |
|----------------------------------|---------|---------|---------|---------|---------|
| Number of Trees                  | 2       | 10      | 6       | 3       | 7       |

**Solution:**

Since the height of the trees are lying between 100-350, we shall start by marking the heights on x-axis in groups of 50cm each and the number of trees will be mentioned on y-axis.

Therefore, if a tree has a height of 230 cm, it will lie in the rectangle 200-250.

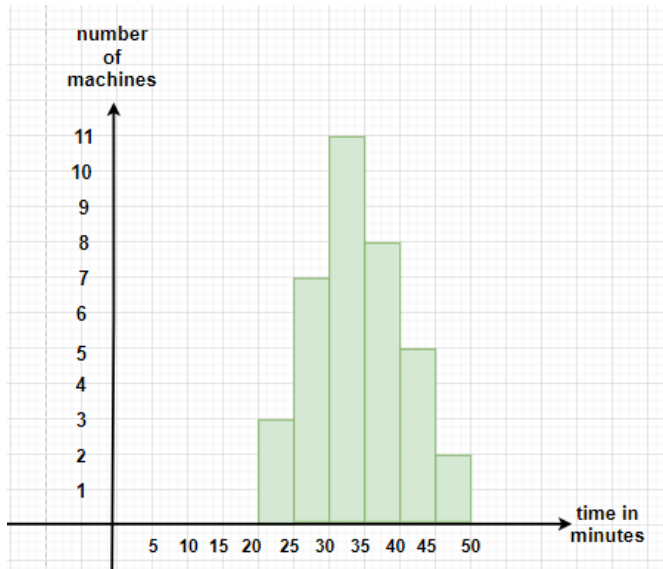


**Question 2:** In the below-given table, the number of machines working in a factory for their fixed amount of time is shown. Draw the Histogram for the following data.

| Time in Minutes | Number of machines |
|-----------------|--------------------|
| 20-25           | 03                 |
| 25-30           | 07                 |
| 30-35           | 11                 |
| 35-40           | 08                 |
| 45-50           | 05                 |
| 50-55           | 02                 |

**Solution:**

The Histogram for the table mentioned above will have class intervals of 5 minutes on x-axis and the number of machines used should be on y-axis



### Pie chart

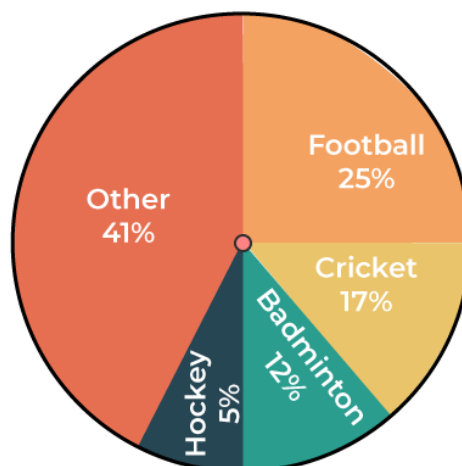
A pie chart is a popular and visually intuitive tool used in data representation, making complex information easier to understand at a glance.

This circular graph divides data into slices, each representing a proportion of the whole, allowing for a clear comparison of different categories, making it easier to digest complex information through a straightforward, intuitive format.

### Pie Chart Examples

In a class of 200 students, a survey was conducted to collect each student's favorite sport. The pie chart of the data is given below:

### Number of Students



Since the pie chart is provided and the total number of students is given, we can easily take the original data out for each sport.

- **Cricket** =  $17/100 \times 200 = 34$  students
- **Football** =  $25/100 \times 200 = 50$  students
- **Badminton** =  $12/100 \times 200 = 24$  students
- **Hockey** =  $5/100 \times 200 = 10$  students
- **Other** =  $41/100 \times 200 = 82$  students

The original data for the pie chart shown above is given below:

| Sport     | Number of Students |
|-----------|--------------------|
| Cricket   | 34                 |
| Football  | 50                 |
| Badminton | 24                 |
| Hockey    | 10                 |
| Other     | 82                 |

### Pie Chart Formula

The total value or percentage of the pie is always 100%. Here, it contains different sectors and segments in which each sector or segment of the chart corresponds to a certain portion of the net or total percentage (or data). The total or sum of all the data can be summed up to 360 degrees.

- Convert the data into degrees on a pie chart. The formula for a pie chart can be summed up as:

**(Given Data / Total Value of Data)  $\times$  360°**

- Calculate the percentage of each sector from the degrees in a pie chart.

**To work out with degrees in a pie chart, we need to follow the following steps:**

- First, we need to measure every slice of the chart.
- Then, we need to divide it by 360°.
- Finally, multiply the obtained result by 100.

**The pie chart formula is given below:**

$(\text{Frequency})/(\text{Total Frequency}) \times 100$

### Calculating the Number of Sectors on a Pie Chart

To calculate the total number of slices or sectors on a pie chart, we need to multiply the sector's percentage by the total value of the data and, finally, divide the result by 100.

### Types of Pie Charts

There are various variations or types of pie charts. Some Common Types of Pie Chart Include-

- **3D Pie Chart:** A 3D pie chart adds depth to the traditional two-dimensional pie chart by rendering it in three dimensions.
- **Doughnut Chart:** A doughnut chart is similar to a pie chart but with a hole in the center.
- **Exploded Pie Chart:** In an exploded pie chart, one or more slices are separated from the rest of the pie to emphasize their importance or to make them stand out.

- **Nested Pie Chart:** Also known as a multi-level pie chart or hierarchical pie chart, this type of chart consists of multiple rings of pie charts, with each ring representing a different level of data hierarchy.
- **Ring Chart:** Similar to a doughnut chart, but with multiple concentric rings representing different data categories. Each ring illustrates a separate dataset, allowing for layered comparison.

### How to Create a Pie Chart?

We will learn how to create a pie chart step by step with the help of an example. A teacher surveyed a group of students to see what their favorite hobby of each student is. Let's take a look at the pie chart example with an explanation. The data collected is listed as follows:

| Hobbies       | Number of students |
|---------------|--------------------|
| Singing       | 16                 |
| Reading books | 20                 |
| Dancing       | 10                 |
| Painting      | 30                 |
| Others        | 24                 |

Now, we will see how to construct a pie chart step by step.

**Step 1:** The first step requires us to write down the available data in tabular form as follows:

| Singing | Reading Books | Dancing | Painting | Others |
|---------|---------------|---------|----------|--------|
| 16      | 20            | 10      | 30       | 24     |

**Step 2:** Now, the sum of all the given data. Here, the Sum of All Data =  $(16 + 20 + 10 + 30 + 24) = 100$

**Step 3:** Now, calculate the percentage of each sector. We need to divide each sector value by the sum or total and then multiply it by 100.

| Singing                        | Reading Books                  | Dancing                        | Painting                       | Others                         |
|--------------------------------|--------------------------------|--------------------------------|--------------------------------|--------------------------------|
| $(16/100) \times 100$<br>= 16% | $(20/100) \times 100$<br>= 20% | $(10/100) \times 100$<br>= 10% | $(30/100) \times 100$<br>= 30% | $(24/100) \times 100$<br>= 24% |

**Step 4:** The Next step is to calculate the degrees corresponding to each slice of the Pie Chart. The values can be calculated as:

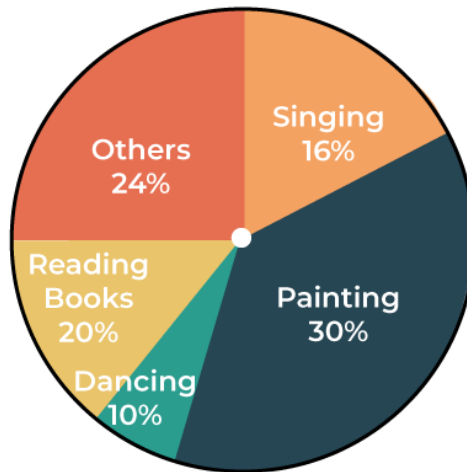
**Central Angle of Each Component = (Given Data / Total Value of Data)  $\times$  360**

Hence, the values are as follows:

| Singing                         | Reading Books                 | Dancing                       | Painting                       | Others                          |
|---------------------------------|-------------------------------|-------------------------------|--------------------------------|---------------------------------|
| $(16/100) \times 360$<br>= 57.6 | $(20/100) \times 360$<br>= 72 | $(10/100) \times 360$<br>= 36 | $(30/100) \times 360$<br>= 108 | $(24/100) \times 360$<br>= 86.4 |

**Step 5:** Now, with the help of a protractor, we will measure each angle from a single point or central point and draw the circle's sectors. The resultant pie chart will be:

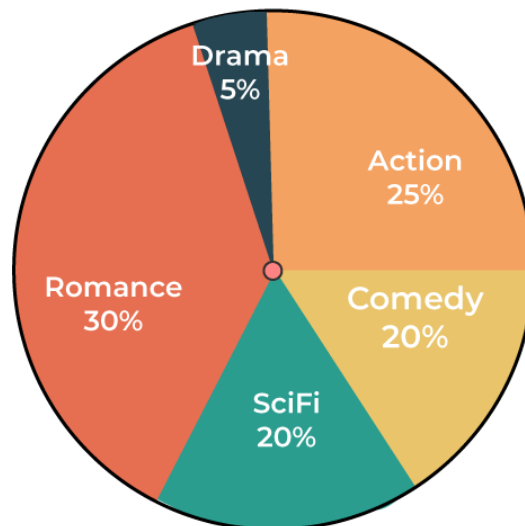
## Number of Students



### How to Read a Pie Chart

In order to read a pie chart, the first thing to notice is the data presented in the pie chart. If the data is given in percentages, it should be converted accordingly in order to analyze and interpret the data. Let's take a look at an example to learn how to interpret pie charts.

**Example:** In a survey done among 300 people, it was observed which type of genre each person prefers. The pie chart of the same is mentioned below. Analyze and interpret the pie chart accordingly to find the original data.



### Solution:

While observing the pie chart, it came to notice that the data is present in percentage. Let's convert the data to obtain the original value.

- Number of people who like comedy =  $20/100 \times 300 = 60$  people.
- Number of people who like action =  $25/100 \times 300 = 75$  people.
- Number of people who like romance =  $30/100 \times 300 = 90$  people.
- Number of people who like drama =  $5/100 \times 300 = 15$  people.
- Number of people who like sci-fi =  $20/100 \times 300 = 60$  people.

## Pie Chart vs Bar Graph

The key differences between a pie chart and a bar graph are listed in the following table:

| Pie Chart                                       | Bar Graph                                      |
|-------------------------------------------------|------------------------------------------------|
| Circular display of data                        | rectangular display of data                    |
| Shows parts of a whole                          | Compares discrete categories                   |
| Depicts percentages or proportions              | Shows exact values or quantities.              |
| Typically one variable                          | Can represent multiple variables.              |
| Easily shows relative proportions               | Effective for comparing quantities.            |
| It might be difficult to compare precise values | Allows for easy comparison between categories. |
| Works well with simple datasets                 | Suitable for complex datasets                  |
| Provides a holistic view                        | Allows for detailed analysis.                  |
| Not efficient with large datasets               | Efficient for displaying large datasets        |

### Also Check:

- Pie Chart in MATLAB
- Bar Graph
- Measurement of Angles
- Bar Graph and Histogram

### Uses of Pie Chart

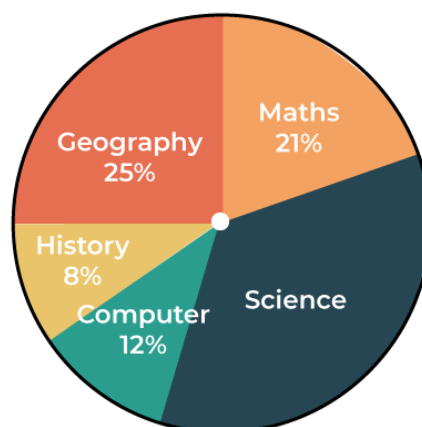
Whenever a fraction or fractions are represented as a part of the whole, pie charts are used. Pie charts are used to compare the data and to analyze which data is bigger or smaller. Hence, when dealing with discrete data, pie charts are preferred. Let's take a look at the uses of the pie chart:

- Pie charts are used to compare the profit and loss in businesses.
- In schools, the grades can be easily compared using a pie chart.
- The relative sizes of data can be compared using a pie chart.
- The marketing and sales data can be compared using a pie chart.

### Solved Examples of Pie Chart

**Example 1:** The given pie chart shows the subject of interest of each student in a class.

### Number of Students





Answer the following question concerning the given pie diagram.

1. If 30 students' subjects of interest are history, how many total students were surveyed?
2. Which subject is liked the most?
3. Which subject is disliked the most?

**Solution:**

1. According to the given question, 8% of the total number of students is 30. i.e.  $(8/100) \times$   
Total = 30

Therefore, Total =  $30 \times (100/8) = 375$

Hence 375 students were surveyed.

2. According to the given pie chart, science is liked the most.

3. According to the given pie chart, history is disliked the most.

**Example 2:** For a science camp, students from different states have enrolled. Construct a pie chart for the given table:

| States      | Number of students |
|-------------|--------------------|
| West Bengal | 10                 |
| Assam       | 5                  |
| Tamil Nadu  | 5                  |
| Gujarat     | 10                 |
| Karnataka   | 10                 |

**Solution:**

**Step 1:** The first step requires us to jot down the available data into tabular form as follows:

| West Bengal | Assam | Tamil Nadu | Gujarat | Karnataka |
|-------------|-------|------------|---------|-----------|
| 10          | 5     | 5          | 10      | 10        |

**Step 2:** The next task is to calculate the sum of all the given data.

Here, the Sum of All Data =  $(10 + 5 + 5 + 10 + 10) = 40$

**Step 3:** Now, the next task is to calculate the percentage of each sector. We need to divide each sector value by the sum or total and then multiply it by 100.

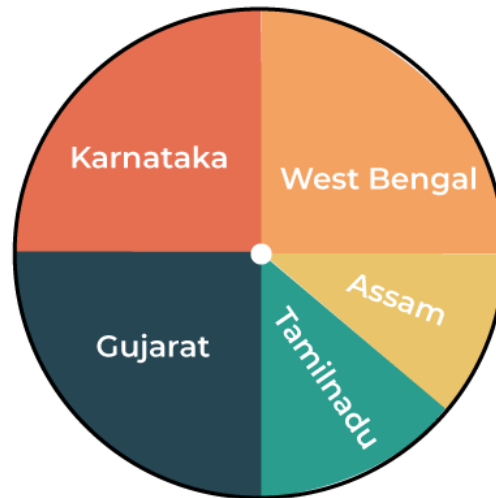
| West Bengal                   | Assam                          | Tamil Nadu                     | Gujarat                       | Karnataka                     |
|-------------------------------|--------------------------------|--------------------------------|-------------------------------|-------------------------------|
| $(10/40) \times 100$<br>= 25% | $(5/40) \times 100$<br>= 12.5% | $(5/40) \times 100$<br>= 12.5% | $(10/40) \times 100$<br>= 25% | $(10/40) \times 100$<br>= 25% |

**Step 4:** Next step is to calculate the degrees corresponding to each slice. The values can be calculated as:

| West Bengal                  | Assam                       | Tamil Nadu                  | Gujarat                      | Karnataka                    |
|------------------------------|-----------------------------|-----------------------------|------------------------------|------------------------------|
| $(10/40) \times 360$<br>= 90 | $(5/40) \times 360$<br>= 45 | $(5/40) \times 360$<br>= 45 | $(10/40) \times 360$<br>= 90 | $(10/40) \times 360$<br>= 90 |

**Step 5:** Now, with a protractor's help, we will measure each angle from a single point or central point and draw the circle's sectors. The resultant pie chart will be:

## Number of Students



**Example 3:** A pie chart is divided into four parts, and the values are given as  $x$ ,  $3x$ ,  $4x$ , and  $4x$ . Find the value of  $x$  in degrees.

**Solution:**

As it is known that a pie chart has  $360^\circ$ . Therefore, if all the angles are added, it will give  $360^\circ$ .

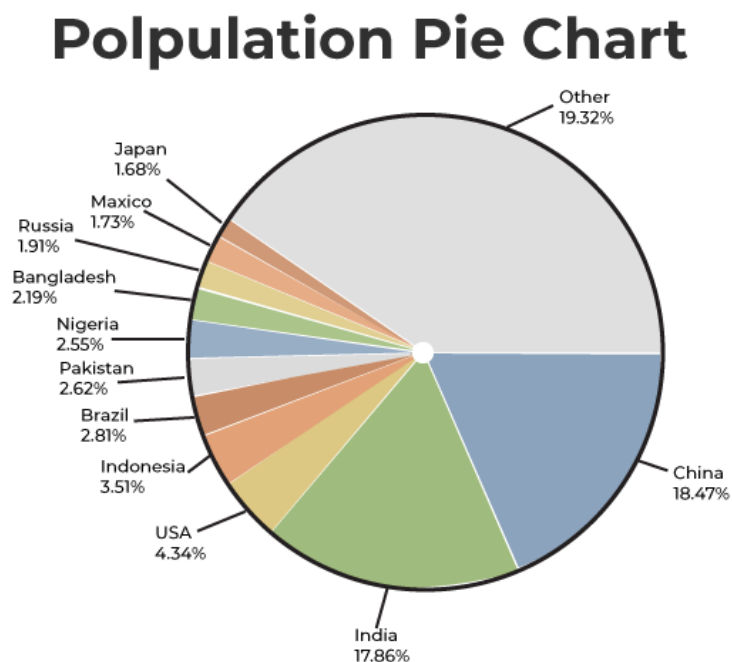
$$x + 3x + 4x + 4x = 360^\circ$$

$$12x = 360^\circ$$

$$x = 30^\circ$$

### Area Chart

A population pie chart represents the population distribution of a region or the world, as shown in the image below.



## Scatter Plot

Scatter plots are one of the most fundamental tools for visualizing relationships between two numerical variables. `matplotlib.pyplot.scatter()` plots points on a Cartesian plane defined by X and Y coordinates. Each point represents a data observation, allowing us to visually analyze how two variables correlate, cluster or distribute.

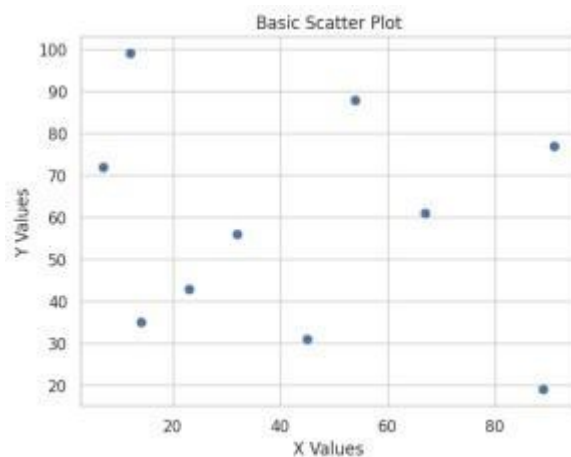
**For example:**

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array([12, 45, 7, 32, 89, 54, 23, 67, 14, 91])
y = np.array([99, 31, 72, 56, 19, 88, 43, 61, 35, 77])
```

```
plt.scatter(x, y)
plt.title("Basic Scatter Plot")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()
```

**Output**



Using `matplotlib.pyplot.scatter()`

**Explanation:** `plt.scatter(x, y)` creates a scatter plot on a 2D plane to visualize the relationship between two variables, with a title and axis labels added for clarity and context.

**Syntax**

```
matplotlib.pyplot.scatter(x, y, s=None, c=None, marker=None, cmap=None, alpha=None,
edgecolors=None, label=None)
```

**Parameters:**

| Parameter | Description                                   |
|-----------|-----------------------------------------------|
| x, y      | Sequences of data points to plot              |
| s         | Marker size (scalar or array-like)            |
| c         | Marker color                                  |
| marker    | Shape of the marker                           |
| cmap      | Colormap for mapping numeric values to colors |
| alpha     | Transparency (0 = transparent, 1 = opaque)    |

|            |                              |
|------------|------------------------------|
| edgecolors | Color of marker edges        |
| label      | Legend label for the dataset |

**Returns:** This function returns a PathCollection object representing the scatter plot points. This object can be used to further customize the plot or to update it dynamically.

### Examples

**Example 1:** In this example, we compare the height and weight of two different groups using different colors for each group.

```
x1 = np.array([160, 165, 170, 175, 180, 185, 190, 195, 200, 205])
```

```
y1 = np.array([55, 58, 60, 62, 64, 66, 68, 70, 72, 74])
```

```
x2 = np.array([150, 155, 160, 165, 170, 175, 180, 195, 200, 205])
```

```
y2 = np.array([50, 52, 54, 56, 58, 64, 66, 68, 70, 72])
```

```
plt.scatter(x1, y1, color='blue', label='Group 1')
```

```
plt.scatter(x2, y2, color='red', label='Group 2')
```

```
plt.xlabel('Height (cm)')
```

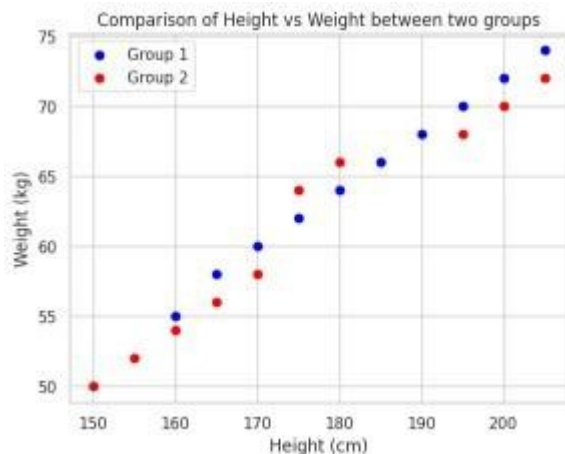
```
plt.ylabel('Weight (kg)')
```

```
plt.title('Comparison of Height vs Weight between two groups')
```

```
plt.legend()
```

```
plt.show()
```

### Output



Using matplotlib.pyplot.scatter()

**Explanation:** We define NumPy arrays x1, y1 and x2, y2 for height and weight data of two groups. Using plt.scatter(), Group 1 is plotted in blue and Group 2 in red, each with labels. The x-axis and y-axis are labeled "Height (cm)" and "Weight (kg)" for clarity.

**Example 2:** This example demonstrates how to customize a scatter plot using different marker sizes and colors for each point. Transparency and edge colors are also adjusted.

```
x = np.array([3, 12, 9, 20, 5, 18, 22, 11, 27, 16])
```

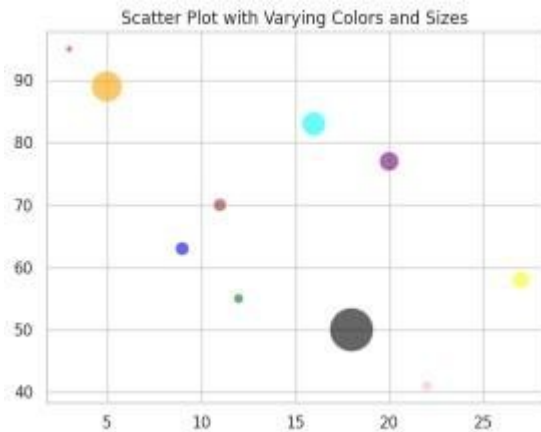
```
y = np.array([95, 55, 63, 77, 89, 50, 41, 70, 58, 83])
```

```
a = [20, 50, 100, 200, 500, 1000, 60, 90, 150, 300] # size
```

```
b = ['red', 'green', 'blue', 'purple', 'orange', 'black', 'pink', 'brown', 'yellow', 'cyan'] # color
```

```
plt.scatter(x, y, s=a, c=b, alpha=0.6, edgecolors='w', linewidths=1)
plt.title("Scatter Plot with Varying Colors and Sizes")
plt.show()
```

### Output



Using matplotlib.pyplot.scatter()

Explanation: NumPy arrays `x` and `y` set point coordinates, `a` defines marker sizes and `b` assigns colors. `plt.scatter()` plots the points with transparency, white edges and linewidth. A title is added before displaying the plot.

**Example 3:** This example shows how to create a bubble plot where the size of each point (bubble) represents a variable's magnitude. Edge color and alpha transparency are also used.

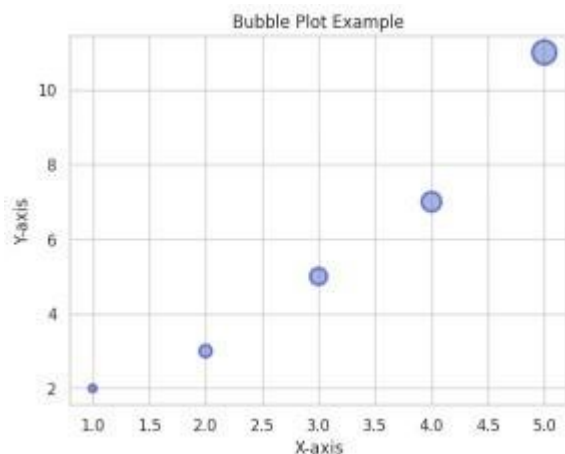
```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 3, 5, 7, 11]
```

```
sizes = [30, 80, 150, 200, 300] # Bubble sizes
```

```
plt.scatter(x, y, s=sizes, alpha=0.5, edgecolors='blue', linewidths=2)
plt.title("Bubble Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

### Output



Using `matplotlib.pyplot.scatter()`

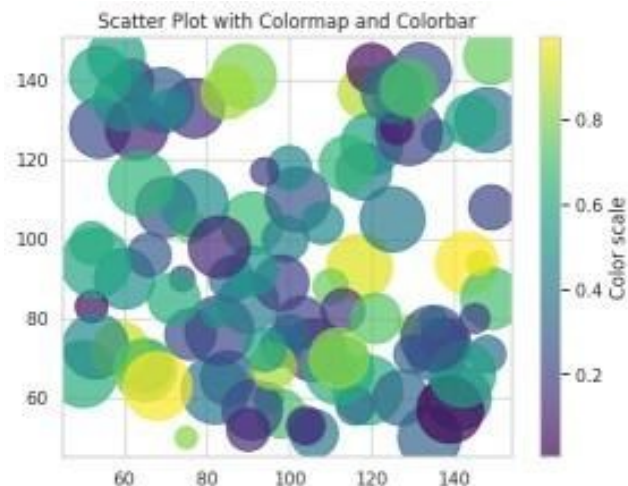
**Explanation:** Lists `x` and `y` define point coordinates, while `sizes` sets the marker (bubble) sizes. The `plt.scatter()` plots the bubbles with 50% transparency (`alpha=0.5`), blue edges and edge width of 2. Axis labels and a title are added before displaying the plot.

**Example 4:** In this example, we map data values to colors using a colormap and add a colorbar. This helps in visualizing a third variable via color intensity.

```
x = np.random.randint(50, 150, 100)
y = np.random.randint(50, 150, 100)
colors = np.random.rand(100) # Random float values for color mapping
sizes = 20 * np.random.randint(10, 100, 100)
```

```
plt.scatter(x, y, c=colors, s=sizes, cmap='viridis', alpha=0.7)
plt.colorbar(label='Color scale')
plt.title("Scatter Plot with Colormap and Colorbar")
plt.show()
```

**Output**



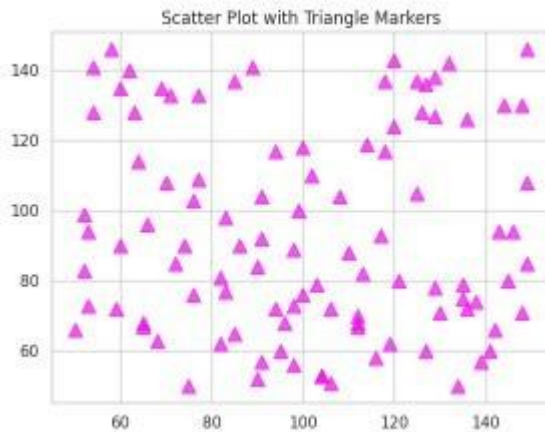
Using `matplotlib.pyplot.scatter()`

**Explanation:** Random arrays `x` and `y` set 100 points, with colors mapped using 'viridis' and varying sizes. `plt.scatter()` plots them with 0.7 transparency and `plt.colorbar()` adds a color legend.

**Example 5:** This final example illustrates how to change the marker style using the marker parameter. Here, triangle markers are used with magenta color.

```
plt.scatter(x, y, marker='^', color='magenta', s=100, alpha=0.7)
plt.title("Scatter Plot with Triangle Markers")
plt.show()
```

**Output**



Using `matplotlib.pyplot.scatter()`

**Explanation:** This code plots points with triangle markers ('^') in magenta color, size 100 and 0.7 transparency. A title is added before displaying the plot.

## Box Plot

A **Box Plot** (or **Whisker plot**) display the summary of a data set, including minimum, first quartile, median, third quartile and maximum. it consists of a box from the first quartile to the third quartile, with a vertical line at the median. the x-axis denotes the data to be plotted while the y-axis shows the frequency distribution. The `matplotlib.pyplot` module of `matplotlib` library provides `boxplot()` function with the help of which we can create box plots.

### Syntax

`matplotlib.pyplot.boxplot(data)`

The data values given to the `ax.boxplot()` method can be a Numpy array or Python list or Tuple of arrays. Let us create the box plot by using `numpy.random.normal()` to create some random data, it takes mean, standard deviation, and the desired number of values as arguments.

### Example:

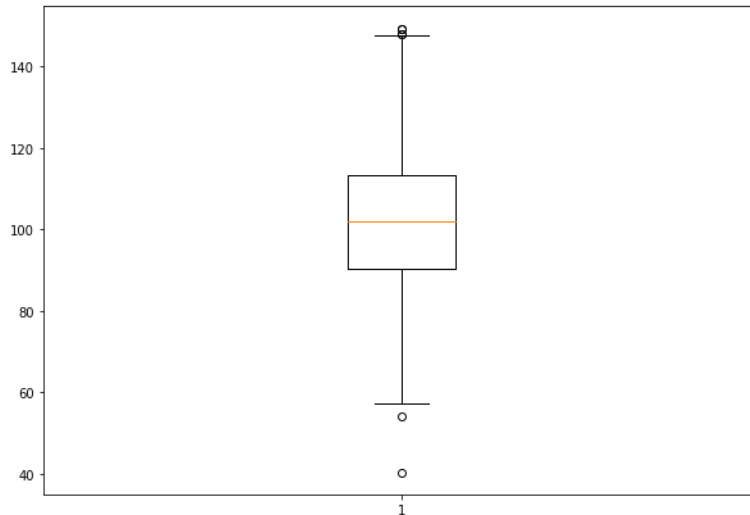
```
import matplotlib.pyplot as plt
import numpy as np
```

```
np.random.seed(10)
d = np.random.normal(100, 20, 200)
```

```
fig = plt.figure(figsize=(10, 7))
```

```
plt.boxplot(d)
plt.show()
```

### Output:



The basic box plot that displays the distribution of the randomly generated data.

### Customizing Box Plot

The `matplotlib.pyplot.boxplot()` provides endless customization possibilities to the box plot. some of the key customization parameters include:

- **The notch:** True attribute creates the notch format to the box plot
- **patch\_artist:** True fills the boxplot with colors, we can set different colors to different boxes.
- **vert:** 0 attribute creates horizontal box plot.
- **labels:** specifies custom labels for the boxes.

### Example 1: Multiple Datasets Box Plot

```
import matplotlib.pyplot as plt
import numpy as np
```

```
np.random.seed(10)
```

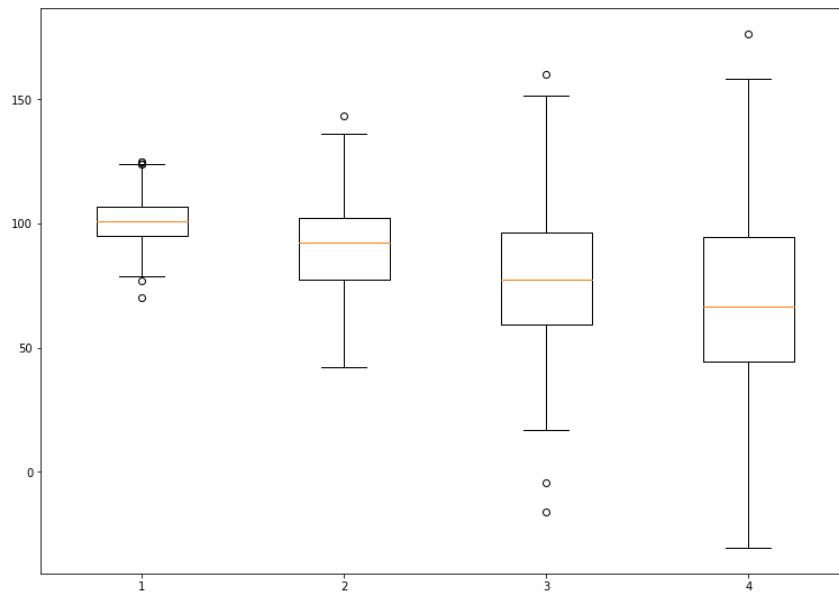
```
d_1 = np.random.normal(100, 10, 200)
d_2 = np.random.normal(90, 20, 200)
d_3 = np.random.normal(80, 30, 200)
d_4 = np.random.normal(70, 40, 200)
d = [d_1, d_2, d_3, d_4]
```

```
fig = plt.figure(figsize=(10, 7))
ax = fig.add_axes([0, 0, 1, 1])
bp = ax.boxplot(d)
```

```
plt.show()
```

**Output:**





**Example 2: Let's try to modify the above plot with some of the customizations:**

We will customize the plot by adding a notch, filling the boxes with colors, and modifying the whisker and median styles.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
np.random.seed(10)
d_1 = np.random.normal(100, 10, 200)
d_2 = np.random.normal(90, 20, 200)
d_3 = np.random.normal(80, 30, 200)
d_4 = np.random.normal(70, 40, 200)
d = [d_1, d_2, d_3, d_4]
```

```
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111)
```

```
bp = ax.boxplot(d, patch_artist = True,
 notch = 'True', vert = 0)
```

```
colors = ['#0000FF', '#00FF00',
 '#FFFF00', '#FF00FF']
```

```
for patch, color in zip(bp['boxes'], colors):
 patch.set_facecolor(color)
```

```
for whisker in bp['whiskers']:
 whisker.set(color = '#8B008B',
 linewidth = 1.5,
 linestyle = ":")
```

```
changing color and linewidth of
for cap in bp['caps']:
 cap.set(color = '#8B008B',
 linewidth = 2)
```

```
for median in bp['medians']:
 median.set(color = 'red',
 linewidth = 3)
```

```
changing style of fliers
for flier in bp['fliers']:
 flier.set(marker = 'D',
 color = '#e7298a',
 alpha = 0.5)
```

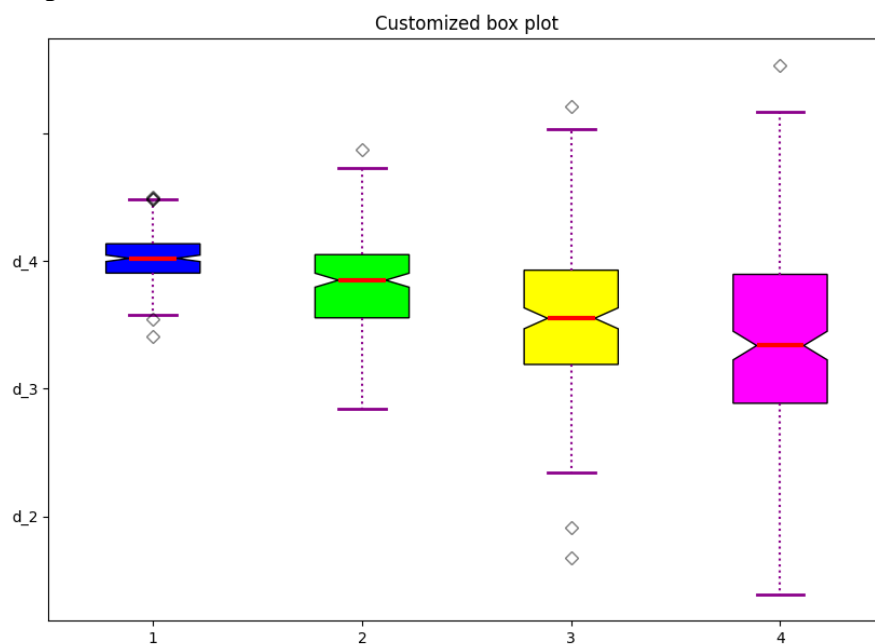
```
ax.set_yticklabels(['d_1', 'd_2',
 'd_3', 'd_4'])
```

```
plt.title("Customized box plot")
```

```
ax.get_xaxis().tick_bottom()
ax.get_yaxis().tick_left()
```

```
plt.show()
```

**Output:**



customizes box plot

## Area Plot

Area line plots are an effective tool in the data visualization toolbox for illustrating relationships and trends across time. They provide a comprehensive and visually captivating method of presenting numerical data by expertly combining the readability of line charts with the eye-catching attractiveness of filled areas.

In this article, we will learn more about creating an area line chart in Python. As Area line plots offer a valuable tool for data visualization, providing a clear and engaging way to convey trends and relationships over time.

### What is an Area Line Plot?

An Area Line Plot, also known as an Area Chart or Stacked Area Chart, is a data visualization technique that is used to represent data over time or across categories. It is an extension of the basic line chart and is particularly useful when you want to show the composition of a whole, along with the individual components, as well as how they change over time or across categories. In this article, we will explore how to create area line plots in Python using the matplotlib library and explain their significance in visualizing data.

Here are the key components and characteristics of an Area Line Plot.

**X-Axis:** The horizontal axis represents the independent variable, typically time or categories. It is a continuous or categorical scale that provides context for the data points.

**Y-Axis:** The vertical axis represents the dependent variable, usually a numeric value that measures the quantity or magnitude of what you are visualizing.

**Lines:** The individual lines in an area line plot represent different categories, groups, or components. Each line starts at the baseline (usually the X-axis) and goes up to show the value of that category or component at a particular point in time or along the category axis.

**Area Filling:** The area between the line and the baseline is filled with colour to make it visually distinct. The area's colour is often used to represent the category or component it represents.

**Stacking:** In a stacked area chart, each line is stacked on top of the previous one. This stacking illustrates how the total changes over time or across categories, as well as the contribution of each category to the whole.

### Creating an Area Line Plot

First, let's make a basic area line plot using Python. To create the plot and show how various categories change over time, we will use Matplotlib.

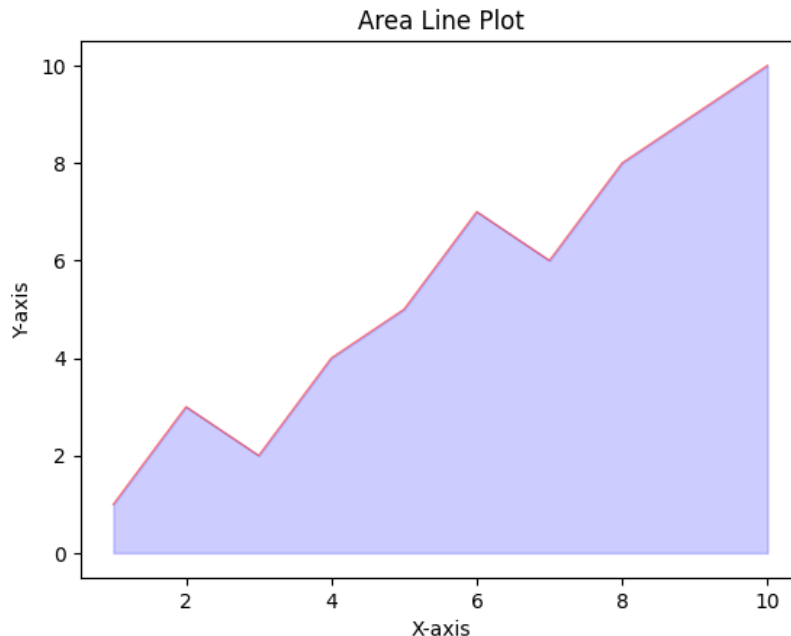
```
import pandas as pd
import matplotlib.pyplot as plt
```

```
Sample data
df = pd.DataFrame({
 'x': list(range(1, 11)),
 'y': [1, 3, 2, 4, 5, 7, 6, 8, 9, 10]
})

Create the area line plot
plt.fill_between(df['x'], df['y'], color='blue', alpha=0.2)
plt.plot(df['x'], df['y'], color='red', alpha=0.5, linewidth=0.9)
```

```
plt.title("Area Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

**Output:**



Area line Plot

### Area Line Plot with markers and Labels

Add some more function to make it attractive

- Import necessary libraries: pandas for data manipulation and matplotlib.pyplot for data visualization.
- Prepare sample data: Create a DataFrame with columns for 'x' and 'y' containing numerical values.
- Generate area line plot: Use `plt.fill_between()` to create a semi-transparent blue area line plot and `plt.plot()` to create a faint red line.
- Enhance plot with markers and labels: Add red circular markers (`s=30`) at data points using `plt.scatter()`. Add black labels above each data point using `plt.text()`, positioning them horizontally centered (`ha='center'`) and vertically aligned at the bottom (`va='bottom'`). Customize axes, add title, and display the plot.

```
import pandas as pd
import matplotlib.pyplot as plt
```

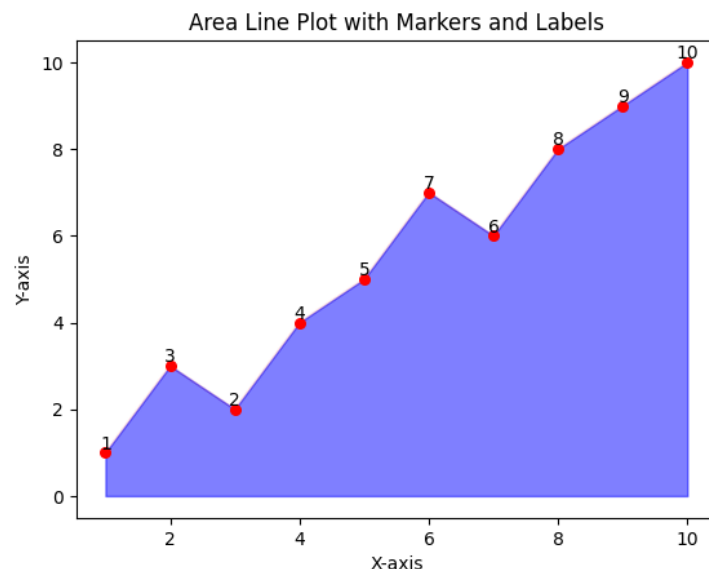
```
Sample data
df = pd.DataFrame({
 'x': list(range(1, 11)),
 'y': [1, 3, 2, 4, 5, 7, 6, 8, 9, 10]
})
```

```
Create the area line plot
plt.fill_between(df['x'], df['y'], color='blue', alpha=0.5)
plt.plot(df['x'], df['y'], color='red', alpha=0.1)

Add red markers at data points
plt.scatter(df['x'], df['y'], color='red', s=30)

Add labels above data points
for i, row in df.iterrows():
 plt.text(row['x'], row['y'], str(row['y']), ha='center', va='bottom', color='black', size=10)

plt.title("Area Line Plot with Markers and Labels")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

**Output:****Area Line Plot****Stacked Area Line Plot**

- Import necessary libraries: pandas for data manipulation and matplotlib.pyplot for data visualization.
- Prepare sample data: Create a DataFrame with columns for 'x', 'Category A', 'Category B', and 'Category C' containing numerical values.
- Define custom colors: Define a list of colors ('yellow', 'purple', 'pink') to be used for each category.
- Generate stacked area line plot: Use plt.stackplot() to create a stacked area line plot with the defined colors and transparency. Additionally, plot individual lines for each category with different colors, linewidths, and transparency. Add labels, customize axes, and display the plot.

```
import pandas as pd
import matplotlib.pyplot as plt
```

```

Sample data
df = pd.DataFrame({
 'x': list(range(1, 11)),
 'Category A': [1, 3, 2, 4, 5, 7, 6, 8, 9, 10],
 'Category B': [2, 4, 3, 5, 6, 8, 7, 9, 10, 11],
 'Category C': [3, 5, 4, 6, 7, 9, 8, 10, 11, 12]
})

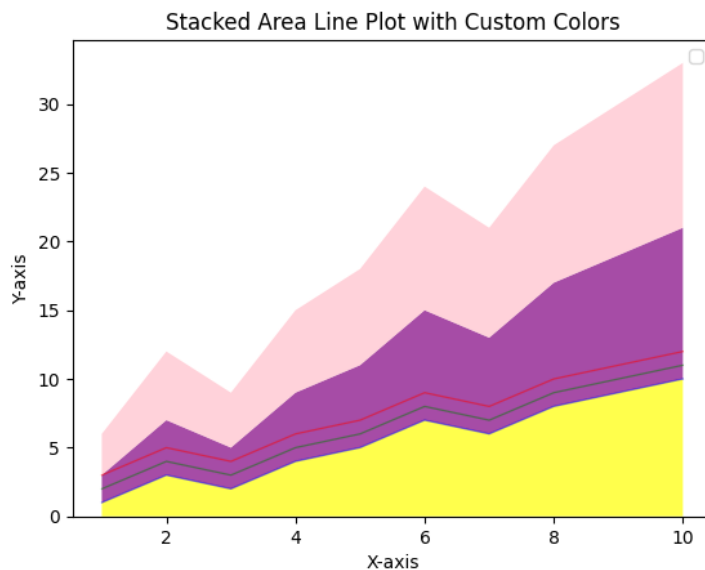
Define custom colors for each category
colors = ['yellow', 'purple', 'pink']

Create the stacked area line plot with custom colors
plt.stackplot(df['x'], df['Category A'], df['Category B'], df['Category C'], colors=colors,
alpha=0.7)

Plot lines for each category with custom colors
plt.plot(df['x'], df['Category A'], color='blue', alpha=0.5, linewidth=0.9)
plt.plot(df['x'], df['Category B'], color='green', alpha=0.5, linewidth=0.9)
plt.plot(df['x'], df['Category C'], color='red', alpha=0.5, linewidth=0.9)

plt.title("Stacked Area Line Plot with Custom Colors")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.show()

```

**Output:****Filling between lines**

- Import necessary libraries: matplotlib.pyplot for data visualization and numpy for numerical operations.

- Prepare sample data: Generate x-axis values using `np.linspace` and calculate corresponding y-values using `sin` and `cos` functions.
- Create plot elements: Create a figure and axis using `plt.subplots()` and plot two lines using `ax.plot()` with different colors and labels.
- Fill between lines: Use `ax.fill_between()` to fill the area between the lines with a specific color and transparency, applying conditions and interpolation. Add labels, customize axes, and display the plot.

```
import matplotlib.pyplot as plt
import numpy as np

Sample data for demonstration
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

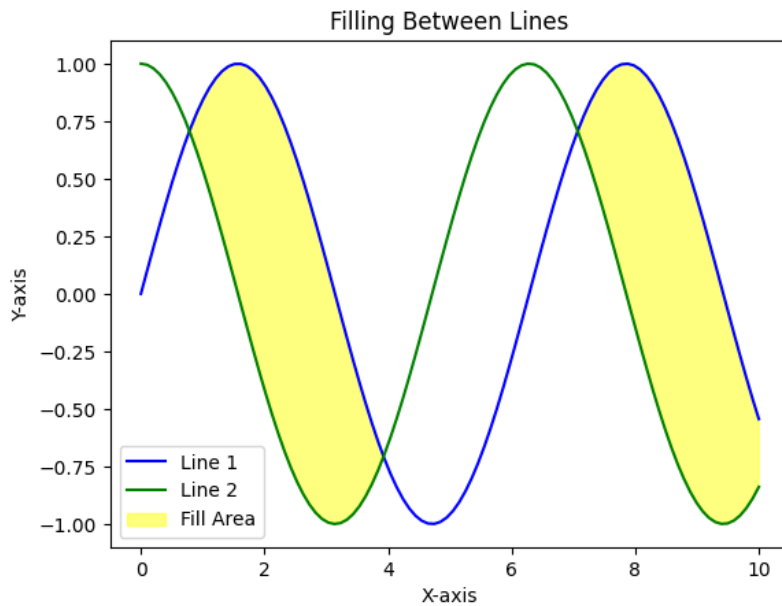
Create a figure and axis
fig, ax = plt.subplots()

Plot the two lines
ax.plot(x, y1, label='Line 1', color='blue')
ax.plot(x, y2, label='Line 2', color='green')

Fill the area between the lines
ax.fill_between(x, y1, y2, where=(y1 > y2), interpolate=True, alpha=0.5, color='yellow',
label='Fill Area')

Customize the plot
ax.set_xlabel('X-axis')
ax.set_ylabel('Y-axis')
ax.set_title('Filling Between Lines')
ax.legend()

Display the plot
plt.show()
Output:
```



### Area Line plot with Plotly

- Import essential libraries: Plotly Express for creating charts and Pandas for data manipulation.
- Prepare sample data: Create a DataFrame with 'x' and 'y' columns containing integer values.
- Generate area line plot: Use `px.area()` to create an area line plot with fill-to-zero, linear line shape, and green line color.
- Customize layout: Add title, axis labels, and hide legend using `update_layout()`.

```
import plotly.graph_objects as go
import pandas as pd
```

```
Sample data
```

```
df = pd.DataFrame({
 'x': list(range(1, 11)),
 'y': [1, 3, 2, 4, 5, 7, 6, 8, 9, 10]
})
```

```
Create the Area Line Plot
```

```
fig = go.Figure(data=go.Scatter(x=df['x'], y=df['y'], fill='tozeroy', mode='lines',
line=dict(color='green')))
```

```
fig.update_layout(
 title="Area Line Plot with Plotly",
 xaxis_title="X-axis",
 yaxis_title="Y-axis",
 showlegend=False # Hide legend
)
```

```
fig.show()
```



**Output:**

Area Line Plot with Plotly

**Heatmap**

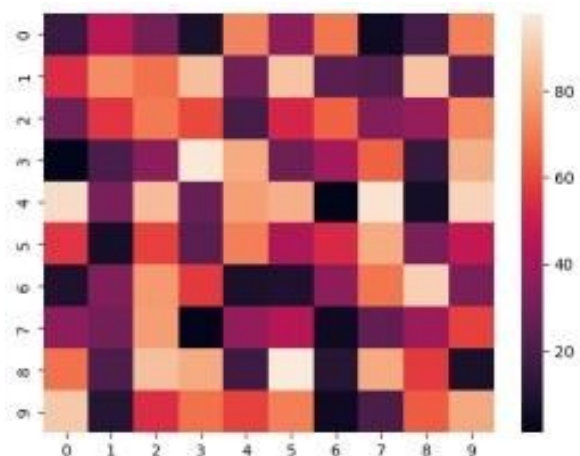
A **heatmap** is a graphical representation of data where individual values are represented by color intensity. It is widely used in data analysis and visualization to identify patterns, correlations and trends within a dataset. **Heatmaps in Seaborn** can be plotted using the **seaborn.heatmap()** function, which offers extensive customization options. Let's explore different methods to create and enhance heatmaps using Seaborn.

**Example:** The following example demonstrates how to create a simple heatmap using the Seaborn library.

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
Generating a 10x10 matrix of random numbers
data = np.random.randint(1, 100, (10, 10))
```

```
sns.heatmap(data)
plt.show()
```



Basic heatmap

**Explanation:** This will produce a heatmap where the intensity of color represents the magnitude of values in the matrix.

### Syntax of `seaborn.heatmap()`

```
seaborn.heatmap(data, *, vmin=None, vmax=None, cmap=None, center=None,
annot_kws=None, linewidths=0, linecolor='white', cbar=True, **kwargs)
```

### Parameters:

- **data:** A 2D dataset that can be coerced into a NumPy ndarray.
- **vmin, vmax:** Values to anchor the colormap; if not specified, they are inferred from the data.
- **cmap:** The colormap for mapping data values to colors.
- **center:** Value at which to center the colormap when plotting divergent data.
- **annot:** If True, displays numerical values inside the cells.
- **fmt:** String format for annotations.
- **linewidths:** Width of the lines separating cells.
- **linecolor:** Color of the separating lines.
- **cbar:** Whether to display a color bar.

Note: All parameters except data are optional.

**Returns:** returns an object of type `matplotlib.axes.Axes`

### Examples

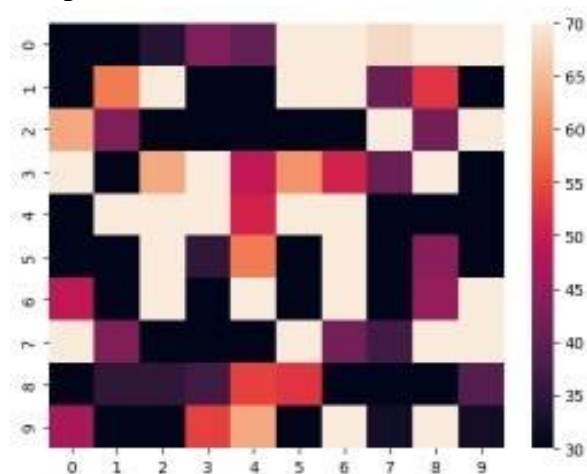
**Example 1:** By setting **vmin** and **vmax**, we explicitly define the range of values that influence the color scaling in the heatmap. This ensures consistency across multiple heatmaps, preventing extreme values from distorting the visualization.

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
data = np.random.randint(1, 100, (10, 10))
```

```
sns.heatmap(data, vmin=30, vmax=70)
plt.show()
```

### Output:



Anchoring the Colormap

**Explanation:** The parameters `vmin=30` and `vmax=70` define the limits of the color scale range. Values lower than 30 are represented by the lowest intensity color, while values higher than 70 are mapped to the highest intensity color.

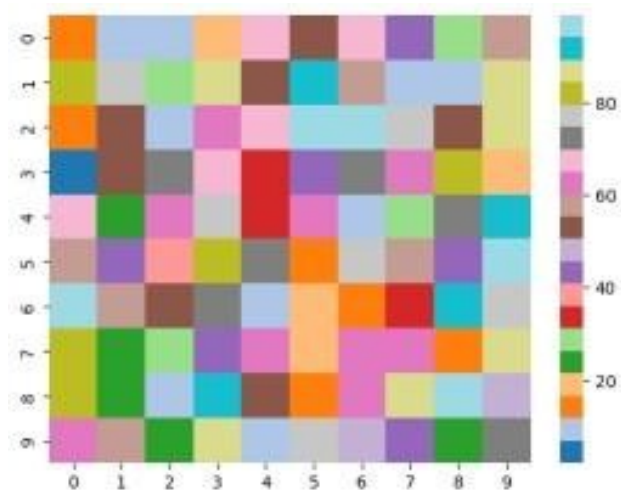
**Example 2:** The **`cmap`** parameter allows us to modify the color scheme of the heatmap, improving its visual appeal and making it easier to distinguish between different values.

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
data = np.random.randint(1, 100, (10, 10))
```

```
sns.heatmap(data, cmap='tab20')
plt.show()
```

**Output:**



Choosing a Colormap

**Explanation:** The argument **`cmap='tab20'`** sets a categorical colormap. Sequential colormaps (Blues, Reds, Greens) suit continuous data, while diverging colormaps (coolwarm, RdBu) highlight variations.

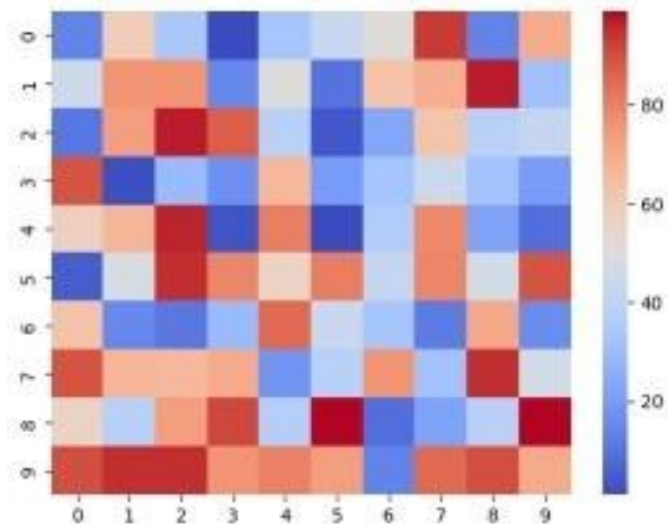
**Example 3:** By setting the **`center`** parameter, we ensure that a specific value (such as the mean or median) appears neutral in the colormap. This adjustment makes deviations above and below this value more distinguishable.

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
data = np.random.randint(1, 100, (10, 10))
```

```
sns.heatmap(data, cmap='coolwarm', center=50)
plt.show()
```

**Output:**



### Centering the Colormap

**Explanation:** The `center=50` parameter sets 50 as a neutral color. Values above it are shaded in one color (e.g., red) and below in another (e.g., blue), making it useful for highlighting deviations from a reference point.

### Advanced customizations in seaborn heatmap

Seaborn's `heatmap()` function provides various options for customization, allowing users to enhance visualization by adjusting color schemes, labels, scaling and spacing.

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import matplotlib.colors as mcolors
```

```
data = np.random.rand(10, 10) * 100
```

```
plt.figure(figsize=(12, 8)) # Adjust figure size
```

```
sns.heatmap(
 data,
 xticklabels=list("ABCDEFGHIJ"), # Custom x-axis labels
 yticklabels=False, # Hide y-axis labels
 norm=mcolors.LogNorm(), # Logarithmic scaling
 cmap="coolwarm", # Color map
 linewidths=0.5 # Cell spacing
)
```

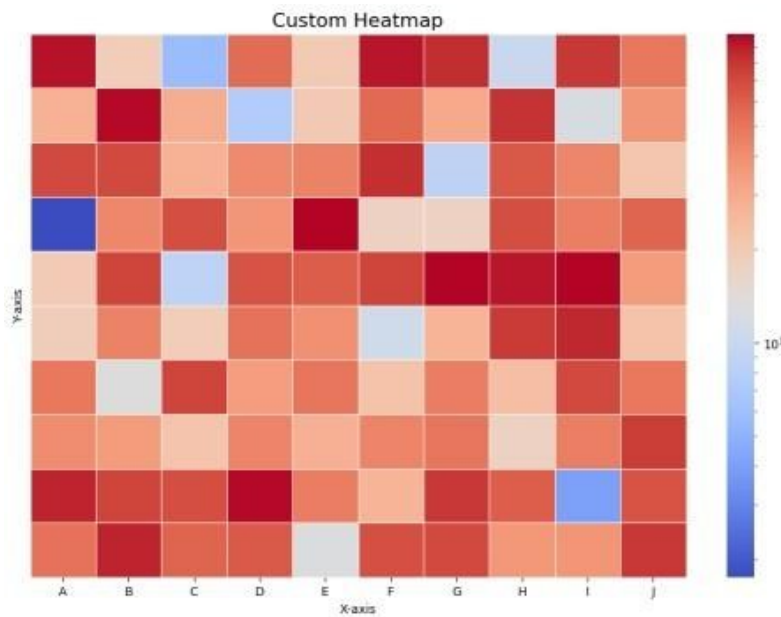
```
Add title and labels
```

```
plt.title("Custom Heatmap", fontsize=16)
```

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

```
plt.show()
```

**Output:**

Advanced customization

**Explanation:**

- **plt.figure(figsize=(12, 8))** enlarges the figure for better visibility.
- **xticklabels=list("ABCDEFGHIIJ")** replaces default labels with A-J.
- **yticklabels=False** removes y-axis labels for clarity.
- **norm=mcolors.LogNorm()** enhances contrast for varied data.
- **cmap="coolwarm"** uses a blue-to-red gradient.
- **linewidths=0.5** adds thin separation lines.
- **plt.title("Custom Heatmap", fontsize=16)** adds a title with a readable font size.
- **plt.xlabel("X-axis") & plt.ylabel("Y-axis")** labels the axes for clear understanding.

## **Python Programming Lab Manual**

- 1) Write a python program to demonstrate the usage of bitwise operators.
- 2) Write a python program to find largest and smallest of three numbers.
- 3) Write a python program to print 'n' terms of Fibonacci series of numbers
- 4) Write a python program to find factorial of a number using Recursion.
- 5) Write a Python program to find number of occurrences of each vowel present in the given string.
- 6) Write a program to create, concatenate and print a string and accessing sub-string from a given string.
- 7) Write a Python program to find whether the given pattern is available in the given string or not?
- 8) Write a python program to check whether the given mail id is valid gmail id or not?
- 9) Write a program to create a Lambda Function to find biggest of given values.
- 10) Write a program to create, append, and remove lists in python.
- 11) Write a program to demonstrate working with tuples in python.
- 12) Write a program to demonstrate working with dictionaries in python.
- 13) Write a python program to perform read and write operations on a file.
- 14) Write a python program to copy the contents of one file to another file
- 15) Write a Python program to visualize marks scored by five students using a bar chart

### 1. Write a python program to demonstrate the usage of bitwise operators.

#### Explanation:

*& (AND): Sets each bit to 1 if both bits are 1*  
*| (OR): Sets each bit to 1 if at least one bit is 1*  
*^ (XOR): Sets each bit to 1 if bits are different*  
*~ (NOT): Inverts all bits (2's complement)*  
*<< (Left Shift): Shifts bits left, multiplying by 2*  
*>> (Right Shift): Shifts bits right, dividing by 2*

# Python program to demonstrate bitwise operators

a = 10 # 1010 in binary  
b = 4 # 0100 in binary

print("a =", a)  
print("b =", b)

# Bitwise AND

print("\nBitwise AND (a & b):", a & b) # 1010 & 0100 = 0000 → 0

# Bitwise OR

print("Bitwise OR (a | b):", a | b) # 1010 | 0100 = 1110 → 14

# Bitwise XOR

print("Bitwise XOR (a ^ b):", a ^ b) # 1010 ^ 0100 = 1110 → 14

# Bitwise NOT

print("Bitwise NOT (~a):", ~a) # -(a+1) → -11

# Left Shift

print("Left Shift (a << 2):", a << 2) # 1010 → 101000 → 40

# Right Shift

print("Right Shift (a >> 2):", a >> 2) # 1010 → 0010 → 2

#### Output

a = 10  
b = 4  
Bitwise AND (a & b): 0  
Bitwise OR (a | b): 14  
Bitwise XOR (a ^ b): 14  
Bitwise NOT (~a): -11  
Left Shift (a << 2): 40  
Right Shift (a >> 2): 2

### **2) Write a python program to find largest and smallest of three numbers.**

#### ***a) Python program to find largest and smallest of three numbers***

```
Taking input from user
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))
```

```
Finding largest
if a >= b and a >= c:
 largest = a
elif b >= a and b >= c:
 largest = b
else:
 largest = c
```

```
Finding smallest
if a <= b and a <= c:
 smallest = a
elif b <= a and b <= c:
 smallest = b
else:
 smallest = c
```

```
Display results
print("Largest number:", largest)
print("Smallest number:", smallest)
```

#### ***b). Using built-in functions***

```
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))
```

```
print("Largest number:", max(a, b, c))
print("Smallest number:", min(a, b, c))
```

#### **Output:**

```
Enter first number: 50
Enter second number: 40
Enter third number: 65
Largest number: 65.0
Smallest number: 40.0
```



**3) Write a python program to print 'n' terms of Fibonacci series of numbers**

***a) Using a loop:***

```
Python program to print n terms of Fibonacci series
n = int(input("Enter number of terms: "))
a, b = 0, 1
print("Fibonacci Series:")
for i in range(n):
 print(a, end=" ")
 a, b = b, a + b
```

***b) Using recursion***

```
Recursive function for Fibonacci

def fib(n):
 if n <= 1:
 return n
 return fib(n-1) + fib(n-2)

n = int(input("Enter number of terms: "))

print("Fibonacci Series:")
for i in range(n):
 print(fib(i), end=" ")
```

**Output:**

```
Enter number of terms: 7
Fibonacci Series:
0 1 1 2 3 5 8
```

### **4) Write a python program to find factorial of a number using Recursion.**

# Python program to find factorial using recursion

```
def factorial(n):
 if n == 0 or n == 1: # Base case
 return 1
 else:
 return n * factorial(n - 1) # Recursive call

Taking input from user
num = int(input("Enter a number: "))

Check for negative numbers
if num < 0:
 print("Factorial is not defined for negative numbers.")
else:
 result = factorial(num)
 print("Factorial of", num, "is:", result)
```

#### **Output:**

```
Enter a number: 5
Factorial of 5 is: 120
```

**5) Write a Python program to find number of occurrences of each vowel present in the given string.**

***a) Using dictionary:***

# Python program to count occurrences of each vowel

# Input string

text = input("Enter a string: ")

# Initialize dictionary

vowels = {'a':0, 'e':0, 'i':0, 'o':0, 'u':0}

# Convert string to lowercase

text = text.lower()

# Count vowels

for ch in text:

if ch in vowels:

vowels[ch] += 1

# Display results

print("Vowel occurrences:")

for vowel, count in vowels.items():

print(vowel, ":", count)

***b) Using built-in count method***

text = input("Enter a string: ").lower()

for v in "aeiou":

print(v, ":", text.count(v))

**Output:**

*Enter a string: Hello World*

*Vowel occurrences:*

*a : 0*

*e : 1*

*i : 0*

*o : 2*

*u : 0*

### **6) Write a program to create, concatenate and print a string and accessing sub-string from a given string.**

```
Creating strings
str1 = "Hello"
str2 = "World"

Concatenation
str3 = str1 + " " + str2

Printing the concatenated string
print("Concatenated String:", str3)

Accessing characters
print("First character:", str3[0])
print("Last character:", str3[-1])

Accessing substring (slicing)
print("Substring (0 to 4):", str3[0:5]) # Hello
print("Substring (6 to end):", str3[6:]) # World
print("Substring (last 5 chars):", str3[-5:])

Taking user input
text = input("\nEnter a string: ")

Printing entered string
print("Entered String:", text)

Accessing substring from user string
start = int(input("Enter start index: "))
end = int(input("Enter end index: "))

print("Substring:", text[start:end])
```

#### **Output:**

```
Concatenated String: Hello World
First character: H
Last character: d
Substring (0 to 4): Hello
Substring (6 to end): World
Substring (last 5 chars): World
```

**7) Write a Python program to find whether the given pattern is available in the given string or not?**

***a) Using in operator (simple method):***

# Python program to check if a pattern exists in a string

```
text = input("Enter the main string: ")
pattern = input("Enter the pattern to search: ")
```

```
if pattern in text:
 print("Pattern found in the string.")
else:
 print("Pattern not found in the string.")
```

***b) Using find() method:***

```
text = input("Enter the main string: ")
pattern = input("Enter the pattern to search: ")

if text.find(pattern) != -1:
 print("Pattern found at index:", text.find(pattern))
else:
 print("Pattern not found.")
```

**Output:**

```
Enter the main string: Hello World
Enter the pattern to search: World
Pattern found in the string.
```

**8) Write a python program to check whether the given mail id is valid gmail id or not?**

***a) Using simple string checks:***

```
Python program to validate Gmail ID

email = input("Enter your email ID: ")

Check conditions
if email.endswith("@gmail.com") and "@" in email and email.index("@") > 0:
 print("Valid Gmail ID")
else:
 print("Invalid Gmail ID")
```

***b) Using Regular Expressions:***

```
import re
email = input("Enter your email ID: ")
Regex pattern for Gmail validation
pattern = r'^[a-zA-Z0-9._%+-]+@gmail\.com$'

if re.match(pattern, email):
 print("Valid Gmail ID")
else:
 print("Invalid Gmail ID")
```

**Output:**

```
Enter your email ID: example123@gmail.com
Valid Gmail ID
```

**9) Write a program to create a Lambda Function to find biggest of given values.**

**a) *For three numbers:***

```
Lambda function to find biggest of three numbers
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

largest = (lambda x, y, z: x if x > y and x > z else (y if y > z else z))(a, b, c)

print("Largest number is:", largest)
```

**b) *For multiple values using lambda + max***

```
Lambda with multiple values
nums = list(map(int, input("Enter numbers separated by space: ").split()))
largest = (lambda x: max(x))(nums)
print("Largest number is:", largest)
```

**Output:**

```
Enter numbers separated by space: 10 25 7 40 18
Largest number is: 40
```

**10) Write a program to create, append, and remove lists in python.**

```
Creating a list
my_list = [10, 20, 30, 40]
print("Original List:", my_list)
Appending elements
my_list.append(50)
print("After appending 50:", my_list)
Adding multiple elements
my_list.extend([60, 70])
print("After extending [60, 70]:", my_list)

Removing elements
my_list.remove(30) # removes specific value
print("After removing 30:", my_list)

Removing last element
my_list.pop()
print("After popping last element:", my_list)

Removing element by index
my_list.pop(1)
print("After popping element at index 1:", my_list)
```

**Output:**

```
Original List: [10, 20, 30, 40]
After appending 50: [10, 20, 30, 40, 50]
After extending [60, 70]: [10, 20, 30, 40, 50, 60, 70]
After removing 30: [10, 20, 40, 50, 60, 70]
After popping last element: [10, 20, 40, 50, 60]
After popping element at index 1: [10, 40, 50, 60]
```



**11) Write a program to demonstrate working with tuples in python.**

```
Creating a tuple
t = (10, 20, 30, 40, 50)

print("Tuple:", t)

Accessing elements
print("First element:", t[0])
print("Last element:", t[-1])

Slicing
print("Elements from index 1 to 3:", t[1:4])

Tuple length
print("Length of tuple:", len(t))

Iterating through tuple
print("Tuple elements:")
for item in t:
 print(item, end=" ")

Concatenation
t2 = (60, 70)
t3 = t + t2
print("\nAfter concatenation:", t3)

Repetition
t4 = t * 2
print("After repetition:", t4)

Membership test
print("Is 30 present in tuple?", 30 in t)

Nested tuple
nested = (1, (2, 3), (4, 5))
print("Nested tuple:", nested)
print("Access nested element:", nested[1][0])

Tuple unpacking
a, b, c, d, e = t
print("Unpacked values:", a, b, c, d, e)
```

### **Output:**

*Tuple: (10, 20, 30, 40, 50)*

*First element: 10*

*Last element: 50*

*Elements from index 1 to 3: (20, 30, 40)*

*Length of tuple: 5*

*Tuple elements: 10 20 30 40 50*

*After concatenation: (10, 20, 30, 40, 50, 60, 70)*

*After repetition: (10, 20, 30, 40, 50, 10, 20, 30, 40, 50)*

*Is 30 present in tuple? True*

*Nested tuple: (1, (2, 3), (4, 5))*

*Access nested element: 2*

*Unpacked values: 10 20 30 40 50*

**12) Write a program to demonstrate working with dictionaries in python.**

```
Creating a dictionary
student = {
 "name": "Adishesha",
 "age": 25,
 "course": "MCA "
}

print("Original Dictionary:", student)

Accessing values
print("Name:", student["name"])
print("Age:", student.get("age"))

Adding new key-value pair
student["grade"] = "A"
print("After adding grade:", student)

Updating value
student["age"] = 26
print("After updating age:", student)

Removing elements
student.pop("course") # removes specific key
print("After removing 'course':", student)

Iterating through dictionary
print("Dictionary elements:")
for key, value in student.items():
 print(key, ":", value)

Keys and Values separately
print("Keys:", student.keys())
print("Values:", student.values())

Checking key existence
print("Is 'name' present?", "name" in student)

Copying dictionary
student_copy = student.copy()
print("Copied Dictionary:", student_copy)

Clearing dictionary
student.clear()
```

## *Python Programming*

```
print("After clearing:", student)
```

### **Output:**

*Original Dictionary: {'name': 'Adishesha', 'age': 25, 'course': 'MCA'}*  
*Name: Adishesha*  
*Age: 25*  
*After adding grade: {'name': 'Adishesha', 'age': 25, 'course': 'MCA', 'grade': 'A'}*  
*After updating age: {'name': 'Adishesha', 'age': 26, 'course': 'MCA', 'grade': 'A'}*  
*After removing 'course': {'name': 'Adishesha', 'age': 26, 'grade': 'A'}*  
*Dictionary elements:*  
*name : Adishesha*  
*age : 26*  
*grade : A*  
*Keys: dict\_keys(['name', 'age', 'grade'])*  
*Values: dict\_values(['Adishesha', 26, 'A'])*  
*Is 'name' present? True*  
*Copied Dictionary: {'name': 'Adishesha', 'age': 26, 'grade': 'A'}*  
*After clearing: {}*

### **13) Write a python program to perform read and write operations on a file.**

#### ***a) Write to a file and then read from it***

```
Writing to a file
file = open("sample.txt", "w")
file.write("Hello, this is a file handling example.\n")
file.write("Python makes file operations easy.")
file.close()

print("Data written to file successfully.")

Reading from the file
file = open("sample.txt", "r")
content = file.read()
file.close()

print("\nContent of the file:")
print(content)
```

**Output:**

Data written to file successfully.

Content of the file:

Hello, this is a file handling example. Python makes file operations easy.

#### ***b) Using with statement***

```
Writing to a file
with open("sample.txt", "w") as file:
 file.write("Welcome to Python File Handling.\n")
 file.write("This file is created using 'with' statement.")

print("Data written successfully.")

Reading from the file
with open("sample.txt", "r") as file:
 print("\nFile Content:")
 for line in file:
 print(line, end="")
```

**Output:**

Data written successfully.

File Content:

Welcome to Python File Handling.

This file is created using 'with' statement.

### **14) Write a python program to copy the contents of one file to another file.**

#### ***a) Using basic file handling:***

# Python program to copy contents of one file to another

# Open source file in read mode

```
src = open("source.txt", "r")
```

# Open destination file in write mode

```
dest = open("destination.txt", "w")
```

# Read content from source and write to destination

```
content = src.read()
```

```
dest.write(content)
```

# Close files

```
src.close()
```

```
dest.close()
```

```
print("File copied successfully.")
```

#### ***b) Using with statement***

# Copy file using 'with' statement

```
with open("source.txt", "r") as src, open("destination.txt", "w") as dest:
```

```
 for line in src:
```

```
 dest.write(line)
```

```
print("File copied successfully.")
```

### **Output:**

*File copied successfully.*

### **15) Write a Python program to visualize marks scored by five students using a bar chart.**

```
import matplotlib.pyplot as plt
```

```
students = []
```

```
marks = []
```

```
Input for 5 students
```

```
for i in range(5):
```

```
 name = input(f'Enter name of student {i+1}: ")
```

```
 score = int(input(f'Enter marks of {name}: "))
```

```
 students.append(name)
```

```
 marks.append(score)
```

## *Python Programming*

```
plt.bar(students, marks)
plt.xlabel("Students")
plt.ylabel("Marks")
plt.title("Marks Scored by Students")
plt.show()
```

### **Output:**

*Enter name of student 1: A*  
*Enter marks of A: 77*  
*Enter name of student 2: B*  
*Enter marks of B: 67*  
*Enter name of student 3: C*  
*Enter marks of C: 80*  
*Enter name of student 4: D*  
*Enter marks of D: 36*  
*Enter name of student 5: E*  
*Enter marks of E: 90*

